

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2022

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, tháng 2 năm 2022

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2022

IOI China candidate team papers / National training team 2022 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2022. Bản dịch bao gồm phần nhận diện tập sách, mục lục và toàn bộ mười bốn bài trong tập.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2022*. Trang bìa ghi đơn vị China Computer Federation và thời điểm phát hành là tháng 2 năm 2022. Tập PDF gồm 246 trang; phần thân bắt đầu từ bài về duy trì thông tin đường đi trên một lớp DAG đặc biệt.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Bàn về duy trì thông tin đường đi trên một lớp DAG đặc biệt	Dai Jiangqi
13	DFT tổng quát hơn	Zhou Hangrui
28	Bàn về ứng dụng của một tư tưởng tham lam dựa trên so sánh trong thi tin học	Zhang Junkai
49	Bàn về các thuật toán liên quan tới đồ thị phẳng	Tang Shaoxuan
72	Bàn về bài toán tương tác dạng tìm tham số tương tác	Hu Hao
82	Bàn về thuật toán xấp xỉ trong OI	Xu Tingqiang
99	Khảo sát bước đầu về bài toán tô màu đồ thị	Peng Bo
118	Bàn về bài toán tối ưu không gian trong thi tin học	Chen Zhixuan
129	Bàn về một lớp ma trận Monge	Zhang Jingxing
145	Bàn về phân tích độ phức tạp của một lớp bảng băm	Chen Yusi
159	Ứng dụng của đại số tuyến tính trong OI	Chang Ruinian
182	Bàn về các bài toán tổ hợp chuỗi liên quan tới lý thuyết Lyndon	Wan Chengzhang
204	Báo cáo ra đề <i>Chuỗi của Cutie Duo</i>	Feng Shiyuan
213	Bàn về bài toán mô phỏng luồng chi phí trong OI	Chen Kuowen

3 Bàn về duy trì thông tin đường đi trên một lớp DAG đặc biệt

Dai Jiangqi, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Trong OI, không ít bài toán liên quan tới cấu trúc dữ liệu có thể được xem như bài toán đường đi trên DAG. Bài viết này giới thiệu ba thuật toán có thể duy trì thông tin đường đi trên DAG trong thời gian $O(n \text{ poly}(\log S))$, trong đó n là kích thước đầu vào, S là tổng số đường đi của DAG và $\text{poly}(x)$ biểu thị một đa thức theo x . Bài viết cũng khảo sát ứng dụng của các thuật toán này trong cấu trúc dữ liệu, chuỗi và số học.

Mở đầu

Trong OI, rất nhiều bài toán có thể quy về mô hình liên quan tới đường đi trên DAG, bao gồm nhưng không giới hạn ở đồ thị, cấu trúc dữ liệu và chuỗi. Trên DAG tổng quát, việc duy trì nhanh thông tin đường đi là không thực tế, vì chỉ riêng mô tả một đường đi đã cần $\Omega(n)$ bit. Do đó bài viết tập trung vào các DAG có tổng số đường đi tương đối nhỏ; nhiều mô hình DAG thực dụng, chẳng hạn máy tự động hữu tố, thỏa tính chất này.

Bài viết chủ yếu giới thiệu ba thuật toán có thể duy trì thông tin đường đi trên DAG trong thời gian $O(\text{polylog } S)$, với S là tổng số đường đi: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng. Trong đó, phân rã chuỗi trên DAG có cài đặt gọn, thao tác sửa đổi tương đối dễ, nên thường thực dụng hơn trong OI. Cây cân bằng bền vững có độ phức tạp thời gian tối ưu và khả năng mở rộng mạnh, nhưng tốn bộ nhớ và khó cài đặt hơn. Độ phức tạp thời gian của phương pháp tách-trộn cây cân bằng vẫn còn mở; hiện chỉ có chứng minh cận trên $O(\log S \log q)$ mà chưa xây dựng được dữ liệu làm chặt cận này. Tuy vậy, vì quan hệ giữa cập nhật và truy vấn của nó ngược với hai thuật toán trước, tức truy vấn là offline còn DAG có thể bị bắt buộc đưa ra online, nó vẫn có một số ứng dụng khó thay thế.

Phần 1 trình bày mô hình cơ bản của duy trì thông tin đường đi trên DAG. Phần 2 lần lượt trình bày ba thuật toán: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng. Phần 3 nêu các ứng dụng trong một số bối cảnh.

3.1 Mô hình cơ bản

Cho một DAG có n đỉnh, trong đó các cạnh ra của mỗi đỉnh có thứ tự. Sắp các đỉnh mà đỉnh i ($1 \leq i \leq n$) trở tới thành dãy G_i . Đỉnh i có trọng số nguyên dương c_i , và cạnh ra thứ j của nó có trọng số cạnh j .

Sắp tất cả các đường đi bắt đầu tại 1 và kết thúc ở một đỉnh có bậc ra bằng 0 theo thứ tự từ điển của dãy trọng số cạnh; cách sắp này là duy nhất. Ký hiệu dãy đường đi là P_i ($1 \leq i \leq S$), trong đó S là tổng số đường đi thỏa điều kiện.

Có q truy vấn. Mỗi truy vấn cho một số nguyên dương k , yêu cầu tính tổng trọng số đỉnh trên P_k . Giả sử số cạnh của DAG không vượt quá $2n$, và

$$1 \leq n \leq 10^5, \quad 1 \leq k \leq 10^{18}, \quad 1 \leq c_i \leq 10^9.$$

3.2 Lời giải

Vì $k \leq 10^{18}$, chỉ cần giữ lại 10^{18} đường đi đầu của DAG. Trên thực tế, có thể dùng DP $O(n)$ để tính số đường đi từ mỗi đỉnh tới các đỉnh bậc ra 0, đồng thời lấy min với 10^{18} , rồi chỉ giữ các đỉnh có giá trị DP nhỏ hơn 10^{18} . Ngoài ra cần sao chép các đỉnh nằm trên đường đi nhỏ thứ 10^{18} theo thứ tự từ điển và giữ với các cạnh ra của chúng một tiền tố trở tới nhóm đỉnh trước. Vì vậy chỉ cần xét trường hợp $S \leq 10^{18}$.

Bằng cách nhị phân hóa bậc ra, dễ thấy ta chỉ cần xét trường hợp bậc ra của mỗi đỉnh là 0 hoặc 2. Trong phân tích dưới đây giả sử $S \geq n$.

3.2.1 Phân rã chuỗi trên DAG

Nếu DAG là một cây có hướng, hoặc một rừng có hướng, tức bậc vào của mỗi đỉnh không vượt quá 1, hiển nhiên có thể dùng phân rã nặng–nhẹ để duy trì thông tin trên chuỗi.

Ta mở rộng cách làm này lên DAG tổng quát hơn. Với mỗi đỉnh có bậc ra khác 0, ký hiệu f_i là số đường đi bắt đầu từ i , và định nghĩa hậu duệ nặng nxt_i là một đỉnh trong G_i có giá trị f lớn nhất. Tương tự, ký hiệu h_i là số đường đi kết thúc ở i , và trong các đỉnh trở tới i , lấy một đỉnh có giá trị h lớn nhất làm tiền nhiệm nặng pre_i .

Cạnh $u \rightarrow v$ là cạnh nặng khi và chỉ khi $v = \text{nxt}_u$ và $u = \text{pre}_v$. Chỉ giữ các cạnh nặng thì phần còn lại của đồ thị gồm một số chuỗi rời nhau, gọi là chuỗi nặng.

Xét một đường đi bất kỳ trên DAG, $p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_k$. Dọc đường đi đó, f_{p_i} giảm dần còn h_{p_i} tăng dần. Với mỗi cạnh không nặng $p_i \rightarrow p_{i+1}$, ít nhất một trong hai bất đẳng thức

$$f_{p_i} > 2f_{p_{i+1}}, \quad 2h_{p_i} < h_{p_{i+1}}$$

đúng. Vì vậy trên một đường đi chỉ có $O(\log S)$ cạnh nhẹ.

Do đó, từ đỉnh 1 ta có thể mỗi lần tìm bằng nhị phân phần chung giữa đường đi được hỏi và chuỗi nặng hiện tại; trên mỗi chuỗi nặng tiền xử lý tổng tiền tố. Nhờ vậy bài toán ở trên được giải trong thời gian $O(n + q \log S \log n)$ và bộ nhớ $O(n)$. Nút thắt nằm ở phép nhị phân trong mảng f .

Tối ưu. Tham khảo kỹ thuật cây nhị phân cân bằng toàn cục trong phân rã cây, độ phức tạp có thể hạ xuống $O(n + q \log S)$. Với một chuỗi nặng cố định $q_1 \rightarrow q_2 \rightarrow \dots \rightarrow q_m$, đặt $w_i = f_{q_i} - f_{q_{i+1}}$, với quy ước $f_{q_{m+1}} = 0$. Dựng một cây nhị phân trên toàn bộ chuỗi sao cho trong một đoạn $[l, r]$, gốc con là vị trí đầu tiên có tổng trọng số hai phía đều không vượt quá một nửa tổng đoạn; nếu không tồn tại thì lấy đầu đoạn. Tiền xử lý LCA trên cây nhị phân này.

Nếu u là vị trí vào chuỗi nặng hiện tại và v là vị trí ra, ta tiền xử lý LCA giữa mỗi đỉnh với cuối chuỗi nặng q_m , rồi nhị phân từ $\text{LCA}(u, q_m)$ đi xuống. Độ phức tạp của phép nhị phân trên cây nhị phân chính là khoảng cách giữa $\text{LCA}(u, q_m)$ và v trên cây đó.

Bắt đầu từ v và đi lên hai bước mỗi lần, tổng trọng số của cây con ít nhất tăng gấp đôi, trừ tình huống lá biên. Sau một bước đi xuống từ LCA, tổng cây con không vượt quá đại lượng tương ứng ở điểm vào. Cộng trên mọi chuỗi nặng của một truy vấn, vì đại lượng của chuỗi hiện tại không nhỏ hơn chuỗi kế tiếp, tổng độ phức tạp nhị phân là $O(\log S)$.

Dùng splay hoặc treap cũng đạt cùng bậc độ phức tạp; bài viết không nhắc lại.

Mở rộng tối ưu. Dùng chia để trị, tức thông tin nửa nhóm trên đoạn với tiền xử lý $O(n \log n)$ và truy vấn $O(1)$ dựa trên hợp nhất hai đoạn, kết hợp DP, có thể hỗ trợ truy vấn giá trị lớn nhất của tổng trọng số đỉnh trên đoạn $P_l, P_{l+1}, \dots, P_r$ trong thời gian $O(n \log n + q \log S)$. Cụ thể, với mỗi đỉnh tính giá trị đường đi lớn nhất bắt đầu ở nó, rồi trên mỗi chuỗi nặng lưu cực trị đoạn của các thông tin đó.

Với các bài toán có sửa điểm đơn, sửa chuỗi đơn giản và truy vấn thông tin trên một chuỗi, cũng có thể tổng quát hóa cây đoạn có trọng số để giảm độ phức tạp mỗi thao tác xuống $O(\log S)$.

Ngoài $w_i = f_i - f_{i+1}$, đặt $x_i = h_i - h_{i-1}$, trong đó $h_0 = 0$. Xét cách dựng cây nhị phân sau. Với một khoảng $[l, r]$, $l < r$, gốc của cây con là chỉ số nhỏ nhất i , $l \leq i < r$, thỏa điều kiện theo độ sâu hiện tại d :

$$\begin{cases} 2 \sum_{j=l}^i w_j \geq \sum_{j=l}^r w_j, & d \equiv 1 \pmod{2}, \\ 2 \sum_{j=l}^i x_j \geq \sum_{j=l}^r x_j, & d \equiv 0 \pmod{2}. \end{cases}$$

Nếu không tồn tại chỉ số như vậy thì lấy $i = l$, rồi dựng đệ quy hai cây con trái/phải trên $[l, i]$ và $[i + 1, r]$.

Khi đó độ phức tạp của phép nhị phân trên cây này chính là khoảng cách giữa u và v trên cây nhị phân, trong đó u là vị trí đi vào chuỗi nặng hiện tại, còn v là vị trí đi ra.

Khoảng cách từ u tới $LCA(u, v)$ không vượt quá

$$O\left(\log \frac{h_v}{h_u - h_{u-1}}\right),$$

vì bắt đầu từ u , cứ đi lên bốn bước thì tổng h của cây con ít nhất tăng gấp đôi, và trong bốn bước đó có hai đỉnh có độ sâu chẵn nên có thể áp dụng kết luận của Mục 2.1.1. Phân tích tương tự cho v cho thấy độ phức tạp truy vấn hoặc sửa trên khoảng không vượt quá

$$O\left(\log \frac{f_u}{f_v - f_{v+1}} + \log \frac{h_v}{h_u - h_{u-1}}\right),$$

nên tổng trên mọi chuỗi nặng vẫn không vượt quá $O(\log S)$.

Ngoài ra, với thông tin khả nghịch có sửa điểm đơn, xây cây đoạn có trọng số riêng theo hai phía cũng đạt truy vấn đơn $O(\log S)$. Tuy nhiên hằng số lớn; nếu thông tin cần duy trì đơn giản thì thuật toán này không có ưu thế rõ rệt so với BIT.

3.2.2 Cây cân bằng bền vững

Xét bài toán gốc theo thứ tự tô-pô đảo của DAG. Với mỗi đỉnh i , xét dãy Q_i gồm mọi đường đi bắt đầu từ i .

Rõ ràng, nếu bậc ra của i bằng 0 thì $Q_i = ((i))$. Nếu bậc ra của i bằng 2, với hai cạnh ra theo thứ tự, Q_i nhận được bằng cách ghép hai dãy đường đi của hai con, rồi chèn i vào đầu mỗi phần tử.

Ta chỉ duy trì tổng trọng số đỉnh của từng đường đi trong Q_i . Khi đó cần hỗ trợ các thao tác sau trong $O(\log S)$:

1. ghép hai dãy số nguyên độ dài không vượt quá S thành một dãy mới;
2. cộng cùng một hằng số vào mọi phần tử của một dãy;
3. cho k , hỏi phần tử thứ k của một dãy.

Dùng cây cân bằng để duy trì. Do có thao tác tự sao chép, ta cần một loại cây không có con trỏ cha và chiều cao chặt $O(\log S)$. Bài viết dùng WBLT, tức Weight Balanced Leafy Tree. Trong WBLT, mỗi lá biểu diễn một phần tử; mỗi nút trong có cây con trái và phải, và kích thước mỗi cây con không nhỏ hơn một hằng số cân bằng nhân với tổng kích thước. Mỗi nút có thể lưu thông tin đoạn. Ở đây, lá biểu diễn đường đi và mỗi nút giữ một nhãn cộng lười cho cây con.

Với thao tác ghép, chỉ cần merge hai WBLT. Khi ghép hai cây A, B và giả sử $|A| \geq |B|$, nếu tỉ lệ kích thước đã cân bằng thì tạo một gốc mới với hai cây con A, B . Nếu không, xét cây con phải của A ; khi ghép trực tiếp vẫn thỏa cân bằng thì gắn B vào phía phải. Nếu vẫn chưa thỏa, tiếp tục tách cấu trúc bên phải của A , ghép các phần tương ứng rồi hợp nhất trở lại. Bằng quy nạp, ghép hai WBLT có tỉ lệ kích thước là hằng số tổn $O(1)$; vì các đệ quy còn lại chỉ đi về một phía, tổng thời gian ghép không vượt quá $O(\log S)$. Trong bài này còn có thể lợi dụng tính khả trừ và giao hoán của thông tin để làm nhãn gần như vĩnh viễn, giảm hằng số số lần *pushdown*.

Thao tác cộng toàn dãy chỉ sửa nhãn ở gốc. Thao tác hỏi phần tử thứ k đệ quy từ gốc xuống lá. Vì vậy bài toán ban đầu được giải trong thời gian $O(n + q \log S)$ và bộ nhớ $O(n \log S)$.

Mở rộng của lời giải WBLT. Ngoài ghép, WBLT và một số cây cân bằng khác còn hỗ trợ tách: cho cây A và $k \in [0, |A|]$, trả về hai WBLT lần lượt biểu diễn k phần tử đầu và $|A| - k$ phần tử sau.

Cụ thể, đệ quy từ trên xuống sẽ tách A thành một số cây con có độ sâu tăng dần và một số cây con có độ sâu giảm dần; sau đó ghép chúng từ dưới lên để thu được hai cây kết quả. Vì chi phí ghép tỉ lệ với logarit tỉ lệ kích thước của hai phần, để chứng minh thao tác tách tốn $O(\log S)$.

Nói cách khác, ngoài ghép dãy, sửa toàn cục và hỏi điểm đơn, cây cân bằng bền vững còn hỗ trợ cắt lấy một đoạn dãy. Tất nhiên cũng có thể hỗ trợ sửa đoạn và hỏi đoạn; trong bài toán gốc, đoạn này tương ứng với một đoạn của dãy đường đi P .

Do cấu trúc phức tạp, cây cân bằng bền vững khó hỗ trợ sửa trọng số đỉnh của DAG.

Thảo luận liên quan. Có thể thấy mỗi DAG mà mọi đỉnh có bậc ra 0 hoặc 2 tương đương về cấu trúc với một cây Leafy Tree bền vững.

Điều này có nghĩa: khi không có tính chất đặc biệt và số thao tác ít, ta có thể duy trì trực tiếp DAG để cài ghép dãy và cắt dãy, bỏ qua các thao tác giữ cân bằng. Định nghĩa độ sâu dep_v của đỉnh v là số cạnh trên đường đi dài nhất bắt đầu ở v . Khi ghép A, B thành C , chỉ cần tạo một đỉnh mới có hai con A, B và đặt $\text{dep}_C = \max(\text{dep}_A, \text{dep}_B) + 1$. Khi tách A thành B, C , thời gian không vượt quá dep_A , vì trong quá trình đệ quy mỗi độ sâu chỉ bị truy cập ở trước và sau nhiều nhất một lần. Bằng cách ghép từ sau ra trước dọc ranh giới giữa B và C , ta đảm bảo $\max(\text{dep}_B, \text{dep}_C) \leq \text{dep}_A$. Sau $O(q)$ thao tác ghép và tách, độ sâu không vượt quá q , nên tổng độ phức tạp là $O(q^2)$. Đây cũng đạt cận dưới lý thuyết trong mô hình này, vì sau q bước số đường đi có thể đạt $\exp(\Theta(q))$. Ưu thế của cây cân bằng bền vững nằm ở các trường hợp DAG có cấu trúc đặc biệt hơn.

Ngoài ra, cấu trúc phân rã chuỗi trên DAG kết hợp cây đoạn có trọng số ở phần trước cũng có thể được xem như một DAG “cân bằng hơn” nhưng tương đương với DAG gốc theo ý nghĩa dãy đường đi.

3.2.3 Tách–trộn cây cân bằng

Xét phiên bản offline của bài toán gốc: tại đỉnh 1, duy trì tập các chỉ số k xuất hiện trong mọi truy vấn.

Duyệt các đỉnh theo thứ tự tô-pô từ trước ra sau. Khi tới đỉnh i , mọi truy vấn đang nằm tại i được cộng thêm c_i vào đáp án. Nếu i có bậc ra 2, chuyển toàn bộ truy vấn này tới các đỉnh mà i trở tới; nửa sau cần trừ đi kích thước của nửa trước khỏi chỉ số k tương ứng.

Cần duy trì nhiều dãy không giảm và hỗ trợ ba thao tác sau; sau khi bị thao tác, dãy cũ biến mất:

1. tách một dãy thành hai dãy mới theo ngưỡng k ;
2. cộng k vào mọi phần tử của một dãy;
3. trộn hai dãy đã sắp xếp thành một dãy mới.

Dùng cây cân bằng để duy trì các dãy này thì hai thao tác đầu là trực tiếp.

Với thao tác trộn hai dãy A và B thành C , chia các vị trí $1, 2, \dots, |A| + |B|$ thành t đoạn liên tiếp cực đại sao cho các phần tử trong mỗi đoạn của C trước khi trộn đều thuộc cùng một dãy cũ. Dùng nhị phân từ trái sang phải hoặc đệ quy từ gốc xuống đều cho lời giải đơn giản $O(t \log q)$.

Tiếp theo chứng minh tổng t không vượt quá $O((n + q) \log S)$. Với một dãy A , định nghĩa thế năng

$$\Phi(A) = \sum_i \log(2 + A_{i+1} - A_i).$$

Tổng thế năng luôn không âm và không vượt quá $O(q \log S)$. Thao tác tách làm thế năng thay đổi nhiều nhất $O(\log S)$, thao tác cộng toàn dãy không đổi thế năng. Vì vậy chỉ cần chứng minh trong một thao tác trộn, thế năng giảm ít nhất $\Omega(t) - O(\log S)$.

Bản quét của chuỗi bất đẳng thức chi tiết bị nhiễu ở chỉ số, nhưng lập luận đọc được như sau: gọi các đoạn cực đại sau trộn là $[l_i, r_i]$ và đặt $d_i = l_i - r_{i-1}$ giữa hai đoạn kề nhau. Khi ghép hai đoạn kề, số hạng

logarit tương ứng được thay bằng logarit của khoảng cách lớn hơn; dùng $\min(d_i, d_{i+1})$ và $\max(d_i, d_{i+1})$ để chặn, mỗi lần đổi đoạn đóng góp một lượng giảm hằng số, còn hai biên chỉ tạo sai số $O(\log S)$. Do đó tổng số đoạn cực đại trên mọi lần trộn bị chặn bởi tổng thể năng tăng.

Ta thu được lời giải thời gian $O((n + q) \log S \log q)$ và bộ nhớ $O(n + q)$.

3.2.4 Mở rộng và thảo luận

Nhìn lại bốn thao tác mà cây cân bằng bền vững ở phần trên hỗ trợ: ghép dây, cắt dây, cộng toàn cục và hỏi điểm đơn. Nếu xem các thao tác này như một DAG, rồi duyệt ngược và dùng cây cân bằng để duy trì mọi truy vấn, nút thất vẫn là tách cây, cộng toàn cục và trộn cây. Nói cách khác, vấn đề mà cây cân bằng bền vững và vấn đề mà tách-trộn cây cân bằng giải quyết phần nào là chuyển vị của nhau.

Đáng tiếc là cận trên tốt nhất đã biết của tách-trộn cây cân bằng là $O(\log S \log q)$, dù cũng chưa có dữ liệu đạt chặt cận này, còn cây cân bằng bền vững đạt $O(\log S)$. Trong các bài toán liên quan tới bài viết này, thuật toán tách-trộn chỉ hơn cây cân bằng bền vững khi có yêu cầu đặc biệt về bộ nhớ, hoặc khi một số thông tin trong DAG thao tác bị bắt buộc đưa ra online, chẳng hạn khi thao tác tách không phải tách theo “ $\geq k$ ” và “ $< k$ ” mà là tách theo k phần tử đầu và phần còn lại.

3.3 Ứng dụng

3.3.1 Duy trì quá trình thao tác trên dây

Theo phần 2.2.2, xem DAG như một Leafy Tree bền vững giúp cài đặt thuận tiện các thao tác cắt dây và ghép dây.

Ví dụ 1. Có n dây. Ban đầu dây thứ i chỉ chứa một số.

Cần thực hiện q thao tác, mỗi thao tác thuộc một trong ba loại:

1. tạo một dây mới bằng cách ghép hai dây đã có;
2. tạo hai dây mới, lần lượt là k phần tử đầu của một dây đã có và các phần tử còn lại, trong đó k là số nguyên lớn không vượt quá độ dài dây;
3. xuất số nghịch thế của một dây modulo 998244353.

Ràng buộc đọc được: $1 < n < 100$, $1 < q < 600$.

Lời giải ví dụ 1. Nếu không có thao tác loại 2, hiển nhiên có thể duy trì số lần xuất hiện của mỗi giá trị trong từng dây và tổng số nghịch thế, mỗi thao tác tốn $O(n)$.

Dùng cách làm ở phần 2.2.2: với thao tác loại 2, ta đệ quy từ trên xuống để tách rồi ghép dần từ dưới lên, trong quá trình duy trì kích thước mỗi nút. Khi đó mỗi thao tác tách có thể viết lại thành $O(q)$ thao tác ghép, còn mỗi thao tác ghép tốn $O(n^2 + n\omega)$, trong đó ω là số bit của kích thước, thường xem là 32 hoặc 64. Tổng độ phức tạp là $O(n^2 q^2 + n\omega q^2)$.

3.3.2 Ứng dụng trên máy tự động hậu tố

Trong lý thuyết chuỗi của OI, máy tự động hậu tố (SAM) là một thuật toán hiệu quả và thực dụng.

Cụ thể, với chuỗi S trên bảng chữ cái Σ , xét tập vị trí kết thúc của xâu con T trong S , ký hiệu là R_T . Với các T khác nhau, quan hệ giữa các R_T chỉ có thể là bao hàm hoặc rời nhau; vì vậy chỉ có $O(|S|)$ tập

R_T khác nhau về bản chất và chúng tạo thành một cấu trúc cây, thực chất là cây hậu tố của chuỗi đảo sau khi nén các đỉnh bậc hai.

Định nghĩa tự động A có tập đỉnh là các R_T , với T là xâu con của S , và có cạnh ký tự a từ R_T tới R_{Ta} nếu Ta cũng là xâu con của S . Từ các tính chất của SAM, A có kích thước $O(|S|)$ và có thể xây dựng cây cùng tự động trong $O(|S|)$.

Các xâu con khác nhau của S tương ứng một-một với các đường đi bắt đầu từ xâu rỗng trong A . Nhờ đó, thông tin về xâu con khác nhau được chuyển thành thông tin đường đi trên tự động, giúp xử lý hiệu quả một số bài toán khá khó nếu chỉ nhìn từ góc độ cây hậu tố.

Ví dụ 2. Nguồn bài là UOJ Long Round #1, bài D¹, đã lược bỏ chi tiết không cần thiết.

Cho chuỗi S và các mảng a, b . Với xâu con T của S , ký hiệu tập vị trí kết thúc bên phải của T trong S là R_T , và định nghĩa

$$f(T) = |R_T| \sum_{i \in R_T} b_i.$$

Với mỗi i , hãy tính

$$\sum_{j=i}^{|S|} f(S[i \dots j]).$$

Ràng buộc đọc được: $1 \leq |S| \leq 5 \cdot 10^5$, bảng chữ cái có 26 ký tự.

Lời giải ví dụ 2. Xây cây hậu tố của chuỗi đảo và SAM của S . Trên mỗi đỉnh của SAM, giá trị f là cố định, còn truy vấn đúng là tính tổng f trên một đường đi của SAM.

Dùng phân rã chuỗi trên DAG và duy trì tổng tiền tố trên mỗi chuỗi nặng là giải được. Nút thắt là tìm độ dài phân chung giữa đường đi truy vấn và chuỗi nặng hiện tại; dùng bảng thưa LCP với tiền xử lý $O(|S| \log |S|)$ và truy vấn $O(1)$.

Thảo luận ví dụ 2. Thực ra SAM chỉ có $O(|S|)$ đường đi tối đại, tương ứng với mọi hậu tố của S . Dựa trên tính chất này, tồn tại thuật toán đơn giản hơn.

Khi trải mọi đường đi của SAM ra, ta thu được một cây chỉ có $O(|S|)$ lá, thực chất là cây hậu tố của chuỗi thuận. Mỗi truy vấn tương đương tính tổng trọng số đỉnh trên một chuỗi của cây này; mặt khác các đỉnh này đều tương ứng với đỉnh SAM.

Nén các đỉnh bậc hai của cây này và xét thao tác nén trên tự động: xóa mọi đỉnh có bậc ra 1 trên tự động, với mọi cạnh trở tới chúng thì đích cạnh thành đỉnh mà đích cũ trở tới, lặp đến khi đích có bậc ra khác 1, đồng thời đổi ký tự trên cạnh thành một chuỗi. Bằng cách lưu mảng trên cạnh, tự động sau nén vẫn duy trì được tổng đoạn; khi trải tự động ra sẽ trực tiếp thu được một cây hậu tố chuỗi thuận đã nén có $O(|S|)$ đỉnh. Xử lý tổng tiền tố trực tiếp trên cây này là đủ. Tổng độ phức tạp là $O(|S| + q)$.

Đáng chú ý, tập đỉnh của SAM nén của chuỗi thuận và chuỗi đảo là giống nhau về bản chất. Điều này đem lại một góc nhìn mới cho các bài toán xâu con liên quan tới cả đầu trái và đầu phải. Tuy nhiên thuật toán này cũng có hạn chế: cấu trúc SAM nén phụ thuộc vào tính đối xứng giữa chuỗi thuận và chuỗi đảo, nên khả năng mở rộng kém hơn phân rã chuỗi trên DAG.

Ví dụ 3. Cho một trie S có n đỉnh. Định nghĩa xâu con của S là chuỗi tương ứng với một đường đi từ trên xuống.

Có q truy vấn. Mỗi truy vấn cho k , yêu cầu tìm xâu con khác nhau nhỏ thứ k của S hoặc báo không tồn tại. Để tránh đầu ra quá lớn, chỉ cần xuất tổng mã ASCII của mọi ký tự trong xâu đó.

¹<https://uoj.ac/problem/577>

Ràng buộc đọc được: $n, q \leq 10^5$, $k \leq 10^{18}$, bảng chữ cái 26.

Lời giải ví dụ 3. Cần xây SAM tổng quát. Ở đây S được định nghĩa là trie tạo bởi các chuỗi trên đường đi từ mọi đỉnh về gốc, còn tập $right$ của SAM tương ứng một-một với đỉnh của trie.

Phân tích độ phức tạp của thuật toán xây SAM trên chuỗi không còn áp dụng trực tiếp, và số cạnh của SAM lúc này có thể đạt $n|\Sigma|$. Theo các tài liệu được trích dẫn, BFS trực tiếp trên trie rồi dùng thuật toán SAM cho chuỗi là tuyến tính theo kích thước kết quả, nhưng tác giả không biết chứng minh đầy đủ. Cũng có thể dùng suffix array tổng quát bằng nhân đôi để tính cây, rồi từ đó tính SAM, tổng độ phức tạp $O(n \log n + n|\Sigma|)$; nếu dùng mở rộng của DC3 trên cây có thể bỏ thừa số $\log$.

Sau khi xây được SAM, bài toán trở thành mô hình kinh điển của phần 1, và cả ba thuật toán ở phần 2 đều giải được. Chẳng hạn dùng phân rã chuỗi trên DAG thì tổng thời gian là $O(n \log n + n|\Sigma| + q \log^2 n)$, hoặc với tối ưu là $O(n \log n + n|\Sigma| + q \log n)$.

Ví dụ 4. Nguồn bài là UOJ tuyển chọn đội, bài C^2 .

Cho chuỗi S độ dài n và m xâu con của S làm mẫu. Có q truy vấn; mỗi truy vấn yêu cầu tổng số lần xuất hiện của mọi xâu mẫu trong một xâu con của S . Ràng buộc đọc được: $n \leq 4 \cdot 10^5$, $m \leq 10^5$, $q \leq 10^5$, bảng chữ cái 26.

Lời giải ví dụ 4. Xây SAM của S và cây hậu tố của chuỗi đảo, rồi phân rã chuỗi trên DAG của SAM.

Với một xâu con T của S , số xâu mẫu trong các hậu tố của T được quyết định bởi đỉnh tương ứng với T trên SAM và các xâu mẫu tương ứng. Hơn nữa, chỉ cần lấy số xâu mẫu trên đường đi từ đỉnh đó về gốc trong cây hậu tố đảo, rồi trừ đi số xâu mẫu có độ dài lớn hơn $|T|$.

Với mỗi truy vấn, định vị xâu con thành một đường đi trên tự động và tính tổng đóng góp của mọi đỉnh trên đường đi đó. Định vị đường đi có thể làm trong $O((n+q) \log n)$ bằng cách tiền xử lý $O(n \log n)$ trên chuỗi gốc và các chuỗi nặng của DAG rồi hỏi LCP xâu con trong $O(1)$.

Với thống kê đáp án: loại đóng góp thứ nhất chỉ cần lưu tổng tiền tố của w_v trên mỗi chuỗi nặng; loại thứ hai chuyển thành bài toán đếm điểm trong hình bình hành nghiêng 45° trên từng chuỗi nặng, sau đổi tọa độ là bài toán đếm điểm hai chiều thông thường. Tùy cách cài LCP và đếm điểm, tổng độ phức tạp là $O((n+m) \log n + q \log^2 n)$ hoặc $O((n+m+q) \log n)$.

3.3.3 Thuật toán Euclid vận năng

Trên lưới có đường thẳng $y = \frac{a}{b}x$ với $a, b \in \mathbb{N}^+$. Xét một điểm động trên đường thẳng này di chuyển theo hướng $x+y$ tăng trên một đoạn. Mỗi khi đi qua một đường ngang, ghi ký hiệu A ; mỗi khi đi qua một đường dọc, ghi ký hiệu B ; nếu đi qua điểm lưới thì ghi A trước rồi B sau. Cuối cùng thu được một dãy trên $\{A, B\}$, ví dụ với $a=3, b=2$ thì dãy là $ABBAB$.

Thay A và B bằng hai phần tử của một nửa nhóm, tức thông tin ghép được trong $O(1)$, mục tiêu của thuật toán Euclid vận năng là tính phần tử nửa nhóm ứng với toàn bộ dãy.

Trước hết giả sử đoạn di chuyển là từ $(0,0)$ tới (a,b) và $\gcd(a,b)=1$. Nếu $a > b$, đặt $k = \lfloor a/b \rfloor$. Dễ thấy sau mỗi A đều có k ký hiệu B . Thay mọi đoạn AB^k trong dãy gốc bằng một ký hiệu mới tương ứng; về mặt hình học, điều này tương đương tác động một phép biến đổi tọa độ lên mặt phẳng và đường thẳng, đưa bài toán về kích thước nhỏ hơn. Lặp quá trình $O(\log a)$ lần sẽ giải được bài toán.

Dùng DAG để mô tả thuật toán trên: ban đầu có hai đỉnh A, B . Mỗi lần tạo một đỉnh C , nối cạnh tới A và B theo phép thay thế tương ứng, rồi thay (a,b) bằng $(a-b, b)$ hoặc $(a, b-a)$ tùy bên lớn hơn, cho

²<https://uoj.ac/problem/697>

tới khi $a = b$.

Số cạnh của DAG có thể đạt độ phức tạp của phép trừ lặp, tức $O(a)$. Chỉ cần dùng nhân đôi để tối ưu các thao tác trừ liên tiếp là có thể giảm kích thước đồ thị xuống $O(\log(a + b))$.

Dãy AB do quá trình Euclid vận năng sinh ra chính là dãy các đỉnh cuối của mọi đường đi khi trải DAG theo thứ tự DFS. Vì vậy phiên bản tổng quát của thuật toán Euclid vận năng tương đương truy vấn đoạn trên Leafy Tree bền vững ứng với DAG này. Nhiều bài toán Euclid cổ điển, như tổng $\lfloor ai/b \rfloor$ hoặc $\min_i(ai \bmod b)$, có thể chuyển thành mô hình này; hơn nữa DAG này cho phép giải một số bài toán phức tạp hơn liên quan tới quá trình Euclid.

Ví dụ 5. Đây là bài mẫu thuật toán Euclid vận năng trên LOJ³.

Cho các số nguyên dương n, m, l và hai ma trận vuông A, B kích thước không vượt quá 10. Cần tính

$$\sum_{i=0}^{l-1} A^i B^{\lfloor im/n \rfloor}.$$

Ràng buộc đọc được: $1 \leq n, m, l \leq 10^{18}$.

Lời giải ví dụ 5. Xét dãy AB ứng với đường thẳng liên quan trong đề. Duy trì ma trận M , ban đầu $M = I$. Từ vị trí thứ hai của dãy trở đi, mỗi khi gặp A thì đặt $M = MB$; mỗi khi gặp B thì cộng M vào đáp án rồi đặt $M = AM$. Khi kết thúc, đáp án là giá trị cần tìm.

Tác động của một đoạn của dãy có thể mô tả bằng một bộ ma trận: nếu trước khi đi vào đoạn có trạng thái M , sau đoạn đáp án được cộng thêm một biểu thức tuyến tính theo M và trạng thái mới cũng là một biến đổi tuyến tính của M . Vì vậy thông tin đoạn ghép được trong $O(1)$, và có thể áp dụng trực tiếp phiên bản truy vấn đoạn của thuật toán Euclid vận năng.

Ví dụ 6. Nguồn bài là CTT 2021 Day 4 T3, đã lược bỏ chi tiết không cần thiết.

Xét đồ thị vô hướng n đỉnh, trong đó i nối với $(i + d) \bmod n$. Trọng số đỉnh lấy theo một dãy tuần hoàn độ dài m : $w_i = a_{i \bmod m}$.

Cần thực hiện q lần sửa điểm đơn trên trọng số, sau mỗi lần sửa tính kích thước tập độc lập có trọng số lớn nhất của đồ thị.

Ràng buộc đọc được: $d < n \leq 10^9$, $m \leq 2 \cdot 10^5$, $w_i \leq 400$, $q \leq 10^5$.

Lời giải ví dụ 6. Dễ thấy đồ thị gồm $\gcd(n, d)$ chu trình, và chỉ có một số loại chu trình khác nhau về bản chất. Sau khi xóa mọi đỉnh từng bị sửa ít nhất một lần, đồ thị còn lại gồm $O(q)$ chuỗi và một số loại chu trình. Chỉ cần tiền xử lý đáp án của mỗi chuỗi và mỗi loại chu trình rồi cộng lại; với mỗi chuỗi, dùng một ma trận 2×2 biểu diễn trạng thái chọn hoặc không chọn hai đầu để tính tập độc lập có trọng số lớn nhất, rồi duy trì cây đoạn trên các điểm quan trọng.

Xét thông tin trên một chu trình. Trên lưới, vẽ đoạn $y = \frac{d}{n}x$ với $x \in [0, n)$ và xây dãy cùng DAG tương ứng của thuật toán Euclid vận năng. Đồng thời duy trì một giá trị x ; các giá trị x khác nhau tương ứng với các chu trình khác nhau. Duyệt dãy và duy trì ma trận 2×2 ; mỗi khi gặp một loại ký hiệu thì cập nhật x theo modulo m , đồng thời ghép thông tin tương ứng với w_x . Sau khi thao tác xong, ma trận nhận được là đáp án của chu trình; nếu chỉ thao tác trên một đoạn thì nhận được đáp án của chuỗi.

Thông tin đoạn của dãy có thể định nghĩa là m ma trận 2×2 , biểu diễn tác động khi giá trị ban đầu của x lần lượt là $0, 1, \dots, m - 1$, cùng một độ dời cho x sau khi đi qua đoạn. DP trên DAG Euclid tính được mọi thông tin đoạn trong $O(m \log n)$, từ đó lập tức có đáp án chu trình.

³<https://loj.ac/p/6440>

Đáp án chuỗi là truy vấn đoạn trên thông tin đường đi của DAG. Vì DAG này chỉ có $O(\log n)$ đỉnh, DFS trực tiếp từ trên xuống đạt $O(q \log n)$. Tổng độ phức tạp là $O((m + q) \log n)$.

Kết luận

Bài viết đã giới thiệu ba thuật toán: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng, đồng thời đưa ra ứng dụng của chúng trong một số bài OI. Tác giả phỏng đoán phía sau mô hình duy trì đường đi trên DAG có thể còn những tính chất đẹp và sâu hơn; các mở rộng khác của ba thuật toán và mối liên hệ giữa chúng vẫn cần được khai thác thêm. Hy vọng bài viết có thể gợi mở hướng nghiên cứu tiếp theo cho độc giả quan tâm.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn huấn luyện viên Yang Yuliang của đội tuyển quốc gia đã hướng dẫn; cảm ơn cha mẹ, nhà trường, các thầy cô và bạn học đã bồi dưỡng, giảng dạy và hỗ trợ; cảm ơn các anh chị và bạn bè đã trao đổi, góp ý, truyền cảm hứng và đọc bản thảo; cảm ơn mọi nguồn tư liệu tham khảo; và cảm ơn độc giả đã dành thời gian đọc bài viết.

Tài liệu tham khảo

1. Zhang Zheyu. *Bàn về thuật toán chia để trị trên cây*. Luận văn đội tuyển quốc gia IOI 2019, 2019.
2. Wang Siqi. *Leafy Tree và cây cân bằng trọng số được cài đặt từ nó*. Luận văn đội tuyển quốc gia IOI 2018, 2018.
3. SF. *Bàn về máy tự động hậu tố nén*. Luận văn đội tuyển quốc gia IOI 2020, 2020.
4. Liu Yanyi. *Mở rộng máy tự động hậu tố trên trie*. Luận văn đội tuyển quốc gia IOI 2015, 2015.
5. J. A. Blumer. *Algorithms for the directed acyclic word graph and related structures*. Ph.D. Thesis, Denver University, 1985.
6. A. Blumer, J. Blumer, D. Haussler, R. M. McConnell, A. Ehrenfeucht. *Complete inverted files for efficient text retrieval and analysis*. J. ACM 34(3)(1987), pp. 578–595.
7. J. Karkkainen, P. Sanders, S. Burkhardt. *Linear work suffix array construction*. J. ACM 53(6)(2006), pp. 918–936.

4 DFT tổng quát hơn

Zhou Hangrui, Trường Văn Uyên thuộc Tập đoàn Giáo dục Trung học Học Quân Hàng Châu

Tóm tắt

Biến đổi Fourier rời rạc được dùng rất rộng rãi trong thi tin học. Bài viết này mở rộng DFT truyền thống, dùng độ phức tạp hơi cao hơn để thực hiện phép chập do một quan hệ hai ngôi tổng quát hơn định nghĩa.

4.1 Định nghĩa liên quan và quy ước

4.1.1 Phép chập và các phép toán cơ bản

Định nghĩa 1.1. Với một tập G và một quan hệ hai ngôi $\varphi : G \times G \rightarrow G$, cho hai ánh xạ $A, B : G \rightarrow \mathbb{C}$. Định nghĩa phép chập $C = A \times B$ bởi

$$C(v) = \sum_{\varphi(x,y)=v} A(x)B(y).$$

Tổng của hai ánh xạ $C = A + B$ thỏa

$$C(v) = A(v) + B(v).$$

Tích của một ánh xạ với một số phức $B = cA$ thỏa

$$B(v) = cA(v).$$

Bài viết chỉ xét trường hợp (G, φ) tạo thành một nửa nhóm hữu hạn. Với ánh xạ $A : G \rightarrow \mathbb{C}$, trong bài này A_x và $A(x)$ có cùng ý nghĩa.

4.1.2 DFT

Định nghĩa 1.2. Với dãy kích thước $(a_i)_{i=1}^k$, định nghĩa tập dãy ma trận tương ứng là R , thỏa:

- $R = \{(m_i)_{i=1}^k \mid m_i \in M_{a_i}\}$, trong đó M_n chứa mọi ma trận phức $n \times n$;
- $C = A + B$ thỏa $C_i = A_i + B_i$;
- $C = AB$ thỏa $C_i = A_i B_i$;
- $B = cA$ thỏa $B_i = cA_i$.

Ở đây $A, B, C \in R$ là các dãy ma trận có cùng dãy kích thước, còn c là một số phức.

Định nghĩa 1.3. Với một quan hệ chập, định nghĩa DFT tương ứng của nó là một biến đổi tuyến tính khả nghịch

$$\mathcal{F} : (G \rightarrow \mathbb{C}) \rightarrow R,$$

trong đó R là tập dãy ma trận do một dãy kích thước $(a_i)_{i=1}^k$ định nghĩa, đồng thời thỏa:

- $\mathcal{F}(A) \times \mathcal{F}(B) = \mathcal{F}(AB)$;
- $\mathcal{F}(A) + \mathcal{F}(B) = \mathcal{F}(A + B)$;
- $c\mathcal{F}(A) = \mathcal{F}(cA)$;
- $\sum_{i=1}^k a_i^2 = |G|$.

Có thể xem $\mathcal{F}$ là một đẳng cấu.

Với phép nhân dãy ma trận, ta làm được trong $O(\sum_{i=1}^k a_i^\omega)$, trong đó ω là số sao cho phép nhân ma trận $n \times n$ có độ phức tạp $O(n^\omega)$; hiện đã biết $\omega \leq 2.3728596$ [5].

Không gian $(G \rightarrow \mathbb{C})$ có một hệ cơ sở $(A_x)_{(x \in G)}$, trong đó A_x là ánh xạ

$$t \mapsto [t = x] = \begin{cases} 1, & t = x, \\ 0, & t \neq x. \end{cases}$$

Vì mọi phần tử đều phân rã được thành tổ hợp tuyến tính của cơ sở, ta chỉ cần quan tâm tới $\mathcal{F}(A_x)$. Đặt $\mathcal{F}_x = \mathcal{F}(A_x)$.

Bổ đề 1.1. Một quan hệ DFT $\mathcal{F}$ tương ứng một-một với họ $(\mathcal{F}_x)$ thỏa:

- $\mathcal{F}_x \times \mathcal{F}_y = \mathcal{F}_{\varphi(x,y)}$;
- các $\mathcal{F}_x$ tạo thành một hệ cơ sở của R .

Chứng minh. Với một quan hệ DFT $\mathcal{F}$, do tính đẳng cấu, họ $(\mathcal{F}_x)$ mà nó thu được hiển nhiên hợp lệ. Ngược lại, với một họ $(\mathcal{F}_x)$, có thể dựng

$$\mathcal{F}(A) = \sum_{x \in G} A(x) \mathcal{F}_x.$$

Đối với phép nhân:

$$\begin{aligned} \mathcal{F}(A) \times \mathcal{F}(B) &= \sum_{x \in G} \sum_{y \in G} A_x B_y \mathcal{F}_x \times \mathcal{F}_y \\ &= \sum_{v \in G} \sum_{\varphi(x,y)=v} A_x B_y \mathcal{F}_x \times \mathcal{F}_y \\ &= \sum_{v \in G} \mathcal{F}_v \sum_{\varphi(x,y)=v} A_x B_y \\ &= \mathcal{F}(A \times B). \end{aligned}$$

Phép cộng và nhân vô hướng hiển nhiên thỏa. Tính song ánh suy ra từ việc $(\mathcal{F}_x)$ tạo thành một hệ cơ sở. ■

Do đó, khi đã biết $(\mathcal{F}_x)$, DFT có thể xem như phép nhân một véc-tơ hàng với ma trận biến đổi, độ phức tạp $O(|G|^2)$. Nếu cần IDFT, tức biến đổi ngược của DFT, việc tìm ma trận nghịch đảo cần tiền xử lý $O(|G|^\omega)$.

4.2 DFT truyền thống và nhóm cyclic

4.2.1 Phép chập dãy

Với phép chập dãy thông thường, quan hệ hai ngôi tương ứng là $(\mathbb{Z}, +)$. Tuy nhiên khi thao tác thực tế, ta thường lấy 2^k sao cho 2^k vượt quá độ dài dãy n , rồi chuyển quan hệ hai ngôi thành $(\mathbb{Z}_{2^k}, +)$; ở đây $\mathbb{Z}_p$ biểu thị nhóm cyclic cấp p .

4.2.2 Nhóm cyclic

Bổ đề 2.1. Với nhóm cyclic $\mathbb{Z}_p$, một DFT tương ứng là

$$\mathcal{F}_i = \{[1], [\omega_p^i], [\omega_p^{2i}], \dots, [\omega_p^{(p-1)i}]\},$$

trong đó ω_p là một căn đơn vị nguyên thủy bậc p .

Độc giả có thể tự kiểm tra rằng cách dựng này thỏa các điều kiện trên.

4.2.3 FFT và biến đổi chirp-Z

Với nhóm cyclic $\mathbb{Z}_p$ thỏa $p = 2^k$, có thể dùng FFT để tối ưu, đưa DFT về $O(p \log p)$.

Với nhóm cyclic $\mathbb{Z}_p$ tổng quát hơn, có thể dùng biến đổi chirp-Z để chuyển về trường hợp trên, cũng đạt $O(p \log p)$. Cụ thể:

$$\begin{aligned}\mathcal{F}(A)_i &= \left[\sum_{j=0}^{p-1} A_j \omega_p^{ij} \right] \\ &= \left[\frac{1}{\omega_p^{\binom{i}{2}}} \sum_{j=0}^{p-1} \frac{A_j}{\omega_p^{\binom{j}{2}}} \omega_p^{(i+j)\binom{j}{2}} \right].\end{aligned}$$

Đây là dạng của một phép chập trừ; sau khi đảo thứ tự có thể chuyển thành phép chập dãy, rồi chuyển tiếp thành bài toán trên $\mathbb{Z}_{2^k}$. Độ phức tạp là $O(p \log p)$.

Vì vậy với nhóm cyclic, ta đều có thể hoàn thành DFT trong $O(|G| \log |G|)$; độ phức tạp của phép nhân ma trận hiển nhiên là $O(|G|)$.

4.3 Tổng trực tiếp và DFT tương ứng

4.3.1 Tổng trực tiếp

Định nghĩa 3.1. Với hai quan hệ hai ngôi $(G_1, \times)$ và $(G_2, \times)$, định nghĩa tổng trực tiếp $(G, \times)$ của chúng bởi:

- $G = \{(x, y) \mid x \in G_1, y \in G_2\}$;
- $(x_1, y_1) \times (x_2, y_2) = (x_1 \times x_2, y_1 \times y_2)$.

4.3.2 Xây dựng DFT

Trước hết chứng minh một bổ đề.

Bổ đề 3.1. Với không gian tuyến tính hữu hạn chiều, ký hiệu tích tensor là $\otimes$. Đặt $V = V_1 \otimes V_2 = \text{span}\{x \mid x = a \otimes b, a \in V_1, b \in V_2\}$. Nếu một cơ sở của V_1 là $(a_i)_{i=1}^n$, một cơ sở của V_2 là $(b_j)_{j=1}^m$, thì một cơ sở của V là

$$c = \{x \mid x = a_i \otimes b_j, i \in [1, n], j \in [1, m]\}.$$

Chứng minh. Hiển nhiên c là một hệ sinh của V , nên chỉ cần chứng minh các phần tử của nó độc lập tuyến tính. Phản chứng, đặt $c_{(i-1)m+j} = a_i \otimes b_j$ và giả sử

$$\sum_{i=1}^n \sum_{j=1}^m c_{(i-1)m+j} k_{i,j} = 0$$

với không phải mọi $k_{i,j}$ đều bằng 0. Đặt $S_i = \sum_{j=1}^m k_{i,j} b_j$, ta có $\sum_{i=1}^n S_i \otimes a_i = 0$. Vì các a_i tạo thành một cơ sở, suy ra $\forall i, S_i = 0$. Vì các b_j tạo thành một cơ sở, nếu $S_i = 0$ thì $k_{i,j} = 0$ với mọi $j \in [1, m]$. Do đó mọi $k_{i,j}$ đều bằng 0, mâu thuẫn. ■

Định nghĩa 3.2. Với hai dãy ma trận R và R' có dãy kích thước lần lượt là $(a_i)_{i=1}^n$ và $(a'_i)_{i=1}^m$, định nghĩa tích tensor của chúng $R \otimes R'$ bởi

$$(R \otimes R')_{(i-1)m+j} = R_i \otimes R'_j.$$

Định lý 3.1. Nếu $G_1 \rightarrow \mathbb{C}$ và $G_2 \rightarrow \mathbb{C}$ đều có DFT, thì $G \rightarrow \mathbb{C}$ cũng có DFT.

Chứng minh. Giả sử hai DFT lần lượt là $\mathcal{A}, \mathcal{B}$, với dãy kích thước tương ứng $(a_i)_{i=1}^n$ và $(b_i)_{i=1}^m$. Giả sử DFT của $G \rightarrow \mathbb{C}$ là $\mathcal{F}$, dãy kích thước tương ứng là $(c_i)_{i=1}^{nm}$. Có thể dựng như sau:

- $c_{(i-1)m+j} = a_i b_j$, với $i \in [1, n], j \in [1, m]$;
- $\mathcal{F}_{(x,y),(i-1)m+j} = \mathcal{A}_{x,i} \otimes \mathcal{B}_{y,j}$, trong đó $\otimes$ là tích tensor.

Với phép cộng và nhân vô hướng, kết luận hiển nhiên.

Do tính chất của tích tensor $(x_1 \otimes y_1)(x_2 \otimes y_2) = (x_1 x_2) \otimes (y_1 y_2)$, trong đó x_1, x_2 là ma trận $n \times n$ và y_1, y_2 là ma trận $m \times m$, với phép nhân ta có:

$$\begin{aligned} \mathcal{F}_{(x_1, y_1), v} \times \mathcal{F}_{(x_2, y_2), v} &= (\mathcal{A}_{x_1, a} \otimes \mathcal{B}_{y_1, b})(\mathcal{A}_{x_2, a} \otimes \mathcal{B}_{y_2, b}) \\ &= (\mathcal{A}_{x_1, a} \mathcal{A}_{x_2, a}) \otimes (\mathcal{B}_{y_1, b} \mathcal{B}_{y_2, b}) \\ &= \mathcal{F}_{(x_1, y_1) \times (x_2, y_2), v}, \end{aligned}$$

trong đó $v = (a-1)m + b$, $a \in [1, n], b \in [1, m]$.

Theo Bổ đề 3.1, các $(\mathcal{F}_x)$ tạo thành một cơ sở. ■

Suy ra hệ quả sau.

Bổ đề 3.2. Nếu

$$G \rightarrow \mathbb{C} = (G_1 \times G_2 \times \cdots \times G_k) \rightarrow \mathbb{C}$$

và mọi $G_i \rightarrow \mathbb{C}$ ($i \in [1, k]$) đều có DFT, thì có thể dựng DFT của $G \rightarrow \mathbb{C}$.

4.3.3 Độ phức tạp tốt hơn

Bổ đề 3.3. Ma trận biến đổi ứng với G là tích tensor của ma trận biến đổi ứng với G_1 và ma trận biến đổi ứng với G_2 .

Điều này suy ra ngay từ quá trình dựng DFT của G .

Định lý 3.2. Với $G \rightarrow \mathbb{C} = (G_1 \times G_2) \rightarrow \mathbb{C}$, ánh xạ $a : G \rightarrow \mathbb{C}$, việc áp dụng DFT ứng với G lên a tương đương lần lượt áp dụng DFT ứng với G_1 và G_2 . Cụ thể:

- Với $x \in G_1$, đặt G'_x thỏa $G'_{x,y} = G_{(x,y)}$. Áp dụng DFT trên G_2 cho từng G'_x , ghi kết quả là H_x .
- Gọi tập dãy ma trận ứng với G_2 là R , dãy kích thước tương ứng là $(b_i)_{i=1}^k$. Với $i \in [1, k]$, đặt $H'_{i,x} = H_{x,i}$. Áp dụng DFT trên G_1 cho từng H'_i . Chú ý ánh xạ lúc này trở thành $G_1 \rightarrow M_{b_i}$, nhưng quá trình tương tự.

Chỉ cần xét tính chất của ma trận biến đổi là đủ.

Tương tự, có hệ quả:

Bổ đề 3.4. Nếu

$$G \rightarrow \mathbb{C} = (G_1 \times G_2 \times \cdots \times G_k) \rightarrow \mathbb{C},$$

thì áp dụng DFT ứng với G lên a tương đương lần lượt áp dụng DFT ứng với G_i ($i \in [1, k]$), có thể cần chia lại G .

Bổ đề 3.5. Độ phức tạp của cách làm này là

$$T = O\left(|G| \sum_{i=1}^k \frac{T_i}{|G_i|}\right),$$

trong đó T_i biểu thị độ phức tạp DFT trên G_i .

Ta có các hệ quả:

- nếu $T_i = O(|G_i| \log |G_i|)$, thì $T = O(|G| \log |G|)$;
- nếu $T_i = O(|G_i|^2)$, thì $T = O(|G| \sum |G_i|)$;
- nếu mọi $|G_i|$ đều là hằng số, có thể xem $T = O(k|G|) = O(|G| \log |G|)$.

Ví dụ, FWT n chiều là tổng trực tiếp của n bản sao $\mathbb{Z}_2 \rightarrow \mathbb{C}$, độ phức tạp là $O(|G| \log |G|) = O(n2^n)$.

Ví dụ 1 (Numbers). Nguồn bài: Zhang Huaihuai, <https://ioihw21.duck-ac.cn/problem/101>.

Cho n, k và hai mảng độ dài k là m, p với $\sum_{i=1}^k p_i = 1$. Trạng thái là một k -bộ c , ban đầu mọi phần tử đều bằng 0. Mỗi lần thao tác, với xác suất p_i ta đặt $c_i := (c_i + 1) \bmod m_i$. Hãy tính kỳ vọng của số trạng thái khác nhau về bản chất đã đi qua trước lần đầu quay lại trạng thái ban đầu. $T = \prod m_i \leq 5 \cdot 10^5$. Dữ liệu được đảm bảo ngẫu nhiên, không cần xét các trường hợp biên.

Lời giải. Đặt $G = \mathbb{Z}_{m_1} \times \mathbb{Z}_{m_2} \times \cdots \times \mathbb{Z}_{m_k}$, và 0 là trạng thái ban đầu.

Xét việc tính xác suất mỗi trạng thái được đi qua; dưới đây giả sử cần tính xác suất trạng thái $x \in G$ được đi qua. Giả định rằng khi đạt tới trạng thái ban đầu hoặc x , thao tác lập tức kết thúc. Gọi f_i là số lần kỳ vọng đi qua trạng thái i trước khi đạt tới trạng thái ban đầu hoặc x . Đặc biệt, cường bức đặt $f_0 = 1$, $f_x = 0$. Với trạng thái khác y , ta có

$$f_y = \sum_{i=1}^k f_{d(y,i)} p_i,$$

trong đó $d(y, i)$ biểu thị trạng thái thu được khi giảm phần tử thứ i của trạng thái y đi 1. Khử Gauss trực tiếp làm được trong $O(T^4)$.

Đưa vào biến d_0, d_x thỏa:

$$f_0 = \sum_{i=1}^k f_{d(0,i)} p_i - d_0 = 1, \quad f_x = \sum_{i=1}^k f_{d(x,i)} p_i - d_x = 0.$$

Kiểm tra cho thấy d_x chính là giá trị cần tìm.

Gọi F là ánh xạ $x \mapsto f_x$, và P là ánh xạ

$$x \mapsto \begin{cases} p_i, & x_j = 1 \iff i = j, \\ 0, & \text{ngược lại.} \end{cases}$$

Tương tự, với d_0, d_x định nghĩa $D : y \mapsto d_y$. Khi đó các phương trình trên có thể viết thành:

$$F = F \times P - D, \quad f_0 = 1, \quad f_x = 0.$$

Mặt khác, $F = F \times P - D$ tương đương

$$\mathcal{F}(F) = \mathcal{F}(F) \times \mathcal{F}(P) - \mathcal{F}(D),$$

tức tương đương T phương trình:

$$\mathcal{F}_i(P_i - 1) = \mathcal{D}_i \quad (i \in G).$$

Ở đây $\mathcal{F}_g$ ($g_i \in G$) và $\mathcal{F}(F)_i$ ($i \in [1, |G|]$) tương ứng một-một; $\mathcal{P}, \mathcal{D}$ được định nghĩa tương tự.

Ta thấy $\mathcal{P}_0 = \sum p_i = 1$; thực tế, với nhóm hữu hạn, mọi kết quả DFT đều có cộng dồn. Phương trình tương ứng là $\mathcal{D}_0 = d_0 + d_x = 0$. Do dữ liệu ngẫu nhiên, xác suất để $\mathcal{P}_i$ ($i \neq 0$) bằng 0 là rất nhỏ.

Định nghĩa $t(a, b)$ bởi

$$\mathcal{F}(C)_b = \sum C_a t(a, b),$$

tức là hệ số của ma trận biến đổi. Khi đó $t(a, b) = t(b, a)$ và $t(a, b)^{-1} = t(a^{-1}, b)$. Ngoài ra, $f_0 = 1$ tương đương $\sum \mathcal{F}_i = |G|$, còn $f_x = 0$ tương đương $\sum \mathcal{F}_i / t(i, x) = 0$.

Lúc này hệ phương trình phức tạp ban đầu được chuyển thành dạng:

$$\sum \mathcal{F}_i = |G|, \quad (1)$$

$$\sum \frac{\mathcal{F}_i}{t(x, i)} = 0, \quad (2)$$

$$\mathcal{F}_i(\mathcal{P}_i - 1) = \mathcal{D}_i = d_x(t(x, i) - 1) \Rightarrow \mathcal{F}_i = \frac{t(x, i) - 1}{\mathcal{P}_i - 1} d_x \quad (i \in G \setminus \{0\}). \quad (3)$$

Thay phương trình (3) vào (1), (2) thu được:

$$\mathcal{F}_0 + \left(\sum \frac{1}{\mathcal{P}_i - 1} - \sum \frac{1}{t(x, i)(\mathcal{P}_i - 1)} \right) d_x = 0,$$

$$\mathcal{F}_0 = \left(\sum \frac{1}{t(x, i)(\mathcal{P}_i - 1)} - \sum \frac{1}{\mathcal{P}_i - 1} \right) d_x,$$

$$\mathcal{F}_0 + \sum \frac{t(x, i) - 1}{\mathcal{P}_i - 1} d_x = |G|,$$

$$\left(\sum \frac{1}{t(x, i)(\mathcal{P}_i - 1)} - \sum \frac{1}{\mathcal{P}_i - 1} \right) d_x + \sum \frac{t(x, i) - 1}{\mathcal{P}_i - 1} d_x = |G|,$$

$$\left(\sum \frac{1}{t(x, i)(\mathcal{P}_i - 1)} + \sum \frac{t(x, i)}{\mathcal{P}_i - 1} - 2 \sum \frac{1}{\mathcal{P}_i - 1} \right) d_x = |G|.$$

Tính trực tiếp theo công thức cuối cùng đạt $O(|G|^2)$.

Xét tối ưu. Nút thắt là ba tổng ở vế trái. Với ánh xạ $x \mapsto 1/(\mathcal{P}_x - 1)$, thực hiện DFT; phần thứ nhất và thứ hai lần lượt là phần tử thứ x và x^{-1} của kết quả. Phần thứ ba là một hằng số. Do đó ta tối ưu được độ phức tạp về độ phức tạp DFT, tức $O(T \log T)$, đủ để qua bài.

Đây là một ứng dụng kinh điển của DFT: dùng DFT chuyển hệ phương trình chập phức tạp thành các phương trình nhân điểm đơn giản, rồi dùng độ dời và giá trị đặc biệt để suy ra phương trình giải độ dời, cuối cùng giải được đáp án.

4.4 DFT trên nửa nhóm giao hoán hữu hạn

4.4.1 DFT trên nhóm giao hoán hữu hạn

Phân rã nhóm giao hoán hữu hạn.

Định nghĩa 4.1. Nếu một bộ phần tử $[a_1, a_2, \dots, a_m]$ của nhóm giao hoán hữu hạn G thỏa:

- $\forall x \in G, \exists (k_i)_{i=1}^m$ sao cho $\prod_{i=1}^m a_i^{k_i} = x$;
- $\forall (k_i)_{i=1}^m$, nếu $\prod_{i=1}^m a_i^{k_i} = e$ thì $a_i^{k_i} = e$ với mọi $i \in [1, m]$,

thì bộ phần tử này là một phân rã của G .

Bổ đề 4.1. $[a_1, a_2, \dots, a_m]$ là một phân rã của G khi và chỉ khi

$$(a_1) \times (a_2) \times \dots \times (a_m) = G.$$

Xét hai tính chất trên, chứng minh khá đơn giản.

Tiếp theo ta chứng minh mọi nhóm giao hoán hữu hạn đều tồn tại phân rã, đồng thời đưa ra cách dựng.

Bổ đề 4.2. Mọi nhóm giao hoán hữu hạn G đều có thể phân rã thành tổng trực tiếp của các nhóm p giao hoán hữu hạn.

Chứng minh. Gọi $o(x)$ là cấp của x . Giả sử cấp của G là $p^r m$, với p nguyên tố và $p \nmid m$. Đặt

$$G_p = \{x \mid o(x) \mid p^r\}, \quad G_m = \{x \mid o(x) \mid m\}.$$

Ta chứng minh $G_p \times G_m = G$.

Dựng ánh xạ $\varphi : (x, y) \mapsto xy$. Hiển nhiên đây là một đồng cấu nhóm; chỉ cần chứng minh nó toàn ánh và $\ker \varphi = \{e\}$.

Với phần tử bất kỳ x , theo định lý Lagrange, $o(x) \mid p^r m$. Đặt $o(x) = p^s m'$ với $p \nmid m'$, khi đó $o(x^{p^s}) = m' \mid m$ và $o(x^{m'}) = p^s \mid p^r$. Vì $(p^s, m) = 1$, tồn tại u, v sao cho $up^s + vm = 1$, nên $x = x^{up^s + vm} = x^{up^s} x^{vm}$. Do đó φ là toàn ánh.

Nếu $x \in G_p, y \in G_m$ và $xy = e$, vì $o(x) = o(x^{-1}) = o(y)$, ta có $o(x) \mid (p^r, m) = 1$, nên $x = y = e$. Vậy $\ker \varphi = \{e\}$.

Do đó φ là một đẳng cấu nhóm. Lặp lại quá trình này, ta có thể phân rã G thành tổng trực tiếp của một số nhóm p giao hoán hữu hạn. ■

Phân rã nhóm p giao hoán hữu hạn.

Bổ đề 4.3. Với nhóm p giao hoán hữu hạn G , lấy một phần tử a có cấp lớn nhất và đặt $H = \langle a \rangle$. Khi đó với mọi $xH \in G/H$, tồn tại $y \in xH$ sao cho $o(xH) = o(y)$.

Chứng minh. Đặt $o(a) = p^n, o(x) = p^t, o(xH) = p^s$; hiển nhiên $t \geq s \geq k$. Xét $x^{p^s} = (xH)^{p^s} = H$, nên $x^{p^s} \in H$. Đặt $x^{p^s} = a^r$, khi đó

$$p^{t-s} = o(x^{p^s}) = o(a^r) = \frac{o(a)}{(o(a), r)} = \frac{p^n}{(p^n, r)}.$$

Suy ra $(o(a), r) = p^{n-t+s}$. Không mất tính tổng quát, đặt $r = up^{n-t+s}$.

Dựng $y = xa^{p^{n-t}v} \in xH$, trong đó $v = -up^{n-t}$. Khi đó

$$(yH)^{p^s} = y^{p^s} H = x^{p^s} a^{p^s p^{n-t}v} H = H.$$

Từ đó suy ra có thể chọn đại diện trong lớp xH có cấp đúng bằng $o(xH)$. ■

Bổ đề 4.4. Mọi nhóm p giao hoán hữu hạn G đều có thể được phân rã.

Chứng minh. Quy nạp theo $|G| = p^n$, giả sử mọi nhóm G' có $|G'| < p^n$ đều tồn tại phân rã. Tìm phần tử có cấp lớn nhất $a \in G$, $o(a) = p^k$. Nếu $k = n$ thì $[a]$ chính là một phân rã.

Đặt $H = \langle a \rangle$ và $G' = G/H$. Theo giả thiết quy nạp, G' tồn tại phân rã; giả sử là $[a_1H, a_2H, \dots, a_mH]$. Theo Bổ đề 4.3, chọn các đại diện sao cho $o(a_i) = o(a_iH)$ với $i \in [1, m]$.

Ta chứng minh $[a, a_1, a_2, \dots, a_m]$ là một phân rã.

Với mọi $x \in G$, xét $xH \in G/H$. Có

$$xH = \prod_{i=1}^m (a_iH)^{k_i} = \left(\prod_{i=1}^m a_i^{k_i} \right) H.$$

Do đó tồn tại t sao cho $x = a^t \prod_{i=1}^m a_i^{k_i}$.

Ngược lại, nếu $a^t \prod_{i=1}^m a_i^{k_i} = e$, thì $\prod_{i=1}^m (a_iH)^{k_i} = H$. Theo tính chất của a_iH , suy ra $(a_iH)^{k_i} = H$ với mọi i , tức $a_i^{k_i} \in H$. Vì $o(a_i) = o(a_iH)$, ta có $a_i^{k_i} = e$. Khi đó cũng suy ra $a^t = e$. ■

Định lý 4.1. Mọi nhóm giao hoán hữu hạn đều có thể được phân rã.

Điều này suy ra ngay từ Bổ đề 4.2 và Bổ đề 4.4.

Một cách cài đặt đơn giản hơn. Chứng minh trên thực ra đã cho một quá trình dựng, nhưng vẫn quá phức tạp.

Định lý 4.2. Với nhóm giao hoán hữu hạn G , duy trì dãy B , ban đầu rỗng, và $S = \text{span}(B)$. Đặt $f(x) = \min\{k \mid x^k \in S\}$. Lập các thao tác:

- nếu $G = S$ thì kết thúc;
- tìm mọi $x \in G, x \notin S$, và chọn phần tử a có $o(x)/f(x)$ lớn nhất;
- $B := B + a, S = \text{span}(B)$.

Dãy B thu được là một phân rã.

Xét mệnh đề mạnh hơn sau.

Bổ đề 4.5. Với mọi $n \geq 0$, tại một thời điểm nào đó nếu $B = [a_1, a_2, \dots, a_n]$, thì tồn tại một phân rã $[a_1, a_2, \dots, a_m]$ ($m \geq n$) sao cho $o(a_i) = p_i^{k_i}$ với $n < i \leq m$, p_i nguyên tố và $k_i \in \mathbb{Z}^+$.

Chứng minh. Quy nạp theo n ; với $n = 0$ hiển nhiên đúng. Giả sử trước khi phần tử thứ n được thêm vào, một phân rã hợp lệ là $B' = [a_1, \dots, a_r]$.

Với phần tử x , nếu $x = \prod_{i=1}^r (a_i')^{t_i}$, thì

$$f(x) = \text{lcm}_i \frac{o(a_i')}{(o(a_i'), t_i)}.$$

Khi $f(x)$ lớn nhất, có thể tìm một lũy thừa p^k và một chỉ số i sao cho $(o(a_i')/p^k, t_i) = 1$ và k đạt lớn nhất; từ đó dựng được một phần tử x có $o(x) = f(x)$, $x \notin S$ và $f(x) = \max_{y \in G \setminus S} f(y)$.

Dựng

$$B = (a_1, \dots, a_{n-1}, x) + R,$$

trong đó R nhận được bằng cách thay trong phân rã cũ một phần tử tương ứng bởi các phần còn lại. Không khó chứng minh $\text{span}(B)$ chứa phần tử bị xóa, nên $\text{span}(B) = G$; hơn nữa cấp của các phần tử trong B vẫn đúng dạng trên, vì vậy đây là một phân rã. ■

Bổ đề 4.6. Với cài đặt thích hợp, cách làm này có độ phức tạp $O(|G|^2)$.

Chứng minh. Nút thắt độ phức tạp nằm ở việc tính $f(x)$. Ở vòng thứ n , ta có

$$f(x) \leq \prod_i o(a_i), \quad f(x) < \frac{|G|}{\prod_i o(a_i)}.$$

Vì vậy tổng chi phí tính $f(x)$ không vượt quá $O(|G|)$ cho mỗi phần tử, dẫn tới tổng $O(|G|^2)$. ■

Nửa nhóm giao hoán hữu hạn.

Định lý 4.3. $G \rightarrow \mathbb{C}$ tồn tại DFT khi và chỉ khi G thỏa luật tuần hoàn.

Chứng minh và cách dựng được trình bày trong tài liệu [6], bài viết này không triển khai chi tiết.

4.5 Nhóm đối xứng

4.5.1 Định nghĩa

Định nghĩa 5.1. Định nghĩa tích của hai biến đổi tuyến tính là tác động liên tiếp của hai biến đổi tuyến tính, tức phép nhân ma trận. Khi đó, với không gian véc-tơ V n chiều, mọi biến đổi tuyến tính không suy biến, tức có định thức khác 0, tạo thành nhóm tuyến tính tổng quát n chiều, ký hiệu là $GL(V)$. Nhóm con $L(V)$ của nó được gọi là nhóm biến đổi tuyến tính trên V .

Định nghĩa 5.2. Với nhóm G , nếu tồn tại một đồng cấu p từ G tới nhóm biến đổi tuyến tính $L(V)$ trên không gian tuyến tính V , thì p được gọi là một biểu diễn tuyến tính của nhóm G . Không gian V là không gian biểu diễn; số chiều của V là số chiều của biểu diễn, ký hiệu là d_p .

Định nghĩa 5.3. Giả sử hai biểu diễn p, p' của nhóm G trên không gian V lần lượt lấy cơ sở

$$e = (e_1, e_2, \dots, e_n), \quad e' = (e'_1, e'_2, \dots, e'_n).$$

Nếu $e = e'X$, $\det X \neq 0$, và với mọi $g \in G$ có $p(g) = X^{-1}p'(g)X$, thì gọi p, p' là tương đương.

Định nghĩa 5.4. Nếu p là một biểu diễn của nhóm G trên không gian tuyến tính V , và không tồn tại không gian con $W \subset V$ thỏa $W \neq 0$, $W \neq V$ và với mọi $g \in G$, $x \in W$ đều có $p(g)x \in W$, thì p là một biểu diễn bất khả quy.

Ký hiệu tập mọi biểu diễn bất khả quy đôi một không tương đương của G là $R(G)$.

4.5.2 DFT của nhóm đối xứng

Tiếp theo bài viết đưa ra thuật toán DFT cho nhóm S_n . Do giới hạn dung lượng và trình độ của tác giả, phần này không đưa ra chứng minh và cài đặt cụ thể; xem các tài liệu [1], [2], [3].

Định lý 5.1 (định lý Burnside). Với nhóm hữu hạn G , ta có

$$\sum_{p \in R(G)} d_p^2 = |G|.$$

Định lý này chứng minh rằng sau DFT, lượng thông tin là đủ.

Bổ đề 5.1. Với nhóm hữu hạn G , cách dựng sau là một DFT hợp lệ:

- $a_i = d_{p_i}$;
- $\mathcal{F}(A)_i = \sum_{g \in G} A_g p_i(g)$.

Điều này cho thấy biểu diễn nhóm và DFT có liên hệ rất chặt. Nếu đã biết $R(G)$, ta có thể hoàn thành DFT trong $O(|G|^2)$. Đáng tiếc là hiện không có một thuật toán tổng quát để tìm $R(G)$ của nhóm hữu hạn tùy ý G .

Bổ đề 5.2. Với cách dựng trên, một cách tính IDFT khác là

$$A_x = \frac{1}{|G|} \sum_{p_i \in R(G)} d_{p_i} \text{Tr}(p_i(x^{-1}) \mathcal{F}(A)_i).$$

Bổ đề 5.3. $R(S_n)$ có quan hệ một-một với các phân hoạch của n .

Tiếp theo ta đưa ra một cách dựng gọi là biểu diễn bán chuẩn Young.

Định nghĩa 5.5. Với hai bảng Young chuẩn bậc n có cùng hình dạng T_1, T_2 , định nghĩa thứ tự không nhân của chúng như sau. Nếu số n nằm ở hai hàng khác nhau trong T_1, T_2 , thì $T_1 < T_2$ khi và chỉ khi hàng của n trong T_1 nhỏ hơn. Nếu không, đặt T'_1, T'_2 là hai bảng Young thu được sau khi xóa phần tử n khỏi T_1, T_2 , và đặt $T_1 < T_2$ khi và chỉ khi $T'_1 < T'_2$.

Định nghĩa 5.6. Với một bảng Young T , giả sử phần tử n, m lần lượt nằm ở các hàng x_1, x_2 và các cột y_1, y_2 . Định nghĩa khoảng cách trực

$$d(n, m) = x_2 - x_1 + y_1 - y_2,$$

chú ý không lấy trị tuyệt đối.

Bổ đề 5.4. Hình dạng của các bảng Young bậc n tương ứng một-một với các phân hoạch của n . Chỉ cần xét độ dài từng hàng của bảng Young là thấy.

Định nghĩa 5.7. Với phân hoạch a của n , sắp các bảng Young tương ứng theo thứ tự không nhân thành T . Ký hiệu khoảng cách trực của a, b trong T_i là $d_i(a, b)$. Dưới đây cho cách dựng $p((t, t+1))$; đặt $p((t, t+1)) = a$.

- Với mọi i , $a_{i,i} = d_i(t, t+1)^{-1}$.
- Nếu đổi chỗ t và $t+1$ trong T_i vẫn là bảng Young chuẩn hợp lệ, gọi bảng thu được là T_j , thì

$$a_{i,j} = \begin{cases} 1 - d_i(t, t+1)^{-2}, & i < j, \\ 1, & i > j. \end{cases}$$

- Các vị trí còn lại đều đặt bằng 0.

Với hoán vị bất kỳ p , dùng sắp xếp nổi bọt có thể chứng minh nó phân rã được thành tích của một số hoán vị dạng $(i, i+1)$; từ đó hợp nhất để thu được $p(p)$. Có thể chứng minh đây là một họ biểu diễn bất khả quy và đôi một không tương đương. Tính trực tiếp, chi phí DFT là $O(|G|^2)$.

4.5.3 Độ phức tạp tốt hơn

Xét S_n và đặt

$$H = \{x \mid x(n) = n\} \sim S_{n-1}.$$

Khi đó $\{s_i H \mid s_i \in S_n\}$ tạo thành một phân hoạch theo các lớp kề của S_n . Biến đổi công thức định nghĩa DFT:

$$\begin{aligned} \mathcal{F}(A) &= \sum_{g \in G} A_g p_i(g) \\ &= \sum_t p_i(s_t) \sum_{x \in H} A_{s_t x} p_i(x). \end{aligned}$$

Đặt $A'_x = A_{s_t x}$.

Định lý 5.2. Với $x \in H$, dạng của $p(x)$ là

$$\begin{bmatrix} p_{i_1}(x) & 0 & \cdots & 0 \\ 0 & p_{i_2}(x) & \cdots & 0 \\ 0 & 0 & \ddots & p_{i_k}(x) \end{bmatrix},$$

trong đó i_j biểu thị hình dạng phân hoạch hoặc bảng Young bậc $n-1$ có thể thu được sau khi xóa một phần tử khỏi hình dạng phân hoạch hoặc bảng Young a_j .

Chỉ cần khảo sát vị trí của n trong bảng Young. Các bảng có cùng vị trí của n chắc chắn liên tiếp trong T . Vì $x(n) = n$, xóa n không ảnh hưởng quá trình dựng. Quy nạp là chứng minh được.

Do đó với $A' : S_n \rightarrow \mathbb{C}$, ta có thể giải riêng từng $\mathcal{F}(A')$, rồi đệ quy thành bài toán con $S_{n-1} \rightarrow \mathbb{C}$, sau đó hợp nhất bằng vài lần nhân dãy ma trận.

Bổ đề 5.5. Trong các ma trận $C_t = p((t, t+1))$ với $t \in [1, n-2]$ và $D_t = D_{\text{prev}} A_t p(t)$ với $t \in [1, n)$, tổng số phần tử khác 0 là $O(n!)$.

Chứng minh. Xét mỗi phân hoạch hoặc hình dạng bảng Young a của $n-1$. Nó đóng góp nhiều nhất một lần vào mỗi C_t và D_t , tức xuất hiện nhiều nhất $2n-2$ lần. Ta biết $\sum a_i = (n-1)!$, nên tổng đóng góp không vượt quá $O(n!)$. ■

Với s_i , ta chọn $s_i = (n, n-1, \dots, i)$. Khi đó $s_i = s_{i-1} \times (i, i+1)$; như vậy chỉ cần n lần nhân dãy ma trận là tính được $p(s_i)$. Chú ý rằng nhờ Bổ đề 5.5, tổng độ phức tạp của phép cộng và phép nhân ma trận không vượt quá $O(n!n)$.

Vì vậy độ phức tạp DFT của S_n là

$$T(n) = nT(n-1) + O(n!n^2) = O(n!n^3),$$

tốt hơn rõ rệt so với cách thô bạo $O((n!)^2)$.

Thuật toán này dùng chia để trị tương tự FFT, đệ quy các trường hợp khác nhau về cùng một bài toán con để giảm độ phức tạp. Với các nhóm tổng quát hơn, nếu tìm được cách dựng tương tự, ta cũng có thể hoàn thành DFT với độ phức tạp thấp.

Kết luận

DFT đã được dùng rộng rãi trong OI, nhưng trong thời gian dài ứng dụng của DFT hầu như chỉ giới hạn ở phép chập dãy; những năm gần đây mới mở rộng tới chuỗi lũy thừa tập hợp. Tuy nhiên với các định nghĩa phép chập tổng quát hơn, cộng đồng OI chưa nghiên cứu nhiều. Bài viết này cho thấy nhóm không

giao hoán cũng có thể thực hiện phép chập với độ phức tạp cao hơn, đồng thời chỉ ra quan hệ giữa DFT và biểu diễn nhóm. Tác giả chỉ trình bày DFT của nhóm đối xứng, chưa trình bày ứng dụng của nó; ý nghĩa sâu hơn và ứng dụng đa dạng hơn của DFT vẫn cần các bạn đọc quan tâm tiếp tục nghiên cứu.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Xu Xianyou đã quan tâm và hướng dẫn; cảm ơn bạn Xu Tingqiang và anh Ma Yaohua đã giúp đỡ về nội dung; cảm ơn bạn Xu Tingqiang, bạn Luo Kai, anh Jin Ce và anh Xu Yinzhan đã đọc kiểm tra bản thảo.

Tài liệu tham khảo

1. Michael Clausen; Ulrich Baum. *Fast Fourier transforms for symmetric groups: theory and implementation*. Math. Comp. 61 (1993), pp. 833–847, 1993.
2. Risi Kondor. *A C++ library for fast Fourier transforms on the symmetric group*.
3. Persi Diaconis; Daniel Rockmore. *Efficient computation of the Fourier transform on finite groups*. J. Amer. Math. Soc. 3 (1990), pp. 297–332, 1990.
4. Audrey Terras. *Fourier Analysis on Finite Groups and Applications*. Cambridge University Press, 1999.
5. Josh Alman; Virginia Vassilevska Williams. *A refined laser method and faster matrix multiplication*. The 32nd Annual ACM-SIAM Symposium on Discrete Algorithms, 2020.
6. Liu Cheng'ao. *Bàn về ứng dụng của DFT trong thi tin học*. Tập luận văn đội tuyển ứng viên quốc gia Trung Quốc IOI 2018, 2018.

5 Bàn về ứng dụng của một tư tưởng tham lam dựa trên so sánh trong thi tin học

Zhang Junkai, Trường Ngoại ngữ Thành Đô

Tóm tắt

Bài viết giới thiệu ứng dụng trong thi tin học của một tư tưởng tham lam dựa trên so sánh. Trước hết bài viết trình bày dạng bài toán, quá trình của phương pháp và một điều kiện cần dùng để áp dụng phương pháp này. Sau đó bài viết đưa vào một lớp bài toán trên cây có dạng tương tự, rồi đưa ra thuật toán dựa trên tư tưởng tham lam nói trên. Cuối cùng bài viết phân tích một số mô hình thường gặp trong thi tin học thỏa các điều kiện đó, cùng một phần tính chất của thuật toán.

5.1 Mở đầu

Trong thi tin học, tư tưởng tham lam là một phần cực kỳ quan trọng. Trong các lời giải tham lam có một lớp phương pháp phân tích ảnh hưởng của việc hoán đổi hai phần tử trong nghiệm lên đáp án, từ đó suy ra tính chất mà nghiệm tối ưu phải thỏa. Phương pháp này thường được gọi là *exchange arguments*. Nó đã xuất hiện trong thi tin học từ hơn mười năm trước, những năm gần đây cũng có một số ví dụ mở rộng, nhưng việc phân tích và nghiên cứu hệ thống về nó vẫn còn khá ít. Bài viết này phân tích phương pháp đó tương đối chi tiết.

Phần đầu dùng một ví dụ để dẫn nhập phương pháp, rồi phân tích một điều kiện đủ để phương pháp thu được nghiệm tối ưu đúng. Phần thứ hai giới thiệu một mở rộng cổ điển: trên dạng bài toán trước đó thêm một ràng buộc có dạng cây có gốc; sau đó đưa ra một điều kiện tương tự nhưng mạnh hơn và hai thuật toán khi bài toán thỏa điều kiện ấy. Trong cả hai phần, bài viết thảo luận một số mô hình thường gặp và phân tích một số tính chất, ứng dụng của thuật toán.

5.2 Một lớp bài toán tối ưu liên quan tới hoán vị và phương pháp exchange arguments

5.2.1 Dẫn nhập bài toán

Trong phần này ta chủ yếu xét dạng bài toán sau.

Cho n phần tử $x_1, \dots, x_n$ và một hàm F có miền xác định là các dãy gồm các phần tử này, miền giá trị là một tập có thứ tự. Với mọi hoán vị bậc n là p , hãy tìm giá trị nhỏ nhất của

$$F((x_{p_1}, x_{p_2}, \dots, x_{p_n})).$$

Ví dụ 1. Cho n cặp số nguyên dương (a_i, b_i) , hãy sắp chúng theo thứ tự tùy ý thành một dãy. Định nghĩa chi phí của dãy là tổng, trên mọi cặp trong dãy, của “ a của cặp đó nhân với tổng các b của mọi cặp đứng sau nó”. Hãy tìm chi phí nhỏ nhất.

$$n \leq 10^6, \quad 1 \leq a_i, b_i \leq 10^6.$$

Bài toán này là một trường hợp của dạng trên, với

$$F(((a_1, b_1), (a_2, b_2), \dots, (a_k, b_k))) = \sum_{1 \leq i < j \leq k} a_i b_j.$$

Rõ ràng các số thực có thể so sánh được.

Ta phân tích tính chất của một hoán vị tối ưu. Xét một hoán vị $(p_1, \dots, p_n)$ và hoán vị nhận được bằng cách đổi hai vị trí kề nhau $(i, i+1)$:

$$(q_1, \dots, q_n) = (p_1, \dots, p_{i-1}, p_{i+1}, p_i, p_{i+2}, \dots, p_n).$$

Khi đó

$$\begin{aligned} & F(((a_{p_1}, b_{p_1}), \dots, (a_{p_n}, b_{p_n}))) - F(((a_{q_1}, b_{q_1}), \dots, (a_{q_n}, b_{q_n}))) \\ &= a_{p_i} b_{p_{i+1}} - a_{p_{i+1}} b_{p_i}. \end{aligned} \tag{1}$$

Vì vậy nếu $a_{p_i} b_{p_{i+1}} - a_{p_{i+1}} b_{p_i} > 0$, sau khi đổi chỗ hai phần tử đó thì F nhỏ hơn, nên $(p_1, \dots, p_n)$ không thể là nghiệm tối ưu. Do đó chỉ các hoán vị thỏa

$$\forall 1 \leq i < n, \quad a_{p_i} b_{p_{i+1}} - a_{p_{i+1}} b_{p_i} \leq 0, \quad \text{tức} \quad \frac{a_{p_i}}{b_{p_i}} \leq \frac{a_{p_{i+1}}}{b_{p_{i+1}}},$$

mới có thể tối ưu.

Nếu một hoán vị p thỏa điều kiện đó thì với mọi $1 \leq i < j \leq n$ ta có

$$\frac{a_{p_i}}{b_{p_i}} \leq \frac{a_{p_j}}{b_{p_j}}.$$

Do đó, với mọi giá trị v , các vị trí i thỏa $a_{p_i}/b_{p_i} = v$ tạo thành một đoạn liên tiếp trong hoán vị, và các đoạn khác nhau được sắp tăng dần theo a/b . Nếu hai phần tử trong cùng một đoạn có cùng tỉ số, theo (1), đổi chỗ hai phần tử kề nhau không làm thay đổi F ; suy ra mọi hoán vị thỏa điều kiện trên có cùng giá trị F và đều tối ưu.

Vì thế với bài toán này, chỉ cần sắp mọi cặp theo a_i/b_i là thu được một cách sắp tối ưu. Độ phức tạp là thời gian sắp xếp $O(n \log n)$.

5.2.2 Phân tích bài toán và phương pháp exchange arguments

Xét cách làm trong ví dụ trên. Nó có thể được xem là xây dựng một quan hệ so sánh giữa mỗi hai phần tử x_i, x_j , cụ thể là

$$x_i < x_j \iff F((x_i, x_j)) \leq F((x_j, x_i)).$$

Trong bài đó, quan hệ so sánh được định nghĩa như vậy và bản thân bài toán thỏa một số tính chất đặc biệt, khiến việc sắp mọi phần tử theo quan hệ so sánh cho ra nghiệm tối ưu. Nhưng trong tình huống tổng quát hơn, điều này hiển nhiên không còn đúng; thực tế với một số hàm F , dạng bài toán trên hiện chỉ có thuật toán mũ. Ta phân tích các tính chất của ví dụ trước để tìm một điều kiện đủ cho thuật toán sau:

Gọi S là tập tất cả phần tử đã cho. Định nghĩa một quan hệ nhị nguyên $<$ trên S , rồi sắp mọi phần tử theo quan hệ đó.

Trong ví dụ trước, dễ thấy tồn tại các tính chất sau.

Tính chất 2.1. Với mọi $a, b \in S$, ít nhất một trong $a < b$ và $b < a$ đúng; tức quan hệ so sánh có tính toàn phần mạnh.

Tính chất 2.2. Với mọi $a, b, c \in S$, nếu $a < b$ và $b < c$ thì $a < c$; tức quan hệ có tính bắc cầu.

Tính chất 2.3. Với mọi $a, b \in S$, nếu $a < b$ thì với mọi dãy phần tử chứa $\{a, b\}$ như một dãy con liên tiếp, ta có

$$F((s_1, \dots, s_{k-1}, a, b, s_{k+2}, \dots, s_\ell)) \leq F((s_1, \dots, s_{k-1}, b, a, s_{k+2}, \dots, s_\ell)).$$

Định lý 2.1. Nếu tồn tại một quan hệ nhị nguyên $<$ trên S thỏa các Tính chất 2.1, 2.2 và 2.3, thì dùng quan hệ này trong thuật toán sắp xếp trên sẽ thu được một nghiệm tối ưu.

Chứng minh. Theo giả thiết, $<$ tạo thành một thứ tự yếu (*weak ordering*, còn gọi là quan hệ ưu tiên) trên S . Do đó, nếu $|S| = n$, tồn tại một cách xếp các phần tử thành dãy $(x_1, \dots, x_n)$ sao cho

$$\forall 1 \leq i < n, \quad x_i < x_{i+1}.$$

Với một quan hệ so sánh thỏa thứ tự yếu, thuật toán sắp xếp dựa trên so sánh có thể tìm một dãy như vậy bằng $O(n \log n)$ lần so sánh.

Bổ đề 2.1. Nếu một hoán vị $(x_1, \dots, x_n)$ của các phần tử trong S thỏa $x_i < x_{i+1}$ với mọi $1 \leq i < n$, và $<$ thỏa Tính chất 2.3, thì không tồn tại hoán vị $(y_1, \dots, y_n)$ của S sao cho

$$F((y_1, \dots, y_n)) < F((x_1, \dots, x_n)).$$

Nói cách khác, $(x_1, \dots, x_n)$ đạt giá trị nhỏ nhất trong mọi hoán vị.

Chứng minh. Từ tính bắc cầu của $<$, ta có $x_i < x_j$ với mọi $i < j$. Xét một hoán vị bất kỳ $(y_1, \dots, y_n) = (x_{p_1}, \dots, x_{p_n})$. Ta biến đổi nó về $(x_1, \dots, x_n)$ bằng các lần đổi chỗ kề nhau: ở lượt i , đưa x_i về vị trí thứ i bằng cách liên tục đổi x_i với phần tử đứng ngay trước nó. Trong mỗi lần đổi, phần tử đứng trước x_i là một x_j với $j > i$, nên $x_i < x_j$. Theo Tính chất 2.3, đổi từ (x_j, x_i) sang (x_i, x_j) không làm F tăng. Sau tất cả các lần đổi, F không tăng, nên

$$F((x_1, \dots, x_n)) \leq F((x_{p_1}, \dots, x_{p_n})).$$

■

Từ bổ đề, mọi hoán vị $(x_1, \dots, x_n)$ thỏa $x_i < x_{i+1}$ đều là nghiệm tối ưu. Điều này cũng chứng minh tính đúng đắn của cách giải trong ví dụ đầu.

5.2.3 Một số ví dụ và mô hình tương ứng

Một số ví dụ. Ngoài ví dụ trên còn có các bài toán khác có dạng tương tự; dưới đây là một vài ví dụ.

Ví dụ 2. Cho n xâu $s_1, \dots, s_n$. Hãy chọn một hoán vị p của n phần tử sao cho xâu nhận được bằng cách nối $s_{p_1}, s_{p_2}, \dots, s_{p_n}$ theo thứ tự có thứ tự từ điển nhỏ nhất.

$$\sum_i |s_i| \leq 5 \times 10^5.$$

Lấy F là kết quả nối các xâu theo thứ tự; so sánh từ điển giữa các xâu là hợp lệ. Vẫn đặt $x < y$ khi và chỉ khi $F((x, y)) \leq F((y, x))$. Khi đó, với hai xâu s, t , ta có $s < t$ khi và chỉ khi $s + t$ không lớn hơn $t + s$ theo thứ tự từ điển (dấu + biểu thị phép nối xâu). Rõ ràng $<$ thỏa Tính chất 2.1. So sánh từ điển không đổi nếu thêm cùng một xâu vào đầu hoặc cuối hai xâu cùng độ dài, nên $<$ thỏa Tính chất 2.3. Chỉ cần chứng minh tính bắc cầu.

Bổ đề 2.2. Ký hiệu s^∞ là xâu vô hạn nhận được bằng cách lặp s vô hạn lần. Khi đó $s + t$ không lớn hơn $t + s$ theo thứ tự từ điển khi và chỉ khi s^∞ không lớn hơn t^∞ theo thứ tự từ điển.

Chứng minh bổ đề này tương đối phức tạp và ít liên quan tới phần còn lại của bài viết, nên được lược bỏ trong nguồn. Từ bổ đề, do so sánh từ điển có tính bắc cầu, quan hệ $<$ nói trên cũng bắc cầu, vì vậy nó thỏa ba tính chất. Dùng cách làm này, với mảng hậu tố hoặc thuật toán tương tự, sau tiên xử lý có thể so sánh $s + t$ và $t + s$ trong $O(1)$. Khi đó độ phức tạp là độ phức tạp dựng cấu trúc và sắp xếp; với cách cài đặt mảng hậu tố thường gặp là $O(L \log L)$ với $L = \sum_i |s_i|$.

Ví dụ 3. Có n ổ đĩa cần định dạng. Ổ thứ i có dung lượng trước khi định dạng là a_i và sau khi định dạng là b_i . Ban đầu mỗi ổ đều chứa đầy dữ liệu; trong quá trình định dạng một ổ, cần chuyển toàn bộ dữ liệu trên ổ đó sang các ổ khác hoặc sang bộ nhớ ngoài. Dữ liệu có thể chuyển tùy ý và chia tùy ý. Hỏi cần ít nhất bao nhiêu bộ nhớ ngoài để định dạng toàn bộ ổ mà vẫn giữ được dữ liệu.

$$n \leq 10^6, \quad 0 < a_i, b_i < 10^6.$$

Xét một thứ tự định dạng $p_1, \dots, p_n$. Bài toán có thể viết thành: cho n cặp (a_i, b_i) , tìm một hoán vị tối đa hóa

$$\min_j \left\{ \sum_{i=1}^j b_{p_i} - \sum_{i=1}^j a_{p_i} \right\}.$$

Tương đương, đặt

$$F(((a_1, b_1), \dots, (a_k, b_k))) = - \min_j \left\{ \sum_{i=1}^j b_i - \sum_{i=1}^j a_i \right\}$$

và tối thiểu hóa F .

Tiếp tục định nghĩa $x < y$ khi $F((x, y)) \leq F((y, x))$. Với $(a_1, b_1), (a_2, b_2)$, điều kiện này tương đương

$$\min\{-a_1, b_1 - a_1 - a_2\} \geq \min\{-a_2, b_2 - a_1 - a_2\}.$$

Để kiểm tra Tính chất 2.3 bằng cách xét đóng góp của hai phần tử kề nhau bị đổi chỗ; phần tiền tố và hậu tố ngoài hai phần tử đó giữ nguyên, còn bất đẳng thức trên đúng chính là điều kiện làm giá trị F không tăng.

Để phân tích quan hệ so sánh, xét dấu của $b_i - a_i$. Ký hiệu

$$\text{sgn}(x) = \begin{cases} 1, & x > 0, \\ 0, & x = 0, \\ -1, & x < 0. \end{cases}$$

Bổ đề 2.3. Với hai cặp $(a_1, b_1), (a_2, b_2)$:

1. nếu $\text{sgn}(b_1 - a_1) > \text{sgn}(b_2 - a_2)$ thì

$$\min\{-a_1, b_1 - a_1 - a_2\} \geq \min\{-a_2, b_2 - a_1 - a_2\};$$

2. nếu cả hai dấu đều bằng 1, bất đẳng thức trên đúng khi $a_1 \leq a_2$;
3. nếu cả hai dấu đều bằng 0, bất đẳng thức trên luôn đúng;
4. nếu cả hai dấu đều bằng -1 , bất đẳng thức trên đúng khi $b_1 \geq b_2$.

Từ bổ đề, nếu chia mọi phần tử thành ba lớp theo $\text{sgn}(b_i - a_i)$, thì trong từng lớp riêng lẻ quan hệ $<$ là một thứ tự yếu hợp lệ. Tuy nhiên giữa các lớp, quan hệ định nghĩa trực tiếp bởi $F((x, y)) \leq F((y, x))$ chưa chắc bắc cầu. Ví dụ có thể có $(2, 2) < (1, 1) < (1, 2)$ nhưng $(2, 2) \not< (1, 2)$.

Mặt khác, nếu $\text{sgn}(b_1 - a_1) > \text{sgn}(b_2 - a_2)$ thì nhất định $(a_1, b_1) < (a_2, b_2)$. Vì vậy ta định nghĩa lại $<$ như sau: $(a_1, b_1) < (a_2, b_2)$ khi và chỉ khi một trong hai điều kiện sau đúng:

1. $\text{sgn}(b_1 - a_1) > \text{sgn}(b_2 - a_2)$;
2. $\text{sgn}(b_1 - a_1) = \text{sgn}(b_2 - a_2)$ và

$$F(((a_1, b_1), (a_2, b_2))) \leq F(((a_2, b_2), (a_1, b_1))).$$

Để kiểm tra quan hệ này thỏa ba tính chất, nên sắp xếp theo nó giải được bài toán trong $O(n \log n)$.

Ví dụ này cho thấy vì Tính chất 2.3 chỉ cho một chiều hàm ý, không nhất thiết có thể dùng trực tiếp $F((x, y)) \leq F((y, x))$ để định nghĩa quan hệ so sánh. Nhưng từ so sánh do phép đổi chỗ gợi ra vẫn là một hướng xuất phát tốt.

Ví dụ 4. Có n hộp, hộp thứ i có trọng lượng w_i và sức chịu tải v_i ($w_i, v_i > 0$). Hãy xếp chúng thành một chồng sao cho tổng trọng lượng các hộp nằm trên mỗi hộp không vượt quá sức chịu tải của hộp đó.

$$n \leq 10^6.$$

Với một dãy hộp $((w_1, v_1), \dots, (w_n, v_n))$, cần

$$\min_j \left\{ v_j - \sum_{i < j} w_i \right\} \geq 0.$$

Nếu tối đa hóa về trái, ta được một bài toán cùng dạng với Ví dụ 3 bằng cách đặt $a_i = -v_i$ và $b_i = -v_i - w_i$. Khi đó mọi $b_i - a_i < 0$, nên một phương án tối ưu là sắp mọi hộp theo b_i giảm dần, tức theo $v_i + w_i$ tăng dần.

Tương tự, trong Ví dụ 1, nếu thay ràng buộc bằng $a_i b_i > 0$, hoặc $a_i > 0$, hoặc các điều kiện gần giống, sau khi xử lý một số phần tử đặc biệt, ta vẫn có thể so sánh (a_i, b_i) và (a_j, b_j) bằng cách tương tự so sánh $a_i b_j$ với $a_j b_i$, để quan hệ thỏa Tính chất 2.1, 2.2 và 2.3. Nhưng nếu không có ràng buộc nào và a_i, b_i là các số thực tùy ý, dễ dựng phản ví dụ cho thấy ba tính chất không thể đồng thời thỏa, nên không còn dùng được thuật toán trên.

Một số mô hình. Ba ví dụ ở trên tương ứng với ba kiểu phần tử, hàm và quan hệ so sánh khác nhau. Gọi một bộ gồm phần tử, hàm và quan hệ so sánh như vậy là một *mô hình*; các ví dụ trên cho ba mô hình:

1. Phần tử có dạng (a, b) với $a, b \in \mathbb{R}$ và $a, b \geq 0$,

$$F(((a_1, b_1), \dots, (a_n, b_n))) = \sum_{1 \leq i < j \leq n} a_i b_j,$$

so sánh F theo thứ tự thông thường trên số thực; cách so sánh phần tử là như Ví dụ 1.

2. Phần tử có dạng (a, b) với $a, b \in \mathbb{R}$,

$$F(((a_1, b_1), \dots, (a_n, b_n))) = -\min_j \left\{ \sum_{i=1}^j b_i - \sum_{i=1}^j a_i \right\},$$

so sánh theo số thực; cách so sánh phần tử cụ thể như Ví dụ 3.

3. Phần tử là xâu s , $F((s_1, \dots, s_n)) = s_1 + \dots + s_n$, so sánh theo thứ tự từ điển; hai phần tử s, t được so sánh bằng thứ tự từ điển của $s + t$ và $t + s$.

Các mô hình này, đặc biệt hai mô hình đầu, xuất hiện khá rộng trong thi tin học. Còn nhiều bài toán khác có thể chuyển thành các mô hình trên hoặc các mô hình khác thỏa ba tính chất; nguồn không triển khai thêm.

5.2.4 Một số ứng dụng khác của phương pháp

Dựa trên các tính chất và cách xử lý trong thuật toán trên, ta còn giải được nhiều bài toán hơn. Ví dụ xét lớp bài toán sau: cho n phần tử $x_1, \dots, x_n$ và một hàm F trên các dãy phần tử, thỏa ba tính chất trên. Chọn k phần tử, sắp chúng tùy ý thành $y_1, \dots, y_k$, và tối ưu hóa $F((y_1, \dots, y_k))$.

Theo thuật toán trên, ta có thể xây dựng một thứ tự yếu giữa các x_i ; giả sử $x_1 < x_2 < \dots < x_n$. Nếu các phần tử được chọn là $x_{s_1} < x_{s_2} < \dots < x_{s_k}$, thì dãy $(x_{s_1}, \dots, x_{s_k})$ là một nghiệm tối ưu cho tập đã chọn. Vì vậy luôn tồn tại nghiệm tối ưu là một dãy con của $(x_1, \dots, x_n)$. Tính chất này biến bài toán chọn một số phần tử theo thứ tự thành bài toán chọn một dãy con, trong nhiều trường hợp làm bài toán dễ hơn.

Ví dụ 5. Có n vật phẩm, vật phẩm thứ i có chi phí không âm c_i và giá trị w_i . Hai người chơi trò chơi sau:

1. Người thứ nhất chọn một vật phẩm và trả chi phí c_i . Người thứ nhất cũng có thể trực tiếp kết thúc trò chơi.
2. Người thứ hai có thể xóa vật phẩm này, khi đó trò chơi trở lại bước đầu nhưng chi phí người thứ nhất đã trả không được hoàn lại. Người thứ hai được làm thao tác này nhiều nhất k lần. Người thứ hai cũng có thể không thao tác, khi đó người thứ nhất nhận lợi ích w_i và trò chơi kết thúc.

Tổng lợi ích của người thứ nhất là lợi ích nhận được trừ tổng chi phí đã trả. Người thứ nhất muốn tối đa hóa, người thứ hai muốn tối thiểu hóa. Hãy tìm kết quả khi cả hai chơi tối ưu.

$$n < 1.5 \times 10^5, \quad k < 9.$$

Nếu người thứ nhất chọn không lấy vật phẩm, lợi ích là 0. Nếu có chiến lược tốt hơn, cuối cùng anh ta chắc chắn nhận lợi ích của một vật phẩm. Khi đó chiến lược của người thứ nhất có thể xem là chọn một dãy gồm $k + 1$ vật phẩm $(s_1, \dots, s_{k+1})$, lần thứ i chọn vật phẩm s_i . Với chiến lược cố định, lợi ích nhỏ nhất mà người thứ hai có thể buộc được là

$$\min_i \left\{ w_{s_i} - \sum_{j=1}^i c_{s_j} \right\}.$$

Vì vậy bài toán trở thành tối đa hóa giá trị này. Đây tương tự Ví dụ 4; từ cách làm của Ví dụ 4, một thứ tự tối ưu là sắp vật phẩm theo w_i tăng dần. Do đó chiến lược của người thứ nhất luôn có thể xem là chọn một dãy con sau khi sắp theo giá trị. Đặt $f_{i,j}$ là giá trị lớn nhất khi chọn j vật phẩm trong hậu tố bắt đầu từ i sau khi sắp; ta có chuyển tiếp

$$f_{i,j} = \max\{f_{i+1,j}, \min(w_i - c_i, f_{i+1,j-1} - c_i)\}.$$

Độ phức tạp là $O(nk)$; trong nguồn, do k là hằng nhỏ nên được ghi là tuyến tính theo n .

5.2.5 Tổng kết

Điểm xuất phát của thuật toán là phân tích sự thay đổi của giá trị phương án khi đổi chỗ hai phần tử kề nhau, từ đó suy ra tính chất mà phương án tối ưu cần thỏa. Vì vậy nó được gọi là *exchange argument*. Bản chất của thuật toán là tìm một quan hệ so sánh tạo thành thứ tự, sao cho mỗi khi đổi một cặp kề nhau mà trong thứ tự phần tử sau không lớn hơn phần tử trước, phương án sau đổi không kém đi; từ đó suy ra dãy thỏa thứ tự là tối ưu. Dùng trực tiếp điều kiện “đổi xong không kém đi” làm quan hệ so sánh có thể gặp vấn đề, nhưng bắt đầu từ quan hệ do phép đổi chỗ gợi ra vẫn là một phương án tốt.

Các ví dụ trên nhìn qua tưởng không liên quan, nhưng đều giải được bằng cùng một phương pháp, cho thấy thuật toán có phạm vi ứng dụng rộng. Đồng thời, tư tưởng phân tích tính chất cục bộ của nghiệm tối ưu để suy ra tính chất toàn cục cũng là một cách mạnh để phân tích bài toán.

5.3 Mở rộng bài toán lên cây có gốc và lời giải

5.3.1 Dẫn nhập bài toán

Phần này xét một lớp bài toán có dạng tương tự phần trước, nhưng thêm một ràng buộc kiểu thứ tự bộ phận liên quan tới cây có gốc. Với một hoán vị p , nói rằng x xuất hiện trước y khi tồn tại $i < j$ sao cho $p_i = x$ và $p_j = y$.

Cho n phần tử $x_1, \dots, x_n$, một hàm F có miền xác định là dãy các phần tử này và miền giá trị là một tập có thứ tự. Cho thêm một cây có gốc T gồm n đỉnh, gốc là 1; cha của đỉnh i ($2 \leq i \leq n$) là fa_i . Để mô tả gọn, ta nói “đỉnh i ứng với phần tử x_i ”.

Một hoán vị được gọi là thỏa ràng buộc của T nếu với mọi $2 \leq i \leq n$, fa_i xuất hiện trước i trong hoán vị. Hãy tìm giá trị nhỏ nhất của

$$F((x_{p_1}, x_{p_2}, \dots, x_{p_n}))$$

trên mọi hoán vị bậc n thỏa ràng buộc của T .

Ví dụ 6. Cho một cây có gốc gồm n đỉnh, gốc là 1. Mỗi đỉnh i có trọng số $w_i \in \{0, 1\}$. Cần sắp các đỉnh thành dãy $p_1, \dots, p_n$ sao cho với mọi đỉnh x , mọi tổ tiên khác chính nó của x đều xuất hiện trước x . Hãy tối thiểu hóa số nghịch thế của dãy $(w_{p_1}, w_{p_2}, \dots, w_{p_n})$, tức số cặp $1 \leq i < j \leq n$ thỏa $w_{p_i} > w_{p_j}$.

$$n < 2 \times 10^5.$$

Trong bài này, nghịch thế chỉ phát sinh khi $w_{p_i} = 1$ và $w_{p_j} = 0$. Nếu xem phần tử $w = 1$ là $(1, 0)$ trong mô hình thứ nhất, còn $w = 0$ là $(0, 1)$, bài toán có thể biểu diễn dưới dạng trên.

Nếu không có ràng buộc T , theo phần trước chỉ cần tồn tại một quan hệ so sánh giữa các phần tử thỏa ba tính chất là dùng được phương pháp exchange arguments. Nhưng khi thêm ràng buộc cây, dù F thỏa điều kiện đó, vẫn có thể không tồn tại thuật toán tốt theo hướng này. Xét mô hình sau: cho n số thực $w_1, \dots, w_n$, sắp chúng sao cho tổng k số đầu là lớn nhất. Định nghĩa $x < y$ theo so sánh số thực thông thường. Quan hệ này rõ ràng thỏa Tính chất 2.1 và 2.2, và cũng dễ chứng minh thỏa Tính chất 2.3.

Một phản ví dụ. Cho một cây có gốc gồm n đỉnh, mỗi đỉnh có trọng số. Cho số nguyên dương k , hãy chọn một khối liên thông kích thước k trên cây, chứa gốc, sao cho tổng trọng số trong khối là lớn nhất.

Bài toán tương đương chọn một hoán vị đỉnh thỏa ràng buộc của T và tối đa hóa tổng trọng số của k đỉnh đầu. Khi không có ràng buộc cây, nó tương đương mô hình vừa nêu. Bài toán có thể giải bằng quy hoạch động $O(n^2)$; nếu cây có dạng gốc có ba cây con và mỗi cây con là một đường, thì bài toán trở thành: cho ba dãy tùy ý a, b, c , tìm

$$\max_{i+j+\ell=k} \{a_i + b_j + c_\ell\},$$

nên khó giải dưới $O(n^2)$. Điều này cho thấy nếu muốn tiếp tục dùng tư tưởng tham lam dựa trên đổi chỗ, cần yêu cầu mạnh hơn về F và cần phân tích thêm.

5.3.2 Phân tích bài toán trên cây

Giả sử tồn tại một quan hệ $<$ thỏa Tính chất 2.1, 2.2 và 2.3. Ta có bổ đề sau.

Bổ đề 3.1. Giả sử các phần tử là $x_1, \dots, x_n$, gốc của cây là 1. Gọi v là đỉnh có phần tử nhỏ nhất trong $x_2, \dots, x_n$ và u là cha của v trên cây. Khi đó tồn tại một hoán vị tối ưu trong đó u và v kề nhau, tức tồn tại $i \in \{1, \dots, n-1\}$ sao cho $p_i = u, p_{i+1} = v$ hoặc $p_i = v, p_{i+1} = u$.

Chứng minh. Xét một hoán vị p bất kỳ thỏa ràng buộc của T . Giả sử $p_a = u$ và $p_b = v$. Do u là cha của v , ta có $a < b$. Vì v là phần tử nhỏ nhất ngoài gốc, mọi đỉnh nằm giữa u và v trong hoán vị đều không nhỏ hơn v . Dùng Tính chất 2.3 có thể lần lượt đổi v sang trái qua các phần tử này mà không làm F tăng. Hơn nữa, trong đoạn giữa u và v không thể có tổ tiên của u : nếu có một đỉnh như vậy, quan hệ tổ tiên trong cây sẽ mâu thuẫn với việc p thỏa ràng buộc của T . Do đó sau khi dịch v tới ngay sau u , hoán vị vẫn thỏa ràng buộc của T . Vì với mọi hoán vị hợp lệ đều tìm được một hoán vị hợp lệ có u, v kề nhau và F không lớn hơn, tồn tại một nghiệm tối ưu có tính chất này. ■

Theo Bổ đề 3.1, nếu chỉ xét các hoán vị hợp lệ trong đó u, v kề nhau, nghiệm tối ưu thu được vẫn là nghiệm tối ưu của bài toán gốc. Hãy hợp nhất u, v trên cây thành một đỉnh đại diện cho dãy hai phần tử (x_u, x_v) . Khi đó ta được một bài toán mới trên cây có $n-1$ đỉnh, mỗi đỉnh ứng với một dãy phần tử; chọn một hoán vị hợp lệ của các đỉnh, nối các dãy theo thứ tự hoán vị và tối thiểu hóa F trên dãy nối đó. Các dãy thu được ở bài toán mới đúng là các dãy của bài toán cũ trong đó x_u, x_v kề nhau, nên nghiệm tối ưu của bài toán mới ứng với một nghiệm tối ưu của bài toán cũ.

Bài toán mới có cùng dạng, nhưng mỗi đỉnh không còn ứng với một phần tử mà ứng với một dãy phần tử. Do đó nếu muốn tiếp tục dùng bổ đề trên, cần mở rộng quan hệ so sánh từ phần tử sang dãy phần tử.

Gọi S là tập các phần tử đã cho, và $\mathcal{T}$ là tập mọi dãy không rỗng gồm các phần tử của S . Giả sử tồn tại một quan hệ nhị nguyên $<$ trên $\mathcal{T}$ thỏa:

- **Tính chất 3.1.** Với mọi $a, b \in \mathcal{T}$, ít nhất một trong $a < b$ và $b < a$ đúng.
- **Tính chất 3.2.** Với mọi $a, b, c \in \mathcal{T}$, nếu $a < b$ và $b < c$ thì $a < c$.
- **Tính chất 3.3.** Với mọi $a, b \in \mathcal{T}$, nếu $a < b$ thì với mọi dãy phần tử s_1, s_2 ,

$$F(s_1 + a + b + s_2) \leq F(s_1 + b + a + s_2),$$

trong đó dấu $+$ là phép nối dãy.

Nếu bài toán thỏa các tính chất này, có hai thuật toán tìm một nghiệm tối ưu bằng $O(n \log n)$ lần so sánh dãy và $O(n)$ lần hợp nhất dãy. Phần sau phân tích hai thuật toán đó.

5.3.3 Hai thuật toán cho bài toán trên cây

Trong phần này giả sử các tính chất trên đều đúng. Khi đó các dãy thỏa cùng kiểu tính chất như phần tử, nên có thể thay mỗi đỉnh ban đầu ứng với một phần tử x_i bằng đỉnh ứng với dãy $s_i = (x_i)$ và chỉ xét so sánh, hợp nhất giữa các dãy.

Thuật toán 1. Tiếp tục dùng ý tưởng của Bổ đề 3.1, nhưng thay phần tử bằng dãy; bổ đề vẫn đúng. Vì vậy khi mỗi đỉnh ứng với một dãy phần tử, ta vẫn có thể hợp nhất hai đỉnh. Mỗi thao tác hợp nhất hai đỉnh trên cây và nối hai dãy tương ứng; sau $n - 1$ lần hợp nhất, cây chỉ còn một đỉnh, dãy của đỉnh đó là một nghiệm tối ưu.

Thuật toán cụ thể:

1. Lặp $n - 1$ lần.
2. Mỗi lần, tìm trong các đỉnh không phải gốc đỉnh x có dãy nhỏ nhất.
3. Nối dãy của x vào cuối dãy của cha x , rồi hợp nhất x với cha của nó trên cây.

Sau $n - 1$ thao tác, dãy ứng với gốc là một nghiệm tối ưu.

Bổ đề 3.1 bảo đảm sau mỗi lần hợp nhất, nghiệm tối ưu của bài toán mới ứng với một nghiệm tối ưu của bài toán trước, nên thuật toán đúng. Khi cài đặt, có thể dùng DSU trên cây để duy trì các đỉnh đã co, lưu thông tin của một khối liên thông tại gốc của khối đó, và dùng một cấu trúc dữ liệu hỗ trợ chèn, xóa phần tử nhỏ nhất (chẳng hạn `priority_queue` của C++) để duy trì mọi đỉnh không phải gốc. Khi đó thuật toán dùng $O(n \log n)$ lần so sánh dãy, $O(n)$ lần hợp nhất dãy; phần còn lại cũng $O(n \log n)$.

Thuật toán 2. Phân tích bài toán từ một góc nhìn khác cho ta thuật toán thứ hai. Nó dựa trên các bổ đề sau.

Bổ đề 3.2. Nối một hoán vị p thỏa quan hệ thứ tự của dãy $(v_1, \dots, v_k)$ nếu trong p , v_1 xuất hiện trước v_2 , v_2 xuất hiện trước v_3 , ..., v_{k-1} xuất hiện trước v_k .

Giả sử trong cây có gốc T , một đỉnh có các con $v_1, \dots, v_k$, và các con này đều là lá. Nếu $s_{v_1} < s_{v_2} < \dots < s_{v_k}$ thì tồn tại một hoán vị phần tử tối ưu, thỏa ràng buộc của T , đồng thời thỏa quan hệ thứ tự của dãy $(v_1, \dots, v_k)$.

Bổ đề 3.3. Giả sử một đỉnh u trong cây có hai con lá v_1, v_2 và $s_{v_1} < s_{v_2}$. Với bất kỳ hoán vị hợp lệ p , nếu $p_i = v_1, p_j = v_2$ và $j < i$, thì có thể chỉ thay đổi đoạn $p_j, p_{j+1}, \dots, p_i$ để được một hoán vị hợp lệ q trong đó v_1 xuất hiện trước v_2 và

$$F((s_{q_1} + \dots + s_{q_n})) \leq F((s_{p_1} + \dots + s_{p_n})).$$

Chứng minh. Vì $s_{v_1} < s_{v_2}$, từ Tính chất 3.1 và 3.2, khi so sánh s_{v_1} với các dãy trong đoạn giữa v_2 và v_1 , một trong hai hướng dịch chuyển tương ứng sẽ không làm F tăng theo Tính chất 3.3. Do v_1, v_2 là lá cùng cha, đưa v_1 lên trước v_2 trong đoạn này không phá vỡ ràng buộc cây. Trường hợp đối xứng xử lý tương tự. ■

Từ Bổ đề 3.3, xét hàm $g(v_1, \dots, v_k)$ là giá trị nhỏ nhất của F trên mọi hoán vị hợp lệ thỏa quan hệ thứ tự $(v_1, \dots, v_k)$. Mỗi khi trong dãy thứ tự có hai phần tử kề nhau v_i, v_{i+1} với $s_{v_i} < s_{v_{i+1}}$, đổi chúng theo chiều đúng không làm g tăng. Vì < thỏa Tính chất 3.1 và 3.2, áp dụng exchange arguments của phần trước cho hàm g suy ra dãy con lá được sắp theo thứ tự không giảm là tối ưu. Do đó Bổ đề 3.2 đúng.

Bổ đề 3.4. Giả sử trong cây T , mọi con $v_1, \dots, v_k$ của đỉnh u đều là lá và $s_{v_1} < s_{v_2} < \dots < s_{v_k}$. Nếu $s_u < s_{v_i}$, thì với mọi hoán vị hợp lệ thứ tự $(v_1, \dots, v_k)$, tồn tại một hoán vị hợp lệ khác vẫn thỏa thứ tự đó, có giá trị F không lớn hơn, và trong đó u được đặt sát trước v_i theo cách tương ứng.

Chứng minh. Ý tưởng giống Bổ đề 3.3. Nếu trong một hoán vị u đứng trước v_i , dùng Tính chất 3.3 để đổi u qua các dây nằm giữa mà không làm F tăng. Vì hoán vị ban đầu đã thỏa thứ tự của các lá, việc dịch chuyển này không phá ràng buộc cây và không phá thứ tự $(v_1, \dots, v_k)$. Trường hợp đối xứng xử lý tương tự. ■

Bổ đề 3.5. Với mỗi đỉnh u của cây T , có thể chia các phần tử trong cây con subtree_u thành các dãy $c_1, \dots, c_k$ sao cho:

1. $c_1 \leq c_2 \leq \dots \leq c_k$;
2. nếu thay cây con của u bằng bài toán mới trong đó u có thêm k con lá $v_1, \dots, v_k$ lần lượt ứng với các dãy $c_1, \dots, c_k$, thì mọi nghiệm tối ưu của bài toán mới thỏa thứ tự $(v_1, \dots, v_k)$ đều ứng với một nghiệm tối ưu của bài toán ban đầu.

Chứng minh. Quy nạp trên cây. Với lá u , lấy $k = 1$ và $c_1 = (x_u)$ là hiển nhiên. Với đỉnh không phải lá, giả sử các con của u đều đã có các dãy thỏa bổ đề. Trộn các dãy của các con theo thứ tự không giảm, đồng thời giữ thứ tự nội bộ của từng con. Theo Bổ đề 3.2, tồn tại nghiệm tối ưu thỏa thứ tự đã trộn. Nếu dãy đầu tiên trong thứ tự đó nhỏ hơn dãy hiện tại của u , dùng Bổ đề 3.4 để cho hai dãy tương ứng kề nhau rồi hợp nhất chúng. Lặp thao tác này cho đến khi dãy đầu tiên không nhỏ hơn dãy của u , hoặc mọi lá đã bị hợp nhất. Khi đó các dãy còn lại, cùng dãy mới của u , thỏa hai điều kiện của bổ đề. ■

Từ cách xây dựng trong chứng minh Bổ đề 3.5, ta có thuật toán thứ hai. Với mỗi đỉnh u , tính dãy S_u gồm các dãy c_i của Bổ đề 3.5 theo thứ tự. Với một đỉnh u , thực hiện:

1. Trước hết lấy tất cả S_v của các con v của u , rồi trộn chúng theo quan hệ so sánh dãy để được S_u .
2. Đặt $C = (x_u)$.
3. Lặp: lấy phần tử đầu c của S_u . Nếu S_u rỗng hoặc $c \not\leq C$ thì dừng; nếu không, xóa c khỏi S_u và nối c vào cuối C .
4. Sau khi dừng, thêm C vào đầu S_u .

Duyệt DFS từ dưới lên để tính mọi S_u . Sau khi có S_1 của gốc, nối mọi dãy trong S_1 theo thứ tự là một nghiệm tối ưu. Thuật toán cần $O(n)$ lần trộn hai danh sách S , $O(n)$ lần hỏi/xóa phần tử đầu của một S và $O(n)$ lần hợp nhất dãy. Nếu dùng cây cân bằng hoặc cấu trúc tương tự để duy trì S , thuật toán dùng $O(n \log n)$ lần so sánh dãy, cùng bậc với thuật toán 1.

5.3.4 Phân tích các mô hình cụ thể

Với các bài toán mà tồn tại quan hệ so sánh thỏa Tính chất 3.1, 3.2 và 3.3, hai thuật toán trên đều giải được bằng $O(n \log n)$ lần so sánh dãy và $O(n)$ lần hợp nhất dãy. Sau đây xét ba mô hình đã nêu ở phần trước.

Mô hình thứ nhất. Trong mô hình thứ nhất, phần tử có dạng (a, b) với $a, b \in \mathbb{R}$, $a, b \geq 0$, và

$$F(((a_1, b_1), \dots, (a_n, b_n))) = \sum_{1 \leq i < j \leq n} a_i b_j.$$

Với hai dãy $c_1 = ((a_1, b_1), \dots, (a_n, b_n))$ và $c_2 = ((a'_1, b'_1), \dots, (a'_m, b'_m))$, ta có

$$F(c_1 + c_2) - F(c_2 + c_1) = \left(\sum_{i=1}^n a_i \right) \left(\sum_{i=1}^m b'_i \right) - \left(\sum_{i=1}^m a'_i \right) \left(\sum_{i=1}^n b_i \right).$$

Vì vậy một dãy có thể xem như phần tử tương đương

$$\left(\sum_i a_i, \sum_i b_i \right).$$

Dùng cách so sánh của mô hình thứ nhất trên hai phần tử tương đương này để so sánh hai dãy. Dễ thấy quan hệ đó thỏa Tính chất 3.1, 3.2 và 3.3.

Mô hình thứ hai. Trong mô hình thứ hai, phần tử có dạng (a_i, b_i) với $a_i, b_i \in \mathbb{R}$, và

$$F(((a_1, b_1), \dots, (a_n, b_n))) = -\min_j \left\{ \sum_{i=1}^j b_i - \sum_{i=1}^j a_i \right\}.$$

Với hai phần tử liên tiếp $(a_1, b_1), (a_2, b_2)$, có thể hợp nhất chúng thành một phần tử tương đương

$$(-\min\{-a_1, b_1 - a_1 - a_2\}, b_1 + b_2 - a_1 - a_2 - \min\{-a_1, b_1 - a_1 - a_2\}),$$

mà giá trị F đối với mọi ngữ cảnh bên ngoài không đổi. Quy nạp cho thấy mọi dãy đều có thể hợp nhất thành một phần tử tương đương khi tính F . Do đó trong Tính chất 3.3 có thể thay các dãy ở hai vế bằng phần tử tương đương của chúng, rồi dùng quan hệ so sánh phần tử của mô hình thứ hai. Vì quan hệ phần tử thỏa Tính chất 2.1, 2.2 và 2.3, quan hệ dãy thu được thỏa Tính chất 3.1, 3.2 và 3.3.

Mô hình thứ ba. Với mô hình này, nối một số xâu trong dãy vẫn thu được một xâu, nên các tính chất trên hiển nhiên đúng. Chi tiết về cài đặt nối và so sánh xâu phức tạp hơn, nguồn không triển khai.

Ngược lại, với phần ví dụ ở đầu phần này, không tồn tại quan hệ < thỏa điều kiện trên. Chẳng hạn lấy $k = 2$, khi đó có thể có $F((3, 3)) > F((4, 1))$ nhưng $F((4, 3, 2)) < F((4, 4, 1))$. Theo Tính chất 3.1, hai dãy $(3, 3)$ và $(4, 1)$ phải có một dãy không lớn hơn dãy kia, nhưng cả hai khả năng đều mâu thuẫn với Tính chất 3.3. Vì vậy điều kiện nói trên chắc chắn không thỏa, và bài toán đó không giải được bằng thuật toán này.

5.3.5 Tính chất và ứng dụng của thuật toán

Hai mô hình đầu có một tính chất tốt: khi hợp nhất hai dãy c_1, c_2 , chỉ cần biết $O(1)$ thông tin của mỗi dãy là có thể tính thông tin của dãy hợp nhất và $F(c_1 + c_2)$. Với mô hình thứ nhất, chỉ cần biết $\sum a_i$, $\sum b_i$ và $F(c)$; với mô hình thứ hai, chỉ cần biết phần tử tương đương (a, b) . Vì thế đối với các bài toán thuộc hai mô hình này, hai thuật toán trên có thể tính đáp án trong $O(n \log n)$. Mô hình thứ nhất ứng với Ví dụ 6; bài toán sau ứng với mô hình thứ hai.

Ví dụ 7. Cho một cây n đỉnh. Mỗi đỉnh trừ đỉnh 1 có trọng số v_i . Một người ban đầu ở đỉnh 1 và có một giá trị hp , ban đầu bằng 0. Người này có thể di chuyển dọc theo các cạnh của cây; khi lần đầu tới đỉnh i , hp được cộng thêm v_i . Nếu $hp < 0$ thì người đó thất bại. Hỏi có thể tới được một đỉnh cho trước t mà không thất bại hay không.

$$n < 10^6, \quad |v_i| < 10^6.$$

Thêm một lá nối với 1 có trọng số rất lớn. Nếu đi được tới t , sau khi đi tới lá này ta chắc chắn có thể đi qua mọi đỉnh còn lại; vì vậy bài toán trở thành có thể đi qua mọi đỉnh mà không thất bại hay không.

Nếu ghi lại dãy đỉnh theo lần đầu tiên đi qua, ta được một hoán vị thỏa ràng buộc của cây T , và quá trình thay đổi của hp chính là đi qua các phần tử theo hoán vị đó.

Bài toán trở thành tìm một hoán vị hợp lệ sao cho mọi thời điểm $hp \geq 0$. Tương đương, tối đa hóa giá trị nhỏ nhất của hp trong quá trình. Đây là mô hình thứ hai: phần tử $x \geq 0$ ứng với $(0, x)$, còn $x < 0$ ứng với $(-x, 0)$. Dùng thuật toán trên cho độ phức tạp $O(n \log n)$.

Tìm một nghiệm tối ưu. Như đã nói, trong hai mô hình đầu chỉ cần duy trì một phần thông tin của dãy để so sánh và tính đáp án. Nếu muốn khôi phục phương án, trong cả hai thuật toán đều có n lần hợp nhất dãy, mỗi lần dạng “nối dãy t vào cuối dãy s ”. Ghi lại các thao tác hợp nhất đó; sau khi thuật toán kết thúc, có thể khôi phục dãy phương án trong $O(n)$.

Tìm đáp án cho bài toán con của mỗi cây con. Xét dạng bài toán sau: với mỗi đỉnh x , bài toán con của cây con gốc x là lấy các đỉnh trong cây con của x làm cây mới T' , lấy x làm gốc, và giữ nguyên dạng bài toán. Hãy tìm đáp án của bài toán con với mọi đỉnh x .

Ví dụ 8. Cho một cây có gốc gồm n đỉnh, gốc là 1, mỗi đỉnh có trọng số v_i . Ta có một số quân cờ và được thực hiện hai loại thao tác:

1. Chọn một đỉnh đang có quân cờ và thu lại toàn bộ quân cờ trên đỉnh đó.
2. Chọn một đỉnh x chưa có quân cờ, đồng thời mọi con của x đều đang có quân cờ, rồi đặt v_x quân cờ lên x .

Với mỗi đỉnh, hỏi nếu cần đặt quân cờ lên đỉnh đó thì ban đầu cần ít nhất bao nhiêu quân cờ.

$$n < 2 \times 10^5, \quad 1 < v_i < 10^9.$$

Phân tích bài toán cho hai tính chất:

1. Nếu cần đặt quân cờ lên đỉnh i , quá trình chỉ đặt quân cờ trong cây con của i và đặt đúng một lần lên mỗi đỉnh trong cây con đó.
2. Tồn tại một cách thao tác tối ưu sao cho sau khi đặt quân cờ lên một đỉnh x , thao tác tiếp theo là thu lại toàn bộ quân cờ ở các con của x .

Gọi $p_1, \dots, p_k$ là dãy các đỉnh được đặt quân cờ. Với bài toán của đỉnh i , tồn tại nghiệm tối ưu mà dãy này là một hoán vị của cây con i . Đặt s_i là tổng v của các con của i ; sau khi đặt quân cờ lên p_j , số quân đang dùng là

$$\sum_{\ell=1}^j v_{p_\ell} - \sum_{\ell < j} s_{p_\ell}.$$

Vì vậy bài toán tương đương tối thiểu hóa giá trị lớn nhất của biểu thức này. Sau khi đảo thứ tự dãy, dạng hàm thu được tương tự mô hình thứ hai, và hướng ràng buộc trên cây cũng trở về dạng cha xuất hiện trước con. Do đó bài toán được chuyển thành tìm đáp án bài toán con của mỗi cây con.

Với thuật toán 2, Bổ đề 3.5 cho ngay tính chất: nếu S_x là dãy các dãy được xây dựng cho đỉnh x , thì nối các phần tử trong S_x theo thứ tự là một nghiệm tối ưu của bài toán con gốc x . Do đó, nếu dùng cây cân bằng hoặc cấu trúc tương tự để duy trì S_x , đồng thời có thể duy trì nghiệm tối ưu của cây con gốc x ; độ phức tạp cùng bậc với thuật toán 2.

Thuật toán 1 cũng giải được yêu cầu này, nhưng cần phân tích thêm.

Bổ đề 3.6. Với một đỉnh u bất kỳ trong cây có gốc T , xét một tập con S của các con của u . Gọi $U(u, S)$ là tập gồm u và mọi đỉnh thuộc các cây con của các đỉnh trong S . Với bất kỳ hoán vị tối ưu p thu được bằng thuật toán 1 trên T , tồn tại một hoán vị q thu được bằng thuật toán 1 trên cây cảm sinh bởi $U(u, S)$, lấy u làm gốc, sao cho q giữ thứ tự tương ứng của các đỉnh trong p khi chỉ xét $U(u, S)$.

Chứng minh. Quy nạp theo số đỉnh của T . Xét thao tác hợp nhất đầu tiên trong thuật toán 1. Nếu thao tác không liên quan tới $U(u, S)$, cấu trúc phần này không đổi và dùng giả thiết quy nạp. Nếu thao tác chỉ ảnh hưởng tới gốc u của phần đang xét, quá trình thuật toán 1 trong phần đó không phụ thuộc vào dãy đang nằm tại gốc, nên kết luận vẫn đúng. Nếu thao tác hợp nhất hai đỉnh đều thuộc $U(u, S)$, thì đỉnh được hợp nhất cũng là một đỉnh có dãy nhỏ nhất trong bài toán cảm sinh bởi $U(u, S)$, nên có thể chọn cùng thao tác đầu tiên trong thuật toán 1 của bài toán con; sau đó áp dụng quy nạp. ■

Từ Bổ đề 3.6, với mọi hoán vị tối ưu p do thuật toán 1 sinh ra và mọi đỉnh x , nếu trong p chỉ giữ các đỉnh thuộc cây con của x , ta thu được một nghiệm tối ưu của bài toán con cây con x theo thuật toán 1. Vì vậy có thể dùng cấu trúc dữ liệu hỗ trợ hợp nhất, duyệt DFS từ dưới lên để duy trì một nghiệm tối ưu cho mỗi đỉnh. Một cách cài đặt là dùng cây đoạn động và hợp nhất cây đoạn; độ phức tạp cùng bậc với thuật toán 1.

Bổ đề 3.5 cũng cho thấy hoán vị tối ưu do thuật toán 2 sinh ra có tính chất tương tự. Tuy nhiên nếu lấy một hoán vị tối ưu bất kỳ không do hai thuật toán trên sinh ra, để dựng phản ví dụ cho tính chất này; vì vậy kết luận chỉ được bảo đảm với nghiệm do hai thuật toán trong bài tạo ra.

Một số ứng dụng khác. Cuối cùng xét một bài toán liên quan tới cách làm của thuật toán 2.

Ví dụ 9. Có n đồng đá xếp thành một hàng, đồng thứ i có a_i viên đá. Cần hợp nhất mọi đồng thành một đồng; mỗi lần chọn hai đồng kề nhau và hợp nhất chúng, chi phí là tổng số đá của hai đồng. Có thêm ràng buộc: trong hai đồng được chọn ở mỗi lần, phải có đồng chứa đồng đá ban đầu thứ x . Hãy tìm chi phí nhỏ nhất.

$$n < 2 \times 10^5, \quad 1 < a_i < 10^9.$$

Với một vị trí x cố định, quá trình có thể xem là mỗi lần nhập một đồng vào đồng thứ x . Nếu lần thứ $i - 1$ nhập đồng p_i ($p_1 = x$), ta thu được một hoán vị. Ràng buộc của hoán vị tương đương với ràng buộc của cây là đường $1 - 2 - \dots - n$ lấy x làm gốc. Tổng chi phí chỉ phụ thuộc vào hoán vị; sau khi trừ một hằng số, nó có dạng

$$\sum_{1 \leq i < j \leq n} a_{p_j},$$

tức mô hình thứ nhất với phần tử $(a_i, 1)$. Vì vậy một bài toán cho x cố định giải được trong $O(n \log n)$.

Nếu dùng thuật toán 2, có thể xử lý mọi x liên tiếp. Gọi L_x là dãy các dãy nhận được từ đoạn $1 - 2 - \dots - (x - 1)$ khi lấy $x - 1$ làm gốc, và R_x tương tự cho đoạn $(x + 1) - \dots - n$ khi lấy $x + 1$ làm gốc. Theo Bổ đề 3.2 và 3.4, nếu trộn L_x và R_x theo quan hệ so sánh thành S_x , rồi nối mọi dãy trong S_x và thêm x vào đầu, ta được một nghiệm tối ưu cho gốc x .

Khi x tăng dần từ 1 tới n , các thay đổi của L_x và R_x chỉ gồm $O(n)$ lần chèn hoặc xóa ở đầu và truy vấn dãy đầu. Duy trì kết quả trộn hiện tại bằng cây cân bằng: khi xóa dãy đầu thì xóa trực tiếp; khi chèn một dãy vào đầu L hoặc R , nhờ cách xây dựng, dãy mới không lớn hơn mọi dãy hiện có ở phía đó, nên có thể chèn vào vị trí thích hợp trong kết quả trộn mà vẫn giữ cả thứ tự toàn cục lẫn thứ tự nội bộ. Đồng thời duy trì giá trị đáp án trên cây cân bằng. Vì so sánh, nối dãy và hợp nhất thông tin đáp án đều duy trì được trong $O(1)$ cho mô hình này, tổng độ phức tạp là $O(n \log n)$.

Trong bài này chỉ cần đáp án. Nếu hai dãy a, b thỏa $a < b$ và $b < a$, đổi chỗ hai dãy này không làm thay đổi F , nên khi chèn có thể sắp các dãy tương đương theo thứ tự tùy ý mà đáp án vẫn tối ưu. Tuy nhiên

nếu chèn tùy ý, phương án đang duy trì có thể không còn thỏa ràng buộc cây, dù giá trị đáp án vẫn đúng.

5.3.6 Tổng kết

Lớp bài toán này có cấu trúc tương tự phần trước, nhưng thêm ràng buộc lên hoán vị có dạng cây có gốc. Về điều kiện cần thỏa và cách giải, hai lớp bài toán có liên hệ rất mạnh. Tuy nhiên các ví dụ cho thấy chỉ thỏa yêu cầu của phần trước không đủ để giải bài toán có ràng buộc cây. Bài viết tăng cường điều kiện: thay vì chỉ so sánh phần tử, cần so sánh cả dãy phần tử. Trên cơ sở đó, bài viết đưa ra hai thuật toán từ hai góc nhìn khác nhau. Thuật toán thứ nhất bắt đầu từ tính chất cục bộ của một đỉnh, liên tục hợp nhất đỉnh có dãy nhỏ nhất với cha của nó để thu nhỏ bài toán. Thuật toán thứ hai trước hết giải các cây con của con, rồi dùng các tính chất đã chứng minh để hợp nhất lời giải cây con thành lời giải của cây con hiện tại. Hai thuật toán có cùng độ phức tạp. Từ quá trình của hai thuật toán có thể suy ra thêm nhiều ứng dụng; bài viết chỉ phân tích một số ứng dụng mà tác giả biết.

5.4 Lời kết và triển vọng

Trong thi tin học, tư tưởng tham lam có phạm vi ứng dụng rất rộng và còn nhiều vấn đề đáng nghiên cứu sâu. Các thuật toán liên quan trong bài viết đã nhiều lần xuất hiện trong thi tin học, nhưng việc phân tích chi tiết chúng không đơn giản. Do trình độ tác giả có hạn, vẫn còn nhiều câu hỏi liên quan tới lớp thuật toán này chưa có câu trả lời tốt, chẳng hạn:

1. Hai phần trong bài đều đặt ra ba tính chất cho quan hệ so sánh. Có tồn tại các tính chất yếu hơn hoặc đơn giản hơn, nhưng vẫn bảo đảm thuật toán trong bài thu được nghiệm tối ưu hay không?
2. Ngoài ba mô hình và các biến thể đã nêu, có tồn tại nhiều mô hình khác thỏa các tính chất trên hay không? Có mô tả bản chất hơn cho các mô hình đó hay không?

Tác giả hy vọng bài viết có thể gợi mở thêm nghiên cứu, để các bạn đọc quan tâm tìm được nhiều cách làm và ứng dụng thú vị hơn cho lớp bài toán này.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn huấn luyện viên đội tuyển quốc gia Yang Ziliang đã hướng dẫn; cảm ơn cha mẹ đã nuôi dưỡng và giáo dục; cảm ơn nhà trường đã bồi dưỡng; cảm ơn huấn luyện viên Li Xiang và các huấn luyện viên của Trường Ngoại ngữ Thành Đô đã chỉ bảo nhiều mặt; cảm ơn bạn Dai Jiangqi đã trao đổi và gợi mở; cảm ơn Liu Yiping, Jin Tian, Song Jiaying và nhiều bạn khác đã đọc kiểm tra bản thảo; cảm ơn anh Wen Kairui đã động viên và giúp đỡ tác giả trong quá trình học thi tin học.

Tài liệu tham khảo

1. Wikipedia contributors. *Weak Orderings*. https://en.wikipedia.org/wiki/Weak_ordering.
2. Errichto. *Lecture #3 – Exchange arguments (sorting with dp)*. <https://codeforces.com/blog/entry/63533>.
3. AtCoder Grand Contest 023 Editorial. <https://atcoder.jp/contests/agc023/editorial>.
4. CERC 2013: Presentation of solutions. <https://cerc.tcs.uj.edu.pl/2013/data/cerc2013solutions.pdf>.

6 Bàn về các thuật toán liên quan tới đồ thị phẳng

Tang Shaoxuan, Trường Trung học số 1 Pinyi, tỉnh Sơn Đông

Tóm tắt

Đồ thị phẳng là một lớp đồ thị đặc biệt và quan trọng trong lý thuyết đồ thị. Bài viết này chủ yếu giới thiệu các thuật toán liên quan tới đồ thị phẳng. Trước hết bài viết thảo luận cách tìm đồ thị đối ngẫu, sau đó đưa ra hai thuật toán kiểm tra tính phẳng của đồ thị. Tiếp theo, thông qua tô màu đỉnh và đếm ghép hoàn hảo, bài viết trình bày các tính chất đặc thù của đồ thị phẳng. Trong khi giới thiệu thuật toán, bài viết xen kẽ các bài ví dụ và chỉ ra cách ứng dụng những thuật toán đó trong bài cụ thể.

6.1 Mở đầu

Đồ thị phẳng được đưa vào thi tin học khá sớm và đã xuất hiện nhiều lần trong các kỳ thi lớn. Tuy vậy, kiến thức và thuật toán liên quan tới đồ thị phẳng không thật sự phổ biến trong cộng đồng thí sinh; ấn tượng của phần lớn thí sinh về đồ thị phẳng thường chỉ dừng lại ở đồ thị đối ngẫu. Bài viết hy vọng đưa thêm nhiều kiến thức về đồ thị phẳng vào tầm nhìn của bạn đọc.

Mục 2 giới thiệu định nghĩa và các tính chất cơ bản của đồ thị phẳng. Mục 3 giới thiệu tính chất và cách tìm đồ thị đối ngẫu. Mục 4 trình bày hai thuật toán kiểm tra tính phẳng: thuật toán DMP và thuật toán Tarjan. Mục 5 giới thiệu tô màu đỉnh trên đồ thị phẳng, chủ yếu nêu tính chất tô màu của đồ thị tam giác hóa. Mục 6 giới thiệu thuật toán FKT để giải bài toán đếm số ghép hoàn hảo của đồ thị phẳng.

6.2 Kiến thức cơ sở

6.2.1 Định nghĩa

Định nghĩa 2.1 (Đồ thị phẳng). Nếu đồ thị G có thể được vẽ trên mặt phẳng sao cho hai cạnh khác nhau chỉ giao nhau tại đầu mút, thì G được gọi là đồ thị phẳng. Một hình vẽ không có cạnh cắt nhau như vậy được gọi là biểu diễn phẳng hoặc nhúng phẳng của G .

Định nghĩa 2.2 (Mặt). Một nhúng phẳng của đồ thị chia toàn bộ mặt phẳng thành một số miền không liên thông với nhau; mỗi miền gọi là một *mặt*. Miền không bị chặn gọi là mặt ngoài, các miền bị chặn gọi là mặt trong. Tập cạnh bao quanh một mặt tạo thành biên của mặt đó.

Thật ra mặt ngoài và mặt trong không khác nhau về bản chất. Ta luôn có thể dùng phép chiếu cầu để vẽ đồ thị phẳng lên mặt cầu; khi đó mọi mặt có vai trò như nhau.

Từ thảo luận trên, theo chiều ngược lại ta cũng biết các đỉnh và cạnh của một đa diện đều tạo thành một đồ thị phẳng, còn các mặt của đa diện chính là các mặt trên đồ thị.

Định nghĩa 2.3 (Đồ thị đối ngẫu). Cho một nhúng phẳng G của đồ thị phẳng. Định nghĩa đồ thị đối ngẫu G^* của nó như sau:

1. với mỗi mặt của G , lấy một điểm làm đỉnh của G^* ;
2. với mỗi cạnh e của G , đặt một cạnh e^* của G^* nối hai đỉnh ứng với hai mặt kề với e , và cạnh e^* cắt e trên mặt phẳng.

Chú ý rằng dù nhúng phẳng G có là đồ thị đơn hay không, đồ thị đối ngẫu G^* vẫn có thể có khuyên và cạnh song song. Cụ thể, nếu G có cầu e , thì trong G^* sẽ có khuyên e^* ; nếu biên chung của hai mặt trong G gồm nhiều cạnh, thì giữa hai đỉnh tương ứng trong G^* sẽ có nhiều cạnh song song.

Dễ thấy với một nhúng phẳng liên thông G , đối ngẫu của đối ngẫu lại là chính nó, tức $(G^*)^* = G$.

6.2.2 Tính chất

Định lý 2.1 (Công thức Euler). Với một nhúng phẳng liên thông G , ký hiệu số đỉnh, số cạnh và số mặt lần lượt là V, E, F . Khi đó

$$V - E + F = 2.$$

Chứng minh. Lấy một cây khung T bất kỳ của đồ thị liên thông G . Lần lượt xóa từng cạnh không thuộc cây; mỗi lần xóa làm số cạnh E và số mặt F cùng giảm 1, nên $V - E + F$ không đổi trong toàn bộ quá trình. Sau khi xóa xong, phần còn lại là cây khung T , hiển nhiên thỏa $V = E + 1$ và $F = 1$, vì vậy $V - E + F = 2$. ■

Chú ý rằng nếu đồ thị phẳng G gồm hai thành phần liên thông khác nhau G_1, G_2 , thì $V = V_1 + V_2$, $E = E_1 + E_2$ và $F = F_1 + F_2 - 1$. Do đó trong trường hợp này $V - E + F = 3$. Tổng quát hơn, nếu C là số thành phần liên thông, thì

$$V - E + F = C + 1.$$

Bổ đề 2.1. Với một nhúng phẳng liên thông đơn G , nếu $V > 3$ thì $E < 3V - 6$.

Chứng minh. Nếu $E < 2$ thì điều kiện hiển nhiên đúng. Ngược lại, vì G là đồ thị đơn nên biên của mỗi mặt có ít nhất ba cạnh; mỗi cạnh lại kề đúng hai mặt. Do đó $3F \leq 2E$. Thay vào công thức Euler ta thu được kết luận. ■

6.3 Đồ thị đối ngẫu

Có thể thấy đồ thị đối ngẫu G^* của một nhúng phẳng G thật ra cung cấp nhiều thông tin hữu ích về G . Kết quả thường dùng hơn cả là kết quả sau.

Định nghĩa 3.1. Chia tập đỉnh V của đồ thị vô hướng $G(V, E)$ thành hai tập không rỗng V_1, V_2 . Khi đó các cạnh có một đầu mút thuộc V_1 và đầu mút còn lại thuộc V_2 tạo thành một *tập cắt* của G . Nếu mọi tập con thực sự của một tập cắt đều không còn là tập cắt, thì tập cắt đó gọi là *tập cắt cực tiểu*.

Bổ đề 3.1. Mỗi tập cắt cực tiểu của nhúng phẳng liên thông G tương ứng một-một với một chu trình đơn của đồ thị đối ngẫu G^* . Đặc biệt, cắt nhỏ nhất của G tương ứng với chu trình ngắn nhất của G^* .

Nếu hai đỉnh s, t nằm trên biên của cùng một mặt f trong nhúng phẳng G , ta có thể nối thêm cạnh $e = (s, t)$ để chia f thành hai mặt f_1, f_2 . Khi đó chạy chu trình ngắn nhất chứa cạnh e^* trên đồ thị đối ngẫu, tức bỏ e^* rồi tìm đường đi ngắn nhất giữa f_1 và f_2 , sẽ tính được cắt nhỏ nhất giữa s và t . Cách này có ưu thế đáng kể về độ phức tạp thời gian so với dùng luồng cực đại để tìm cắt nhỏ nhất.

Ví dụ 1 (Liên thông sau khi xóa cạnh trên lưới). Cho một đồ thị lưới $n \times m$ và q thao tác. Mỗi thao tác thuộc một trong hai dạng:

1. xóa cạnh giữa hai đỉnh u, v ; bảo đảm trước thao tác này còn tồn tại cạnh giữa u và v ;

2. hỏi hai đỉnh u, v có liên thông hay không.

Ràng buộc trong bản quét đọc được là $1 \leq nm, q \leq 10^5$.

Lời giải. Nếu dùng trực tiếp thuật toán liên thông động trên đồ thị vô hướng, độ phức tạp là $O((nm + q) \log^2 nm)$ và không qua được bài này.

Xét một thời điểm mà việc xóa cạnh làm thay đổi tính liên thông. Khi đó tồn tại một tập đỉnh S sao cho mọi cạnh trong tập cắt do S và $V \setminus S$ sinh ra đều đã bị xóa. Theo Bổ đề 3.1, lúc này trên đồ thị đối ngẫu mới hình thành ít nhất một chu trình gồm các cạnh đã xóa.

Đồ thị đối ngẫu của đồ thị lưới rất dễ tìm. Dùng DSU để duy trì tính liên thông trên đồ thị đối ngẫu; giữa hai đỉnh của đồ thị đối ngẫu có cạnh e^* khi và chỉ khi cạnh tương ứng e trong đồ thị gốc đã bị xóa.

Mỗi khi xóa cạnh $e = (u, v)$, ta thêm cạnh đối ngẫu e^* vào đồ thị đối ngẫu. Nếu cạnh này tạo thành chu trình, nghĩa là việc xóa e trong đồ thị gốc đã tách ra hai thành phần liên thông.

Khi đó bắt đầu BFS đồng thời từ u và v , mỗi bên lần lượt mở rộng một đỉnh cho tới khi duyệt hết một thành phần liên thông. Giả sử hai thành phần được tách ra là V_1, V_2 , thì bước này tốn $O(\min(|V_1|, |V_2|))$. Tổng độ phức tạp của các bước như vậy là $O(nm \log nm)$.

Tô thành phần vừa duyệt xong bằng một màu mới; khi trả lời truy vấn, chỉ cần so sánh màu của hai đỉnh. Độ phức tạp thời gian là $O(q + nm \log nm)$.

Bài này còn có lời giải $O(q + nm)$; xem tài liệu tham khảo [2].

6.3.1 Cách tìm đồ thị đối ngẫu

Cho một nhúng phẳng liên thông G có n đỉnh. Cụ thể, ta biết vị trí của mỗi đỉnh trên mặt phẳng, và mỗi cạnh là đoạn thẳng nối hai đỉnh. Cần tìm đồ thị đối ngẫu G^* của nó.

Tách mỗi cạnh vô hướng (u, v) thành hai cạnh có hướng $u \rightarrow v$ và $v \rightarrow u$, rồi sắp xếp các cạnh đi ra từ mỗi đỉnh theo góc cực.

Xuất phát từ một cạnh $e : u \rightarrow v$. Mỗi lần tìm cạnh đầu tiên gặp được khi xoay cạnh e quanh tâm v theo chiều kim đồng hồ, gọi cạnh đó là e' , rồi tiếp tục từ e' cho tới khi tạo thành một vòng. Vòng này chính là một mặt f của G . Với mặt trong, các cạnh trên vòng đều đi ngược chiều kim đồng hồ; với mặt ngoài, các cạnh đều đi theo chiều kim đồng hồ.

Sau khi tìm được mọi mặt, với mỗi cạnh $e = (u, v)$ của G , nối hai đỉnh trong đồ thị đối ngẫu ứng với hai mặt nằm hai bên e .

Độ phức tạp thời gian là $O(n \log n)$, chủ yếu nằm ở bước sắp xếp cạnh theo góc cực.

6.3.2 Định vị điểm

Cho một nhúng phẳng liên thông G có n đỉnh, và m điểm trên mặt phẳng. Cần nhanh chóng xác định mỗi điểm nằm trong mặt nào của G .

Nếu với mỗi mặt đều chạy thuật toán kiểm tra điểm nằm trong đa giác, độ phức tạp sẽ là $O(nm)$.

Trước hết chạy thuật toán tìm đồ thị đối ngẫu để biết hai mặt nằm hai bên của mỗi cạnh. Sau đó quét theo hoành độ. Tại mỗi thời điểm $x = t$, duy trì thứ tự tương đối của các cạnh bị đường thẳng $x = t$ cắt qua. Các thao tác gồm chèn một cạnh, xóa một cạnh, và truy vấn mặt chứa điểm hiện tại. Ta có thể dùng cây cân bằng để duy trì; khi truy vấn chỉ cần tìm hai cạnh gần nhất phía trên và phía dưới điểm đó, từ đó xác định được mặt chứa điểm.

Độ phức tạp thời gian là $O((n + m) \log n)$.

Khi cài đặt, có thể dùng set để duy trì thứ tự tương đối của các cạnh; trong hàm so sánh vị trí của hai cạnh tại thời điểm hiện tại. Vì hai cạnh không cắt nhau ngoài đầu mút, thứ tự tương đối của chúng không thay đổi nên cách này không gây lỗi.

Ví dụ 2 (WC2013, đồ thị phẳng). Cho một nhúng phẳng liên thông G có n đỉnh, cạnh có trọng số. Có q truy vấn; mỗi truy vấn cho hai điểm x, y trên mặt phẳng và hỏi trong mọi đường đi từ x tới y , giá trị nhỏ nhất có thể của trọng số cạnh lớn nhất bị đi qua là bao nhiêu. Quá trình đi không được đi qua mặt ngoài.

$$1 \leq n, q \leq 10^5, \quad \text{trọng số cạnh} \in (0, 10^7].$$

Lời giải. Trước hết dùng thuật toán trên để tìm đồ thị đối ngẫu của G , đồng thời xác định mặt chứa từng điểm trong truy vấn.

Đề bài yêu cầu không đi qua mặt ngoài. Xét cách xác định một mặt f có phải mặt ngoài hay không. Một cách đơn giản là dùng tích có hướng theo các cạnh có hướng bao quanh f để tính diện tích của f . Vì chỉ ở mặt ngoài các cạnh có hướng mới đi theo chiều kim đồng hồ, nếu diện tích tính ra là số âm thì f là mặt ngoài.

Xóa đỉnh ứng với mặt ngoài trong đồ thị đối ngẫu G^* . Phần còn lại trở thành bài toán kinh điển trên đồ thị: nhiều lần hỏi trên G' đường đi giữa hai đỉnh sao cho cạnh lớn nhất trên đường đi là nhỏ nhất. Chỉ cần tìm cây khung nhỏ nhất rồi dùng nhân đôi để xử lý cạnh lớn nhất trên đường đi giữa hai đỉnh.

Độ phức tạp thời gian là $O((n + q) \log n)$.

6.4 Kiểm tra tính phẳng

Bây giờ xét cách kiểm tra một đồ thị cho trước có phải đồ thị phẳng hay không. Ta có định lý quen thuộc sau.

Định lý 4.1 (Định lý Kuratowski). Một đồ thị đơn là đồ thị phẳng khi và chỉ khi nó không chứa một đồ thị con là phân hoạch của K_5 hoặc $K_{3,3}$.

Ở đây phân hoạch của một đồ thị nghĩa là thực hiện một số lần thao tác sau: chọn một cạnh (u, v) , tạo đỉnh mới x , rồi thay (u, v) bằng hai cạnh (u, x) và (x, v) . Nói đơn giản, đó là chèn thêm một số đỉnh bậc hai trên các cạnh của đồ thị.

Nếu kiểm tra trực tiếp theo định lý này, độ phức tạp thời gian thậm chí là mũ, rõ ràng không chấp nhận được. Vì vậy ta cần những thuật toán hiệu quả hơn.

6.4.1 Giảm hóa

Để tiện trình bày về sau, ta thực hiện một số giản hóa.

Gọi đồ thị cần kiểm tra là $G(V, E)$. Chú ý rằng khuyên và cạnh song song không ảnh hưởng tới tính phẳng, nên ta có thể giả sử G là đồ thị đơn.

Tiếp theo, chỉ cần xét từng thành phần song liên thông theo đỉnh của G . Trước hết, các thành phần liên thông của G độc lập với nhau. Với một thành phần liên thông, nếu nó có điểm khớp u , ta có thể tách G tại điểm khớp thành một số đồ thị con $G_1, G_2, \dots, G_k$; chú ý rằng đỉnh u xuất hiện trong mọi G_i . Nếu các đồ thị này đều phẳng, ta đặt u lên mặt ngoài trong từng nhúng phẳng rồi ghép các bản sao của u lại với nhau, từ đó thu được một nhúng phẳng của G .

6.4.2 Thuật toán DMP

Một ý tưởng thô là xác định dần vị trí của các đỉnh và cạnh trên mặt phẳng. Giả sử hiện tại đã xác định được vị trí của đồ thị con $H(V_H, E_H)$ trên mặt phẳng. Khi đó G bị H chia thành một số khối chưa xác định như sau.

Định nghĩa 4.1 (Đoạn). Các đoạn do đồ thị G bị đồ thị con H chia ra được định nghĩa như sau:

- nếu cạnh $e = (u, v)$ thỏa $u, v \in H$ nhưng $e \notin H$, thì đỉnh u, v và cạnh e tạo thành một đoạn;
- xét các thành phần liên thông $G_1, G_2, \dots, G_k$ của đồ thị thu được sau khi xóa các đỉnh trong H khỏi G . Với một G_i , lấy G_i cùng các cạnh nối từ G_i tới H và các đầu mút của chúng; phần đó tạo thành một đoạn.

Các đỉnh đồng thời nằm trong đoạn và trong đồ thị con H gọi là *điểm bám* của đoạn. Tập các điểm bám gọi là *biên* của đoạn. Nếu một đường đi đơn trong đoạn chứa các điểm bám đúng bằng hai đầu mút của nó, thì đường đi đó gọi là *đường bám*.

Vì mỗi đoạn đều là đồ thị con liên thông của G , trong một nhúng phẳng nó phải nằm trọn trong một mặt nào đó của H , nếu không sẽ sinh giao cắt. Nói cách khác, biên của mỗi đoạn phải nằm trên cùng biên của một mặt của H . Ký hiệu $F(A)$ là tập các mặt chứa hoàn toàn biên của đoạn A . Thuật toán như sau:

1. Ban đầu đặt H là một chu trình đơn bất kỳ trong G , rồi tìm mọi đoạn của G theo H .
2. Nếu tồn tại đoạn A sao cho $|F(A)| = 0$, thì G không phẳng; kết thúc.
3. Nếu tồn tại đoạn A sao cho $|F(A)| = 1$, mặt f chứa đoạn này được xác định duy nhất; nhúng A vào mặt f .
4. Nếu không, chọn tùy ý một đoạn A và một mặt $f \in F(A)$.
5. Chọn tùy ý một đường bám P của A , nhúng nó vào mặt f , chia f thành hai mặt, rồi cập nhật H và các đoạn do H chia ra trong G .
6. Nếu G không còn đoạn nào, thì G phẳng; kết thúc. Ngược lại quay về bước 2.

Nếu cài đặt trực tiếp, thuật toán có thể đạt tới độ phức tạp $O(n^3)$.

Ta tối ưu một số chi tiết. Duy trì tập $F(A)$ cho mỗi đoạn A . Mỗi lần có thể tìm một đoạn A và mặt f trong $O(n)$; sau đó xóa mặt f , chèn hai mặt f_1, f_2 sau khi chia, xóa đoạn A và chèn một số đoạn mới. Vì vậy chỉ còn cần xử lý hai loại câu hỏi:

- với một mặt cụ thể f , xác định với mọi đoạn A xem $f \in F(A)$ hay không;
- với một đoạn cụ thể A , xác định với mọi mặt f xem $f \in F(A)$ hay không.

Tổng số đỉnh xuất hiện trên các mặt và tổng số điểm bám của các đoạn đều là $O(n)$, nên hai loại câu hỏi trên cũng xử lý được trong $O(n)$. Tổng số vòng lặp là $O(n)$, vì vậy tổng độ phức tạp thời gian là $O(n^2)$.

Sau đây chứng minh tính đúng đắn của thuật toán. Chỉ cần chứng minh nếu G là đồ thị phẳng thì thuật toán sẽ đưa ra một nhúng phẳng.

Định nghĩa 4.2. Hai đoạn S, T gọi là xung đột nếu thỏa các điều kiện sau:

- $F(S) \cap F(T) \neq \emptyset$;
- tồn tại một mặt $f \in F(S) \cap F(T)$ và hai đường bám $P \subset S, Q \subset T$ sao cho P, Q không thể đồng thời nhúng vào mặt f .

Bổ đề 4.1. Với hai đoạn xung đột S, T , nếu $|F(S)| \geq 2$ và $|F(T)| \geq 2$, thì $F(S) = F(T)$ và $|F(S)| = 2$.

Chứng minh. Phản chứng. Giả sử $F(S) \neq F(T)$; khi đó có thể chọn ba mặt khác nhau f, g, h sao cho $h \in F(S) \cap F(T)$ và f, g lần lượt thuộc hai tập mặt của S, T . Vì S, T xung đột, tồn tại hai đường bám $P \subset S, Q \subset T$ không thể đồng thời nhúng vào h .

Nhúng P vào mặt f và Q vào mặt g . Khi đó hai đường đi được nhúng ở bên ngoài mặt h ; điều này tương đương với việc chúng có thể đồng thời được nhúng vào bên trong h , chẳng hạn bằng phép nghịch đảo qua một đường tròn có tâm tại một điểm bất kỳ trong h . Mâu thuẫn với định nghĩa xung đột.

Do đó $F(S) = F(T)$. Nếu $|F(S)| > 2$, ta cũng chọn được ba mặt khác nhau và lập luận tương tự, nên phải có $|F(S)| = 2$. ■

Định nghĩa đồ thị C : tập đỉnh của C là tập mọi đoạn của G , và hai đỉnh trong C có cạnh khi và chỉ khi hai đoạn tương ứng xung đột.

Bổ đề 4.2. Khi mọi đoạn A đều thỏa $|F(A)| \geq 2$, đồ thị C là đồ thị hai phía.

Chứng minh. Phản chứng. Giả sử C có một chu trình lẻ, ký hiệu là $A_1, A_2, \dots, A_{2k+1}$. Theo Bổ đề 4.1, các $F(A_i)$ đều giống nhau và có kích thước 2, giả sử là $\{f, g\}$. Vì hai đoạn kề nhau trong chu trình là hai đoạn xung đột, chúng không thể nhúng vào cùng một mặt. Vì vậy việc nhúng vào f hay g tạo thành một tô hai màu của chu trình lẻ, mâu thuẫn. ■

Định lý 4.2. Thuật toán DMP là đúng.

Chứng minh. Nếu thuật toán đưa ra một nhúng phẳng, hiển nhiên G là đồ thị phẳng. Giả sử G có một nhúng phẳng G' , ta xem đó là nhúng mục tiêu.

Khi bắt đầu chọn chu trình đơn, nếu chiều của chu trình khác với trong G' , ta lật G' lại vẫn thu được một nhúng phẳng.

Nếu tồn tại đoạn A thỏa $|F(A)| = 1$, thì cách nhúng đường bám được chọn thực ra là duy nhất, nên chắc chắn trùng với cách nhúng trong G' .

Ngược lại, mọi đoạn A đều thỏa $|F(A)| \geq 2$; chỉ cần xét trường hợp cách nhúng đường bám P của đoạn A khác với trong G' . Xét thành phần liên thông M chứa A trong đồ thị xung đột C ; chỉ các đoạn trong M mới có thể xung đột với A . Gọi các đỉnh của M là $A_1, A_2, \dots, A_k$.

Theo Bổ đề 4.1 và Bổ đề 4.2, M là đồ thị hai phía và các $F(A_i)$ đều giống nhau, có kích thước 2. Trong nhúng G' , đổi mặt nhúng của mọi đoạn thuộc một phía thích hợp của M sang mặt còn lại; nhúng vẫn hợp lệ và lúc này cách nhúng của P trùng với nhúng mục tiêu. Vì vậy thuật toán không làm mất khả năng đi tới một nhúng phẳng. ■

6.4.3 Thuật toán Tarjan

Để tiện kiểm tra một nhúng phẳng có hợp lệ hay không, trước hết nêu bổ đề sau.

Bổ đề 4.3. Trong một nhúng phẳng G , nếu giữa hai đỉnh x, y có ba đường đi p_1, p_2, p_3 đôi một không giao nhau ngoài hai đầu mút. Gọi $(x, v_1), (x, v_2), (x, v_3)$ là các cạnh đầu tiên trên p_1, p_2, p_3 và $(w_1, y), (w_2, y), (w_3, y)$ là các cạnh cuối cùng. Nếu quanh x ba cạnh theo chiều kim đồng hồ có thứ tự $(x, v_1), (x, v_2), (x, v_3)$, thì quanh y ba cạnh theo chiều kim đồng hồ có thứ tự ngược lại tương ứng.

Chứng minh. Bổ đề này là hệ quả trực tiếp của định lý đường cong Jordan; chứng minh định lý Jordan khá phức tạp, xem [5]. ■

Ta thường dùng phản đảo của Bổ đề 4.3: nếu giữa hai đỉnh x, y có ba đường đi không giao nhau nhưng thứ tự quanh hai đầu không thỏa kết luận trên, thì G không phải một nhúng phẳng.

Trước khi giới thiệu thuật toán, xét một ví dụ liên quan trực tiếp tới thuật toán.

Ví dụ 3 (HNOI2010, phiên bản tăng cường kiểm tra đồ thị phẳng). Cho một đồ thị đơn vô hướng G có n đỉnh và m cạnh, đồng thời cho một chu trình Hamilton của G . Hãy kiểm tra đồ thị vô hướng này có phẳng hay không.

$$1 < n \leq 5 \times 10^5, \quad 1 < m \leq 10^6.$$

Lời giải. Gọi chu trình Hamilton là $v_1, v_2, \dots, v_n$, trong đó với mọi i có cạnh từ v_i tới $v_{i \bmod n+1}$. Để tiện trình bày, đặt $v_0 = v_n$ và $v_{n+1} = v_1$.

Xét chu trình này như một đường tròn. Mỗi cạnh ngoài chu trình, chẳng hạn (v_i, v_j) với $i < j$, phải được vẽ ở một trong hai phía của chuỗi $v_i, v_{i+1}, \dots, v_j$. Nếu nó nằm bên trái, thứ tự theo chiều kim đồng hồ quanh v_i là $(v_i, v_{i-1}), (v_i, v_j), (v_i, v_{i+1})$; nếu nằm bên phải thì thứ tự là $(v_i, v_{i+1}), (v_i, v_j), (v_i, v_{i-1})$. Các cạnh ở hai phía khác nhau không ảnh hưởng lẫn nhau; chỉ cần xét quan hệ giữa các cạnh nằm cùng một phía.

Bổ đề 4.4. Nhúng phẳng của G hợp lệ khi và chỉ khi không tồn tại hai cạnh cùng phía (v_i, v_k) và (v_j, v_l) thỏa $j < i < l < k$.

Chứng minh. Nếu tồn tại hai cạnh như vậy, xét ba đường đi giữa v_i và v_l được tạo từ các đoạn trên chu trình Hamilton và hai cạnh ngoài chu trình; áp dụng Bổ đề 4.3 suy ra nhúng không hợp lệ.

Ngược lại, nếu không có cặp cạnh như vậy, thì trong mỗi phía, các khoảng ứng với cạnh ngoài chu trình hoặc rời nhau hoặc lồng nhau. Do đó mỗi lần chọn cạnh có khoảng dài nhất và vẽ nó đủ sát chuỗi tương ứng, nó sẽ không cắt các cạnh đã vẽ trước. ■

Xây dựng đồ thị G' , trong đó mỗi đỉnh là một cạnh của G không thuộc chu trình Hamilton; giữa hai đỉnh có cạnh nếu theo bổ đề trên, hai cạnh tương ứng không được nằm cùng một phía.

Một nhúng phẳng hợp lệ tương ứng với một tô hai màu của G' : mỗi đỉnh được gán nhãn trái hoặc phải, biểu thị cạnh tương ứng trong G nằm ở phía nào. Nếu dựng tường minh G' rồi kiểm tra hai phía, độ phức tạp có thể là $O(m^2)$.

Xét quét các cạnh (v_i, v_j) với $i > j$, sắp theo j giảm dần; nếu j bằng nhau thì sắp theo i tăng dần. Duy trì hai ngăn xếp L, R , tương ứng các cạnh hiện ở phía trái và phía phải.

Theo thứ tự quét, ta chỉ cần kiểm tra liệu có cạnh đã thêm (v_k, v_l) thỏa $j < l < i < k$ hay không. Trong mỗi ngăn xếp, thông tin của mỗi cạnh chỉ cần duy trì giá trị l . Luôn giữ tính đơn điệu theo l của L, R , tức từ đáy tới đỉnh ngăn xếp, các giá trị l không giảm.

Khi thêm cạnh (v_i, v_j) với $i > j$, thực hiện:

1. Trong cả L và R , liên tục bật các cạnh có $l > i$, vì từ nay về sau chúng không thể thỏa ràng buộc $j < l < i < k$.
2. Thử thêm (v_i, v_j) vào L ; các cạnh trong L xung đột với nó phải được chuyển sang R .
3. Việc chuyển sang R có thể tạo xung đột mới, nên tiếp tục chuyển các cạnh xung đột ban đầu trong R sang L .
4. Nếu vẫn không thỏa, kết luận vô nghiệm; ngược lại tiếp tục thêm cạnh kế tiếp.

Để chứng minh và cài đặt nhanh các bước chuyển do xung đột, tạo thêm một ngăn xếp B . Mỗi phần tử của B đại diện cho một thành phần liên thông của G' đã xuất hiện. Trong một thành phần liên thông, chỉ cần xác định phía của một cạnh là xác định được mọi cạnh còn lại; các thành phần liên thông khác nhau độc lập với nhau.

Ta cũng giữ tính đơn điệu của B theo giá trị l : với hai phần tử kề nhau từ đáy lên đỉnh, giá trị l lớn nhất của phần tử trước không vượt quá giá trị l nhỏ nhất của phần tử sau. Trong L và R , các phần tử thuộc cùng một thành phần liên thông luôn chiếm một đoạn liên tiếp. Để gộp nhanh hai thành phần liên thông và bật phần tử ở cuối, mỗi thành phần lưu thêm hai danh sách liên kết đôi L_S, R_S , lần lượt xâu các cạnh của thành phần theo thứ tự trong L và R .

Với ngăn xếp này, quy trình có thể viết lại ở mức thành phần liên thông:

1. Nếu giá trị l ở cuối danh sách của phần tử đỉnh B lớn hơn i , bật phần tử đó, cho tới khi mọi thành phần còn lại đều có $l < i$.
2. Nếu giá trị l nhỏ nhất của phần tử đỉnh lớn hơn j , cần chuyển toàn bộ thành phần đó sang R .
3. Nếu trong thành phần này đồng thời đã có phần tử thuộc L và R , báo vô nghiệm; nếu không, chuyển cả thành phần sang R , bật nó khỏi B và quay lại bước 2.
4. Ngược lại, nếu giá trị l lớn nhất của phần tử đỉnh lớn hơn j , kiểm tra có thể hoán đổi L và R trong thành phần này hay không; nếu không thể thì báo vô nghiệm, sau đó bật nó khỏi B .
5. Gộp mọi phần tử vừa bật cùng cạnh đang xét thành một thành phần liên thông mới, đẩy vào B , rồi quay lại bước 1 với cạnh tiếp theo.

Cách làm này giống ngăn xếp đơn điệu, nên L, R, B đều giữ được tính đơn điệu. Thực tế không cần duy trì tường minh L, R ; chỉ cần duy trì danh sách trong mỗi thành phần liên thông là đủ. Mỗi lần bật ngăn xếp dẫn tới một lần gộp thành phần liên thông, nên tổng số lần bật là $O(m)$. Tổng độ phức tạp thời gian là $O(m)$.

Thuật toán Tarjan tổng quát vẫn bắt đầu bằng cách tìm một chu trình và chia đồ thị thành các đoạn. Sau đó đệ quy kiểm tra từng đoạn có phẳng hay không, rồi dùng kỹ thuật tương tự ví dụ trên để xử lý các ràng buộc giữa các đoạn.

Như trước, giả sử đồ thị cần kiểm tra G là song liên thông theo đỉnh. Trước hết chạy DFS trên G , gọi cây DFS là T . Cạnh của G được chia thành hai loại: cạnh cây và cạnh ngược lên tổ tiên. Định hướng các cạnh như sau: cạnh cây đi từ cha tới con, cạnh ngược đi từ con tới tổ tiên. Nếu không nói khác, mọi đường đi đều theo hướng đã định.

Ký hiệu cạnh cây từ u tới v là $u \rightarrow v$, cạnh ngược từ u tới v cũng là $u \rightarrow v$, và đường đi từ u tới v là $u \rightarrow v$. Để tiện trình bày, đánh lại số đỉnh theo thứ tự DFS.

Đặt low_u là số hiệu nhỏ nhất của một đỉnh có thể đi tới từ u bằng cách đi qua một số cạnh cây theo hướng xuống, rồi nhiều nhất một cạnh ngược. Đặt low'_u là giá trị nhỏ thứ hai nghiêm ngặt theo cách tương tự. Riêng nếu $\text{low}_u = u$, định nghĩa $\text{low}'_u = u$; đây cũng là trường hợp duy nhất $\text{low}_u = \text{low}'_u$. Hai giá trị này có thể tính trong quá trình DFS.

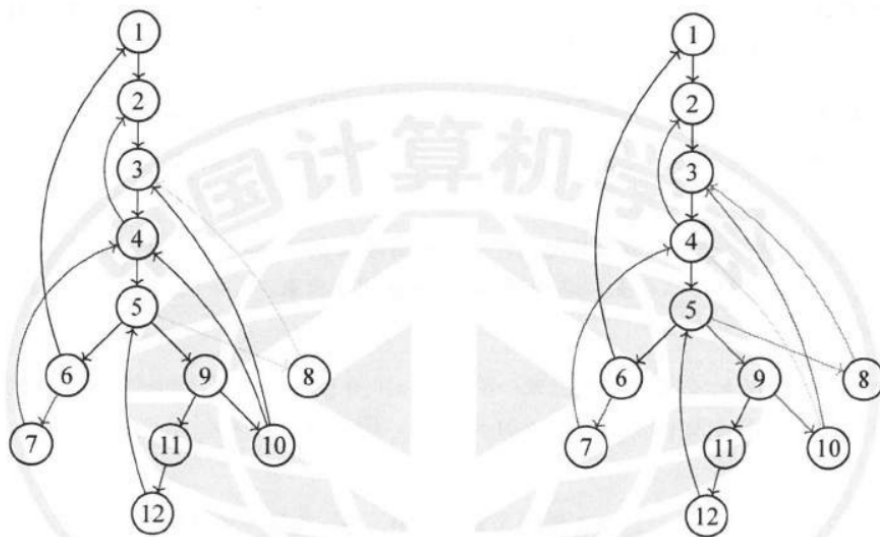
Tiếp theo ta chia G thành một số đường đi và nhúng các đường đi đó vào mặt phẳng theo một thứ tự đặc biệt. Để tìm phân hoạch đường đi, gán trọng số cho mỗi cạnh.

Định nghĩa 4.3. Trọng số $d((u, v))$ của cạnh có hướng (u, v) được định nghĩa bởi

$$d((u, v)) = \begin{cases} 2v, & u < v, \\ 2\text{low}_v, & u > v, \text{low}_v > u, \\ 2\text{low}_v + 1, & u > v, \text{low}_v < u. \end{cases}$$

Ý nghĩa của các trọng số này sẽ thể hiện trong các bổ đề sau. Với mỗi đỉnh, sắp xếp các cạnh đi ra theo trọng số tăng dần. Để không ảnh hưởng tổng độ phức tạp, cần dùng sắp xếp theo thùng, đạt $O(m)$.

Sau đó chạy DFS một lần nữa trên G . Giả sử hiện ở đỉnh u , ta duyệt các cạnh đi ra từ u theo thứ tự đã sắp. Nếu đi qua cạnh cây $u \rightarrow v$, thêm cạnh này vào đường đi hiện tại rồi đệ quy tới v . Nếu $u \rightarrow v$ là cạnh ngược, thêm cạnh này làm cạnh cuối của đường đi hiện tại, rồi mở một đường đi mới. Kết thúc thuật toán, G được chia thành một số đường đi, mỗi cạnh xuất hiện đúng một lần. Mỗi đường đi được chia ra đều đi qua một số cạnh cây trước rồi đi qua một cạnh ngược; ký hiệu một đường đi từ s tới t là $s \rightarrow t$.



Hình 1: Ví dụ về các đoạn và các đường đi được chia từ cây DFS. Hai hình đặt cạnh nhau cho thấy cùng một đồ thị trước và sau khi các đường đi phân hoạch được làm nổi bật bằng những cung ngược.

Bổ đề 4.5. Với một đường đi được chia ra $p : s \rightarrow t$, xét các cạnh ngược chưa đi qua tại thời điểm bắt đầu chia đường đi này. Khi đó t là đỉnh có số hiệu nhỏ nhất mà hậu duệ của s có thể đi tới bằng các cạnh ngược đó. Đặc biệt, nếu v là một đỉnh trên p và $v \neq s, t$, thì $t = \text{low}_v$.

Chứng minh. Trọng số của một cạnh thực ra biểu diễn số hiệu nhỏ nhất của đỉnh có thể tới được nếu bắt đầu từ cạnh đó và đi qua nhiều nhất một cạnh ngược, kể cả chính cạnh đó. Mỗi lần ta chọn cạnh chưa đi qua có trọng số nhỏ nhất, nên kết luận hiển nhiên. ■

Bổ đề 4.6. Nếu $p : s \rightarrow t$ là một đường đi được chia ra, thì t là tổ tiên của s có thể trùng với s , nên $t \rightarrow s$ hợp lệ. Nếu p là đường đi đầu tiên được chia ra, thì p là một chu trình đơn; nếu không, p là một đường đi đơn. Ngoài ra, nếu p không phải đường đi đầu tiên, nó chứa đúng hai đỉnh s, t đã xuất hiện trong các đường đi được chia trước đó.

Chứng minh. Nếu p chỉ gồm một cạnh ngược, kết luận rõ ràng. Ngược lại, xét cạnh đầu tiên $s \rightarrow v$ của p . Khi đó v là lần đầu được đi qua; theo Bổ đề 4.5 có $t = \text{low}_v$. Vì G song liên thông theo đỉnh, t là tổ tiên của s , và $t = s$ khi và chỉ khi $s = 1$. Thực tế chỉ có t có thể trùng với đỉnh khác trên đường đi; chỉ đường đi đầu tiên mới thỏa $s = t = 1$. Do đó đường đi đầu tiên là chu trình đơn, còn các đường đi khác là đường đi đơn. Theo tính chất của DFS, với một đỉnh $v \neq 1$, lần đầu v được đi qua luôn nằm trong phần giữa của một đường đi $s \rightarrow t$, tức $v \neq s, t$; các đỉnh ở giữa đường đi đều là lần đầu được đi qua. Vì vậy mỗi đường đi không đầu tiên chứa đúng hai đỉnh không phải lần đầu được đi qua. ■

Bổ đề 4.7. Nếu hai đường đi được chia ra $p_1 : s_1 \rightarrow t_1$ và $p_2 : s_2 \rightarrow t_2$ thỏa s_1 nằm trên p_2 và p_2 được chia trước p_1 , thì $t_1 < t_2$.

Chứng minh. Cạnh ngược trong p_1 xuất phát từ một hậu duệ của s_1 , và tại thời điểm bắt đầu chia p_2 cạnh đó vẫn chưa được đi qua. Theo Bổ đề 4.5, t_2 không lớn hơn đỉnh mà cạnh này đi tới; vì p_1 được chia sau, ta thu được quan hệ nghiêm ngặt $t_1 < t_2$. ■

Bổ đề 4.8. Với hai đường đi được chia ra có cùng điểm đầu và điểm cuối $p_1 : s \rightarrow t$ và $p_2 : s \rightarrow t$, gọi v_1, v_2 lần lượt là đỉnh thứ hai của p_1, p_2 . Nếu p_1 được chia trước p_2 , thì $v_1 \neq t$, $\text{low}'_{v_1} < s$, còn nếu $v_2 \neq t$ thì $\text{low}'_{v_2} < s$.

Chứng minh. Vì p_1 được chia trước p_2 , trọng số cạnh đầu $d(s, v_1)$ nhỏ hơn $d(s, v_2)$. Từ công thức trọng số suy ra $d(s, v_1) = 2t + 1$ và $d(s, v_2) \leq 2t + 1$. Do hai đường đi khác nhau nên trường hợp bằng dẫn tới điều kiện về low' nêu trên. ■

Tiếp theo trình bày quá trình nhúng phẳng. Gọi c là đường đi đầu tiên. Theo Bổ đề 4.6, c là chu trình đơn. Chu trình này gồm một số cạnh cây $1 \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ và một cạnh ngược $v_k \rightarrow 1$. Theo định nghĩa đoạn trong thuật toán DMP, G bị c chia ra thành một số đoạn. Mỗi đoạn hoặc là một cạnh ngược (u, v) , hoặc gồm một cạnh cây (u, v) cộng với các cạnh trong cây con của v và các cạnh ngược xuất phát từ cây con đó. Vì vậy một cạnh (u, v) xuất phát từ chu trình c nhưng không thuộc c tương ứng đúng một đoạn của G .

Với mỗi đoạn, các đường đi được chia ra nằm trong nó xuất hiện trong một khoảng liên tiếp theo thời gian chia. Hơn nữa, sau khi chia đường đi đầu tiên, quá trình DFS đi vào các đoạn theo thứ tự u giảm dần.

Theo định lý đường cong Jordan, mỗi đoạn hoặc nằm bên trái hoặc nằm bên phải chuỗi $1 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$. Tương tự ví dụ trước, nếu đoạn ứng với cạnh (u, v) nằm bên trái, thì thứ tự theo chiều kim đồng hồ quanh u là $(u, \text{prev}(u)), (u, v), (u, \text{next}(u))$; nếu nằm bên phải, thứ tự là $(u, \text{next}(u)), (u, v), (u, \text{prev}(u))$.

Với một đoạn S , gọi đường đi đầu tiên được chia trong nó là p . Giả sử mọi đường đi trước p đã được nhúng vào mặt phẳng. Bổ đề sau cho điều kiện cần và đủ để p có thể nhúng vào một phía; đây là điểm mấu chốt của thuật toán.

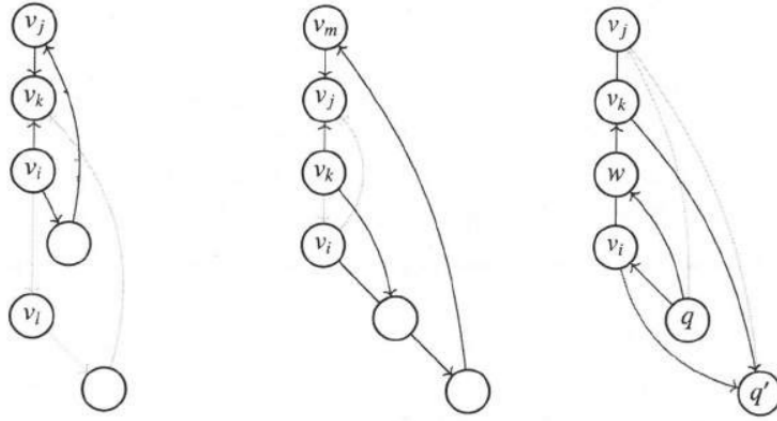
Bổ đề 4.9. Đường đi $p : v_i \rightarrow v_j$ có thể được nhúng ở phía trái, tương ứng phía phải, khi và chỉ khi không có cạnh ngược (x, y) đã được nhúng ở cùng phía đó thỏa $v_j < y < v_i$.

Chứng minh. Nếu không có cạnh ngược y nằm giữa v_j và v_i như trên, thì trong các cạnh đã nhúng không có cạnh đi vào hoặc đi ra khỏi đoạn giữa v_i và v_j . Do đó chỉ cần nhúng p đủ sát chuỗi giữa v_i và v_j , nhúng chắc chắn hợp lệ.

Ngược lại, nếu tồn tại cạnh ngược (x, y) với $v_j < y < v_i$, ta chứng minh p không thể nhúng phẳng ở phía đó. Giả sử p muốn nhúng bên trái. Gọi S' là đoạn chứa (x, y) , và $q : o \rightarrow y$ là đường đi được chia chứa cạnh ngược đó. Lấy đường đi đầu tiên trong S' là $q' : v_r \rightarrow v_s$. Từ thứ tự chia suy ra $v_r > v_i$ hoặc $v_r = v_i$; trong từng trường hợp, dùng Bổ đề 4.3 để dựng ba đường đi không giao nhau giữa hai đầu thích hợp, từ đó mâu thuẫn với thứ tự quanh đỉnh nếu p cũng nhúng bên trái.

Phần chứng minh trong bản quét gồm một phân tích trường hợp dài với nhiều chỉ số. Ý chính là: trong mọi trường hợp có cạnh ngược đã nhúng cùng phía cắt khoảng (v_j, v_i) , ta dựng được ba đường đi như trong Bổ đề 4.3 để chứng minh không thể nhúng p ở phía đó. ■

Bổ đề này rất giống Bổ đề 4.4 trong ví dụ trên. Thật vậy, sau khi qua bước quét trong ví dụ, hai bổ đề trở thành cùng một dạng ràng buộc đối với một đường đi $p : v_i \rightarrow v_j$.



Hình 2: Ba cấu hình minh họa cho chứng minh Bổ đề 4.9. Các hình biểu diễn những cách dựng ba đường đi khi tồn tại cạnh ngược đã nhúng cùng phía cắt khoảng (v_j, v_i) , từ đó đường đi p không thể được nhúng ở phía đó.

Vấn đề là ở đây đường đi p chỉ là đường đi đầu tiên của đoạn S ; các đường đi khác trong S chưa được xét. Tuy nhiên, vì các đường đi trong S đều phải nhúng ở cùng phía với p , và v_j là vị trí nhỏ nhất mà cạnh ngược trong S có thể đi tới, nếu p không mâu thuẫn với các đường đi đã nhúng trước đó thì mọi đường đi trong S cũng không mâu thuẫn với phần trước. Do đó chỉ cần kiểm tra nội bộ S có thể nhúng hay không.

Vì vậy ta chỉ cần thực hiện thuật toán ở ví dụ trên; sau khi chèn thành công đường đi p vào L , làm thêm các bước sau. Trước hết đặt một dấu đáy trong ngăn xếp R , rồi đệ quy kiểm tra đoạn S có hợp lệ không. Nếu S hợp lệ, lúc này trong ngăn xếp B xuất hiện thêm một số thành phần liên thông. Vì đoạn S chỉ có thể nằm bên trong chu trình đang xét, các thành phần liên thông do đệ quy sinh ra cần được gộp thành một thành phần và đều nằm trong L . Nếu trong các thành phần của đoạn S đồng thời có phần tử thuộc L và R , thì vô nghiệm; ngược lại bật chúng ra khỏi B , thu hồi dấu đáy trong R , gộp các thành phần này cùng đường đi p thành một thành phần lớn rồi đẩy vào B . Dấu đáy trong R giúp phân biệt thành phần nào thuộc đoạn S và ngăn phần đệ quy sinh cạnh với các phần tử trước đó.

Tóm lại, ta thu được một thuật toán kiểm tra tính phẳng trong thời gian $O(n)$.

6.5 Tô màu đỉnh trên đồ thị phẳng

Bài toán tô màu là một lớp bài toán quan trọng trong lý thuyết đồ thị. Với đồ thị phẳng, ta có định lý bốn màu nổi tiếng.

Định nghĩa 5.1 (k -tô màu). Với một đồ thị vô hướng G , nếu gán cho mỗi đỉnh u một màu c_u sao cho với mọi cạnh (u, v) đều có $c_u \neq c_v$, thì phép gán này gọi là một phương án tô màu. Nếu G tồn tại một phương án dùng không quá k màu, ta gọi đó là một k -tô màu và nói G là k -tô màu được. Giá trị k nhỏ nhất để G k -tô màu được gọi là sắc số của G , ký hiệu $\chi(G)$.

Định lý 5.1 (Định lý bốn màu). Mọi đồ thị phẳng G đều 4-tô màu được.

Chứng minh xem [8]. Quá trình chứng minh định lý bốn màu cũng cho một thuật toán dựng 4-tô màu của đồ thị phẳng trong thời gian $O(n^2)$. Tuy nhiên thuật toán đó quá phức tạp, nên không trình bày ở đây.

Thực ra cách tô màu tham lam đã có thể cho một 6-tô màu. Ta cần bổ đề sau.

Bổ đề 5.1. Trong một đồ thị phẳng luôn tồn tại một đỉnh có bậc không vượt quá 5.

Chứng minh. Nếu mọi đỉnh đều có bậc ít nhất 6, thì $3V \leq E$, mâu thuẫn với Bổ đề 2.1. ■

Với đồ thị phẳng G , lấy một đỉnh x có bậc không quá 5 và xóa nó khỏi G , thu được G' . Đệ quy tô màu G' , cuối cùng gán cho x một màu chưa xuất hiện trong các đỉnh kề với nó. Như vậy dùng được không quá 6 màu.

Cải tiến nhẹ thuật toán này có thể cho một 5-tô màu. Chỉ cần xét trường hợp đỉnh x có bậc 5 và năm đỉnh kề với nó có màu đôi một khác nhau.

Giả sử các đỉnh kề với x theo thứ tự ngược chiều kim đồng hồ là v_1, v_2, v_3, v_4, v_5 , trong đó đỉnh v_i có màu i . Xét đồ thị con cảm sinh bởi các đỉnh màu 1 và 3. Nếu v_1 và v_3 không liên thông trong đồ thị con này, ta đảo màu trong thành phần liên thông chứa v_1 (màu 1 thành 3 và ngược lại); khi đó có thể tô x bằng màu 1.

Nếu v_1 và v_3 liên thông, xét một đường đi đơn giữa chúng trong đồ thị con màu 1, 3; đường đi này cùng với x tạo thành một chu trình. Theo định lý đường cong Jordan, chu trình chia mặt phẳng thành trong và ngoài. Lúc này trong đồ thị con cảm sinh bởi các màu 2 và 4, hai đỉnh v_2 và v_4 chắc chắn không liên thông. Tương tự, đảo màu trong thành phần liên thông chứa v_2 rồi tô x bằng màu 2.

Vì vậy thuật toán này có thể dựng một 5-tô màu cho đồ thị phẳng trong thời gian $O(n^2)$.

Dù mọi đồ thị phẳng đều 4-tô màu được và kiểm tra 2-tô màu rất dễ, việc kiểm tra một đồ thị phẳng có 3-tô màu được hay không vẫn là bài toán NPC. Tiếp theo xét một lớp đồ thị có ràng buộc mạnh hơn.

Định nghĩa 5.2 (Đồ thị tam giác hóa). Một nhúng phẳng G gọi là đồ thị tam giác hóa nếu biên của mỗi mặt trong của G đều có đúng 3 cạnh.

Với đồ thị tam giác hóa, ta có thể kiểm tra hiệu quả liệu nó có 3-tô màu được hay không.

Bổ đề 5.2. Đồ thị tam giác hóa G 3-tô màu được khi và chỉ khi G không chứa đồ thị bánh xe lẻ. Ở đây đồ thị bánh xe lẻ gồm một chu trình lẻ và một đỉnh đặc biệt nối cạnh tới mọi đỉnh trên chu trình.

Chứng minh. Trước hết, đồ thị bánh xe lẻ hiển nhiên không thể 3-tô màu. Chỉ cần chứng minh nếu đồ thị tam giác hóa không chứa bánh xe lẻ thì nó 3-tô màu được.

Chứng minh bằng quy nạp. Khi $n < 3$ thì hiển nhiên. Nếu G không song liên thông theo đỉnh, có thể tách nó thành các thành phần song liên thông, tô màu từng phần rồi ghép lại. Vì vậy giả sử G song liên thông.

Ta thử tìm một đỉnh u sao cho sau khi xóa u , đồ thị vẫn song liên thông. Xét chu trình c tạo bởi biên mặt ngoài. Chọn một đỉnh u bất kỳ trên biên mặt ngoài; gọi các cạnh kề với u lần lượt là $(u, v_1), (u, v_2), \dots, (u, v_k)$. Vì G là đồ thị tam giác hóa, giữa các đỉnh kề liên tiếp của u đều tồn tại cạnh; nếu không, mặt chứa hai cạnh tương ứng sẽ có nhiều hơn 3 cạnh. Sau khi xóa u , biên mặt ngoài mới thu được bằng cách thay đoạn quanh u bởi chuỗi các đỉnh kề này. Nếu không có dây cung của c xuất phát từ u , chu trình này vẫn đơn nên xóa u giữ được song liên thông.

Nếu có dây cung (u, v) của c , nó chia G thành hai phần chỉ có chung hai đỉnh u, v . Theo giả thiết quy nạp, hai phần đều có phương án 3-tô màu; đổi tên màu ở một phần nếu cần rồi ghép lại là được.

Tiếp theo, xét các khoảng giữa các đỉnh kề liên tiếp của u . Từ thảo luận trước, các đỉnh nằm giữa hai đỉnh kề ấy phải tạo thành một chuỗi. Màu của các đỉnh trong chuỗi phải luân phiên với màu ở hai đầu. Vì G không chứa bánh xe lẻ, để không tạo thành bánh xe lẻ với u và hai đỉnh kề, độ dài chuỗi phải là lẻ, nên màu ở hai đầu chuỗi giống nhau. Suy ra màu của các đỉnh kề v_i của u luân phiên nhau; do đó có thể tô u bằng màu khác với tất cả chúng. ■

Ví dụ 4 (MIN&MAX II). Với một dãy số nguyên a có độ dài n và các phần tử đôi một khác nhau, ta xây dựng đồ thị vô hướng đơn $G(a)$ như sau. Có n đỉnh xếp từ trái sang phải, đánh số 1 tới n ; mỗi đỉnh nối với đỉnh gần nhất bên trái và bên phải có giá trị lớn hơn nó, đồng thời nối với đỉnh gần nhất bên trái và bên phải có giá trị nhỏ hơn nó. Ở đây khi so sánh hai đỉnh, ta so sánh các giá trị a_i của chúng.

Với mỗi truy vấn khoảng, cần xử lý các đồ thị con do các dãy con liên tiếp $a[l : r]$ sinh ra và tìm giá trị lớn nhất của $\chi(G(a[l' : r']))$ cùng số đoạn con đạt giá trị đó trong khoảng truy vấn. Ràng buộc đọc được từ bản quét là

$$1 \leq n, q \leq 3 \times 10^5.$$

Lời giải. Đặt tọa độ của đỉnh thứ i là (i, a_i) , và mỗi cạnh là đoạn thẳng nối hai đỉnh.

Với dãy bất kỳ a , đồ thị $G(a)$ là đồ thị phẳng. Giả sử có hai cạnh (i, j) và (k, l) cắt nhau, không mất tổng quát $i < k < j < l$, và j là đỉnh gần nhất bên phải của i có giá trị lớn hơn a_i . Do hai cạnh cắt nhau, xét các trường hợp tương quan giá trị a_i, a_j, a_k, a_l ; trong mọi trường hợp, một đầu mút sẽ bị một đỉnh nằm giữa chặn lại, mâu thuẫn với định nghĩa “gần nhất”. Ở đây nói x bị w chặn trên đường tới y nghĩa là $x < w < y$ và a_w nằm giữa a_x, a_y theo đúng phía so sánh, nên x không thể nối trực tiếp tới y .

Tiếp theo dùng một kết luận hình học: với một đỉnh u , giả sử các cạnh kề với nó theo thứ tự góc cực là $(u, v_1), (u, v_2), \dots, (u, v_k)$. Với hai cạnh kề nhau trong thứ tự này, (u, v_i) và $(u, v_{i \bmod k+1})$, hai đỉnh v_i và $v_{i \bmod k+1}$ có cạnh nối khi và chỉ khi trong hệ tọa độ lấy u làm gốc, hai điểm này không nằm ở hai góc phần tư khác nhau và không kề nhau. Chứng minh kết luận này cần phân tích trường hợp khá dài nhưng không khó, nên nguồn lược bỏ.

Từ đó suy ra với mọi dãy a , $G(a)$ là đồ thị tam giác hóa. Nếu tồn tại một mặt có nhiều hơn 3 cạnh, lấy trên biên mặt đó một đỉnh có giá trị nhỏ nhất; kết luận hình học ở trên cho thấy giữa hai đỉnh kề nó trên biên phải có cạnh, mâu thuẫn.

Ngoài ra, với một đỉnh u , $G(a)$ có bánh xe lẻ lấy u làm đỉnh đặc biệt khi và chỉ khi bốn góc phần tư lấy u làm gốc đều có điểm và bậc của u là số lẻ. Đây là hệ quả đơn giản của kết luận trên.

Chú ý rằng $G(a[l : r])$ chính là đồ thị con cảm sinh bởi các đỉnh thuộc khoảng $[l, r]$ trong $G(a)$. Số bánh xe lẻ khác nhau nhiều nhất chỉ là $O(n)$, nên có thể tiền xử lý trực tiếp.

Ta có cách phân loại:

1. $\chi(G(a[l : r])) = 1$ khi và chỉ khi $l = r$;
2. $\chi(G(a[l : r])) = 2$ khi và chỉ khi $l < r$ và dãy $a_l, \dots, a_r$ đơn điệu tăng hoặc đơn điệu giảm. Nếu không, tồn tại i sao cho ba đỉnh liên tiếp tạo thành tam giác, nên không thể 2-tô màu;
3. $\chi(G(a[l : r])) = 3$ khi và chỉ khi $a[l : r]$ không đơn điệu và $G(a[l : r])$ không có bánh xe lẻ;
4. $\chi(G(a[l : r])) = 4$ khi và chỉ khi $G(a[l : r])$ có bánh xe lẻ.

Khi cố định đầu trái của khoảng, nếu đầu phải dịch sang phải thì sắc số không giảm. Vì vậy chỉ cần xử lý các vị trí mà sắc số thay đổi đối với từng đầu trái; sau đó quét từ trái sang phải và dùng cây đoạn để duy trì kết quả.

Độ phức tạp thời gian là $O((n + q) \log n)$.

6.6 Đếm ghép hoàn hảo trên đồ thị phẳng

Đếm số ghép hoàn hảo của một đồ thị là bài toán rất khó; ngay cả với đồ thị hai phía, bài toán này cũng là #P-complete. Tuy nhiên thuật toán FKT có thể tính số ghép hoàn hảo của đồ thị phẳng trong thời gian đa thức.

Ý tưởng chính của FKT là chuyển việc đếm ghép hoàn hảo thành tính pfaffian pf của một ma trận phản đối xứng; đại lượng này lại có thể chuyển thành tính định thức, từ đó có thuật toán hiệu quả.

Trước hết cần tìm một nhúng phẳng G của đồ thị phẳng đã cho. Hai thuật toán kiểm tra tính phẳng đã nêu ở trên đều cho cách dựng một nhúng phẳng khi đồ thị phẳng.

Nếu số đỉnh của G là lẻ thì hiển nhiên vô nghiệm; ngược lại ký hiệu số đỉnh là $2n$. Nếu G không liên thông, thực hiện thuật toán sau riêng cho từng thành phần liên thông. Dưới đây giả sử G liên thông. Định nghĩa ma trận kề $\{G_{ij}\}$: nếu giữa i và j có cạnh thì $G_{ij} = 1$, ngược lại $G_{ij} = 0$. Số ghép hoàn hảo của G là

$$\text{PerfMatch}(G) = \frac{1}{2^n n!} \sum_p \prod_{i=1}^n G_{p_{2i-1}, p_{2i}},$$

trong đó p chạy qua mọi hoán vị của $1, \dots, 2n$. Hệ số phía trước xuất hiện vì mỗi ghép hoàn hảo tương ứng đúng $2^n n!$ hoán vị khác nhau.

Với một ma trận A , định nghĩa

$$\text{pf}(A) = \frac{1}{2^n n!} \sum_p \text{sgn}(p) \prod_{i=1}^n A_{p_{2i-1}, p_{2i}}.$$

Gọi $o(p)$ là số cặp nghịch thế của hoán vị p , khi đó $\text{sgn}(p) = (-1)^{o(p)}$.

Hai công thức này rất giống nhau. Nếu ta dựng được một ma trận A chỉ khác ma trận kề ở dấu của một số vị trí, sao cho các dấu đó triệt tiêu $\text{sgn}(p)$ phía trước, thì có thể tính số ghép hoàn hảo bằng cách tính pfaffian.

Ta định hướng mọi cạnh của G , rồi dựng ma trận A :

$$A_{ij} = \begin{cases} 1, & i \rightarrow j \in E', \\ -1, & j \rightarrow i \in E', \\ 0, & i \neq j, \{i, j\} \notin E', \end{cases}$$

trong đó E' là tập cạnh có hướng sau khi định hướng G . Dễ thấy A là ma trận phản đối xứng, tức $A_{ij} = -A_{ji}$.

6.6.1 Bổ đề then chốt

Bổ đề 6.1. Nếu định hướng E' của nhúng phẳng G thỏa điều kiện: với mỗi mặt trong f , trên biên của f có lẻ số cạnh đi theo chiều kim đồng hồ, thì đóng góp của mọi ghép hoàn hảo vào $\text{pf}(A)$ đều bằng nhau, tức đều là 1 hoặc đều là -1 .

Chứng minh. Ký hiệu

$$f(p) = \text{sgn}(p) \prod_{i=1}^n A_{p_{2i-1}, p_{2i}}.$$

Trước hết với một ghép hoàn hảo M , chứng minh mọi hoán vị p, p' tương ứng với M đều có $f(p) = f(p')$. Có hai loại phép đổi:

1. Trong một cặp ghép (p_{2i-1}, p_{2i}) , đổi chỗ hai phần tử. Khi đó $\text{sgn}(p)$ đổi dấu và phần tử ma trận tương ứng cũng đổi dấu, nên f không đổi.
2. Đổi chỗ hai cặp ghép liên tiếp. Khi đó hoán vị nhận được có cùng dấu, và tích phần tử ma trận cũng không đổi.

Hai loại phép đổi này đủ để biến mọi hoán vị tương ứng với M thành nhau, nên $f(p) = f(p')$.

Định nghĩa hiệu đối xứng của hai tập A, B là $A \Delta B = (A \setminus B) \cup (B \setminus A)$. Với một tập cạnh có hướng $E = (e_1, \dots, e_k)$, ký hiệu $g(E) = \prod_i A_{e_i}$.

Xét hai ghép hoàn hảo M, M' và hai hoán vị tương ứng p, p' . Theo lập luận trên, tỉ số đóng góp chỉ phụ thuộc vào $M \Delta M'$. Đồ thị $G'(V, M \Delta M')$ có bậc mỗi đỉnh bằng 0 hoặc 2, nên nó là hợp của một số chu trình và đỉnh cô lập. Hơn nữa, trên mỗi chu trình, cạnh của M và cạnh của M' xuất hiện xen kẽ, nên mọi chu trình đều có độ dài chẵn.

Xét riêng từng chu trình chẵn. Gọi chu trình hiện tại là $c : v_1, v_2, \dots, v_{2k}$. Ta cần chứng minh số cạnh đi theo chiều kim đồng hồ trên chu trình là lẻ.

Gọi v, e, f lần lượt là số đỉnh, số cạnh và số mặt nằm nghiêm ngặt trong chu trình. Áp dụng định lý Euler cho phần bên trong chu trình, kể cả chính chu trình, thu được

$$(v + 2k) - (e + 2k) + (f + 1) = 2,$$

tức $v - e + f = 1$.

Gọi $C(u)$ là số cạnh đi theo chiều kim đồng hồ trên biên mặt u . Theo điều kiện định hướng, $C(u) \equiv 1 \pmod{2}$ với mọi mặt trong. Khi cộng $C(u)$ trên mọi mặt bên trong chu trình, mỗi cạnh nằm nghiêm ngặt bên trong được tính đúng một lần, còn cạnh trên chu trình được tính khi và chỉ khi nó đi theo chiều kim đồng hồ của chu trình. Nếu gọi a là số cạnh đi theo chiều kim đồng hồ trên chu trình, ta có

$$f \equiv \sum_u C(u) \equiv a + e \pmod{2}.$$

Kết hợp với $v - e + f = 1$ suy ra

$$a \equiv 1 - v \pmod{2}.$$

Vì ghép hoàn hảo chọn xen kẽ cạnh trên chu trình và tách phần trong với phần ngoài, số đỉnh bên trong v phải là chẵn; do đó a là lẻ.

Gọi M'' là ghép hoàn hảo thu được từ M bằng cách đổi lựa chọn trên các chu trình của $M \Delta M'$. Với mỗi chu trình chẵn, dấu hoán vị và tích dấu ma trận cùng cho kết quả khiến đóng góp không đổi. Lập trên mọi chu trình suy ra $f(p) = f(p')$ với mọi ghép hoàn hảo. ■

6.6.2 Cách định hướng

Bây giờ cần tìm một định hướng thỏa điều kiện của Bổ đề 6.1.

Bổ đề 6.2. Cho nhúng phẳng G và đồ thị đối ngẫu G^* . Với một cây khung bất kỳ T_1 của G , lấy mọi cạnh $e \notin T_1$ và cạnh đối ngẫu e^* tương ứng; các cạnh này cùng mọi đỉnh của G^* tạo thành một cây khung T_2 của G^* .

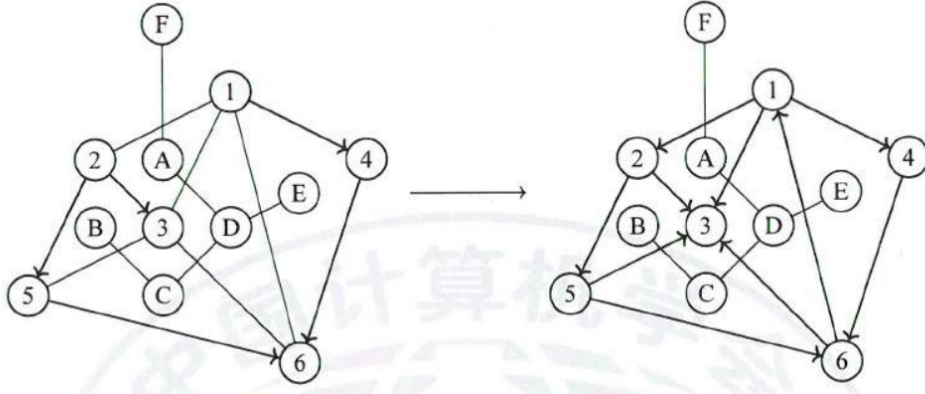
Chứng minh. Trước hết T_2 không có chu trình. Nếu có, lấy một chu trình đơn của T_2 ; theo Bổ đề 3.1, nó tương ứng với một tập cắt của G . Điều này mâu thuẫn với việc G chứa cây khung T_1 nhưng mọi cạnh đối ngẫu trong chu trình đều ứng với cạnh không thuộc T_1 .

Tiếp theo T_2 liên thông. Nếu không, lấy hai thành phần liên thông của nó; các cạnh giữa chúng đều không thuộc T_2 , nên cạnh gốc tương ứng đều thuộc T_1 . Theo Bổ đề 3.1, điều này tương ứng với một chu trình trong G , mâu thuẫn với T_1 là cây. ■

Ta thực hiện thuật toán sau:

1. Tìm đồ thị đối ngẫu G^* của nhúng phẳng G .

2. Tìm một cây khung tùy ý T_1 của G , và định hướng tùy ý các cạnh trong cây này.
3. Xét mọi cạnh của G không thuộc T_1 . Theo Bổ đề 6.2, các cạnh đối ngẫu của chúng tạo thành cây khung T_2 của G^* .
4. Tìm một lá v trong T_2 không đại diện cho mặt ngoài, và lấy cạnh kề e^* của nó.
5. Mặt ứng với lá v hiện chỉ còn một cạnh e chưa định hướng. Nếu trong mặt này đã có lẻ số cạnh đi theo chiều kim đồng hồ, định hướng e ngược chiều kim đồng hồ; ngược lại định hướng e theo chiều kim đồng hồ.
6. Xóa v và e^* khỏi T_2 . Nếu T_2 chỉ còn đỉnh ứng với mặt ngoài, thuật toán kết thúc; ngược lại quay lại bước 4.



Hình 3: Ví dụ định hướng một cạnh còn lại trên mặt. Hình bên trái là trạng thái trước khi xử lý một lá của T_2 ; hình bên phải cho thấy cạnh còn lại trên mặt tương ứng đã được định hướng để số cạnh thuận chiều kim đồng hồ trong mặt đó là lẻ.

Mỗi lần ta xóa một đỉnh, mọi cạnh trong mặt trong tương ứng đều đã được định hướng và có đúng lẻ số cạnh đi theo chiều kim đồng hồ. Vì vậy khi thuật toán kết thúc, ta thu được một định hướng thỏa Bổ đề 6.1.

6.6.3 Tính pfaffian

Bổ đề 6.3. Với một ma trận phản đối xứng A và một ma trận bất kỳ B , ta có

$$\text{pf}(A)^2 = \det(A),$$

$$\text{pf}(BAB^T) = \det(B) \text{pf}(A).$$

Chứng minh xem [11].

Theo Bổ đề 6.1,

$$\text{PerfMatch}(G) = |\text{pf}(A)|.$$

Vì vậy có thể trực tiếp tính định thức của A rồi lấy căn. Độ phức tạp thời gian là $O(n^3)$.

Tuy nhiên, thông thường ta làm việc trên trường $\mathbb{F}_p$; khi đó phép lấy căn có thể cho hai nghiệm, và ta không phân biệt được dấu dương âm trong $\mathbb{F}_p$, gây ra một số rắc rối.

Thực tế quá trình giải đưa vào hai dấu không chắc chắn. Dấu thứ nhất đến từ việc ta không biết mỗi ghép hoàn hảo đóng góp 1 hay -1 cho đáp án, nên cần tìm một ghép hoàn hảo của đồ thị tổng quát để

xác định dấu đóng góp. Dấu thứ hai xuất hiện khi tính pfaffian: thông qua Bổ đề 6.3 ta thu được bình phương của đáp án, nên khi lấy căn lại có dấu.

Để tránh lấy căn, có thể trong Bổ đề 6.3 chọn B là ma trận của phép biến đổi hàng sơ cấp, rồi thực hiện một quá trình tương tự khử Gauss. Chi tiết xem [12] và [13]. Tài liệu [14] đưa ra thuật toán tính pfaffian trong thời gian $O(n^3)$ trên một vành nguyên bất kỳ.

Có thể thấy bản thân thuật toán FKT khá ngắn gọn, nhưng xử lý dấu ở cuối lại tương đối phiền. Khi ra đề, có thể cân nhắc các cách như tính số cặp có thứ tự gồm hai ghép hoàn hảo, để tránh phải phán đoán dấu.

6.6.4 Ví dụ

Ví dụ 5. Alice và Bob chơi cờ trên một bàn cờ $n \times m$. Mỗi ô có một trọng số a_i ; ban đầu có k ô đã đặt quân. Alice và Bob luân phiên chọn một ô chưa có quân và đặt quân lên đó, Alice đi trước; người không còn nước đi hợp lệ sẽ thua. Ô Bob chọn phải có cạnh chung với ô Alice vừa chọn ở lượt trước.

Độ thú vị của một nước đi là tích trọng số của ô vừa đi với trọng số của ô đối phương đi ở nước trước; riêng nước đầu tiên có độ thú vị bằng 0. Độ thú vị của cả ván là tổng độ thú vị của mọi nước đi.

Nếu cả hai người đều chọn ngẫu nhiên đều trong các nước đi hợp lệ, hãy tính giá trị trung bình của độ thú vị cả ván trên mọi cục diện mà Bob thắng. Hai cục diện khác nhau khi và chỉ khi có một nước đi khác nhau. Lấy đáp án theo mô-đun 998244353, bảo đảm số cục diện Bob thắng không chia hết cho 998244353.

$$1 \leq nm \leq 400, \quad 0 \leq k < nm.$$

Lời giải. Xây dựng đồ thị G : đỉnh là các ô ban đầu chưa có quân; nếu hai ô có cạnh chung thì nối cạnh giữa hai đỉnh tương ứng. Gọi số đỉnh của G là $2r$.

Ghép ô Bob chọn với ô Alice chọn ngay trước đó. Khi Bob thắng, ta thu được một ghép hoàn hảo của G .

Ngược lại, xét đóng góp của một ghép hoàn hảo $M = \{(m_{i,0}, m_{i,1})\}_{i=1}^r$ vào đáp án. Trước hết một ghép hoàn hảo sinh ra $2^r r!$ cục diện khác nhau. Sau đó xét đóng góp của từng nước đi trong các cục diện đó; ta chỉ cần tính hai đại lượng sau, ký hiệu p_M, q_M , cùng với số ghép hoàn hảo:

$$p_M = \sum_i a_{m_{i,0}} a_{m_{i,1}},$$

$$q_M = \sum_{i=1}^r \sum_{j=i+1}^r (a_{m_{i,0}} + a_{m_{i,1}})(a_{m_{j,0}} + a_{m_{j,1}}).$$

Nếu s là tổng mọi trọng số ô, thì có thể quan sát $2(p_M + q_M) + \sum_i a_i^2 = s^2$. Vì vậy chỉ cần tính p_M .

Xét cách tính p_M . Đặt trọng số của cạnh (u, v) là $a_u a_v x + 1$; chỉ cần tính hệ số của x trong tích các trọng số cạnh của ghép. Chú ý rằng nếu đồ thị G có trọng số cạnh, thuật toán FKT cho tổng tích trọng số cạnh trên mọi ghép hoàn hảo. Do đó đặt trọng số cạnh (u, v) là $a_u a_v x + 1$ và tính pfaffian trong vành mô-đun x^2 sẽ cho tổng p_M trên mọi ghép hoàn hảo; gọi kết quả đó là p .

Ở đây ta lại gặp vấn đề dấu. Tất nhiên có thể xử lý dấu như trên, nhưng khá phiền. Trước hết dùng định thức để tính bình phương của pfaffian. Nếu số ghép hoàn hảo là u , thì định thức đầu tiên cho

$$(px + u)^2 \equiv 2pux + u^2 \pmod{x^2}.$$

Dù không biết riêng p và u mang dấu nào, ta vẫn biết được tỉ số

$$\frac{p}{u} = \frac{2pu}{2u^2}.$$

Từ đó đáp án có thể biểu diễn dưới dạng $c_1 \frac{p}{u} + c_2$, trong đó c_1, c_2 là các hằng số dễ tính.

Độ phức tạp thời gian là $O(n^3)$.

6.7 Tổng kết

Đồ thị đối ngẫu là công cụ quan trọng để nghiên cứu đồ thị phẳng. Bài viết đã giới thiệu ngắn gọn quan hệ giữa chu trình và cắt trong đồ thị đối ngẫu, cũng như cách tìm đồ thị đối ngẫu và phương pháp định vị điểm.

Kiểm tra tính phẳng là bài toán then chốt của đồ thị phẳng. Bài viết đưa ra hai thuật toán: thuật toán DMP và thuật toán Tarjan. Hai thuật toán này không chỉ kiểm tra một đồ thị có phẳng hay không, mà với đồ thị phẳng còn dựng được những phẳng cụ thể.

Các tính chất đặc thù của đồ thị phẳng giúp nó có lời giải tốt cho một số bài toán khó trên đồ thị tổng quát. Với bài toán tô màu đỉnh, bài viết đưa ra cách tìm sắc số của đồ thị tam giác hóa. Với bài toán đếm ghép hoàn hảo, bài viết trình bày thuật toán FKT.

Trong đồ thị phẳng còn có nhiều vấn đề khác, chẳng hạn bài toán đẳng cấu đồ thị con, vốn đã có các thuật toán khá hoàn chỉnh trong nghiên cứu học thuật. Hy vọng bài viết đóng vai trò gợi mở, thu hút thêm nhiều bạn đọc nghiên cứu các bài toán và thuật toán liên quan tới đồ thị phẳng.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn huấn luyện viên đội tuyển quốc gia Yang Ziliang đã hướng dẫn; cảm ơn thầy Liu Minbo của Trường Trung học số 1 Pinyi đã quan tâm và chỉ bảo; cảm ơn bạn Chang Ruinian và bạn Dai Jiangqi đã đọc kiểm tra bản thảo; cảm ơn các bạn học khác đã trao đổi thảo luận; cảm ơn cha mẹ đã quan tâm và ủng hộ.

Tài liệu tham khảo

1. Wikipedia, the free encyclopedia. *Planar graph*. https://en.wikipedia.org/wiki/Planar_graph.
2. Lacki, J.; Sankowski, P. *Optimal Decremental Connectivity in Planar Graphs*. Theory Comput. Syst. 61, 1037–1053, 2017. doi: 10.1007/s00224-016-9709-x.
3. Qian Qiao. *Phương pháp xử lý đồ thị phẳng*. Bài giảng NOI WC, 2013.
4. Kohnert, A. *Algorithm of Demoucron, Malgrange, Pertuiset*, 2004.
5. Cairns, S. S. *An Elementary Proof of the Jordan-Schoenflies Theorem*. Proceedings of the American Mathematical Society, 2(6), 860–867, 1951. doi: 10.2307/2031698.
6. Hopcroft, J.; Tarjan, R. *Efficient Planarity Testing*. Journal of the ACM (JACM), 21(4), 549–568, 1974. doi: 10.1145/321850.321852.
7. Wikipedia, the free encyclopedia. *FKT algorithm*. https://en.wikipedia.org/wiki/FKT_algorithm.
8. Robertson, N.; Sanders, D.; Seymour, P.; Thomas, R. *A New Proof of The Four-Colour Theorem*. Electron. Res. Announc. Amer. Math. Soc. 2, 17–25, 1996. doi: 10.1090/S1079-6762-96-00003-0.
9. PERE. *Bàn về bài toán tô màu đỉnh của đồ thị*. Tập luận văn đội tuyển ứng viên quốc gia Trung Quốc IOI 2019, 2019.

10. Liu Cheng'ao. *Lời giải MIN&MAX II*. <https://loj.ac/d/322>.
11. Haber, H. E. *Notes on antisymmetric matrices and the pfaffian*, 2015.
12. AK. *Cách tính đúng Pfaffian của ma trận phản đối xứng*. https://blog.csdn.net/EL_Captain/article/details/111016740.
13. Z%. *Báo cáo lời giải Matching*. Bài tập giai đoạn một đội tuyển IOI 2021, 2020.
14. Galbiati, G.; Maffioli, F. *On the computation of pfaffians*. Discrete Applied Mathematics, 51(3), 269–275, 1994. doi: 10.1016/0166-218X(92)00034-J.

7 Bàn về bài toán tương tác dạng tìm tham số tương tác

Hu Hao, Trường Yali

Tóm tắt

Bài toán tương tác đang dần trở thành một điểm kiểm tra quan trọng trong OI. Các cách ra đề thường gặp gồm tìm tham số ẩn của trình tương tác, trò chơi và một số biến thể khác. Bài viết này tập trung vào lớp bài cần tìm một tham số tương tác ẩn, giới thiệu cách mô hình hóa, ba kiểu lời giải thường dùng và một số hướng tối ưu.

7.1 Cách ra đề cơ bản

Trình tương tác giữ một tham số ẩn. Người giải có thể liên tục gửi truy vấn, nhận câu trả lời từ trình tương tác, rồi cuối cùng phải xuất ra tham số ẩn đó. Nếu bỏ qua chi tiết giao tiếp, quá trình này có thể xem là:

tham số tương tác $\rightarrow$ truy vấn $\rightarrow$ câu trả lời truy vấn $\rightarrow$ chương trình của thí sinh.

Trong lớp bài này, trọng tâm không chỉ là “hỏi gì”, mà còn là tổ chức thông tin đã biết sao cho mỗi truy vấn tiếp theo thật sự dùng được thông tin ấy.

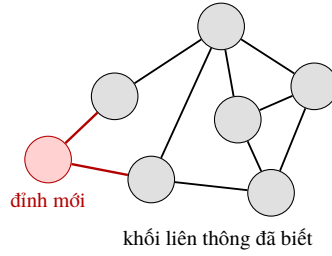
7.2 Tăng dần lượng thông tin đã biết để sử dụng

7.2.1 Tư tưởng

Ở đầu bài toán, ta có thể xem thông tin đã biết là rỗng, hoặc chọn một phần tử làm thông tin ban đầu. Sau đó, mục tiêu của mỗi thao tác là làm cho phần thông tin đã biết và dễ sử dụng ngày càng lớn hơn.

Ví dụ, nếu mục tiêu là tìm một dãy ẩn và đề bảo đảm rằng có thể dùng vài phần tử đầu để truy vấn ra các phần tử phía sau, ta có thể xác định dãy từ trái sang phải. Khi đó thông tin dễ dùng là một tiền tố của dãy. Ngược lại, nếu chỉ biết vài vị trí rời rạc, thông tin ấy thường khó dùng cho các truy vấn sau và không phải là dạng thông tin thuận tiện.

Một ví dụ khác là tìm một đồ thị vô hướng ẩn. Ta có thể chọn một đỉnh làm thành phần liên thông ban đầu, rồi liên tục hỏi để mở rộng thành phần này. Khi đó thông tin dễ dùng là tập đỉnh của một đồ thị con cảm sinh liên thông đã biết. Nếu chỉ biết vài cạnh rời rạc, các truy vấn sau khó tận dụng chúng, thậm chí có thể tìm lại cùng một cạnh nhiều lần.



Hình 4: Minh họa thao tác thêm một đỉnh mới vào thành phần liên thông đã biết, rồi tìm toàn bộ các cạnh nối đỉnh mới với phần đã biết.

7.2.2 Ví dụ 1: Combo

Trình tương tác giữ một xâu S độ dài n trên bảng chữ cái kích thước 4. Cần tìm và xuất ra xâu này. Mỗi lần có thể hỏi một xâu T độ dài nhỏ hơn $4n$; trình tương tác trả về giá trị lớn nhất w sao cho tiền tố độ dài w của S là một xâu con của T . Đặc biệt, ký tự đầu tiên của S chỉ xuất hiện đúng một lần trong S .

Nếu đã biết một tiền tố S' của S , thì khi hỏi $S'a$ và nhận được câu trả lời đúng với độ dài lớn hơn, ta biết $S'a$ là một tiền tố dài hơn của S . Vì vậy tiền tố đã biết chính là thông tin để sử dụng.

Trước hết dùng hai truy vấn để xác định ký tự đầu tiên, sau đó lặp cách trên để mở rộng tiền tố. Tính chất của bài còn cho phép tối ưu: chỉ bằng một truy vấn ghép dạng $S'aS'baS'bbS'bc$ có thể xác định ký tự tiếp theo của S , nên tổng số truy vấn là $n + O(1)$. Do phạm vi áp dụng của mẹo này không rộng, nguồn chỉ nêu tóm tắt.

7.2.3 Ví dụ 1, biến thể

Trình tương tác giữ một xâu S độ dài n trên bảng chữ cái kích thước 2. Mỗi lần hỏi một xâu, trình tương tác trả lời xâu đó có phải là xâu con của S hay không. Cần tìm và xuất ra S .

Nếu dùng phương pháp giống ví dụ trước, ta có thể và cũng chỉ có thể tìm được một hậu tố của S : khi nối thêm 0 hoặc 1 sau hậu tố hiện tại mà cả hai đều trả về sai, ta biết đã chạm tới cuối xâu. Khi đó có thể thêm ký tự vào phía trước hậu tố để khôi phục toàn bộ xâu.

Có thể áp dụng ý tưởng tối ưu của ví dụ trước, nhưng cần cẩn thận với trường hợp đang ở cuối xâu. Nếu chỉ thấy truy vấn $S'0$ trả về sai rồi kết luận ký tự tiếp theo là 1, ta có thể sai vì S' đã là hậu tố cuối cùng. Do đó cứ sau một số bước lại kiểm tra xem hiện tại đã ở cuối xâu hay chưa; nhờ vậy có thể giải trong $n + O(\sqrt{n})$ truy vấn.

7.2.4 Ví dụ 2: Natural Park

Trình tương tác giữ một đồ thị vô hướng. Cần tìm và xuất mọi cạnh của đồ thị. Mỗi lần có thể hỏi hai đỉnh và một tập đỉnh; trình tương tác trả lời hai đỉnh đó có nằm trong cùng một thành phần liên thông của đồ thị con cảm sinh bởi tập đỉnh đã cho hay không.

Ta chọn một đỉnh bất kỳ làm tập ban đầu, rồi liên tục tìm một đỉnh mới kề với tập đã biết. Sau đó tìm tất cả các cạnh nối đỉnh mới này với tập đã biết và thêm nó vào tập. Bản chất của cách làm vẫn là mở rộng một thông tin liên thông để sử dụng.

7.2.5 Ví dụ 3: So Mean

Trình tương tác giữ một hoán vị, trong đó bảo đảm phần tử đầu tiên nhỏ hơn n . Cần tìm toàn bộ hoán vị. Mỗi lần có thể hỏi một số vị trí; trình tương tác trả lời trung bình cộng các giá trị ở những vị trí ấy có

phải số nguyên hay không.

Quan sát quan trọng là khi hỏi một đoạn vị trí liên tiếp quanh vị trí x , câu trả lời cho biết x có phải là giá trị lớn nhất hoặc nhỏ nhất trong một tập ứng viên hay không. Nhờ đó ta có thể lần lượt tìm ra phần tử lớn nhất và nhỏ nhất còn chưa biết.

7.2.6 Ví dụ 4: *Real-time Strategy*

Trình tương tác giữ một cây. Cần tìm và xuất ra mọi cạnh. Mỗi lần hỏi hai đỉnh x, y , trình tương tác trả về đỉnh nằm trên đường đi từ x tới y và cách x đúng một cạnh.

Ta có thể chọn một đỉnh làm gốc. Với mỗi đỉnh còn lại, liên tục hỏi để lần theo đường đi từ gốc tới đỉnh đó, từ đó khôi phục được cấu trúc cây.

7.3 Các hướng tối ưu

Trong nhiều bài, lời giải đơn giản nhất không đủ số truy vấn. Để tìm hướng tối ưu, trước hết cần hiểu sâu hơn thuật toán ban đầu. Qua các ví dụ trên, đa số thuật toán đang lặp lại hai bước:

1. tìm một phần tử chưa biết;
2. tìm quan hệ giữa phần tử ấy và phần thông tin đã biết.

Trong ví dụ đồ thị, ta tìm một đỉnh mới rồi tìm các cạnh nối nó với tập đỉnh đen đã biết. Sau bước đó, toàn bộ thông tin giữa đỉnh mới và phần đã biết đều được xác định. Lặp quá trình này sẽ thu được đáp án cuối cùng.

Thông thường một hoặc cả hai bước có thể được tối ưu. Giống khi tối ưu bài cấu trúc dữ liệu, ta cần xác định nút thắt độ phức tạp rồi dùng các kỹ thuật quen thuộc. Nếu nút thắt nằm ở bước tìm phần tử mới, có thể chia để trị phần chưa xác định theo các tính chất rút ra từ truy vấn trước đó. Nếu nút thắt nằm ở bước tìm quan hệ, có thể chia phần đã biết thành nhiều nhóm, lần lượt kiểm tra nhóm nào có quan hệ với phần tử mới, rồi chỉ giữ lại các nhóm còn liên quan.

7.3.1 Tối ưu ví dụ 2

Trong bài *Natural Park*, bước thứ nhất là tìm một đỉnh kề với tập đã biết, bước thứ hai là tìm mọi cạnh từ đỉnh đó vào tập đã biết. Cả hai bước đều có thể là nút thắt; ở đây xét trước bước thứ hai.

Giả sử đã chọn một đỉnh gốc để thuận tiện tìm quan hệ giữa đỉnh mới và tập đã biết. Ta dùng bổ đề sau: cho ba đỉnh x, y, z , trong đó x, y thuộc tập đã biết S còn $z \notin S$. Nếu trong đồ thị con cảm sinh bởi S ta đi được từ z tới y khi cho phép dùng x , nhưng trong đồ thị con cảm sinh bởi $S \setminus \{x\}$ thì không đi được tới y , thì x và z có cạnh trực tiếp.

Chứng minh rất ngắn. Nếu x và z không có cạnh trực tiếp, xét bước đầu tiên trên đường đi từ z tới y , gọi là r . Vì $S \setminus \{x\}$ liên thông, đỉnh r vẫn có thể đi tới y mà không qua x , mâu thuẫn.

Dựa vào bổ đề này, có thể tìm nhanh một cạnh nối với đỉnh mới bằng cách lấy một cây khung của tập đã biết rồi nhị phân trên thứ tự duyệt trước của cây. Mỗi tiền tố trong thứ tự duyệt trước tạo thành một khối liên thông phù hợp với điều kiện của bổ đề.

7.3.2 Tối ưu ví dụ 3

Trong lời giải của bài *So Mean*, ta gần như không dùng quan hệ giữa các phần tử, mà chỉ tìm phần tử có tính chất lớn nhất hoặc nhỏ nhất trong một tập. Vì vậy bước tìm phần tử là nút thắt.

Sau truy vấn đầu tiên, có thể chia các phần tử thành nhóm lẻ và nhóm chẵn bằng cách hỏi liên quan tới từng phần tử. Tương tự, dựa trên phần dư modulo 4 có thể chia thành bốn nhóm, rồi tiếp tục làm cho kích thước bài toán giảm một nửa sau mỗi tầng. Độ phức tạp thỏa truy hồi

$$T(n) = 2T(n/2) + O(n),$$

nên theo định lý chính ta có $T(n) = O(n \log n)$.

7.3.3 Tối ưu ví dụ 4

Trong lời giải ban đầu cho bài cây, nhiều truy vấn bị lãng phí khi đi trong phần thông tin đã biết. Quá trình của phân rã trọng tâm khớp với dạng truy vấn của bài, nên có thể dùng phân rã trọng tâm để nhanh chóng đi tới lá của cây đã biết. Nhờ đó nút thắt này giảm xuống mức logarithm.

7.4 Thử tìm tiền nhiệm duy nhất

Phương pháp thứ hai nhằm tới từng phần tử riêng lẻ: tìm tiền nhiệm của nó, rồi nối mỗi điểm với tiền nhiệm của nó để thu được một chuỗi. Trên một dãy, tiền nhiệm của một phần tử có thể hiểu là phần tử đứng ngay trước; trên cây, tiền nhiệm có thể hiểu là cha của một đỉnh.

Tựu trung, phương pháp này nhằm tìm một chuỗi, còn phương pháp trước nhằm mở rộng một khối liên thông. Có hai tình huống nên ưu tiên cân nhắc cách này: cần tìm một chuỗi có đầu cuối đã biết nhưng không thể lần ngược trực tiếp từ đầu cuối ấy; hoặc đáp án vốn là một chuỗi và các phương pháp khác xử lý không tốt.

Đôi khi không thể trực tiếp tìm tiền nhiệm $p(x)$ của một điểm x , nhưng có thể tìm một điểm $p^k(x)$ phía trước nó. Nếu thỏa hai điều kiện sau thì vẫn có thể khôi phục chuỗi trong số bước tuyến tính theo độ dài chuỗi:

- nếu tồn tại $p(x)$ thì luôn tìm được một $p^k(x)$ nào đó;
- với mọi $v = p^k(x)$, luôn có thể tìm một điểm nằm giữa v và x .

Nói cách khác, nếu tìm được một điểm phía trước bất kỳ và tìm được một điểm nằm giữa hai điểm đã biết, ta có thể khôi phục cả chuỗi: trước hết tìm $p^k(x)$, sau đó xử lý đoạn giữa $p^k(x)$ và x , rồi tiếp tục xử lý phần phía trước.

7.4.1 Ví dụ 2, tối ưu thứ hai

Với mỗi đỉnh trong bài *Natural Park*, có thể tìm một đường đi từ nó tới thành phần liên thông đã biết, rồi thêm lần lượt các đỉnh trên đường đi đó vào thành phần. Để tìm một điểm nằm giữa, ta lại nhị phân trên một đoạn của cây đã biết.

7.4.2 Ví dụ 5: Integer

Trình tương tác giữ một hoán vị $p : [0, n) \rightarrow [0, n)$ và một số nguyên nội bộ I . Ta có thể hỏi một phép toán $Q(i)$; trình tương tác cộng thêm 2^{p_i} vào I và trả về số bit 1 trong biểu diễn nhị phân của I . Mục tiêu cuối cùng là tìm hoán vị của trình tương tác.

Nếu nối mỗi số với số nhỏ hơn nó đúng 1, bài toán có thể xem là tìm một chuỗi. Hai truy vấn liên tiếp $Q(i), Q(i)$ cho biết bit thứ p_i của I có đang bằng 1 hay không, đồng thời đẩy bit thứ $p_i + 1$ thành 1. Lấy một hoán vị ngẫu nhiên q rồi lần lượt hỏi $Q(q_i), Q(q_i)$; mỗi lần như vậy loại được một tiền nhiệm

sai của một số với xác suất khoảng $1/2$. Sau $C \log n$ vòng, mọi tiền nhiệm sai bị loại với xác suất cao, chỉ còn tiền nhiệm đúng. Nối các quan hệ còn lại sẽ cho chuỗi đáp án.

Cuối cùng cần đưa n bit thấp đầu tiên của I về 0 rồi cộng $2 \cdot 2^0$. Vị trí của số 0 có thể tìm bằng hai lượt hỏi tuần tự

$$Q(0), Q(1), \dots, Q(n-1), Q(0), Q(1), \dots, Q(n-1).$$

7.5 Xử lý khéo trạng thái của trình tương tác

Đôi khi trình tương tác không chỉ có tham số ẩn mà còn có một “thanh ghi” nội bộ. Khi đó hai truy vấn giống hệt nhau liên tiếp có thể trả về hai câu trả lời khác nhau, làm phân tích trở nên khó hơn nhiều.

Cũng có tình huống thanh ghi phụ thuộc vào tham số tương tác, và đề yêu cầu trạng thái cuối của thanh ghi thỏa một điều kiện nào đó. Thường thì yêu cầu này tương đương với việc tìm được tham số tương tác, vì nếu biết tham số ẩn, việc điều khiển thanh ghi sẽ rất đơn giản.

7.5.1 Cố gắng đưa thanh ghi về trạng thái lý tưởng

Ban đầu thông tin trong thanh ghi thường không có quy luật, nhưng sau một số thao tác có thể xuất hiện các tính chất tốt. Khi phân tích giá trị trả về để rút ra các tính chất ấy, ta cũng nên nghĩ cách thao tác sao cho chúng được giữ lại.

Hơn nữa, nếu một phần nội dung của thanh ghi do chính ta ghi vào, việc quy ước rõ mỗi giá trị ghi vào biểu thị trạng thái gì sẽ giúp phân tích các thao tác sau dễ hơn. Có thể nói phần quan trọng nhất của lớp bài này chính là kiểm soát thanh ghi chưa biết.

7.5.2 Ví dụ 6: *Indiana Jones and the Uniform Cave*

Trong một hang động có n phòng. Mỗi phòng có m đường một chiều, đích đến có thể là chính phòng đó hoặc phòng khác. Các lối vào đường một chiều phân bố đều trên tường phòng và không phân biệt được bằng hình dạng. Toàn bộ đồ thị có hướng được bảo đảm liên thông mạnh.

Mỗi phòng có một viên đá; đây là công cụ duy nhất để phân biệt phòng và đường. Ban đầu viên đá nằm ở chính giữa trước một lối đi nào đó. Mỗi khi tới một phòng, ta có thể chuyển viên đá tới trước một lối đi, đặt nó ở bên trái hoặc bên phải lối đi đó, rồi chọn một lối đi để rời phòng. Không được mang đá ra khỏi phòng. Ban đầu ta đứng ở một phòng nào đó; mục tiêu là đi qua mọi cạnh của hang động.

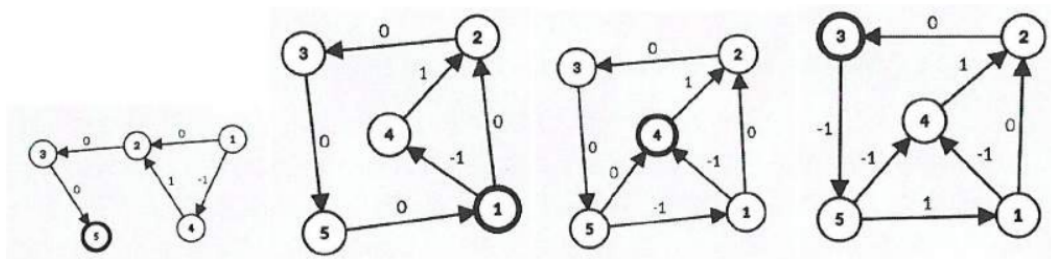
Ở đây thanh ghi chính là vị trí viên đá. Ban đầu mọi viên đá đều ở giữa, một trạng thái rất có quy luật. Ý tưởng tự nhiên là các thao tác sau cũng đặt đá theo một quy luật nhất định để có thể đọc lại nội dung thanh ghi.

Vì viên đá là nguồn thông tin duy nhất, ta có thể xem một cạnh ra của một đỉnh là cạnh mà viên đá đang chỉ tới, và mong muốn sau khi đi ra khỏi một đỉnh ta có thể quay lại đỉnh ấy. Do đó cố gắng để các đỉnh đã biết và cạnh ra do đá chỉ tới tạo thành một rừng hướng vào. Theo đó, vị trí viên đá được phân loại:

- ở giữa: lần đầu đi qua phòng;
- bên trái: cạnh đã xác định;
- bên phải: cạnh chưa xác định chắc chắn đích đến.

Các viên đá đặt bên phải tạo thành một chuỗi, biểu thị những đỉnh chưa tìm hết cạnh ra. Đi theo đá đặt bên trái thì cuối cùng sẽ tới một đỉnh nằm trên chuỗi bên phải. Vì vậy các viên đá có thể đóng vai trò chỉ đường trong phần thông tin đã biết.

Khi xác định thêm một cạnh, ta tiếp tục thử cạnh kế tiếp theo chiều kim đồng hồ. Khi mọi cạnh của một đỉnh đã được đi qua, ta đổi trạng thái viên đá về bên trái và đặt nó sao cho chu trình tạo ra đi qua nhiều đỉnh bên phải nhất. Cuối cùng ta đi tới cuối chuỗi bên phải để tiếp tục mở rộng.



Hình 5: Sơ đồ trạng thái đá cho ví dụ *Indiana Jones and the Uniform Cave*. Đỉnh tô đậm là phòng hiện tại; nhãn 1, 0, -1 lần lượt biểu diễn đá đặt bên trái, bên phải và không đặt đá trên cạnh.

7.5.3 Ví dụ 7: Rotary Laser Lock

Có một ổ khóa xoay laser gồm các nửa vòng đồng tâm đánh số từ 0 tới $n - 1$. Vòng trong cùng là vòng 0, vòng ngoài cùng là vòng $n - 1$. Ổ khóa được chia đều thành nhiều phần; mỗi vòng chứa một cung tròn che phủ đúng một số phần kề nhau. Nhiều vòng khác nhau có thể che cùng một đoạn cung.

Mỗi lần ta có thể xoay một vòng đi một đơn vị phần chia. Trình tương tác trả về số phần của ổ khóa hiện không bị bất kỳ cung tròn nào che phủ. Cần dùng một số lần xoay để tìm và xuất vị trí hiện tại của mọi cung tròn.

Với từng vòng, ta xoay đủ một vòng, ghi nhận giá trị trả về lớn nhất, rồi đưa vòng tới vị trí lớn nhất mà giá trị trả về đạt cực đại. Làm như vậy sẽ bảo đảm mỗi cung tròn đều có một cung tròn khác trùng với nó, đây là một tính chất rất tốt.

Sau đó xem các cung tròn trùng nhau như một đối tượng duy nhất. Vì thao tác trên cả nhóm không phá vỡ tính chất vừa tạo, ta lặp quá trình trên và cuối cùng dừng sau số vòng lặp logarithm.

7.6 Tài liệu tham khảo

Bản dịch tiếng Anh của các đề bài được tham khảo từ:

- <https://loj.ac/p/2398>
- <https://ioihw20.duck-ac.cn/problem/271>
- <https://www.luogu.com.cn/problem/CF1428H>

Lời cảm ơn

Tác giả cảm ơn CCF đã cung cấp cơ hội giao lưu và học tập; cảm ơn thầy Liao Xiaogang đã ủng hộ và hướng dẫn; cảm ơn anh Mao Xiao đã góp ý sửa bài; cảm ơn các bạn Chu Xuanyu và Zhu Tianyi đã cung cấp một phần ví dụ; cảm ơn các thành viên đội tuyển đã cung cấp các bài kiểm tra nội bộ chất lượng cao; và cảm ơn các thầy cô, bạn học khác từng giúp đỡ tác giả.

8 Bàn về thuật toán xấp xỉ trong OI

Xu Tingqiang, Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc

Tóm tắt

Trong nhiều bài OI, thí sinh chỉ cần xuất kết quả có sai số tương đối không vượt quá một ngưỡng ε là được chấp nhận, với các giá trị thường gặp như 10^{-2} , 10^{-4} , 10^{-9} . Bài viết này giới thiệu một số thuật toán xấp xỉ: kết quả của chúng có sai số tương đối hoặc tuyệt đối không vượt quá phạm vi cho trước. Dùng thuật toán xấp xỉ, nhiều bài toán tính giá trị trả về số thực có thể được xử lý tốt; một phần bài toán NP-hard cũng có thể được giải trong thời gian đa thức khi cho phép sai số đủ lớn, thậm chí sai số tùy ý; ngoài ra, một số bài toán vốn đã có thuật toán đa thức chính xác cũng có thể giảm bậc đa thức nếu chấp nhận một sai số nhỏ.

8.1 Lời nói đầu

Trong OI, nhiều bài toán có đáp án là số thực đặt giới hạn sai số tuyệt đối hoặc tương đối cỡ 10^{-9} để tránh lỗi độ chính xác của kiểu `double` trong C++. Trên thực tế, một số bài có thể xấp xỉ đáp án bằng khai triển Taylor, tích phân số và các kỹ thuật tương tự; đổi lại một phần thời gian để đạt đủ độ chính xác, ta có thể thu được độ phức tạp mà cách giải trực tiếp khó đạt tới. Những kỹ thuật này chưa thật sự phổ biến, nên mục 2 sẽ giới thiệu các thuật toán đó.

Mặt khác, với một số bài toán NP, giới nghiên cứu đã nhiều năm tìm cách thu được nghiệm tương đối tốt hơn. Các thuật toán trong bài có thể giải tốt một phần lớp bài này. Những thuật toán ngẫu nhiên khác, chẳng hạn thuật toán điều chỉnh cục bộ, cũng có thể tìm nghiệm khá tốt, nhưng mức tốt của nghiệm không được bảo đảm trên mọi dữ liệu, và do đó tính đúng đắn cũng khó chứng minh. Ngược lại, các thuật toán dưới đây bảo đảm với mọi dữ liệu đều cho đáp án tương đối tốt. Mục 3 trình bày nhóm thuật toán này.

Mục 4 mô tả một số thuật toán xấp xỉ trên đồ thị. Những bài toán này bản thân có thể giải chính xác bằng thuật toán đa thức khác, nhưng xấp xỉ cho phép đánh đổi một chút sai số để giảm mạnh độ phức tạp.

8.2 Xấp xỉ độ chính xác

8.2.1 Tích phân số

Trong nhiều bài, đáp án có thể được biến đổi thành một tích phân số. Tức tồn tại một hàm khả tích $f(x)$ và một khoảng $[A, B]$, đề bài yêu cầu tính

$$\int_A^B f(x) dx.$$

Trong đa số bài, $f(x)$ thường biểu thị độ dài đoạn thẳng, diện tích hình hoặc thông tin hình học tương tự. Vì vậy ở đây mặc định $f(x)$ liên tục, khả vi mọi cấp trừ hữu hạn điểm, và không âm trên $[A, B]$. Đôi khi f khá phức tạp, khó tính trực tiếp nguyên hàm, khi đó cần tính gần đúng.

8.2.2 Xấp xỉ bằng tổng Riemann

Theo định nghĩa tích phân xác định, chọn một số nguyên dương N và chia đều $[A, B]$ thành N đoạn. Với mỗi đoạn, dùng giá trị của f tại trung điểm để xấp xỉ giá trị trung bình trên đoạn ấy, rồi cộng lại:

$$S_N = \frac{B-A}{N} \sum_{i=0}^{N-1} f\left(A + \left(i + \frac{1}{2}\right) \frac{B-A}{N}\right).$$

Khi $N \rightarrow \infty$, S_N tiến tới giá trị tích phân thật. Thuật toán trả giá bằng $O(N)$ lần tính hàm, nên việc ước lượng sai số theo N rất quan trọng.

Với một số hàm thường gặp:

- nếu $f(x) = ax + b$ thì công thức trung điểm cho kết quả chính xác;
- nếu $f(x) = x^2$ thì sai số tương đối là bậc $O(N^{-2})$;
- nếu $f(x) = \cos x$ trên khoảng không vượt qua điểm bất thường của hàm trị tuyệt đối tương ứng, sai số cũng là bậc $O(N^{-2})$.

Định lý 2.1. Nếu f khả vi mọi cấp trên $[A, B]$, sai số tuyệt đối của xấp xỉ Riemann bằng trung điểm trong trường hợp xấu nhất không vượt quá một hằng số nhân với

$$\frac{(B-A)^3}{N^2} \max_{x \in [A, B]} |f'''(x)|.$$

Nguồn chỉ dẫn chứng minh trong tài liệu tham khảo [4].

Ví dụ 1 (Xấp xỉ là đủ). Cho n điểm nguyên trên mặt phẳng. Với mỗi điểm, hãy tính tổng khoảng cách từ nó tới mọi điểm còn lại. Cho phép sai số tương đối $\varepsilon = 10^{-4}$, $n \leq 2 \cdot 10^5$, tọa độ có trị tuyệt đối không vượt quá 10^8 .

Với hai điểm (x_1, y_1) , (x_2, y_2) , khoảng cách giữa chúng có thể xem như

$$\int_0^\pi |(x_2 - x_1) \cos \theta + (y_2 - y_1) \sin \theta| d\theta,$$

tức tích phân của độ dài hình chiếu đoạn thẳng lên mọi hướng. Đổi thứ tự tổng và tích phân, lấy N góc đều nhau $\theta_1, \theta_2, \dots, \theta_N$. Với mỗi góc, chiếu tất cả điểm lên đường thẳng vuông góc tương ứng, rồi tính tổng khoảng cách một chiều từ mỗi điểm tới các điểm còn lại. Sau khi sắp xếp các hình chiếu, bài toán một chiều này xử lý được tuyến tính.

Trên các khoảng không chứa điểm không khả vi của $|\cos \theta|$, phân tích ở trên cho sai số $O(N^{-2})$. Gần các điểm $\cos \theta = 0$, phân tích phân trong một đoạn ngắn cũng chỉ gây sai số bậc $O(N^{-2})$. Vì vậy chọn $N = O(\varepsilon^{-1/2})$ là đủ. Độ phức tạp thời gian là $O(n\varepsilon^{-1/2} \log n)$.

8.2.3 Tích phân Simpson

Ta thử dùng hàm bậc hai để xấp xỉ hàm dưới dấu tích phân. Khi $g(x) = ax^2 + bx + c$, tích phân trên $[A, B]$ được xác định bởi các hệ số bậc hai. Quy tắc Simpson dùng ba điểm

$$(A, f(A)), \quad \left(\frac{A+B}{2}, f\left(\frac{A+B}{2}\right) \right), \quad (B, f(B))$$

để quyết định một đa thức bậc hai, rồi dùng tích phân của đa thức đó xấp xỉ tích phân của f .

Định nghĩa 2.1. Tích phân Simpson của f trên $[A, B]$ là

$$\text{Simpson}(A, B) = \frac{B-A}{6} \left(f(A) + 4f\left(\frac{A+B}{2}\right) + f(B) \right).$$

Hiệu quả xấp xỉ của Simpson tốt hơn công thức trung điểm.

Định lý 2.2. Chia $[A, B]$ thành N đoạn đều nhau và dùng Simpson trên từng đoạn. Nếu f khả vi đủ cấp trên $[A, B]$, sai số tuyệt đối không vượt quá một hằng số nhân với

$$\frac{(B - A)^5}{N^4} \max_{x \in [A, B]} |f^{(4)}(x)|.$$

Chứng minh xem tài liệu tham khảo [4].

8.2.4 Tích phân Simpson thích nghi

Khi $f(x)$ có đoạn khác quá xa hàm bậc hai, chẳng hạn gần điểm không khả vi, sai số Simpson tại vùng đó tăng mạnh. Nếu thu nhỏ mọi đoạn thì chi phí quá lớn, nên ta dùng Simpson thích nghi: nếu ước lượng sai số trên một đoạn đủ nhỏ thì trả luôn giá trị Simpson, ngược lại chia đôi đoạn và đệ quy.

1. Đặt $mid = (A + B)/2$.

2. Nếu

$$|\text{Simpson}(A, mid) + \text{Simpson}(mid, B) - \text{Simpson}(A, B)| < \varepsilon,$$

trả về $\text{Simpson}(A, B)$.

3. Ngược lại, trả về tổng kết quả đệ quy trên $[A, mid]$ và $[mid, B]$.

Simpson thích nghi giải được nhiều bài hình học tính toán trong OI và thường có hiệu năng tốt.

8.2.5 Khai triển Taylor

Với một hàm giải tích thực f , xét khai triển Taylor quanh x_0 :

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \dots + \frac{f^{(n)}(\xi)}{n!}(x - x_0)^n,$$

trong đó ξ nằm giữa x và x_0 .

Bổ đề 2.1. Nếu tồn tại $M > 0$ sao cho với mọi $n > 0$,

$$\left| \frac{f^{(n)}(x_0)}{n!} (x - x_0)^n \right| \leq M 2^{-n},$$

thì chỉ cần tính $O(\log \varepsilon^{-1})$ hạng đầu của khai triển Taylor để sai số tuyệt đối không vượt quá ε .

Chứng minh. Vì f giải tích thực, chuỗi Taylor hội tụ tới $f(x)$. Lấy $N_0 = \lceil \log \varepsilon^{-1} + \log M \rceil + 1$, tổng trị tuyệt đối của các hạng sau N_0 không vượt quá một cấp số nhân nhỏ hơn ε . ■

Một hệ quả thường dùng hơn là: nếu các đạo hàm bị chặn bởi M và $|x - x_0|$ đủ nhỏ, thì cũng chỉ cần giữ $O(\log \varepsilon^{-1})$ hạng. Lợi ích của Taylor là biến một hàm phức tạp thành một đa thức bậc thấp; đa thức có cấu trúc tốt và dễ cộng gộp.

Ví dụ 2 (Lấy suất). Cho dãy số nguyên dương $a_1, a_2, \dots, a_n$ và q truy vấn trên đoạn. Mỗi truy vấn yêu cầu tính một tích/hàm của các a_i trong đoạn, cho phép sai số $\varepsilon = 10^{-9}$; ràng buộc đọc được từ bản quét là $n, q \leq 6 \cdot 10^5$ và các giá trị nằm trong phạm vi đa thức lớn.

Lời giải chia các phần tử thành hai loại. Với các hạng quá lớn so với tham số truy vấn, đóng góp của chúng đã nhỏ hơn ε nên có thể xử lý riêng hoặc bỏ qua an toàn. Với phần còn lại, viết biểu thức cần tính dưới dạng

$$\exp\left(\sum_i \log(1 - u_i)\right),$$

rồi khai triển

$$\log(1 - u) = -u - \frac{u^2}{2} - \frac{u^3}{3} - \dots$$

Do $|u_i|$ đủ nhỏ, theo bổ đề chỉ cần giữ $O(\log \varepsilon^{-1})$ hạng. Mỗi $\log(1 - u_i)$ trở thành một đa thức bậc thấp theo tham số truy vấn; các hạng cùng bậc có thể cộng gộp bằng tiên xử lý tổng tiên tố. Nhờ vậy truy vấn được trả lời nhanh hơn nhiều so với tính trực tiếp từng phần tử.

Ví dụ 3 (Game of Chance). Sau một số biến đổi, bài toán cần tính tổng các giá trị của một hàm phân thức dạng $f(x)$ trên nhiều cặp số dương (a_i, b_i) , với $n \leq 3 \cdot 10^5$ và sai số 10^{-9} .

Nếu $a_i < b_i$, đặt $x = a_i/b_i < 1$ và khai triển quanh một điểm cố định như $x_0 = 1/2$. Giữ $O(\log \varepsilon^{-1})$ hạng rồi dùng nhị thức Newton, ta thu được một đa thức bậc thấp theo x . Khi cộng trên mọi $a_i < b_i$, chỉ cần biết các tổng lũy thừa a_i^k và b_i^{-k} tương ứng.

Nếu $a_i > b_i$, khai triển trực tiếp không hội tụ tốt. Khi đó đổi góc nhìn, dùng $b_i/a_i < 1$ và khai triển hàm tương đương theo biến nghịch đảo. Như vậy bài toán được tách thành hai miền $0 < x < 1$ và $x > 1$, mỗi miền dùng một khai triển Taylor hội tụ. Đây là kỹ thuật rất thường gặp khi phải lấy tổng các hàm phân thức: chia miền để bảo đảm hội tụ, rồi trả thêm hệ số $O(\log \varepsilon^{-1})$ cho bậc đa thức.

8.3 Nghiệm xấp xỉ của một số bài toán NP

Nhiều bài toán NP có thuật toán xấp xỉ, chẳng hạn ba lô và tập độc lập lớn nhất. Cũng có những bài toán NP đã được chứng minh không tồn tại thuật toán xấp xỉ hiệu quả theo nghĩa thường dùng, ví dụ đếm số nghiệm 2-SAT. Dưới đây là một số bài có thể xấp xỉ.

Ta dùng thuật ngữ $(1 + \varepsilon)$ -xấp xỉ để chỉ nghiệm có sai số tương đối không vượt quá ε .

8.3.1 Bài toán ba lô

Xét bài toán ba lô 0–1 kinh điển. Có n vật phẩm, vật phẩm thứ i có thể tích w_i và giá trị v_i . Ba lô có dung lượng W ; cần chọn các vật phẩm có tổng thể tích không vượt quá W và tổng giá trị lớn nhất. Giả sử $w_i \leq W$.

Bài toán này là NP-hard, nên không thể kỳ vọng một thuật toán đa thức theo kích thước đầu vào để giải chính xác trong mọi trường hợp. Tuy nhiên, với mọi $\varepsilon > 0$, ta có thuật toán đa thức cho nghiệm $(1 + \varepsilon)$ -xấp xỉ.

Gọi V là giá trị lớn nhất của một vật phẩm. Thay mỗi giá trị v_i bằng

$$v'_i = \left\lfloor \frac{nv_i}{\varepsilon V} \right\rfloor.$$

Sau đó chạy DP theo tổng giá trị đã làm tròn: $dp[j]$ là tổng thể tích nhỏ nhất để đạt tổng giá trị làm tròn j . Vì tổng v'_i không vượt quá $O(n^2/\varepsilon)$, độ phức tạp là $O(n^3/\varepsilon)$ hoặc tốt hơn với cài đặt chuẩn.

Bổ đề 3.1. Lấy phương án có $dp[j] \leq W$ và j lớn nhất. Phương án tương ứng có sai số tương đối không vượt quá ε so với tối ưu.

Ý chứng minh. Mỗi vật phẩm mất nhiều nhất $\varepsilon V/n$ giá trị khi làm tròn; một phương án chọn nhiều nhất n vật phẩm nên mất không quá εV . Vì nghiệm tối ưu có giá trị ít nhất V , sai số tương đối không vượt quá ε . ■

Trên thực tế, FPTAS tốt nhất cho ba lô 0–1 hiện có độ phức tạp thấp hơn nhiều so với cận đơn giản trên; xem tài liệu [3].

8.3.2 Lấy mẫu ngẫu nhiên

Trong một số bài NP, có thể dùng lấy mẫu ngẫu nhiên để ước lượng một đại lượng. Mô hình cơ bản sau cho thấy mức tốt của phương pháp này.

Định lý 3.1. Cho một bộ sinh ngẫu nhiên trả về số trong $[A, B]$ với kỳ vọng E_0 và phương sai bị chặn bởi σ^2 . Lấy mẫu độc lập N lần và lấy trung bình. Sai số tuyệt đối kỳ vọng so với E_0 là $O(\sigma N^{-1/2})$.

Ý chứng minh. Nếu các kết quả là $a_1, \dots, a_N$, trung bình là $\bar{a} = \frac{1}{N} \sum_i a_i$. Phương sai của $\bar{a}$ bằng σ^2/N , nên sai số tuyệt đối kỳ vọng bị chặn bởi bậc căn của đại lượng này. ■

Trong thi thuật toán, không chỉ cần kỳ vọng nhỏ mà còn cần xác suất sai lớn là rất nhỏ. Dùng bất đẳng thức Hoeffding, ta có:

Định lý 3.2. Với giả thiết kết quả luôn nằm trong $[A, B]$, sau N lần lấy mẫu, sai số tuyệt đối của trung bình so với kỳ vọng gần như chắc chắn không vượt quá $O((B - A)N^{-1/2})$, trong đó xác suất thất bại giảm theo hàm mũ của hằng số ẩn.

Do đó nếu muốn sai số tuyệt đối không vượt quá ε , chỉ cần lấy $N = O((B - A)^2 \varepsilon^{-2})$ mẫu, bỏ qua các hằng số xác suất.

Ví dụ 5 (Number of Close Strings). Cho k xâu nhị phân độ dài n và số nguyên dương m . Hỏi có bao nhiêu xâu nhị phân độ dài n có khoảng cách Hamming không quá m tới ít nhất một xâu đã cho. Cho phép sai số tương đối 10^{-7} , với $1 \leq n, k \leq 50$.

Gọi S_i là tập các xâu cách xâu thứ i không quá m , và cần tính

$$\left| \bigcup_i S_i \right|.$$

Mọi $|S_i|$ đều biết được bằng công thức tổ hợp. Xét một dãy không lờ trong đó mỗi phần tử của S_i được liệt kê theo từng i ; với một xâu x , nếu nó xuất hiện $c(x)$ lần thì đóng góp $1/c(x)$ trong dãy này. Kỳ vọng của biến ngẫu nhiên $1/c(x)$, khi chọn đều một phần tử từ dãy liệt kê, nhân với tổng độ dài dãy, chính là kích thước hợp cần tìm.

Vì có thể lấy mẫu đều một phần tử từ một S_i đã chọn và tính $c(x)$ bằng cách so sánh với các xâu ban đầu, ta dùng lấy mẫu ngẫu nhiên để ước lượng kỳ vọng. Theo định lý trên, lấy $O(k^2 \varepsilon^{-2})$ mẫu là đủ để khống chế sai số tương đối trong phạm vi yêu cầu, với hằng số nhỏ trong thực tế.

8.4 Một số thuật toán xấp xỉ trên đồ thị

Thuật toán xấp xỉ có phạm vi ứng dụng rộng. Ngay cả với bài toán có thuật toán đa thức chính xác, nếu cho phép một phần sai số tương đối hoặc tuyệt đối, ta vẫn có thể đạt độ phức tạp tốt hơn.

Ví dụ 6 (AIK). Cho một DAG $G = (V, E)$. Với mỗi đỉnh, cần tính số đỉnh mà nó tới được. Cho phép sai số tương đối ε . Ràng buộc đọc được: $|V| \leq 10^5, |E| \leq 10^6$.

Trước hết dùng một bộ đề xác suất.

Bổ đề 4.1. Với k biến ngẫu nhiên độc lập phân bố đều trên $[0, 1]$, kỳ vọng của giá trị nhỏ nhất là $\frac{1}{k+1}$.

Vì vậy, mỗi lần ta gán cho mỗi đỉnh một trọng số ngẫu nhiên đều trong $[0, 1]$, rồi trên DAG tính với mỗi đỉnh trọng số nhỏ nhất trong tập các đỉnh nó tới được. Nếu đáp án thật của đỉnh i là ans_i , kỳ vọng của giá trị nhỏ nhất là xấp xỉ $\frac{1}{\text{ans}_i + 1}$. Sau N lần lấy mẫu, lấy trung bình các giá trị nhỏ nhất, rồi nghịch đảo và trừ 1 để ước lượng ans_i .

Phân tích phương sai trong nguồn cho thấy sai số tương đối gần như chắc chắn là $O(N^{-1/2})$, nên chọn $N = O(\varepsilon^{-2})$ là đủ. Độ phức tạp thời gian là

$$O((|V| + |E|)\varepsilon^{-2}).$$

Định nghĩa 4.1. Với một bài toán có đáp án chính xác là số nguyên dương x , nếu chỉ cần tìm y thỏa $x \leq y \leq x + t$, ta gọi đó là xấp xỉ *surplus- t* .

Ví dụ 7. Cho đồ thị vô hướng đơn không trọng số $G = (V, E)$. Hãy tìm xấp xỉ surplus-2 của độ dài chu trình đơn ngắn nhất. Giả sử $|V| \leq 8000$.

Cố định một đỉnh v , chạy BFS từ v , tìm khoảng cách ngắn nhất tới mọi đỉnh và đánh dấu đường ngắn nhất có duy nhất hay không. Duy trì t , ban đầu bằng n :

1. nếu hai đỉnh kề nhau có cùng khoảng cách d tới v , cập nhật $t = \min(t, 2d + 1)$;
2. nếu một đỉnh có nhiều hơn một đường ngắn nhất tới v và khoảng cách là d , cập nhật $t = \min(t, 2d)$.

Bổ đề 4.3. Giá trị t cuối cùng không nhỏ hơn độ dài chu trình đơn ngắn nhất; nếu chu trình ngắn nhất đi qua v hoặc một đỉnh kề v , thì $t \leq g + 2$, trong đó g là độ dài chu trình ngắn nhất.

Ý tưởng chứng minh là: hai trường hợp cập nhật ở trên đều tạo ra một chu trình không suy biến, nên trong đồ thị tồn tại một chu trình đơn không dài hơn giá trị cập nhật. Ngược lại, nếu chu trình ngắn nhất đi qua v hoặc lân cận của v , xét các đỉnh đối diện trên chu trình theo chiều lẻ độ dài, BFS chắc chắn phát hiện một trong hai trường hợp với sai số không quá 2.

Nếu chu trình ngắn nhất đi qua v hoặc lân cận của v , ta xuất t . Nếu không, xóa v cùng lân cận của nó rồi chọn đỉnh mới, lặp lại quá trình. Chọn v là đỉnh bậc lớn nhất hiện tại thì tổng thời gian là $O(|V|^2)$.

Ví dụ 8. Vẫn với đồ thị vô hướng đơn không trọng số, yêu cầu xấp xỉ surplus-1 của độ dài chu trình đơn ngắn nhất.

Ta cải tiến việc tìm chu trình đi qua một đỉnh v . Chạy BFS từng lớp, nhưng mỗi cạnh sau khi được xét sẽ bị xóa khỏi tập cạnh đang xét. Khi đang duyệt cạnh (u, x) , nếu x chưa thăm thì đánh dấu và đưa vào lớp tiếp theo; nếu x đã thăm thì trả về

$$\text{dist}(v, u) + \text{dist}(v, x) + 1.$$

Mỗi đỉnh được đi qua nhiều nhất một lần trước khi trả về, nên chi phí cho một đỉnh gốc là $O(|V|)$. Nếu chu trình ngắn nhất đi qua v , thuật toán trả về xấp xỉ surplus-1; nếu độ dài chu trình ngắn nhất là chẵn, nó còn trả về đúng giá trị chính xác. Chạy với mọi v cho độ phức tạp $O(|V|^2)$.

Ví dụ 9 (Ước lượng là đủ). Cho đồ thị vô hướng đơn không trọng số $G = (V, E)$ và q truy vấn khoảng cách giữa hai đỉnh. Cần trả về khoảng cách ngắn nhất surplus-2 cho mỗi truy vấn, với $|V| \leq 8000$, $q \leq 10^5$.

Dùng kỹ thuật của ví dụ 7: chọn đỉnh bậc lớn nhất v , chạy BFS từ v để có khoảng cách tới mọi đỉnh. Nếu đường đi ngắn nhất giữa x, y đi qua v hoặc lân cận của v , thì $d_v(x) + d_v(y)$ là xấp xỉ surplus-2. Xóa v và lân cận của nó rồi lặp lại. Với mỗi truy vấn, lấy giá trị nhỏ nhất trong các tổng khoảng cách thu được.

Nếu làm tới khi xóa hết đỉnh, số mức có thể lớn. Ta chỉ chọn B mức đầu tiên, rồi xét đồ thị còn lại G' . Vì bậc của mọi đỉnh còn lại không vượt quá khoảng $|E|/B$, có thể chạy BFS từ mọi đỉnh trong G' với tổng chi phí nhỏ hơn. Cân bằng tham số B cho độ phức tạp tổng quát cỡ $O(|V||E|^{1/2}q^{1/2})$ trong phân tích của nguồn.

Ví dụ 10. Cho đồ thị vô hướng đơn không trọng số $G = (V, E)$, cần trả lời xấp xỉ surplus-2 cho mọi cặp khoảng cách ngắn nhất.

Nếu áp dụng trực tiếp ví dụ 9 cho mọi truy vấn, độ phức tạp còn cao. Nguồn giới thiệu một thuật toán xấp xỉ mọi cặp với ý tưởng sau. Đặt $n = |V|$, $m = |E|$, chọn ngưỡng s và một số đỉnh mốc T . Trước hết lấy tập S gồm các đỉnh bậc lớn, rồi nhiều lần chọn một đỉnh che phủ nhiều đỉnh nhất trong S (một đỉnh che phủ chính nó và các láng giềng), đưa nó vào T và xóa các đỉnh được che phủ khỏi S .

Sau đó xây dựng tập cạnh thừa E' gồm:

- các cạnh có ít nhất một đầu bậc nhỏ;
- các cạnh liên quan tới phần đỉnh bậc lớn chưa xử lý;
- các cạnh nối đỉnh được che phủ tới mức che phủ nó.

Chạy BFS từ mỗi mức trên đồ thị gốc để biết khoảng cách từ mức tới mọi đỉnh. Cuối cùng, với mỗi đỉnh nguồn v , chạy BFS trên E' và thêm các cạnh có trọng số từ v tới mọi mức $u \in T$ với trọng số $\text{dist}(v, u)$. Kết quả thu được là khoảng cách surplus-2 từ v tới mọi đỉnh.

Phân tích của nguồn chứng minh hai điểm:

- đường đi ngắn nhất nào đi qua một đỉnh bậc lớn đã được che phủ đều có thể thay bằng đường qua mức che phủ với sai số không quá 2;
- số cạnh trong E' là $O(ns)$, nên tổng chi phí BFS trên đồ thị thừa giảm đáng kể.

Kết hợp tham số phù hợp, nguồn thu được thuật toán mọi cặp surplus-2 có độ phức tạp cỡ $O(n^{2.5})$ theo cách trình bày dành cho OI. Thuật toán tương đối dễ cài đặt và có hằng số tốt.

8.5 Tổng kết

Thuật toán xấp xỉ có triển vọng ứng dụng rộng trong OI. Từ Simpson thích nghi đã được dùng khá phổ biến, tới các thuật toán xấp xỉ cho bài NP hoặc bài đồ thị còn ít được phổ biến, chúng đều có thể xuất hiện trong thi thuật toán. Với người giải, lớp bài này giống quy hoạch động hoặc tham lam ở chỗ khó có một khuôn mẫu chung, đòi hỏi kỹ năng tổ hợp và tư duy thiết kế.

Một nhược điểm của bài xấp xỉ là dữ liệu kiểm thử không dễ xây dựng. Để chấm một lời giải, nhiều khi cần biết đáp án chính xác, trong khi chính đáp án chính xác lại khó tính hơn. Nếu yêu cầu độ chính xác rất cao, có thể chấp nhận rủi ro rằng một số đáp án sai số hơi lớn hơn ngưỡng vẫn được chấm đúng, đổi lại bảo đảm thuật toán chuẩn chạy được.

Các thuật toán trong bài chỉ là phần rất nhỏ của lĩnh vực xấp xỉ. Còn rất nhiều bài toán khi cho phép sai số có thể được xử lý bằng các thuật toán khác nhau. Ngay cả trong học thuật, thuật toán xấp xỉ vẫn là

một hướng rộng và chưa được khai phá hết; có thể trong tương lai độc giả sẽ đặt tên cho một thuật toán xấp xỉ mới.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Ye Jinyi của Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc đã quan tâm và hướng dẫn; cảm ơn gia đình và bạn bè đã ủng hộ, động viên; cảm ơn anh Deng Mingyang, bạn Zhou Hangrui, bạn Li Baitian và bạn Dai Jiangqi đã thảo luận và gợi mở; cảm ơn bạn Zhou Hangrui, bạn Chen Yusi, bạn Yang Juntian và bạn Wu Hang đã đọc kiểm tra bản thảo.

Tài liệu tham khảo

1. Dor, D.; Halperin, S.; Zwick, U. *All-pairs almost shortest paths*. SIAM Journal on Computing, 29(5):1740–1759, 2000.
2. Itai, A.; Rodeh, M. *Finding a minimum circuit in a graph*. SIAM Journal on Computing, 7(4), 413–423, 1978.
3. Jin, C. *An Improved FPTAS for 0-1 Knapsack*. ICALP 2019, Leibniz International Proceedings in Informatics 132, 76:1–76:14.
4. Fazekas Jr., E. C.; Mercer, P. R. *Elementary proofs of error estimates for the midpoint and Simpson's rules*. Mathematics Magazine, 82(5):365–370, 2009.
5. Wikipedia contributors. *Approximation algorithm*, 2021.
6. Wikipedia contributors. *Knapsack problem*, 2021.
7. Wikipedia contributors. *Hoeffding's inequality*, 2022.

9 Khảo sát bước đầu về bài toán tô màu đồ thị

Peng Bo, Trường Trung học trực thuộc Đại học Quảng Châu

Tóm tắt

Bài toán tô màu đồ thị là một chủ đề vừa quan trọng vừa thú vị trong khoa học máy tính. Bài viết này giới thiệu một số biến thể tô màu khác nhau, chia thành hai nhóm offline và online, đồng thời trình bày vài lời giải tương đối tốt cho các biến thể đó.

9.1 Mở đầu

Tìm số màu tối thiểu của một đồ thị tổng quát là bài toán NP-đầy đủ; ngay cả bài toán kiểm tra đồ thị có tô được bằng ba màu hay không cũng là NP-đầy đủ. Vì vậy, nhiều nghiên cứu chuyển sang tìm nghiệm gần tối ưu: trong thời gian đa thức, cố gắng giảm số màu phải dùng càng nhiều càng tốt.

Bài viết này khảo sát những kết quả tốt có thể đạt được trên một số lớp đồ thị đặc biệt, chia thành hai hướng. Trong bài toán offline, toàn bộ thông tin của đồ thị được biết ngay từ đầu. Trong bài toán online, mỗi lần chỉ nhận thêm một đỉnh, thuật toán phải tô màu ngay và không được sửa màu đã gán. Cả hai hướng đều có nhiều kết quả đáng chú ý.

9.2 Ký hiệu và định nghĩa

Các bài toán tô màu đồ thị xét việc tô màu các đối tượng của đồ thị, chẳng hạn đỉnh hoặc cạnh, dưới một số ràng buộc như hai đỉnh kề nhau không được cùng màu. Mô hình cổ điển nhất là tô màu đỉnh, nhưng còn có các mô hình khác. Nếu không nói thêm, đối tượng nghiên cứu là đồ thị vô hướng đơn $G = (V, E)$, đặt $n = |V|$, các đỉnh được đánh số $1, 2, \dots, n$.

Với đỉnh v , ký hiệu

$$N(v) = \{u \mid (u, v) \in E\}, \quad d(v) = |N(v)|.$$

Với tập đỉnh S , đặt

$$D(S) = \sum_{v \in S} d(v), \quad N(S) = \bigcup_{v \in S} N(v).$$

Cần chú ý rằng nói chung $D(S)$ không bằng $|N(S)|$.

Bài toán tô màu đồ thị offline yêu cầu xây dựng một ánh xạ $f : V \rightarrow \mathbb{Z}_{>0}$ sao cho với mọi cạnh $(u, v) \in E$ đều có $f(u) \neq f(v)$, đồng thời tối thiểu hóa $\max_{v \in V} f(v)$.

Bài toán tô màu đồ thị online duy trì một đồ thị vô hướng đơn, ban đầu không có đỉnh và cạnh. Ở lượt thứ i , hệ thống thêm đỉnh số i và các cạnh nối nó với các đỉnh đã xuất hiện trước đó. Thuật toán phải lập tức chọn màu $f(i)$; màu này không được thay đổi về sau. Điều kiện vẫn là hai đầu của mọi cạnh phải khác màu, và mục tiêu là giảm số màu lớn nhất đã dùng. Nếu không nói thêm, ta cho phép bộ sinh tương tác dựa vào các đầu ra trước đó của thuật toán để quyết định lượt tiếp theo, nhưng tổng số đỉnh là đã biết.

9.3 Bài toán tô màu offline

Trên một số lớp đồ thị rất đặc biệt, ví dụ đồ thị dây cung, tồn tại thuật toán đa thức tìm được nghiệm tối ưu. Bài viết không đi sâu vào các trường hợp này, mà tập trung vào vài kết quả có tính đại diện.

9.3.1 Tô màu cạnh

Trong bài toán tô màu cạnh, ta xây dựng ánh xạ $f : E \rightarrow \mathbb{Z}_{>0}$ sao cho hai cạnh kề nhau tại cùng một đỉnh phải nhận hai màu khác nhau, rồi tối thiểu hóa số màu dùng. Vì với mỗi đỉnh v , có $d(v)$ cạnh kề tại v thì một không được cùng màu, cần dưới hiển nhiên là $\Delta = \max_{v \in V} d(v)$ màu.

Định lý Vizing. Số màu cạnh tối thiểu của một đồ thị đơn bất kỳ chỉ có thể là Δ hoặc $\Delta + 1$.

Ý chứng minh là quy nạp theo số cạnh. Bắt đầu từ đồ thị rỗng, mỗi lần thêm một cạnh (v, w_1) và tô nó bằng một trong $\Delta + 1$ màu. Ở mỗi đỉnh luôn có ít nhất một màu chưa xuất hiện trên các cạnh kề. Gọi màu trống tại v là a , còn màu trống tại w_1 là b_1 . Nếu tại w_1 cũng trống màu a thì tô ngay cạnh mới bằng a .

Nếu không, xét đường đi luân phiên hai màu a/b_1 bắt đầu từ w_1 . Nếu đường đi này không kết thúc ở v , đổi chỗ hai màu a, b_1 trên đường đi thì w_1 sẽ trống màu a , rồi tô cạnh mới bằng a . Nếu đường đi quay về v , gọi đỉnh ngay trước v trên đường đó là w_2 ; khi đó cạnh (v, w_2) đang có màu b_1 . Ta xóa tạm cạnh (v, w_2) , tô (v, w_1) bằng b_1 , rồi tiếp tục xử lý cạnh (v, w_2) . Lặp lại quá trình này với $w_2, w_3, \dots$. Vì số màu hữu hạn, sẽ đến lúc gặp lại một màu đã xuất hiện; dùng lập luận đường đi luân phiên khi đó cho phép đổi màu và tô thành công cạnh còn lại. Đây chính là phép đổi Kempe kinh điển trong chứng minh định lý Vizing.

9.3.2 Tô màu đồ thị phẳng

Một đồ thị vô hướng đơn $G = (V, E)$ là đồ thị phẳng nếu có thể vẽ nó trên mặt phẳng sao cho các cạnh không cắt nhau. Định lý bốn màu nói rằng mọi đồ thị phẳng đều tô được bằng bốn màu, nhưng chứng

2. tìm tập V_2 có đồ thị cảm sinh hai phía và $|N(V_2)| = O(f(n)|V_2|)$;
3. tìm hai đỉnh u, v bắt buộc cùng màu trong mọi cách tô ba màu hợp lệ.

Tiến triển thứ nhất trực tiếp cho phép xóa nhiều đỉnh bằng hai màu. Tiến triển thứ ba cho phép nhập hai đỉnh mà không làm mất nghiệm hợp lệ. Với tiến triển thứ hai, ta có thể xóa $V_2 \cup N(V_2)$; khi ghép các tập V_2 qua nhiều bước sẽ thu được một tiến triển kiểu thứ nhất.

Vì vậy, chỉ cần có một thuật toán đa thức mà với mọi đồ thị ba phía đều tìm ra một trong ba tiến triển trên, ta thu được thuật toán đa thức dùng $O(f(n))$ màu. Trong lập luận ghép, nếu thuật toán con trên một đồ thị con trả về kiểu thứ ba, thì hai đỉnh đó cũng bắt buộc cùng màu trong toàn đồ thị ban đầu. Nếu chỉ nhận kiểu một và kiểu hai, ta lặp các kiểu hai cho tới khi hoặc gặp kiểu một, hoặc tập đỉnh còn lại giảm một nửa; khi đó hợp các tập kiểu hai tạo thành một tập lớn có đồ thị cảm sinh hai phía.

Các tính chất được khai thác. Nếu tồn tại đỉnh v có $d(v) = O(f(n))$, thì $\{v\}$ cho một tiến triển kiểu hai. Vì thế có thể giả sử mọi đỉnh đều có bậc $\Omega(f(n))$.

Với hai đỉnh u, v , đặt $S = N(u) \cap N(v)$. Nếu $|S| = \Omega(n/f(n)^2)$, xét $N(S)$. Nếu $N(S)$ là đồ thị hai phía, thì tùy theo $|N(S)|$ lớn hay nhỏ ta thu được tiến triển kiểu một hoặc kiểu hai. Nếu $N(S)$ không hai phía, từ cấu trúc ba phía suy ra u, v phải cùng màu trong mọi cách tô ba màu hợp lệ, tức là tiến triển kiểu ba. Do đó có thể tiếp tục giả sử mọi cặp đỉnh có số láng giềng chung không vượt quá $O(n/f(n)^2)$.

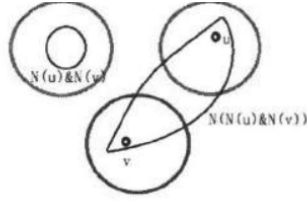


图 2: uv 不同色

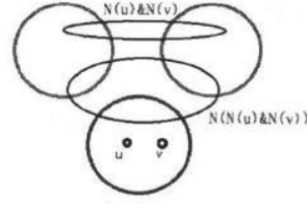


图 3: uv 同色

Hình 7: Hai trường hợp khi xét $S = N(u) \cap N(v)$, tương ứng u, v khác màu và cùng màu.

Trong đồ thị ba phía ngẫu nhiên, nếu lấy một đỉnh đỏ v , thì $N(v)$ gần như phân bố đều trong hai màu còn lại, và tập láng giềng bậc hai $N(N(v))$ chứa khoảng một nửa đỉnh đỏ. Dùng thuật toán xấp xỉ phủ đỉnh Bar-Yehuda–Even / Monien–Speckenmeyer, nếu tập độc lập lớn chiếm ít nhất một nửa thì ta tìm được một tập độc lập đáng kể trong thời gian đa thức.

Khó khăn trong trường hợp xấu nhất nằm ở độ không đều của bậc. Tác giả đưa vào các lớp bậc mịn. Đặt

$$\delta = \frac{1}{5 \log n}, \quad I_i = \{v \in V \mid d(v) \in [(1 + \delta)^i, (1 + \delta)^{i+1}]\},$$

và với tập S ,

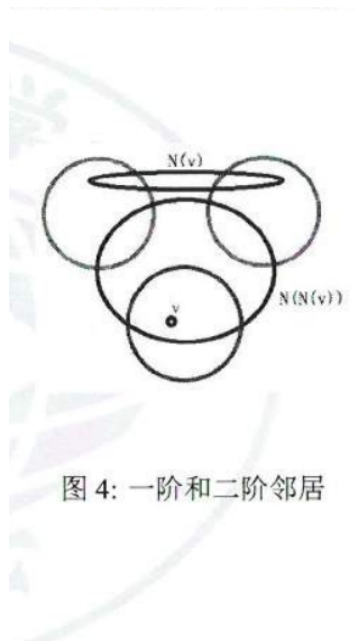
$$N_i(S) = \{v \in N(S) \mid d_S(v) \in [(1 + \delta)^i, (1 + \delta)^{i+1}]\}.$$

Các lớp này làm cho tổng bậc và số đỉnh có thể chuyển đổi cho nhau với sai số nhỏ.

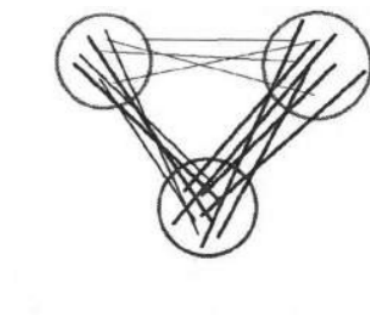
Hai định lý kỹ thuật. Định lý thứ nhất nói rằng nếu một đồ thị có bậc nhỏ nhất d , có tập độc lập R , và $D_R(V \setminus R) \geq AD(V \setminus R)$ với $0 < A < 1$, thì tồn tại $v \in R$ và một lớp I_i sao cho

$$|N(v) \cap I_i| = \Omega(\delta^2 d / \log_{1.5} n)$$

và trong tập $N(v) \cap I_i$, phần cạnh quay về R vẫn chiếm xấp xỉ tỷ lệ A . Định lý thứ hai nói rằng nếu với mọi tập S có $D_R(S) > A'D(S)$, thì tồn tại một lớp $N_i(S)$ vừa giữ được lượng cạnh đáng kể, vừa có tỷ lệ



Hình 8: lân cận bậc một $N(v)$ và lân cận bậc hai $N(N(v))$ trong phân tích đồ thị ba phía ngẫu nhiên.



Hình 9: Ví dụ trực quan cho trường hợp tập đỉnh đỏ có tổng bậc lớn nhưng bậc phân bố không đều.

đỉnh thuộc R ít nhất gần $(1 - 2\delta)A'$. Bản quét chứa nhiều công thức hằng số chi tiết ở đây; ý chính là dùng nguyên lý chia hộp: khi chia các quả bóng đỏ/xanh vào nhiều hộp, luôn có một hộp vừa giữ đủ nhiều bóng đỏ vừa không làm tỷ lệ đỏ giảm quá nhiều.

Từ hai định lý trên suy ra: nếu trong đồ thị ba phía, mọi cặp đỉnh không có quá $n/f(n)^2$ láng giềng chung và bậc nhỏ nhất đủ lớn, thì tồn tại các chỉ số i, j sao cho

$$T = N_i(N(v) \cap I_j)$$

có kích thước đủ lớn, và trong một cách tô ba màu nào đó, hầu hết các đỉnh của T đều cùng một màu, chẳng hạn màu đỏ. Khi đó đồ thị cảm sinh bởi T có đủ đỉnh nhỏ, nên thuật toán xấp xỉ phủ đỉnh tìm được một tập độc lập đủ lớn.

Thuật toán. Thuật toán nhận vào một đồ thị ba phía $G = (V, E)$ và trả về một tiến triển.

1. Nếu tồn tại đỉnh bậc nhỏ, hoặc tồn tại hai đỉnh có quá nhiều láng giềng chung, trả về tiến triển trực tiếp như trên.
2. Ngược lại, duyệt mọi v và các lớp bậc i, j ; với mỗi $T = N_i(N(v) \cap I_j)$, chạy thuật toán Bar-Yehuda–Even / Monien–Speckenmeyer.
3. Nếu tìm được trong T một tập độc lập đủ lớn, trả về tiến triển kiểu một.

Theo các định lý kỹ thuật, bước cuối luôn thành công. Chọn tham số sao cho kích thước tập độc lập tìm được bằng $\Omega(n/f(n))$ dẫn tới $f(n) = n^{2/5} \text{polylog}(n)$. Vì vậy, trong thời gian đa thức có thể tô màu đồ thị ba phía bằng $O(n^{2/5} \text{polylog}(n))$ màu.

9.4 Bài toán tô màu online

9.4.1 Cây

Trước tiên, cây cho thấy khác biệt lớn giữa offline và online. Nếu đảm bảo đồ thị cuối cùng là một cây, mọi thuật toán online trong trường hợp xấu nhất vẫn cần ít nhất $\log n + 1$ màu.

Lấy một thuật toán online bất kỳ A . Chứng minh quy nạp rằng có thể tạo một cây T_k với 2^k đỉnh buộc A dùng $k + 1$ màu. Trường hợp $k = 0$ hiển nhiên. Giả sử đã tạo được $T_0, T_1, \dots, T_k$ sao cho từ mỗi cây chọn được một đỉnh có màu đôi một khác nhau. Thêm một đỉnh mới nối với tất cả các đỉnh được chọn; đỉnh mới không thể dùng bất kỳ màu nào trong $k + 1$ màu đó, nên phải dùng màu thứ $k + 2$. Điều này tạo được T_{k+1} .

Ngược lại, tham lam trực tiếp cho cây dùng không quá $\lceil \log n \rceil + 1$ màu, nên cận trên và cận dưới cùng bậc.

9.4.2 Đồ thị khoảng

Một đồ thị là đồ thị khoảng nếu mỗi đỉnh tương ứng với một đoạn trên trục số, và hai đỉnh kề nhau khi hai đoạn tương ứng giao nhau. Gọi $\omega(G)$ là kích thước clique lớn nhất. Kierstead và Trotter chứng minh rằng với mọi đồ thị khoảng G , tồn tại thuật toán online dùng không quá $3\omega(G) - 2$ màu.

Thuật toán đệ quy $\text{SOLVE}(k)$ xử lý đồ thị khoảng có clique lớn nhất không quá k :

1. Nếu $k = 1$, tô mọi đỉnh bằng cùng một màu.
2. Ngược lại, tạo một bản $\text{SOLVE}(k - 1)$, duy trì hai đồ thị G_1, G_2 . Ba màu mới được dành riêng cho G_2 .

3. Khi đỉnh v xuất hiện, nếu $G_1 + v$ vẫn có clique lớn nhất không quá $k - 1$, đưa v vào $\text{SOLVE}(k - 1)$. Nếu không, đưa v vào G_2 và tô nó bằng màu khả dụng nhỏ nhất trong ba màu mới.

Cần chứng minh ba màu đủ cho G_2 . Với biểu diễn khoảng cố định, mỗi khoảng được đưa vào G_2 vì tồn tại một điểm đã bị $k - 1$ khoảng của G_1 phủ. Nếu hai khoảng trong G_2 bao chứa nhau, hoặc nếu G_2 có tam giác, thì có một điểm trên trục số bị G_1 phủ $k - 1$ lần và còn bị ít nhất hai khoảng của G_2 phủ, mâu thuẫn với $\omega(G) \leq k$. Suy ra G_2 không có tam giác và bậc tối đa không vượt quá 2, nên tô tham lam bằng ba màu là đủ.

Cận $3\omega(G) - 2$ cũng là cận dưới. Ý tưởng đối kháng là ép thuật toán chỉ dùng $3\omega(G) - 3$ màu rồi xây dựng các khối khoảng có thể “co” lại. Với mỗi i , dựng được nhiều khối chứa clique cỡ không quá i nhưng cùng mang một tập $3i - 2$ màu. Từ các khối liên tiếp, thêm hai khoảng chồng lẫn thích hợp để tạo ba màu mới ngoài tập cũ; sau khi co các khối có cùng bộ màu mới, quy nạp từ i lên $i + 1$. Khi $i = \omega(G)$, thuật toán bị buộc dùng màu thứ $3\omega(G) - 2$.



图 6: I_1, I_2 颜色相同

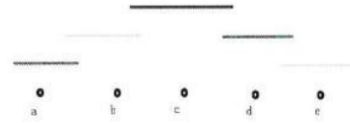


图 7: I_1, I_3 颜色相同

Hình 10: Hai bước dựng khối khoảng trong chứng minh cận dưới $3\omega(G) - 2$, tương ứng I_1, I_2 cùng màu và I_1, I_3 cùng màu.

9.4.3 Đồ thị không có chu trình độ dài 3 và 5

Nếu đồ thị cuối cùng không chứa chu trình đơn độ dài 3 hoặc 5, có thuật toán online dùng không quá $2\sqrt{n}$ màu. Đặt $B = \lceil \sqrt{n} \rceil$ và duy trì các tập W_i với $B < i \leq 2B$.

Khi đỉnh v xuất hiện:

1. thử tô tham lam bằng một màu trong $1, \dots, B$;
2. nếu thất bại nhưng tồn tại W_i sao cho $v \in N(W_i)$, tô v bằng màu i đầu tiên như vậy;
3. nếu không, lấy màu $i > B$ đầu tiên chưa có tập đại diện và đặt

$$W_i = \{u \mid u \in N(v), \text{col}(u) < B\}.$$

Nếu hai đỉnh kề nhau cùng nhận một màu lớn c , cả hai phải được tô ở bước hai. Nếu một trong hai là đỉnh đã tạo W_c , ta có tam giác; nếu cả hai đều chỉ nằm trong lân cận của W_c , thì tránh tam giác buộc chúng có hai láng giềng khác nhau trong W_c , tạo ra một chu trình độ dài 5. Cả hai đều bị cấm. Mặt khác mỗi tập W_i không rộng có kích thước hơn B , các tập này rời nhau, nên có không quá n/B màu lớn được dùng. Tổng số màu không vượt quá $2B$.

9.5 Tô màu online đồ thị k phía

9.5.1 Đồ thị hai phía

Vì cây cũng là đồ thị hai phía, Theorem về cây cho thấy không thể hy vọng số màu $o(\log n)$ trong trường hợp xấu nhất. Một thuật toán đơn giản dùng không quá $2 \log n$ màu như sau. Với mỗi thành phần liên

thông, duy trì một tham số k sao cho phía trái chưa dùng màu $k - 1$, phía phải chưa dùng màu k , và tổng thể chỉ dùng các màu $1, \dots, k$.

Khi một đỉnh mới nối nhiều thành phần, gọi hai giá trị k lớn nhất trong các thành phần bị gộp là k_1, k_2 . Thành phần mới dùng $k = \max(k_1, k_2 + 2)$. Đỉnh mới nhận màu k hoặc $k - 1$ tùy nó nằm ở phía nào. Nếu f_k là số đỉnh ít nhất để thuật toán buộc phải tạo tham số k , thì $f_k \geq \min(f_{k-1} + f_{k-2}, 2f_{k-2}) + 1$, suy ra f_k tăng theo hàm mũ và số màu không quá $2 \log n$.

9.5.2 Đồ thị k phía tổng quát

Giả sử trước hết số đỉnh n và số màu tối ưu k đã biết. Tư tưởng là dùng “tham lam một phần”. Đặt ngưỡng s . Ta thử tô online bằng s màu mới; các đỉnh không tô được vì kề với đủ một đại diện của mỗi màu sẽ được gửi vào các bài toán con $(k - 1)$ phía. Trong offline có thể chọn lớp màu nhỏ nhất để tách, nhưng online không biết lớp nào nhỏ, nên thuật toán chọn ngẫu nhiên một màu r và chỉ dùng các đỉnh màu r để sinh bài toán con. Để các bài toán con có kích thước được khống chế, mỗi khi một bài toán con đạt s đỉnh thì mở bài toán con mới.

Thuật toán $\text{SOLVE}(n, k)$ xử lý đồ thị k phía có không quá n đỉnh:

1. Nếu $k \leq 2$, dùng thuật toán online cho đồ thị hai phía.
2. Đặt $s = S(n, k)$, cấp s màu mới và chọn ngẫu nhiên đều một màu $r \in \{1, \dots, s\}$.
3. Duy trì đồ thị G gồm các đỉnh tô được tham lam bằng s màu.
4. Khi đỉnh v xuất hiện, nếu v có thể nhận một trong s màu trong G , tô bằng màu nhỏ nhất khả dụng và thêm vào G . Nếu màu đó là r , mở một bài toán con $\text{SOLVE}(s, k - 1)$ tương ứng.
5. Nếu v không tô được bằng s màu, nó kề với ít nhất một đỉnh màu r . Gửi v vào bài toán con gắn với đỉnh màu r đó; nếu bài toán con đã có s đỉnh thì mở bản mới.

Ký hiệu $A(n, k)$ là kỳ vọng số màu lớn nhất mà thuật toán dùng trên mọi đồ thị k phía có n đỉnh. Với lựa chọn tham số

$$S(n, k) = \lceil 2k n^{1-1/k} (\log n)^{1/k} \rceil,$$

ta có quy hoạch cận trên dạng

$$A(n, k) \leq s + \frac{2n}{s} A(s, k - 1).$$

Quy nạp theo k cho kết quả

$$A(n, k) = O(k 2^k n^{1-1/k} (\log n)^{1/k}).$$

Khi n, k chưa biết, ta dùng chiến lược nhân đôi. Nếu số đỉnh vượt quá ước lượng hiện tại n' , tăng n' lên gấp đôi và bắt đầu một bản thuật toán mới với bộ màu mới. Nếu thuật toán đi xuống tới bài toán hai phía nhưng phát hiện đồ thị con không hai phía, tăng ước lượng k' thêm một và cũng khởi động lại với bộ màu mới. Các hệ số $n^{1-1/k}$ và 2^k hấp thụ chi phí khởi động lại, nên cận kỳ vọng tiệm cận không đổi.

9.6 Tổng kết

Bài toán tô màu đồ thị có nhiều biến thể, và mỗi biến thể lại có những thuật toán rất khác nhau. Bài viết đã giới thiệu một số thuật toán tiêu biểu với hy vọng đóng vai trò gợi mở, giúp độc giả tiếp tục nghiên cứu các cách làm và ứng dụng thú vị hơn.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Wang Xiaopeng và thầy Cai Zijian của Trường Trung học trực thuộc Đại học Quảng Châu đã quan tâm và hướng dẫn; cảm ơn huấn luyện viên đội tuyển quốc gia Yang Yaoliang; cảm ơn bạn Peng Sijin đã đọc kiểm tra bản thảo; và cảm ơn cha mẹ đã luôn chăm sóc, ủng hộ.

Tài liệu tham khảo

1. R. Bar-Yehuda; S. Even. *A Local-Ratio Theorem for Approximating the Weighted Vertex Cover Problem*. In: Analysis and Design of Algorithms for Combinatorial Problems, 1985.
2. Avrim Blum. *New Approximation Algorithms for Graph Coloring*. Journal of the ACM, 1994.
3. Henry A. Kierstead. *On-line coloring k -colorable graphs*. Israel Journal of Mathematics, 1998.
4. Henry A. Kierstead; William T. Trotter. *An extremal problem in recursive combinatorics*. Congressus Numerantium, 1981.
5. Burkhard Monien; Ewald Speckenmeyer. *Ramsey Numbers and an Approximation Algorithm for the Vertex Cover Problem*. Acta Informatica, 1985.
6. S. Vishwanathan. *Randomized online graph coloring*. Proceedings of the 31st Annual Symposium on Foundations of Computer Science, 1990.
7. Avi Wigderson. *Improving the Performance Guarantee for Approximate Graph Coloring*. Journal of the ACM, 1983.

10 Bàn về bài toán tối ưu không gian trong thi tin học

Chen Zhixuan, Trường Trung học Zhilin, Nhạc Thanh, Chiết Giang

Tóm tắt

Bài viết giới thiệu một vài mô hình xử lý dữ liệu thường gặp, cùng các kỹ thuật tối ưu không gian tương ứng. Tác giả phân tích hiệu quả, phạm vi áp dụng của các kỹ thuật này, rồi minh họa bằng một số bài toán cụ thể.

10.1 Mở đầu

Trong thi tin học, đề bài thường đặt yêu cầu cao về thời gian, đôi khi còn chấp nhận hy sinh bộ nhớ để đổi lấy thời gian tốt hơn. Tuy nhiên trong một số bài, giới hạn bộ nhớ nghiêm ngặt cũng là một cách kiểm tra năng lực thiết kế thuật toán. Bài viết này chọn một số mô hình xử lý dữ liệu phổ biến, trình bày vài cách đơn giản để giảm độ phức tạp không gian, đồng thời đưa ra phương pháp tối ưu cụ thể cho một số bài kinh điển.

10.2 Mô hình 1: xử lý dữ liệu tĩnh trên DAG

Trong nhiều bài toán, dữ liệu cần xử lý có thể chia thành nhiều khối. Giữa các khối có quan hệ phụ thuộc, còn đáp án cuối cùng chỉ liên quan tới khối cuối cùng. Mô hình này có thể mô tả như sau:

1. Có n khối dữ liệu; khối thứ i ký hiệu là data_i , kích thước là L_i .

2. Một cạnh có hướng (i, j) biểu thị việc tính data_i phụ thuộc vào data_j . Các cạnh này tạo thành một DAG. Ngoài ra, mỗi khối cũng có thể phụ thuộc vào dữ liệu đầu vào.
3. Chi phí lưu dữ liệu đầu vào có thể bỏ qua, và đáp án chỉ liên quan tới khối cuối cùng.

Cách xử lý trực tiếp là cấp phát trước n mảng có độ dài $\max_i L_i$, hoặc cấp phát riêng bộ nhớ cho từng khối. Nhưng khi mọi phụ thuộc của một khối đã được xử lý xong, khối đó có thể được giải phóng. Vì vậy có thể chọn một thứ tự topo, chẳng hạn $1, 2, \dots, n$, xử lý từng khối, cấp phát bộ nhớ khi cần và giải phóng khi khối không còn hiệu lực. Trong trường hợp tổng quát, cách này khó ước lượng không gian, phụ thuộc vào thứ tự topo, và việc cấp phát/giải phóng động cũng có hằng số lớn.

Với một số trường hợp đặc biệt, có những kỹ thuật đơn giản vừa giảm bộ nhớ vừa giữ được hằng số thời gian tốt.

10.2.1 Trường hợp đặc biệt 1: mảng lăn

Mảng lăn thường áp dụng được khi:

1. mọi khối dữ liệu có cùng kích thước L ;
2. với mỗi i , các khối mà data_i phụ thuộc vào đều có chỉ số j thỏa $i - m < j < i$, trong đó m là một hằng số nhỏ.

Khi đó dùng mảng lăn đưa không gian xuống $O(mL)$.

Ví dụ 1: tô màu bóng. Có n quả bóng xếp thành một hàng, ban đầu đều màu trắng. Tô quả thứ i tốn chi phí w_i . Cần tìm chi phí tô nhỏ nhất sao cho trong mọi đoạn m quả liên tiếp có ít nhất hai quả được tô. Ràng buộc đọc được từ bản quét là $n \leq 10^6$, $m \leq 10^3$, $w_i \leq 10^9$.

Đặt $f_{i,j}$ là chi phí nhỏ nhất khi quả được tô cuối cùng là i , quả được tô ngay trước đó là j . Chuyển tiếp của quy hoạch động chỉ dùng những trạng thái mà hai vị trí tô gần nhau hơn m , nên có thể xử lý bằng tiền tố hoặc cực trị trượt. Cách trực tiếp có thời gian và không gian $O(nm)$.

Xem mỗi lớp f_i là một khối kích thước m . Khi xử lý f_i , chỉ các lớp f_j với $j > i - m$ còn có thể tham gia chuyển tiếp, nên có thể lưu các lớp này theo chỉ số $i \bmod m$. Cùng tư tưởng hàng đợi vòng này cũng có thể dùng để giảm không gian trong những thuật toán như SPFA.

10.2.2 Trường hợp đặc biệt 2: gộp trên cây

Gộp trên cây thường chỉ trường hợp:

1. các cạnh phụ thuộc tạo thành một cây gốc hướng ra ngoài từ một khối u ;
2. kích thước L_i của mỗi khối bằng tổng kích thước của các khối con.

Khi đó không gian duy trì có thể tối ưu về $O(L_u)$.

Ví dụ 2: ba lô trên cây. Cho một cây vô hướng có các đỉnh đánh số 1 tới n . Mỗi đỉnh có trọng số w_i . Trọng số của một khối liên thông là tích trọng số các đỉnh trong khối. Với mỗi i , cần tính tổng trọng số của mọi khối liên thông đúng i đỉnh và đi qua đỉnh 1, lấy modulo $10^9 + 7$. Ràng buộc đọc được là $n \leq 8000$, $1 \leq w_i < 10^9 + 7$.

Đặt $f_{u,i}$ là tổng trọng số các khối liên thông có i đỉnh, chứa u , và có đỉnh sâu nhất nhỏ nhất là u . Với mỗi cạnh cha–con, ta gộp quy hoạch động của cây con vào quy hoạch động của đỉnh cha. Cài đặt ba lô cây ngây thơ có thời gian và không gian $O(n^2)$.

Nhưng kích thước của khối dữ liệu f_u đúng bằng kích thước cây con của u , đây là mô hình gộp trên cây điển hình. Sau khi gộp thông tin của con v vào u , dữ liệu của v không còn cần riêng nữa. Nếu xem kích thước của f_u là số đỉnh liên quan tới nó, tại mọi thời điểm mỗi đỉnh chỉ thuộc nhiều nhất một khối dữ liệu, nên tổng số giá trị DP hiệu lực không vượt quá n .

Để tránh cấp phát động, có thể lưu các giá trị f trong đoạn thứ tự DFS tương ứng với các con đã duyệt. Khi gộp hai khối, chúng vừa đúng là hai đoạn kề nhau trên thứ tự DFS. Merge sort cũng là ví dụ điển hình của mô hình này: dữ liệu đang duy trì có thể đặt ngay trên đoạn đang sắp xếp; nếu không thể gộp tại chỗ, cần tính thêm không gian tạm thời $O(n)$.

10.3 Mô hình 2: xử lý dữ liệu tĩnh có truy vấn

Một bài xử lý dữ liệu tĩnh thường có thể mô tả như sau:

1. Dữ liệu thỏa mô hình 1. Giả sử sau khi tối ưu theo mô hình 1, không gian xử lý là $\text{process}(L_i)$.
2. Có m truy vấn độc lập. Mỗi truy vấn có thể tách thành $O(c)$ bài toán con, và mỗi bài toán con chỉ liên quan tới thông tin của một khối dữ liệu.

Do phải trả lời truy vấn, ta không thể giải phóng khối ngay sau khi nó hết hiệu lực như mô hình 1 để đạt không gian lý tưởng $O(\text{process}(L_i))$. Nếu xử lý online mọi truy vấn, không gian trực tiếp thường là $O(\sum_i L_i)$. Tùy theo bậc lớn của n, L_i, m, c và tính chất truy hồi/quyết định của dữ liệu, có thể chọn nhiều kỹ thuật giảm không gian nhưng cố gắng không đổi bản chất thuật toán.

10.3.1 Kỹ thuật 1: tách xử lý

Khi c nhỏ và các bài toán con tương đối độc lập, có thể đưa không gian về

$$O(\text{process}(L_i) + n + m \cdot c + k),$$

trong đó k là chi phí phụ trợ của bài cụ thể:

1. Offline mọi bài toán con của các truy vấn, rồi phân phối chúng vào danh sách của khối dữ liệu tương ứng.
2. Xử lý từng khối như trong mô hình 1; khi tới khối hiện tại, lấy các truy vấn trên danh sách của nó ra và trả lời.

Ví dụ 3: ST/RMQ. Cho dãy $a_1, a_2, \dots, a_n$ và m truy vấn, mỗi truy vấn hỏi giá trị lớn nhất trên đoạn $[l, r]$. Ràng buộc đọc được là $n \leq 10^5, m \leq 5 \cdot 10^5$.

Sparse Table cổ điển xử lý RMQ trong thời gian $O(n \log n + m)$ và không gian $O(n \log n)$. Xem mỗi lớp nhân đôi là một khối kích thước $O(n)$; lớp thứ i chỉ phụ thuộc vào lớp thứ $i - 1$, nên có thể dùng mảng lăn với $\text{process}(L) = O(n)$. Mỗi truy vấn RMQ tách thành $O(1)$ bài toán con độc lập. Vì vậy áp dụng tách xử lý đưa không gian xuống $O(n + m)$. Bài RMQ còn có thuật toán tốt hơn cả về thời gian lẫn không gian, nhưng không thuộc hướng tối ưu đang bàn ở đây.

10.3.2 Kỹ thuật 2: xử lý song song

Khi các bài toán con của truy vấn có quan hệ phụ thuộc hoặc quan hệ quyết định, tách xử lý đơn giản không còn trực tiếp áp dụng được. Nếu tham số của mỗi bài toán con được quyết định từ kết quả của bài toán con trước, có thể xử lý song song nhiều truy vấn theo từng vòng. Ví dụ đơn giản nhất là nhị phân đáp án; kỹ thuật “nhị phân cùng nhau” thường được dùng để tránh phải bền vững hóa dữ liệu.

Ví dụ 4: phần tử lớn thứ k trên đoạn. Cho dãy $a_1, \dots, a_n$ và m truy vấn. Mỗi truy vấn hỏi phần tử lớn thứ k trong đoạn $[l, r]$. Ràng buộc đọc được: $n, m \leq 10^5$, $1 \leq a_i \leq n$, $1 \leq k \leq r - l + 1$.

Cách kinh điển là dựng cây đoạn giá trị bền vững theo từng tiền tố; truy vấn trên hai phiên bản r và $l - 1$ để đi xuống cây, mỗi tầng dùng trọng số con để quyết định hướng đi. Vì data_i và data_{i-1} chỉ khác nhau trên một đường dài $O(\log n)$, tổng bộ nhớ là $O(n \log n)$.

Theo tư tưởng xử lý song song, có thể offline mọi truy vấn và mô tả trạng thái nhị phân hiện tại bằng các tham số như (L, R, mid) . Mỗi vòng cần đếm bao nhiêu số trong đoạn thuộc một nửa giá trị. Nếu xử lý mỗi vòng bằng đếm điểm hai chiều ngây thơ rồi quyết định vòng tiếp theo, không gian giảm xuống $O(n)$ nhưng thời gian tăng.

Để giữ bản chất thuật toán của cây đoạn bền vững, mô tả trạng thái truy vấn bằng vị trí nút p trên cây đoạn giá trị. Ở mỗi vòng, bài toán con chỉ cần một phần tử ở hai khối dữ liệu tiền tố $l - 1$ và r . Vì không cần bền vững hóa, không gian xử lý khối là $O(n)$. Tiếp tục chia các tầng của cây đoạn thành các khối dữ liệu: ở vòng thứ i , mỗi tiền tố chỉ dùng khối thứ i . Khi xử lý một tầng, mỗi bước chỉ sửa một vị trí cụ thể, nên thời gian mỗi tầng là tuyến tính. Trong cài đặt, có thể tiền xử lý số hiệu lá rồi dùng phép bit để tìm tổ tiên cấp cần thiết; hoặc bỏ cấu trúc đánh số cây đoạn ban đầu và định vị hoàn toàn bằng bit. Cây đoạn cũng có thể thay bằng Fenwick tree và chia theo lowbit.

10.3.3 Kỹ thuật 3: nén bài toán con

Khi c lớn, tách xử lý giúp giảm bộ nhớ dữ liệu nhưng lại sinh thêm quá nhiều bộ nhớ để lưu các bài toán con của truy vấn. Khi đó cần quan sát chính quá trình tách truy vấn và nén cách lưu bài toán con.

Ví dụ 5: bao lồi trên đoạn. Cho n điểm (x_i, y_i) và m truy vấn. Mỗi truy vấn cho đoạn $[l, r]$ và hệ số k , cần tính

$$\max_{l \leq j \leq r} (kx_j + y_j).$$

Ràng buộc đọc được: $n, m \leq 10^5$, các tọa độ và k không vượt quá 10^9 , coi n và m cùng bậc.

Dùng cây đoạn lưu bao lồi ở mỗi nút, tiền xử lý bằng cách gộp như merge sort. Mỗi truy vấn tách thành $O(\log n)$ nút cây đoạn; nếu truy vấn từng bao bằng nhị phân thì thời gian $O(\log^2 n)$. Offline theo hệ số k có thể dùng con trỏ tối ưu trên từng bao, cho thời gian và không gian $O(n \log n)$.

Thông tin trên cây đoạn gồm $2n - 1$ khối; quá trình xây dựng giống merge sort nên có thể đạt $\text{process}(L_i) = O(n)$. Tuy nhiên nếu trực tiếp áp dụng tách xử lý, bộ nhớ lưu mọi bài toán con truy vấn sẽ quá lớn. Nhận xét quan trọng là việc tách một truy vấn đoạn chỉ là một quá trình truy vấn trên cây đoạn; tại mỗi thời điểm đệ quy chỉ cần $O(1)$ tham số để mô tả trạng thái. Do đó dùng chính tham số của quá trình đệ quy thay cho việc lưu tường minh mọi bài toán con:

1. Duyệt từng nút cây đoạn, đồng thời duy trì tập truy vấn đang đi tới nút đó.
2. Ở nút hiện tại, quyết định truy vấn nào đi sang con trái, con phải, và truy vấn nào được trả lời tại nút hiện tại.
3. Sau khi xử lý hai con, gộp bao lồi của hai con rồi trả lời phần truy vấn thuộc nút này.
4. Khi quay lui, hoàn tác thay đổi của tập truy vấn.

Cách nén này tích hợp tiền xử lý với xử lý truy vấn, nên không đổi bản chất thuật toán: thời gian vẫn $O(n \log n)$, không gian giảm về $O(n)$. Nếu dùng cây đoạn ZKW không đệ quy, cài đặt còn đơn giản hơn: sắp truy vấn theo k , xác định lá bắt đầu của đoạn, chia cây theo độ sâu, xử lý từ dưới lên; ở mỗi tầng, đưa mọi truy vấn lên một tầng rồi tìm các phần cần hỏi, sau đó gộp bao lồi của tầng hiện tại. Phương pháp này còn có thể hỗ trợ sửa đổi dạng chèn điểm.

10.4 Phân khối cho bài toán dãy đặc biệt

Với bài toán đoạn trên đây, chọn cách phân khối hợp lý đôi khi giảm đáng kể bộ nhớ. Khác với thuật toán phân khối thông thường, phân khối để tối ưu bộ nhớ thường dùng số khối hoặc kích thước khối tương đối nhỏ.

10.4.1 Phân khối tăng trên

Phân khối tăng trên chia dãy gốc thành ít khối, mỗi khối là một bài toán con:

1. Dãy gốc dài n , có m thao tác đoạn.
2. Với dãy dài n , xử lý thao tác toàn cục có thời gian/không gian $T(n), M(n)$.
3. Với dãy dài n , xử lý thao tác đoạn có thời gian/không gian $T'(n), M'(n)$.

Nếu chia đều dãy thành S đoạn, một thao tác đoạn được tách thành một số thao tác toàn cục trên các khối nguyên và $O(1)$ thao tác đoạn biên. Thời gian xấp xỉ

$$S \cdot T(n/S) + T'(n/S),$$

còn vì có thể offline xử lý từng khối độc lập, không gian là

$$M(n/S) + M'(n/S).$$

Với bài bao lỗi trên đoạn ở trên, cách ngây thơ có thể xem là $T(n) = O(1)$, $M(n) = O(n)$, $T'(n) = O(\log n)$, $M'(n) = O(n \log n)$. Lấy $S = \log n$, thời gian vẫn cùng bậc $O(\log n)$ cho mỗi truy vấn, còn không gian giảm xuống $O(n)$.

10.4.2 Phân khối tăng dưới

Phân khối tăng dưới thường có mô hình:

1. Chia dãy dài n thành các khối dài S .
2. Tính kết quả gộp thông tin trong từng khối, rồi biến dãy gốc thành dãy mới dài khoảng n/S .
3. Dùng cấu trúc dữ liệu ban đầu trên dãy mới.

Không gian cấu trúc trên dãy mới giảm theo hệ số S . Khi xử lý điểm hoặc đoạn, cần đi vào $O(1)$ khối và xử lý thô bên trong; thời gian mỗi thao tác có dạng $O(\log(n/S) + f(S))$.

Ví dụ 6: DP động. Cho dãy $a_1, a_2, \dots, a_n$. Cần chọn một số phần tử sao cho khoảng cách giữa hai phần tử được chọn kề nhau không vượt quá k . Trọng số của một phương án là tích các phần tử đã chọn; cần tính tổng trọng số mọi phương án hợp lệ modulo $10^9 + 7$. Có m lần sửa, mỗi lần đổi a_x thành y , sau mỗi lần sửa in đáp án mới. Ràng buộc đọc được: $n \leq 10^5$, $m \leq 10^5$, $k \leq 8$.

Đây là mô hình DP động bằng ma trận. Mỗi nút cây đoạn lưu một ma trận chuyển trạng thái; sửa điểm tốn $O(k^3 \log n)$, tiền xử lý $O(nk^3)$, không gian $O(nk)$.

Dùng phân khối tăng dưới, mỗi khối được dựng lại bằng DP tuyến tính trong khối. Chọn S cỡ $\sqrt{k} \log n$ hoặc một hằng số nhỏ hơn $\log n$ trong thực tế, không gian giảm về $O(n)$, tiền xử lý cũng giảm hệ số, đổi lại hằng số sửa điểm tăng. Các kỹ thuật như tối ưu bằng bitset hoặc nén nhiều chữ số trên một ô nhớ cũng có thể nhìn như các dạng phân khối tăng dưới đặc biệt.

10.5 Tối ưu cho trie

10.5.1 Mô hình trie ngây thơ

Giả sử xây trie từ n xâu $s_1, s_2, \dots, s_n$, tổng độ dài là m , bảng chữ cái là Σ . Trie ngây thơ có $O(m)$ nút; nếu mỗi nút lưu một mảng con cho mọi ký tự trong Σ , không gian là $O(m|\Sigma|)$.

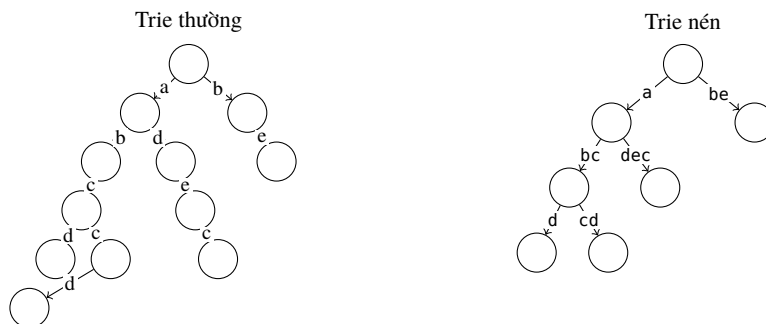
10.5.2 Trie bằng danh sách liên kết

Khi bảng chữ cái lớn, lưu cả mảng hậu duệ ở mỗi nút gây lãng phí. Có thể thay mảng bằng danh sách các cạnh con thực sự tồn tại. Tổng số cạnh con của mọi nút chỉ là $O(m)$, nên không gian còn $O(m)$. Đổi lại, tìm cạnh con phải duyệt danh sách, thời gian xây trie là $O(m|\Sigma|)$ trong cận xấu, nhưng hằng số thường nhỏ.

Nếu sắp xếp mọi xâu theo thứ tự từ điển trước khi chèn, khi chèn vào trie chỉ cần so sánh cạnh con cuối của danh sách xem có khớp hay không; thời gian chèn có thể đạt $O(m \log n + \sum |s_i|)$, và với cài đặt tốt có thể gần tuyến tính. Khi xử lý khớp, cũng có thể duy trì con trỏ ở mỗi nút để đạt tuyến tính khâu hao. Dùng cây cân bằng thay danh sách đưa truy vấn cạnh xuống $O(\log |\Sigma|)$ nhưng tốn bộ nhớ hơn, cài đặt khó hơn, và khi $|\Sigma| \leq 26$ thường không hiệu quả hơn.

10.5.3 Trie nén

Trie nén gom các chuỗi dài bậc một trên trie thường thành một nút, giữ nguyên quan hệ tổ tiên, nhờ đó số nút giảm xuống $O(n)$. Hình dưới so sánh trie thường và trie nén của bốn xâu abcd, adec, abccd, be: các đoạn chỉ có một đường đi được nén thành một cạnh có nhãn dài.



Hình 11: Trie thường và trie nén cho bốn xâu abcd, adec, abccd, be. Ở trie nén, các chuỗi bậc một như be, dec và cd được gộp thành nhãn cạnh.

Giống trie thường, mỗi nút trie nén vẫn cần lưu các cạnh con theo ký tự đầu, đồng thời mỗi nút lưu chuỗi nén trên cạnh nối với cha. Khi chèn hoặc truy cập, ta dùng ký tự đầu để tìm cạnh con, rồi so sánh chuỗi hiện tại với nhãn cạnh. Nếu xuất hiện vị trí không khớp, tạo một nút mới làm điểm tách và đặt cả xâu mới lẫn nút cũ làm con của nó. Mỗi lần chèn một xâu sinh thêm nhiều nhất một nút, nên tổng số nút không vượt quá $2n - 1$. Không gian là $O(m + n|\Sigma|)$ nếu lưu nhãn cạnh trực tiếp và mỗi nút vẫn lưu cấu trúc cạnh con theo bảng chữ cái.

Suffix tree là một trie nén đặc biệt của mọi hậu tố của một xâu. Mỗi nhãn cạnh nén đều là một đoạn con của xâu gốc, nên không cần dùng thêm $O(m)$ bộ nhớ để lưu chuỗi cạnh; chỉ cần lưu cặp chỉ số đoạn.

Ứng dụng: cây đoạn mở nút động. Cây đoạn mở nút động theo miền giá trị, hỗ trợ sửa điểm, về bản chất là trie trên bảng chữ cái $\{0, 1\}$; xâu được chèn chính là biểu diễn nhị phân của chỉ số. Với n lần sửa

điểm, cây đoạn mở nút động gây thơ dùng $O(n \log V)$ bộ nhớ, trong đó V là miền giá trị.

Khi một giá trị số có thể lưu trong $O(1)$ ô nhớ, nhãn chuỗi ứng với mỗi nút trie có thể ghi bằng chỉ số hoặc đoạn bit, tổng không gian còn $O(n)$. Dùng trie nén cho cây đoạn mở nút động vì vậy có thể giảm bộ nhớ xuống $O(n)$. Cùng ý tưởng áp dụng được cho bài toán gộp cây đoạn, nhưng cài đặt cần xét nhiều trường hợp hơn.

10.6 Tổng kết

Bài viết trình bày các hướng tối ưu không gian cho xử lý dữ liệu tĩnh và truy vấn, thao tác đoạn trên dãy, trie và một số cấu trúc liên quan, đồng thời minh họa bằng các bài toán kinh điển.

Trong ứng dụng thực tế, tối ưu không gian thường rất quan trọng; trong thi thuật toán nó lại không phải lúc nào cũng là trọng tâm. Tác giả hy vọng bài viết có thể gợi hứng thú với tối ưu không gian, từ đó giúp thiết kế thêm nhiều thuật toán tốt và đưa góc nhìn tối ưu bộ nhớ vào các đề thi tin học.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng trao đổi và học tập; cảm ơn huấn luyện viên đội tuyển quốc gia Yang Yaoliang đã hướng dẫn; cảm ơn thầy Zhang Chao đã ủng hộ và chỉ bảo; cảm ơn bạn Dai Jiangqi của Trường Ngoại ngữ Nam Kinh và bạn Zhou Pinliang của Đại học Giao thông Tây An đã đọc kiểm tra bản thảo; cảm ơn các thầy cô và bạn bè từng giúp đỡ.

Tài liệu tham khảo

1. Liu Rujia. *Nhập môn kinh điển về thi thuật toán*. Nhà xuất bản Đại học Thanh Hoa.
2. Kurt Maly. *Compressed Tries*. 1976.

11 Bàn về một lớp ma trận Monge

Zhang Jingxing, Trường Trung học trực thuộc Đại học Bắc Kinh

Tóm tắt

Bài viết giới thiệu một lớp ma trận Monge đặc biệt, các thuật toán liên quan và ứng dụng của chúng trong thi tin học.

11.1 Mở đầu

Ma trận Monge, hay mảng Monge, là ma trận có các phần tử thỏa bất đẳng thức tứ giác. Đây là một cấu trúc thường gặp trong thi tin học và có nhiều ứng dụng, đặc biệt trong tối ưu quy hoạch động. Trong số đó, ma trận Monge đơn vị đơn giản có thêm nhiều tính chất mạnh.

Với ma trận tổng quát kích thước $n \times n$, hiện chưa có thuật toán $O(n^{3-\varepsilon})$ cho phép nhân min-plus trong trường hợp chung. Tuy nhiên với ma trận Monge và với ma trận Monge đơn vị đơn giản, lần lượt tồn tại thuật toán $O(n^2)$ và $O(n \log n)$. Bài viết tập trung vào thuật toán nhân nhanh cho ma trận Monge đơn vị đơn giản, cách thuật toán này cải thiện độ phức tạp của một số bài cổ điển, và các ứng dụng của nó trong OI.

Tác giả biết tới thuật toán này từ hai bài trên Library Checker và một bài báo giới thiệu thuật toán. Các thuật toán liên quan tới ma trận Monge đơn vị trong bài đều xuất phát từ bài báo đó. Ở thời điểm bài viết được soạn, theo hiểu biết của tác giả, ma trận Monge đơn vị chưa được dùng rộng rãi trong thi tin học. Hy vọng phần giới thiệu này giúp độc giả thấy được cấu trúc gọn nhưng mạnh của nó, từ đó áp dụng vào nhiều bài toán hơn và gợi ý cho việc ra đề mới.

11.2 Khái niệm cơ bản

Phần này định nghĩa các ký hiệu sẽ dùng, giải thích một số khái niệm và nêu các tính chất nền tảng của đối tượng nghiên cứu.

11.2.1 Định nghĩa cơ bản

Với khoảng chỉ số, bài viết dùng ký hiệu nửa nguyên để biểu diễn biên giữa hai chỉ số: $\hat{i}$ chỉ $i + \frac{1}{2}$, còn $\check{i}$ chỉ $i - \frac{1}{2}$. Ký hiệu $[i : j]$ biểu thị tập các số nguyên $i, i + 1, \dots, j - 1$.

Một ma trận A trên hai tập chỉ số X, Y có phần tử $A_{i,j}$ với $i \in X, j \in Y$. Nếu không nói thêm, chỉ số hàng tăng từ trên xuống, chỉ số cột tăng từ trái sang phải, và các phần tử là số thực. Khi $|Y| = 1$, ma trận trên $X \times Y$ được xem như một vector cột và ký hiệu là a_i .

11.2.2 Mật độ và phân bố

Với ma trận A trên các hàng $[i_1 : i_2]$ và cột $[j_1 : j_2]$, định nghĩa ma trận mật độ A^Δ trên các ô giữa các hàng/cột bởi công thức sai phân hai chiều

$$A_{i,j}^\Delta = A_{i,j} - A_{i,j-1} - A_{i+1,j} + A_{i+1,j-1}.$$

Ngược lại, với một ma trận mật độ D , ma trận phân bố D^Σ là tổng tiền tố hai chiều tương ứng. Có thể xem hai phép này như sai phân hai chiều và tiền tố hai chiều.

Một ma trận A được gọi là ma trận Monge khi mọi phần tử của A^Δ đều không âm. Định nghĩa tương đương quen thuộc trong quy hoạch động là bất đẳng thức tứ giác: với $i_1 < i_2$ và $j_1 < j_2$,

$$A_{i_1,j_1} + A_{i_2,j_2} \leq A_{i_1,j_2} + A_{i_2,j_1}.$$

Chiều thuận lấy đúng một ô sai phân; chiều ngược thu được bằng cách cộng các ô sai phân trong hình chữ nhật.

11.2.3 Ma trận Monge đơn giản, đơn vị và á đơn vị

Một ma trận Monge là *đơn giản* nếu cột đầu tiên và hàng cuối cùng đều bằng 0. Với ma trận đơn giản, phép lấy mật độ rồi lấy phân bố trả lại ma trận ban đầu; ngược lại, ma trận thu được từ phân bố của mật độ luôn có cột đầu và hàng cuối bằng 0.

Một ma trận vuông là ma trận hoán vị nếu mỗi hàng và mỗi cột có đúng một phần tử bằng 1, các phần tử còn lại bằng 0. Một ma trận là á hoán vị nếu nó bằng 0, hoặc sau khi xóa mọi hàng/cột toàn 0 thì trở thành ma trận hoán vị.

Một ma trận Monge là *đơn vị* nếu mật độ của nó là ma trận hoán vị; là *á đơn vị* nếu mật độ của nó là ma trận á hoán vị. Ma trận Monge đơn vị đơn giản và ma trận Monge á đơn vị đơn giản thường được biểu diễn bằng hoán vị hoặc tập điểm ứng với mật độ, không cần lưu toàn bộ ma trận.

Nếu A là ma trận Monge đơn vị đơn giản kích thước phù hợp, thì hàng đầu tiên tăng từ trái sang phải như 0, 1, 2, ..., còn cột cuối cùng tăng từ dưới lên như 0, 1, 2, Ngược lại, các điều kiện biên này cùng

tính Monge và tính nguyên buộc mật độ có đúng một 1 trên mỗi hàng/cột. Với trường hợp á đơn vị, các phần tử nguyên không âm và hiệu giữa hai ô kề nhau không vượt quá 1 là dấu hiệu quan trọng; khi đó mọi phần tử đều nằm trong khoảng

$$0 \leq A_{i,j} \leq \min(j - j_0, i_1 - i).$$

11.2.4 Nhân khoảng cách

Với ma trận A trên $X \times Y$ và B trên $Y \times Z$, định nghĩa tích khoảng cách, tức tích min-plus,

$$C = AB, \quad C_{i,j} = \min_{k \in Y} \{A_{i,k} + B_{k,j}\}.$$

Trong bài này, nếu không nói thêm, mọi phép nhân ma trận đều là phép nhân khoảng cách.

Các lớp ma trận đang xét đóng dưới phép nhân này. Nếu A, B là ma trận Monge thì AB cũng là ma trận Monge. Chứng minh xét hai hàng và hai cột, lấy các vị trí đạt min tương ứng; tùy thứ tự của hai vị trí trung gian, dùng bất đẳng thức tứ giác của A hoặc của B để suy ra bất đẳng thức tứ giác cho C .

Nếu A, B là ma trận Monge đơn vị đơn giản thì AB vẫn là ma trận Monge đơn vị đơn giản. Điều này suy ra từ tính đóng Monge và các điều kiện biên: cột đầu và hàng cuối của tích bằng 0, còn hàng đầu/cột cuối có dạng tuyến tính chuẩn nhờ hiệu kề nhau bị chặn bởi 1. Tương tự, ma trận Monge á đơn vị đơn giản cũng đóng dưới phép nhân khoảng cách.

11.3 Nhân khoảng cách nhanh

Với ma trận tổng quát, phép nhân khoảng cách chưa có thuật toán dưới bậc ba trong trường hợp chung. Với ma trận Monge, có thuật toán $O(n^2)$; với ma trận Monge đơn vị hoặc á đơn vị đơn giản, có thuật toán $O(n \log n)$. Phần này tập trung vào thuật toán cho trường hợp sau.

11.3.1 Nhân ma trận Monge tổng quát với vector

Một ma trận được gọi là *hoàn toàn đơn điệu* nếu với mọi hai hàng $i_1 < i_2$ và hai cột $j_1 < j_2$, quan hệ so sánh giữa hai hiệu $A_{i_1,j_1} - A_{i_2,j_1}$ và $A_{i_1,j_2} - A_{i_2,j_2}$ không vi phạm mẫu đơn điệu. Nói cách khác, vị trí đạt min trên các hàng có tính đơn điệu. Ma trận Monge luôn hoàn toàn đơn điệu, vì nếu vi phạm thì trái với bất đẳng thức tứ giác.

Hệ quả quan trọng là *đơn điệu quyết định*: trong một ma trận hoàn toàn đơn điệu, vị trí cột của min trái nhất trên từng hàng không giảm khi đi từ trên xuống dưới. Tương tự với min phải nhất. Mọi ma trận con lấy từ một ma trận hoàn toàn đơn điệu cũng hoàn toàn đơn điệu.

Thuật toán SMAWK dùng các tính chất trên để nhân một ma trận Monge với một vector cột trong thời gian tuyến tính, nếu chỉ cần truy cập $O(n)$ phần tử ma trận. Ý tưởng là trước hết xét các hàng lẻ, dùng cấu trúc giống ngăn xếp đơn điệu để loại bỏ những cột không thể chứa min trái nhất, rồi đệ quy trên bài toán có số hàng và số cột đều giảm. Sau khi có kết quả cho các hàng lẻ, min của mỗi hàng chẵn chỉ cần tìm trong khoảng giữa hai quyết định lân cận. Tổng số phần tử cần hỏi thỏa truy hồi $T(n) = T(n/2) + O(n) = O(n)$.

11.3.2 Nhân hai ma trận Monge tổng quát

Với hai ma trận Monge A, B kích thước lớn nhất là n , có thể tính $C = AB$ trong $O(n^2)$. Cách trực tiếp là áp dụng SMAWK cho từng cột của B . Bài viết cũng nêu một cách nhìn khác: ký hiệu $p_{i,j}$ là vị trí nhỏ nhất đạt min trong công thức của $C_{i,j}$. Với mỗi cột, các $p_{i,j}$ đơn điệu theo hàng; nhờ đối xứng qua đường

chéo phụ, chúng cũng đơn điệu theo cột. Vì vậy khi quét các ô theo đường chéo phụ, chỉ cần liệt kê các vị trí trung gian giữa hai quyết định biên. Tổng số vị trí phải xét trên mỗi đường chéo là $O(n)$, và có $O(n)$ đường chéo, nên tổng thời gian là $O(n^2)$.

11.3.3 Ma trận Monge á đơn vị đơn giản với vector

Nếu áp dụng trực tiếp SMAWK cho ma trận á đơn vị đơn giản, việc hỏi một phần tử có thể trở thành bài toán đếm điểm hai chiều trên mặt độ. Có thể dùng cấu trúc dữ liệu phức tạp, nhưng bài báo gốc có thuật toán tuyến tính. Bài viết này trình bày một thuật toán gọn hơn, $O(n \log n)$.

Cho ma trận Monge á đơn vị đơn giản A và vector b . Đặt $C_{i,j} = A_{i,j} + b_j$. Khi cần tính một giá trị $A_{i,j}$, ta xem mặt độ A^Δ như tập điểm của một ma trận á hoán vị; từ một vị trí hiện tại (x, y) cùng giá trị đã biết, di chuyển sang ô cần hỏi bằng các bước kề nhau, mỗi bước cập nhật giá trị trong $O(1)$.

Thuật toán chia để trị theo hàng: ở ma trận con hiện tại, lấy hàng giữa, quét từ trái sang phải để tìm cột đạt min, rồi nhờ đơn điệu quyết định chia bài toán thành phần trên chỉ xét các cột không lớn hơn quyết định này và phần dưới chỉ xét các cột không nhỏ hơn nó. Nếu ma trận con có a hàng và b cột, việc quét hàng giữa và di chuyển vị trí truy cập tốn $O(a + b)$. Mỗi tầng đệ quy có tổng $a + b$ là $O(n)$, số tầng $O(\log n)$, nên thời gian là $O(n \log n)$.

11.3.4 Nhân ma trận Monge đơn vị đơn giản

Vì ma trận Monge đơn vị đơn giản được biểu diễn bằng hoán vị, ta định nghĩa phép nhân trực tiếp trên hoán vị. Với hoán vị a của $[1 : n]$, đặt $\text{Mat}(a)$ là ma trận mật độ có 1 tại các vị trí tương ứng của hoán vị. Với hai hoán vị a, b , định nghĩa

$$a \circ b = \text{Mat}^{-1} \left((\text{Mat}(a)^\Sigma \text{Mat}(b)^\Sigma)^\Delta \right).$$

Tính đóng của ma trận Monge đơn vị đơn giản bảo đảm định nghĩa này hợp lệ.

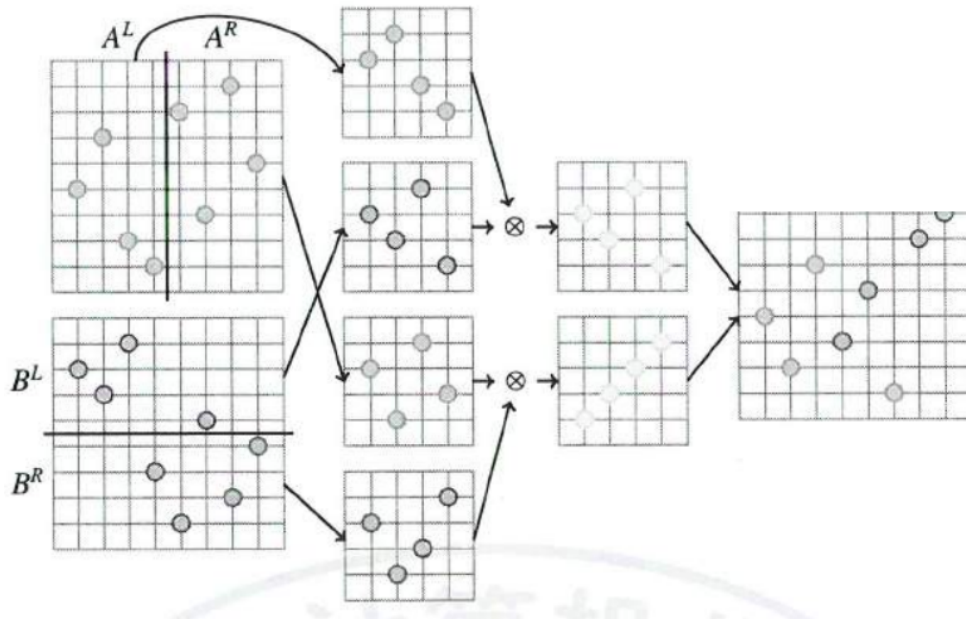
Thuật toán $O(n \log n)$ dùng chia để trị. Chia miền trung gian thành hai nửa: phần trái và phần phải tạo hai bài toán con. Sau khi xóa các hàng/cột toàn 0, mỗi nửa lại tương ứng với một hoán vị nhỏ hơn. Giải hai bài toán con thu được hai ma trận mật độ D^L, D^R . Hợp của chúng là một ma trận hoán vị, nhưng chưa hẳn là đáp án. Cần xét hiệu giữa hai giá trị ứng viên trái/phải; hiệu này đơn điệu theo cả hàng và cột, và hai ô kề nhau chỉ thay đổi nhiều nhất 1. Do đó có thể dùng hai con trỏ để tìm đường biên vùng hiệu bằng 0, thêm các điểm cần thiết, còn các vùng còn lại lấy trực tiếp từ D^L hoặc D^R . Mỗi tầng tốn $O(n)$, nên tổng thời gian là $O(n \log n)$.

Bài viết còn mô tả ý nghĩa của phép toán $\circ$. Hoán vị đơn vị $1, 2, \dots, n$ là đơn vị của phép toán. Một hoán vị chỉ có đúng một cặp nghịch thế kề nhau đóng vai trò như một phép đổi chỗ kề. Mọi hoán vị có thể tách thành tích của các phép đổi chỗ kề như vậy. Khi nhân $a \circ b$, có thể hiểu rằng ta quét các phép đổi chỗ kề của b : nếu hai đường tương ứng trong biểu diễn dây của a đã cắt nhau thì không đổi; nếu chưa cắt thì đổi chúng. Hình minh họa quá trình này bằng hai hàng điểm và các đường nối.

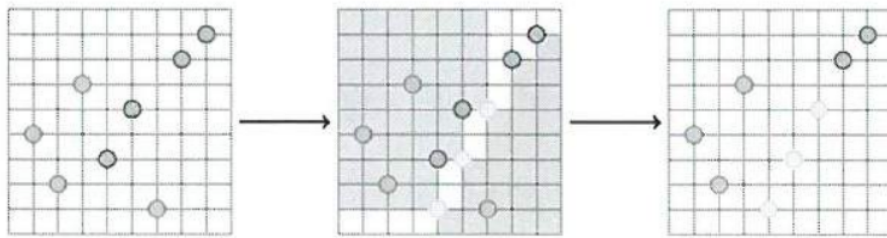
Hoán vị đảo $n, n-1, \dots, 1$ là phần tử hấp thụ: nhân nó với bất kỳ hoán vị nào ở bên trái hoặc bên phải đều trả lại chính nó. Ngoài ra, với mỗi hoán vị a có thể định nghĩa hoán vị quay a° sao cho $a^\circ \circ a$ là hoán vị đảo; về mặt ma trận, đây là phép quay ma trận mật độ đi một phần tư vòng.

11.3.5 Nhân ma trận Monge á đơn vị đơn giản

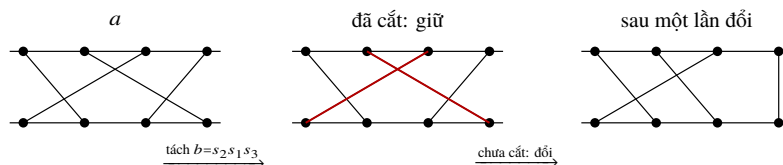
Với ma trận á hoán vị được biểu diễn bằng tập các điểm 1, cũng có thể tính tích của hai ma trận á đơn vị đơn giản trong $O(n \log n)$. Trước hết dùng bổ đề: khi nhân khoảng cách, nếu một hàng của mật độ bên trái toàn 0 hoặc một cột của mật độ bên phải toàn 0, có thể xóa chúng trước, tính tích trên phần còn lại



Hình 12: Quá trình chia đôi miền trung gian để nhận hai ma trận mật độ D^L và D^R ; đường đỏ ứng với D^L , đường xanh ứng với D^R .



Hình 13: Từ D^L, D^R tìm đường biên của vùng hiệu bằng 0, rồi bổ sung các điểm còn thiếu để thu được D .



Hình 14: Minh họa phép nhân $a \circ b$ trên biểu diễn dây của hoán vị. Tách b thành các đổi chỗ kề; với mỗi cặp dây, nếu chúng đã giao nhau trong a thì giữ nguyên, còn nếu chưa giao nhau thì thực hiện đổi chỗ.

rồi chèn lại sau. Sau khi xóa, hai ma trận vẫn có thể chưa phải hoán vị; ta bù thêm các hàng/cột và các điểm 1 thích hợp để biến chúng thành hoán vị. Việc bù ở phía trên hoặc bên phải không làm thay đổi các giá trị phân bố trên vùng gốc, nên có thể áp dụng thuật toán hoán vị ở trên rồi cắt kết quả trở lại.

Cách nhìn hình học cũng tương tự phép toán trên hoán vị: coi hàng toàn 0 của ma trận bên trái như một điểm ở cột $-\infty$, và hàng toàn 0 của ma trận bên phải như một điểm ở cột $+\infty$, rồi thực hiện cùng quá trình đổi chỗ các đường như trong phép $\circ$.

11.4 Ứng dụng

11.4.1 Duy trì ma trận khoảng cách của đồ thị lưới đặc biệt

Một lớp bài toán có bản chất là hỏi thông tin khoảng cách trên đồ thị lưới đặc biệt. Các ma trận khoảng cách của những đồ thị này thường là ma trận Monge á đơn vị đơn giản, nên có thể dùng phép nhân nhanh.

Ví dụ 1: LIS trên đoạn tính. Cho một hoán vị độ dài n và q truy vấn $[l, r]$, hỏi độ dài dãy con tăng dài nhất của hoán vị trong đoạn đó. Có thể xét cỡ ràng buộc $n, q \leq 10^5$.

Xét đồ thị lưới có $n + 1$ hàng và $n + 1$ cột. Từ mỗi ô có cạnh sang phải trọng số 1 và cạnh xuống dưới trọng số 0; với mỗi phần tử hoán vị $a_x = y$, thêm cạnh chéo trọng số 0 từ (y, x) tới $(y + 1, x + 1)$. Gọi $A_{l,r}$ là độ dài đường đi ngắn nhất từ $(0, l)$ tới (n, r) . Khi đó độ dài LIS trong đoạn $[l, r]$ là $r - l - A_{l,r}$. Bằng lập luận hoán đổi hai đường đi ngắn nhất cắt nhau, ta có bất đẳng thức Monge cho A ; hiệu kề nhau không vượt quá 1, nên A là ma trận Monge á đơn vị đơn giản.

Nếu tính nhanh được mật độ A^Δ , vì nó chỉ có $O(n)$ điểm, mỗi truy vấn đoạn có thể trả lời bằng đếm điểm hai chiều. Để tính A^Δ , chia đồ thị lưới theo hàng thành hai nửa. Ma trận khoảng cách toàn cục bằng tích khoảng cách của hai ma trận con. Khi xóa các khoảng cột rỗng trong mỗi nửa, ta nhận được hai ma trận nhỏ hơn; sau khi giải đệ quy, chèn lại các khoảng rỗng tương ứng bằng thao tác tuyến tính trên biểu diễn hoán vị/tập điểm. Do phép nhân ma trận á đơn vị đơn giản tốn $O(n \log n)$ trên mỗi mức, chọn điểm chia cân bằng cho tổng độ phức tạp xây dựng $O(n \log^2 n)$. Sau đó offline truy vấn bằng đếm điểm hai chiều.

Ví dụ 2: LCS giữa tiền tố và xâu con. Cho hai xâu a, b độ dài lần lượt n, m . Mỗi truy vấn gồm x, y, z , yêu cầu độ dài LCS giữa tiền tố độ dài x của a và xâu con $b[y : z]$. Bài gốc nêu $n, m, q \leq 10^4$.

Dựng đồ thị lưới tương tự: nếu $a_i = b_j$, thêm cạnh chéo trọng số 0 từ (i, j) tới $(i + 1, j + 1)$. Gọi $A_{l,r}^{(i)}$ là độ dài đường đi ngắn nhất từ $(0, l)$ tới (i, r) , và đặt $A_{l,r}^{(i)} = 0$ khi $r \leq l$. Khi đó mỗi $A^{(i)}$ là ma trận Monge á đơn vị đơn giản, và truy vấn (x, y, z) có đáp án

$$z - y - A_{y,z}^{(x)}.$$

Nếu $B_{l,r}^{(i)}$ là độ dài đường đi ngắn nhất từ (i, l) tới $(i + 1, r)$, với quy ước $B_{l,r}^{(i)} = 0$ khi $r \leq l$, thì

$$A^{(i)} = A^{(i-1)} B^{(i-1)}.$$

Với $l < r$, ma trận bước này có dạng rất thưa:

$$B_{l,r}^{(i)} = r - l - [\exists p \in [l, r), a_i = b_p].$$

Để dựng biểu diễn hoán vị của $B^{(i)M}$, xét với mỗi $l, l + 1$ vị trí nhỏ nhất r sao cho $B_{l,r}^{(i)} - B_{l+1,r}^{(i)} = 1$. Trường hợp $r \leq l$ không xảy ra; với $r = l + 1$ điều kiện tương đương $a_i \neq b_l$; với $r > l + 1$ nó tương đương hai mệnh đề $[\exists p \in [l, r), a_i = b_p]$ và $[\exists p \in [l + 1, r), a_i = b_p]$ khác nhau. Vì vậy nếu $a_i \neq b_l$ thì vị trí nhỏ nhất là $l + 1$; nếu không, lấy $l_1 > l$ nhỏ nhất sao cho $a_i = b_{l_1}$, vị trí nhỏ nhất là $l_1 + 1$. Quét ngược một lần sẽ tính được $B^{(i)M}$, từ đó tính lần lượt các $A^{(i)}$ và xử lý các truy vấn theo mô tả của bài gốc.

11.4.2 Quy hoạch động một chiều–một chiều

Một số bài toán quy hoạch động một chiều–một chiều có ma trận chi phí là ma trận Monge á đơn vị đơn giản. Nhờ đó các bài toán truy vấn trên nhiều lớp chuyển tiếp có thể đạt độ phức tạp tốt hơn các kỹ thuật tối ưu DP thông dụng.

Ví dụ 3. Cho một dãy độ dài n và q truy vấn. Mỗi truy vấn cho đoạn $[l, r)$ và số k , hỏi nếu chia đoạn này thành nhiều nhất k đoạn con thì tổng số màu khác nhau trong các đoạn con lớn nhất là bao nhiêu. Ràng buộc trong bài gốc ở cỡ $n, q \leq 10^5$.

Đánh số chỉ số từ 0. Xét ma trận A trên $[0 : n] \times [0 : n]$; với $r > l$, đặt $r - l - A_{l,r}$ bằng số màu khác nhau trong đoạn $[l, r)$, còn với $r \leq l$ đặt $A_{l,r} = 0$. Số màu khác nhau thỏa bất đẳng thức tứ giác theo chiều ngược, nên A là ma trận Monge trong miền hợp lệ; các trường hợp biên cũng thỏa điều kiện Monge. Hơn nữa các hiệu kề nhau bị chặn bởi 1, vì vậy A là ma trận Monge á đơn vị đơn giản.

Khi cần chia thành nhiều đoạn, ta cần các lũy thừa theo phép nhân khoảng cách của A . Các chuyển tiếp với $r \leq l$ không tối ưu, nên phép nhân khoảng cách mô tả đúng bài toán. Mật độ A^Δ có thể tính trong $O(n)$: với mỗi vị trí l , điểm làm thay đổi sai phân tương ứng chính là vị trí xuất hiện kế tiếp của màu tại l , cộng thêm một. Sau đó dùng phép nhân nhanh giữa các ma trận Monge á đơn vị đơn giản để tính các lũy thừa, rồi trả lời truy vấn offline bằng đếm điểm hai chiều. Cấu trúc thuật toán là tiền xử lý các lũy thừa ma trận và trả lời từng truy vấn bằng một truy vấn hình học trên mật độ.

Các bài toán tương tự có thể xử lý cùng một khuôn mẫu: nếu ma trận chi phí là Monge á đơn vị đơn giản, ta biểu diễn nó bằng mật độ thưa, dùng phép nhân nhanh và đọc đáp án từ mật độ hoặc phân bố của ma trận kết quả.

11.4.3 Kiểm tra thứ tự Bruhat

Cấu trúc của phép nhân ma trận Monge đơn vị đơn giản còn liên quan tới thứ tự Bruhat trên hoán vị. Với hai hoán vị a, b , viết $a < b$ nếu có thể biến a thành b bằng một số lần đổi chỗ hai phần tử đang tạo thành một cặp đúng thứ tự.

Bổ đề. Với hai hoán vị a, b của $[1 : n]$, ta có $a < b$ khi và chỉ khi mọi phần tử của ma trận phân bố $\text{Mat}(a)^\Sigma$ không vượt quá phần tử tương ứng của $\text{Mat}(b)^\Sigma$.

Chứng minh. Chiều cần thiết xuất phát từ việc đổi một cặp đúng thứ tự chỉ làm các phần tử trong ma trận phân bố tăng hoặc giữ nguyên. Với chiều đủ, xét tổng chênh lệch giữa hai ma trận phân bố và quy nạp theo tổng này. Nếu tổng bằng 0 thì hai hoán vị bằng nhau. Nếu không, chọn từ dưới lên hàng đầu tiên mà hai ma trận mật độ khác nhau; trong hàng đó vị trí của 1 ở a phải nằm bên trái vị trí của 1 ở b . Tiếp tục đi lên để tìm một hàng tạo được cặp có thể đổi chỗ sao cho mọi hiệu phân bố vẫn không âm nhưng tổng chênh lệch giảm. Đưa phép đổi này vào đầu hoặc cuối dãy thao tác quy nạp sẽ hoàn thành chứng minh; lập luận chính là quy nạp bằng hiệu hai ma trận phân bố.

Định lý. Gọi z là hoán vị đảo $n, n-1, \dots, 1$. Với hai hoán vị a, b của $[1 : n]$, ta có $a < b$ khi và chỉ khi $a^\circ \circ b = z$, trong đó a° là hoán vị quay tương ứng với phép quay mật độ của a và $\circ$ là phép nhân hoán vị sinh bởi ma trận Monge đơn vị đơn giản.

Chứng minh. Đặt $A = \text{Mat}(a)$, $B = \text{Mat}(b)$, và để C là ma trận mật độ ứng với a° . Khi $a < b$, bổ đề trên cho $A^\Sigma \leq B^\Sigma$ theo từng phần tử. Nhân khoảng cách bên trái bởi C^Σ bảo toàn thứ tự này. Mặt khác

$C^{\Sigma}A^{\Sigma}$ chính là ma trận của hoán vị đảo; kết hợp với cận biên của ma trận Monge đơn vị suy ra $a^{\circ} \circ b = z$. Chiều ngược dùng cùng bất đẳng thức min-plus: từ $a^{\circ} \circ b = z$ suy ra các phần tử của B^{Σ} không nhỏ hơn A^{Σ} , rồi áp dụng bổ đề.

Nhờ định lý trên, bài toán kiểm tra $a < b$ có thể giải trong $O(n \log n)$ bằng thuật toán nhân hoán vị của ma trận Monge đơn vị. Cũng có thể dùng trực tiếp bổ đề: quét từng cột của hiệu hai ma trận phân bố bằng cây đoạn hỗ trợ cộng đoạn và hỏi giá trị nhỏ nhất, cũng đạt $O(n \log n)$.

Phần này không tạo ra độ phức tạp tốt hơn cho bài toán kiểm tra thứ tự Bruhat, nhưng cho thấy phép toán $\circ$ có nhiều cấu trúc đáng nghiên cứu. Do phạm vi bài viết, tác giả chỉ nêu mối liên hệ này và khuyến nghị độc giả tham khảo bài báo gốc để xem các chứng minh đầy đủ hơn.

11.5 Tổng kết

Bài viết giới thiệu ma trận Monge đơn vị đơn giản như một cấu trúc mạnh: nó biểu diễn được một ma trận khoảng cách $n \times n$ đặc biệt chỉ bằng $O(n)$ không gian, hỗ trợ nhân hai ma trận và nhân với vector trong $O(n \log n)$. Cấu trúc này xuất hiện trong nhiều bài toán OI như LIS, LCS, số lượng phần tử khác nhau trên đoạn và các biến thể quy hoạch động.

Các thuật toán liên quan tương đối dễ cài đặt, hằng số hợp lý và có thể phân biệt rõ với lời giải vét cạn. Tuy vậy bài viết mới chỉ giới thiệu tầng cơ bản; các tính chất sâu hơn, cũng như cách đưa chúng tự nhiên vào đề thi chất lượng, vẫn còn nhiều không gian nghiên cứu.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Xiao Ran của Trường Trung học trực thuộc Đại học Bắc Kinh đã quan tâm và hướng dẫn; cảm ơn gia đình, bạn bè đã ủng hộ; cảm ơn anh Li Baitian đã thảo luận và gợi ý.

Tài liệu tham khảo

1. *Static Range LIS Query*. Library Checker, https://judge.yosupo.jp/problem/static_range_lis_query.
2. *Prefix-substring LCS*. Library Checker: judge.yosupo.jp/problem/prefix_substring_lcs.
3. Tiskin, A. *Fast Distance Multiplication of Unit-Monge Matrices*. *Algorithmica*, 71(4):859–888, 2015.
4. Aggarwal, A.; Klawe, M. M.; Moran, S.; Shor, P.; Wilber, R. *Geometric applications of a matrix-searching algorithm*. *Algorithmica*, 2(1):195–208, 1987.
5. *The Bakery*. Codeforces 834D, <https://codeforces.com/problemset/problem/834/D>.

12 Bàn về phân tích độ phức tạp của một lớp bảng băm

Chen Yusi, Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc

Tóm tắt

Bài viết giới thiệu ngắn gọn về bảng băm, một số cách xử lý xung đột băm, đồng thời phân tích độ phức tạp của chúng dưới các giả thiết khác nhau về hàm băm.

12.1 Mở đầu

Bảng băm là một cấu trúc dữ liệu từ điển quen thuộc. So với cây cân bằng và nhiều cấu trúc từ điển khác, bảng băm thường dễ cài đặt hơn và chạy nhanh hơn, nên được dùng rộng rãi trong thi tin học lẫn hệ thống thực tế. Tuy nhiên, trong tài liệu OI tiếng Trung, việc phân tích chặt độ phức tạp của bảng băm không phổ biến. Bài viết này trình bày vài cách cài đặt thông dụng và phân tích thời gian, bộ nhớ của chúng.

12.2 Bảng băm

Ký hiệu $[s] = \{0, 1, \dots, s-1\}$. Bảng băm là cấu trúc lưu các cặp khóa–giá trị và hỗ trợ:

1. $\text{Insert}(x, v)$: chèn khóa x với giá trị v ;
2. $\text{Modify}(x, v)$: sửa giá trị ứng với khóa x thành v ;
3. $\text{Delete}(x)$: xóa khóa x và giá trị tương ứng;
4. $\text{Query}(x)$: hỏi giá trị ứng với khóa x .

Các tên thao tác được giữ nguyên như mẫu trong bài gốc.

Gọi n là số phần tử trong bảng băm; n không vượt quá tổng số thao tác. Giả sử tập khóa là U và bảng có m vị trí, thường chọn $m = O(n)$ và $m < |U|$. Với một thao tác trên khóa x , bảng băm dùng hàm $h : U \rightarrow [m]$ để tính vị trí $y = h(x)$ rồi thao tác trên vị trí đó.

Nếu hai khóa khác nhau x_1, x_2 có $h(x_1) = h(x_2)$ thì xảy ra xung đột băm. Phần sau giới thiệu vài mức giả thiết về hàm băm và hai cách xử lý xung đột.

12.3 Hàm băm

Định nghĩa. Một ánh xạ ngẫu nhiên từ tập khóa U tới $[m]$ được gọi là một hàm băm.

Định nghĩa. Hàm băm *hoàn toàn ngẫu nhiên* là hàm được chọn đều từ mọi ánh xạ $U \rightarrow [m]$.

Định nghĩa. Một nhóm biến ngẫu nhiên $X_0, X_1, \dots, X_{n-1}$ nhận giá trị trong S được gọi là k -độc lập nếu với mọi k chỉ số phân biệt và mọi bộ giá trị tương ứng, xác suất đồng thời bằng tích xác suất đều, tức là $1/|S|^k$ trong trường hợp đều.

Định nghĩa. Một họ hàm băm $h : U \rightarrow [m]$ là k -độc lập nếu với mọi k khóa phân biệt $x_0, \dots, x_{k-1}$, các biến $h(x_0), \dots, h(x_{k-1})$ là k -độc lập.

Trực giác là: hàm băm càng độc lập thì các giá trị băm của những khóa khác nhau càng giống các biến ngẫu nhiên độc lập, và xung đột càng khó bị cấu trúc xấu chi phối. Nếu một họ hàm là $(k+1)$ -độc lập thì nó cũng là k -độc lập; hơn nữa một họ k -độc lập cũng là k' -độc lập với mọi $k' \leq k$.

Một cách xây dựng chuẩn là dùng đa thức trên trường hữu hạn. Chọn số nguyên tố $p > |U|$, chọn ngẫu nhiên $a_0, a_1, \dots, a_{k-1} \in [p]$ và đặt

$$H(x) = a_{k-1}x^{k-1} + \dots + a_1x + a_0 \pmod{p}, \quad h(x) = H(x) \bmod m.$$

Với k khóa phân biệt, hệ phương trình tuyến tính theo các hệ số a_i có ma trận Vandermonde không suy biến, nên các giá trị $H(x)$ là độc lập; do đó $h(x)$ cho một họ băm k -độc lập sau phép lấy modulo.

Định nghĩa. Một họ hàm băm $h : U \rightarrow [m]$ là *toàn vực* nếu với mọi $x_1 \neq x_2$,

$$\Pr[h(x_1) = h(x_2)] \leq \frac{1}{m}.$$

Mọi họ 2-độc lập đều là toàn vực. Các quan hệ này cho một thang mạnh–yếu giữa hàm băm hoàn toàn ngẫu nhiên, k -độc lập và toàn vực.

12.4 Phương pháp nối chuỗi riêng

Phương pháp nối chuỗi riêng (*separate chaining*) tạo một danh sách liên kết ban đầu rỗng cho mỗi giá trị băm. Khi chèn (x, v) , tính $y = h(x)$ rồi chèn cặp này vào đầu danh sách của y . Các thao tác sửa, xóa và hỏi tìm khóa trong danh sách tương ứng rồi xử lý. Nếu danh sách của y có độ dài c_y , một thao tác đi qua không quá c_y phần tử. Vì vậy cần xét

$$L = \max_{y \in [m]} c_y,$$

độ dài danh sách lớn nhất.

Với hàm băm hoàn toàn ngẫu nhiên, khi $n = \alpha m$ và α là hằng số nhỏ hơn 1, kỳ vọng và trung vị của L đều có bậc

$$\Theta\left(\frac{\log n}{\log \log n}\right).$$

Ý tưởng chứng minh là ước lượng xác suất một danh sách có đúng k phần tử:

$$\Pr[c_y = k] = \binom{n}{k} \left(\frac{1}{m}\right)^k \left(1 - \frac{1}{m}\right)^{n-k}.$$

Từ đó tính xác suất mọi danh sách đều ngắn hơn k , rồi chọn ngưỡng j sao cho xác suất này chuyển từ nhỏ sang lớn. Công thức Stirling cho thấy $j = \Theta(\log n / \log \log n)$.

Kết quả này nói rằng độ phức tạp tệ nhất trong một dãy thao tác của nối chuỗi riêng không phải $O(1)$ như cách nói trực giác thường gặp, mà có kỳ vọng $\Theta(\log n / \log \log n)$. Để đạt cận trên cùng bậc không cần hàm băm hoàn toàn ngẫu nhiên; với họ toàn vực, xác suất va chạm cặp vẫn đủ để có cận $O(\log n / \log \log n)$ với xác suất cao.

Định lý. Với họ hàm băm toàn vực và nối chuỗi riêng, nếu $\alpha = n/m$ là hằng số thì với mọi $c > 0$ tồn tại hằng số a sao cho

$$\Pr\left[L > a \frac{\log n}{\log \log n}\right] < n^{-c}.$$

Chứng minh. Xét một danh sách cố định có kỳ vọng $\mu = \alpha$. Dùng cận Chernoff cho đuôi của độ dài danh sách rồi hợp trên m danh sách. Chọn hằng số a đủ lớn để xác suất mỗi danh sách vượt ngưỡng nhỏ hơn n^{-c-1} , khi đó xác suất tồn tại danh sách vượt ngưỡng nhỏ hơn n^{-c} .

Dù cận tệ nhất không phải $O(1)$, nối chuỗi riêng chỉ cần giả thiết băm yếu, hằng số nhỏ và trong thực tế các truy vấn thường không đạt được số lượng cực hạn của phân tích, nên phương pháp này vẫn rất hiệu quả. C++ `unordered_map` cũng dùng một biến thể dựa trên nối chuỗi riêng.

12.5 Phương pháp dò tuyến tính

Phương pháp dò tuyến tính (*linear probing*) dùng một mảng T độ dài m , mỗi vị trí lưu một cặp khóa–giá trị. Khi chèn (x, v) , tính $y = h(x)$. Nếu T_y rỗng thì đặt cặp vào đó; nếu không, lần lượt kiểm tra

$T_y, T_{y+1}, \dots, T_{m-1}, T_0, T_1, \dots$ cho đến vị trí rỗng đầu tiên. Các thao tác sửa, xóa và hỏi cũng bắt đầu từ $h(x)$ và quét về sau đến khi gặp khóa cần tìm.

Một khóa x sẽ được lưu ở vị trí rỗng đầu tiên không trước $h(x)$ theo vòng tròn. Thời gian của thao tác trên x không vượt quá khoảng cách từ $h(x)$ tới vị trí rỗng kế tiếp.

Giả sử $m > 3n$ và dùng họ hàm băm k -độc lập. Bài viết quan tâm tới kỳ vọng độ phức tạp tệ nhất của một thao tác, và riêng với dò tuyến tính chứng minh rằng 5-độc lập là đủ để kỳ vọng này là $O(1)$.

Định nghĩa. Một đoạn $R = [L, r]$ được gọi là đoạn liên tục cực đại nếu mọi vị trí trong R đều đã bị chiếm, còn hai vị trí ngay trước và ngay sau đoạn đều rỗng.

Đoạn liên tục cực đại chứa $h(q)$ là duy nhất, và thao tác Query(q) không quét quá $|R| + 1$ vị trí.

Định nghĩa. Một đoạn dạng $[i2^\ell, (i+1)2^\ell)$ được gọi là một ℓ -đoạn. Một ℓ -đoạn là *nguy hiểm* nếu có ít nhất $3 \cdot 2^\ell$ khóa có giá trị băm rơi vào đoạn đó.

Nếu một đoạn liên tục cực đại R có độ dài lớn hơn $2^{\ell+2}$, thì trong bốn ℓ -đoạn đầu tiên giao với R phải có một đoạn nguy hiểm: các ô trong phần giao đều bị chiếm, nên số khóa băm vào hợp các đoạn này vượt quá tổng dung lượng an toàn. Do đó, nếu đoạn chứa $h(q)$ có độ dài nằm giữa $2^{\ell+2}$ và $2^{\ell+3}$, thì trong một tập hằng số gồm các ℓ -đoạn gần $h(q)$ phải có một đoạn nguy hiểm.

Gọi p_ℓ là xác suất một ℓ -đoạn cố định nguy hiểm. Từ lập luận trên, kỳ vọng độ dài đoạn quét được chặn bởi

$$O\left(1 + \sum_{\ell=0}^{\log_2 m} 2^\ell p_\ell\right).$$

Cần ước lượng p_ℓ . Với một ℓ -đoạn I , đặt $X_x = 1$ nếu $h(x) \in I$ và 0 nếu ngược lại, rồi $X = \sum_{x \in S} X_x$. Ta có $\mu = E[X] \leq n2^\ell/m < 2^\ell/3$ do $m > 3n$. Nếu I nguy hiểm thì $X > 3 \cdot 2^\ell$, tức là $X - \mu$ lớn hơn hằng số nhân với 2^ℓ .

Với các biến 4-độc lập X_x , bài viết dùng ước lượng moment bậc bốn:

$$E[(X - \mu)^4] \leq 4\mu^2,$$

và bất đẳng thức Markov để suy ra

$$p_\ell = O(2^{-2\ell}).$$

Khi xét thao tác trên khóa q , điều kiện theo $h(q)$ vẫn để các giá trị băm của các khóa khác là 4-độc lập nếu họ ban đầu là 5-độc lập. Vì thế

$$\sum_{\ell \geq 0} 2^\ell p_\ell = \sum_{\ell \geq 0} O(2^{-\ell}) = O(1).$$

Định lý. Nếu $m > 3n$ và chọn một họ hàm băm 5-độc lập $h : U \rightarrow [m]$, thì kỳ vọng độ phức tạp tệ nhất của một thao tác trong dò tuyến tính là $O(1)$.

Thực ra với $m = (1 + \varepsilon)n$ và hàm băm 5-độc lập, có thể chứng minh kỳ vọng độ phức tạp tệ nhất là $O(\varepsilon^{-13/6})$; bài viết chỉ cần trường hợp tải nhỏ hơn để minh họa.

12.6 Tổng kết

Bài viết giới thiệu hai cách xử lý xung đột băm: nối chuỗi riêng và dò tuyến tính. Với nối chuỗi riêng, dưới họ hàm băm toàn vực, kỳ vọng độ phức tạp tệ nhất của một thao tác là $\Theta(\log n / \log \log n)$ trong chế độ tải hằng. Với dò tuyến tính, dưới họ hàm băm 5-độc lập và $m > 3n$, kỳ vọng độ phức tạp tệ nhất là $O(1)$.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn cha mẹ đã nhiều năm nuôi dưỡng và dạy dỗ; cảm ơn thầy Ye Jinyi và thầy Gu Duoyu đã hướng dẫn; cảm ơn anh Deng Mingyang, bạn Xu Tingqiang và bạn Xiang Zhaofan đã đọc kiểm tra bài viết.

Tài liệu tham khảo

1. Mikkel Thorup. *Linear Probing with 5-Independent Hashing*. 2017.
2. Mihai Patrascu and Mikkel Thorup. *The power of simple tabulation-based hashing*. Journal of the ACM, 2012.
3. G. H. Gonnet. *Expected length of the longest probe sequence in hash code searching*. Journal of the ACM, 1981.

13 Ứng dụng của đại số tuyến tính trong OI

Chang Ruinian, Trường Trung học Ba Thục Trùng Khánh

Tóm tắt

Bài viết khái quát khá toàn diện các phương pháp đại số tuyến tính trong OI, đồng thời mở rộng sang một số ứng dụng ít quen thuộc hơn của chúng.

13.1 Mở đầu

Những năm gần đây, đại số tuyến tính xuất hiện ngày càng nhiều trong đề thi OI. Các nội dung được khảo sát thường là định thức, cơ sở tuyến tính và vài kỹ thuật cơ bản khác, nhưng phạm vi của đại số tuyến tính không chỉ dừng ở đó. Bằng cách dùng ngôn ngữ tuyến tính, ta có thể rút ra nhiều kết luận thú vị và giải một số bài toán mới. Bài viết này trình bày vai trò của đại số tuyến tính qua nhiều nhánh ứng dụng.

13.2 Ký hiệu và quy ước

Giả sử độc giả đã biết đại số tuyến tính cơ bản, các phép toán đa thức cơ bản, đếm tổ hợp và một ít kiến thức về quá trình ngẫu nhiên. Nếu không nói thêm, mọi đối tượng được xét trên một trường; trong nhiều đoạn, ta giả sử ma trận đang xét có các giá trị riêng cần dùng trong trường mở rộng thích hợp.

Với ma trận A , ký hiệu $A[S, T]$ là ma trận con lấy các hàng trong dãy hoặc tập S và các cột trong dãy hoặc tập T , theo thứ tự tăng khi S, T là tập. Viết $A[S] = A[S, S]$. Ký hiệu A^* là ma trận phụ hợp, $|A|$ là định thức của ma trận A , còn $|X|$ là lực lượng tập X . Ký hiệu 0 và I lần lượt là ma trận không và ma trận đơn vị; nếu cần chỉ rõ kích thước thì ghi ở chỉ số dưới. Bài viết dùng ngoặc Iverson $[P]$, bằng 1 khi mệnh đề P đúng và bằng 0 khi sai.

13.3 Cơ sở

13.3.1 Định thức

Trước hết nhắc lại một số đẳng thức định thức sẽ được dùng nhiều lần.

Định lý 3.1: khai triển Laplace tổng quát. Với ma trận vuông A và các tập chỉ số I, J có cùng kích thước,

$$|A| = \sum_{|J'|=|I|} \text{sgn}(I, J') |A[I, J']| |A[\bar{I}, \bar{J}']|.$$

Dấu phụ thuộc vào tổng chỉ số của các hàng/cột được chọn. Chứng minh trực tiếp từ định nghĩa định thức.

Định lý 3.2: đồng nhất thức Grassmann. Giả sử A là ma trận có tập hàng R , tập cột C , và $|R| \leq |C|$. Với $J, J' \subseteq C$, $|J| = |J'| = |R|$, và $i \in J \setminus J'$, ta có

$$|A[R, J]| \cdot |A[R, J']| = \sum_{j \in J' \setminus J} |A[R, J - i + j]| \cdot |A[R, J' + i - j]|.$$

Ở đây $J - i + j$ nghĩa là thay cột i trong J bằng cột j , giữ nguyên vị trí của i ; ký hiệu $J' + i - j$ được hiểu tương tự.

Chứng minh. Xét ma trận khối

$$\tilde{A} = \begin{pmatrix} A[R, J] & A[R, J'] \\ A' & A[R, J'] \end{pmatrix},$$

trong đó A' chỉ có cột thứ k bằng $A[R, \{i\}]$, các cột còn lại bằng 0, với k là vị trí của i trong J . Định thức của $\tilde{A}$ hiển nhiên bằng 0; khai triển theo $|R|$ hàng cuối cho đúng đẳng thức trên.

Định lý 3.3: Cauchy–Binet. Nếu A là ma trận $n \times m$, B là ma trận $m \times n$ và $n \leq m$, thì

$$|AB| = \sum_{\substack{S \subseteq [m] \\ |S|=n}} |A[[n], S]| |B[S, [n]]|.$$

Chứng minh có thể làm bằng ma trận khối

$$\begin{pmatrix} 0 & A \\ B & I_m \end{pmatrix}$$

rồi khai triển theo một phía và so với phép biến đổi sơ cấp.

Định lý 3.4: phần bù Schur. Nếu D khả nghịch thì

$$\det \begin{pmatrix} A & B \\ C & D \end{pmatrix} = |D| \cdot |A - BD^{-1}C|.$$

Đẳng thức nhận được bằng biến đổi sơ cấp ma trận khối.

13.3.2 Hạng

Hạng của ma trận cũng là một đại lượng trung tâm. Các bất đẳng thức cơ bản gồm

$$\text{rank}(AB) \leq \min(\text{rank } A, \text{rank } B),$$

và

$$\text{rank}(AB) \geq \text{rank } A + \text{rank } B - \dim C,$$

trong đó C là không gian trung gian của ánh xạ tuyến tính. Các công thức này trở nên tự nhiên khi xem ma trận như ánh xạ tuyến tính.

Một hệ quả của phần bù Schur là nếu khối D khả nghịch thì hạng của ma trận khối có thể viết qua hạng của $A - BD^{-1}C$. Ngoài ra,

$$\text{rank } A = \max_{\substack{I \subseteq R, J \subseteq C \\ |I|=|J|}} \{|J| : |A[I, J]| \neq 0\},$$

tức hạng bằng cấp của định thức con không suy biến lớn nhất. Vì vậy nhiều bất đẳng thức về hạng có thể chứng minh bằng các đồng nhất thức định thức.

Ba tính chất sau là tương đương với ma trận vuông: các hàng độc lập tuyến tính tối đại đủ số, các cột độc lập tuyến tính tối đại đủ số, và ma trận không suy biến. Độ phức tạp tốt nhất của bốn việc tính hạng, tìm định thức, nghịch đảo ma trận không suy biến và nhân hai ma trận thường được ký hiệu bằng cùng một số mũ ω ; trong cài đặt OI, thuật toán $O(n^3)$ vẫn là lựa chọn chủ yếu.

13.3.3 Ma trận tổng quát

Khi phần tử ma trận chứa biến hình thức, ta cần khái niệm độc lập đại số. Một tập phần tử $y_1, \dots, y_k$ là độc lập đại số trên trường K nếu không tồn tại đa thức khác không p trên K sao cho $p(y_1, \dots, y_k) = 0$.

Với ma trận có phần tử chứa biến, *hạng tổng quát* được định nghĩa là kích thước lớn nhất của một định thức con không đồng nhất bằng 0. Tương đương, chọn một mở rộng trường và thay các biến bởi các giá trị độc lập đại số đủ tổng quát; hạng tổng quát là hạng lớn nhất có thể thu được sau phép thế.

13.3.4 Dạng chuẩn Jordan

Khối Jordan $J_n(\lambda)$ là ma trận có λ trên đường chéo chính, 1 trên đường chéo phụ ngay phía trên và 0 ở nơi khác. Dạng chuẩn Jordan nói rằng một ma trận thích hợp tương tự với ma trận khối chéo gồm các khối Jordan. Từ dạng này dễ suy ra các công thức:

$$\text{tr}(A^k) = \sum_i \lambda_i^k, \quad |\exp A| = \exp(\text{tr } A),$$

và nếu f là đa thức hoặc hàm hình thức phù hợp thì giá trị riêng của $f(A)$ là $f(\lambda_i)$, có xét tới cấu trúc khối Jordan khi cần.

13.4 Ma trận có sửa đổi

13.4.1 Ma trận phụ hợp

Với ma trận A , phần bù đại số của phần tử (i, j) là

$$C_{i,j} = (-1)^{i+j} M_{i,j},$$

trong đó $M_{i,j}$ là định thức con sau khi xóa hàng i và cột j . Ma trận phụ hợp A^* có phần tử $A_{i,j}^* = C_{j,i}$, và thỏa

$$AA^* = A^*A = |A|I.$$

Vì phần bù đại số là đạo hàm của định thức theo phần tử tương ứng, ma trận phụ hợp thường dùng để xử lý các bài toán tạm sửa ma trận, truy vấn nghịch đảo động hoặc cập nhật hạng thấp.

Nếu A không suy biến thì $A^* = |A|A^{-1}$. Nếu $\text{rank } A < n - 2$, mọi phần bù đại số đều bằng 0, nên $A^* = 0$. Trường hợp $\text{rank } A = n - 1$ phức tạp hơn: khi đó $\text{rank } A^* = 1$. Ta có thể tìm hai vector khác không u, v sao cho $Au = 0$ và $v^T A = 0$, rồi A^* có dạng kuv^T . Chỉ cần tính một phần bù đại số không bằng 0 để xác định hệ số k .

13.4.2 Công thức Sherman–Morrison–Woodbury

Công thức Sherman–Morrison–Woodbury, viết tắt SMW, xử lý hiệu quả các sửa đổi hạng thấp, sửa đổi ma trận con và nghịch đảo động. Với các ma trận có kích thước phù hợp,

$$(A - UC^{-1}V)^{-1} = A^{-1} + A^{-1}U(C - VA^{-1}U)^{-1}VA^{-1},$$

khi cả A và $C - VA^{-1}U$ khả nghịch. Chứng minh bằng cách xét ma trận khối

$$\begin{pmatrix} A & U \\ V & C \end{pmatrix}$$

rồi biến đổi khối và lấy nghịch đảo hai vế tương ứng.

Hai hệ quả thường dùng là xóa một hàng/cột khỏi ma trận nghịch đảo đã biết, và thêm một hàng/cột vào ma trận.

Với thao tác xóa hàng/cột, đặt $N = M^{-1}$. Nếu A là ma trận thu được từ ma trận khả nghịch M bằng cách xóa hàng và cột cùng chỉ số s , ký hiệu $\bar{s}$ là tập chỉ số còn lại. Khi $N_{s,s} \neq 0$, ma trận A khả nghịch và

$$A^{-1} = N[\bar{s}, \bar{s}] - N[\bar{s}, s] N[s, \bar{s}] N_{s,s}^{-1}.$$

Trong bản nguồn, hệ quả này được viết cho trường hợp xóa hàng/cột cuối rồi lấy khối con chính bậc n .

Với thao tác thêm hàng/cột, trước hết có thể thêm một hàng vào cuối:

$$\begin{pmatrix} A & 0 \\ v^T & 1 \end{pmatrix},$$

rồi thêm cột cuối để thu được

$$\begin{pmatrix} A & u \\ v^T & t \end{pmatrix}.$$

Mỗi bước là một sửa đổi hạng thấp nên có thể cập nhật nghịch đảo bằng công thức SMW.

Cập nhật vùng nhỏ. Giả sử ma trận M chỉ thay đổi trên ma trận con chính $M[S]$, phần thay đổi là $\Delta_{S,S}$ và $N = M^{-1}$ là nghịch đảo trước cập nhật. Khi đó ma trận mới khả nghịch khi và chỉ khi $I + \Delta_{S,S}N[S]$ khả nghịch, và nghịch đảo mới là

$$N - N[*, S](I + \Delta_{S,S}N[S])^{-1}\Delta_{S,S}N[S, *].$$

Đặc biệt, phần nghịch đảo trên chính tập S có thể tính trong $O(|S|^\omega)$, rất hữu ích khi mỗi lần sửa chỉ chạm một vùng nhỏ.

13.5 Liên hệ với đồ thị

13.5.1 Hai không gian tuyến tính quan trọng trên đồ thị

Với đồ thị $G = (V, E)$, xem mỗi tập cạnh như một vector 0/1 trên E , cộng là hiệu đối xứng. *Không gian chu trình* gồm các tập cạnh Euler, tức mỗi đỉnh có bậc chẵn. *Không gian cắt* gồm các tập cạnh cắt giữa một tập đỉnh S và $V \setminus S$.

Một cơ sở của không gian chu trình nhận được bằng cách chọn một rừng sinh, rồi với mỗi cạnh không thuộc rừng lấy chu trình cơ bản tạo bởi cạnh đó và đường đi trên rừng. Một cơ sở của không gian cắt nhận được từ các cạnh cây và các cạnh không cây cắt qua cạnh đó. Suy ra số tập Euler là $2^{|E|-|V|+c}$, số tập cắt là $2^{|V|-c}$, với c là số thành phần liên thông. Hai không gian này trực giao trên $\mathbb{F}_2$.

Ví dụ 1: DZY loves chinese. Cho một đồ thị, mỗi truy vấn cho một tập không quá 15 cạnh và hỏi liệu có tồn tại một tập con là cắt hay không. Cách làm: coi mỗi cạnh không cây là một biến hình thức trên $\mathbb{F}_2$; trọng số của một cạnh cây là tổng các biến của cạnh không cây đi qua nó. Khi đó trong tập cạnh truy vấn có một cắt khi và chỉ khi các vector trọng số tương ứng phụ thuộc tuyến tính. Thay các biến bằng giá trị ngẫu nhiên trên $\mathbb{F}_{2^w}$ cho xác suất sai nhỏ; với $w = 64$ và $k = 15$, xác suất sai rất thấp.

13.5.2 Matching và ma trận Tutte

Đại số tuyến tính thường xuất hiện nhất trong đồ thị qua bài toán matching, vì phương pháp đại số xử lý được bài toán parity của matroid tuyến tính.

Với đồ thị $G = (V, E)$, ma trận Tutte $T(G)$ là ma trận đối xứng lệch có hàng/cột đánh số bởi V :

$$T_{i,j} = \begin{cases} x_{i,j}, & i < j, ij \in E, \\ -x_{j,i}, & i > j, ij \in E, \\ 0, & \text{ngược lại.} \end{cases}$$

Khi không nhầm lẫn, bài viết viết tắt là T . Định lý kinh điển:

$$\text{rank } T = 2\nu(G),$$

trong đó $\nu(G)$ là kích thước matching lớn nhất. Chứng minh có thể xem trong tập luận văn năm 2017.

Pfaffian. Với ma trận đối xứng lệch $2n \times 2n$, Pfaffian $\text{Pf}(M)$ là tổng có dấu trên mọi perfect matching của $[2n]$, mỗi hạng tử là tích các phần tử ma trận trên matching đó. Hai tính chất quan trọng là

$$\text{Pf}(BMB^T) = |B| \text{Pf}(M), \quad |M| = \text{Pf}(M)^2.$$

Do đó có thể tính Pfaffian bằng khử Gaussian chuyên cho ma trận đối xứng lệch: mỗi bước chọn một phần tử khác 0 ở cột đầu để tạo một khối 2×2 , dùng hai hàng/cột đầu để triệt các phần tử còn lại, rồi đệ quy trên khối con bậc giảm hai. Các phép biến đổi là đồng dạng đối xứng nên không làm thay đổi Pfaffian ngoài hệ số đã kiểm soát.

Cũng như dùng det để thay permanent trong một số bài, ta có thể dùng Pfaffian trên ma trận Tutte để đại diện cho việc liệt kê matching. Gán ngẫu nhiên các biến của ma trận Tutte rồi dùng bổ đề Schwartz–Zippel sẽ kiểm tra được sự tồn tại của các cấu hình matching với xác suất cao. Ví dụ CodeChef PRECPAIR chuyển trọng số thành bậc đa thức, tính Pfaffian sau gán ngẫu nhiên rồi nội suy.

Pfaffian còn có khai triển giống định thức:

$$\text{Pf}(A) = \sum_i (-1)^i A_{1,i} \text{Pf}(A \setminus \{1, i\}),$$

với $A \setminus \{1, i\}$ là ma trận sau khi xóa hàng/cột 1 và i .

13.5.3 Dạng ma trận của định lý Tutte–Berge

Bài viết nêu một dạng tăng cường bằng ma trận của định lý Tutte–Berge. Với ma trận Tutte $A(G)$, các tập hàng/cột I, J , và

$$D(I, J) = \{(I', J') \mid A[I \setminus I', J'] = 0, A[I', J \setminus J'] = 0, I' \subseteq I, J' \subseteq J\},$$

hạng của $A[I, J]$ là

$$\text{rank } A[I, J] = \min_{(I', J') \in D(I, J)} (|I \setminus I'| + |J \setminus J'| + |I' \cap J'| - \text{odd}(G[I' \cap J'])),$$

trong đó $\text{odd}(G)$ là số thành phần liên thông có kích thước lẻ của G , còn $G[S]$ là đồ thị con cảm sinh bởi S .

Chứng minh dùng ba bổ đề về tính dưới mô-đun của hạng:

$$\text{rank } A[I, *] + \text{rank } A[I', *] \geq \text{rank } A[I \cap I', *] + \text{rank } A[I \cup I', *],$$

và dạng hai phía cho hàng/cột. Một bổ đề khác chọn được cặp tập con làm chặt độ khuyết hạng, còn bổ đề đồ thị nói rằng nếu trong đồ thị liên thông, xóa bất kỳ đỉnh nào cũng không làm giảm kích thước matching cực đại, thì đồ thị có số đỉnh lẻ và xóa bất kỳ đỉnh nào cũng còn perfect matching. Kết hợp lại cho bất đẳng thức hai chiều của công thức hạng.

Từ dạng ma trận trên suy ra định lý Tutte–Berge quen thuộc:

$$2\nu(G) = \min_S (2|V| - |S| - \text{odd}(G[V \setminus S])).$$

13.6 Quá trình ngẫu nhiên và các hệ quả

13.6.1 Chuỗi Markov và random walk trên đồ thị

Quá trình ngẫu nhiên là chủ đề thường gặp trong OI, và chuỗi Markov là phần liên hệ rất chặt với đại số tuyến tính. Một ma trận hàng ngẫu nhiên P thỏa $\sum_j P_{i,j} = 1$. Ký hiệu $P_{i,j}^n$ là xác suất từ trạng thái i tới trạng thái j sau n bước. Đặt $f_{i,j}^{(n)}$ là xác suất lần đầu tới j ở bước n , và m_i là thời gian quay về trung bình của trạng thái i .

Chu kỳ của trạng thái i là

$$p_i = \gcd\{n : P_{i,i}^n > 0\}.$$

Một trạng thái là thường hồi nếu tổng xác suất quay về bằng 1, ngược lại là tạm thời. Các trạng thái trong cùng thành phần liên thông mạnh có cùng tính thường hồi/tạm thời và cùng chu kỳ.

Định lý cơ bản: nếu trạng thái i thường hồi và có chu kỳ d , thì giới hạn của $P_{i,i}^n$ trên các bước phù hợp modulo d bằng nghịch đảo thời gian quay về trung bình. Với chu kỳ 1, các xác suất chuyển tiếp hội tụ về phân bố ổn định.

Một phân bố hàng π là phân bố ổn định nếu

$$\pi P = \pi, \quad \sum_i \pi_i = 1.$$

Với chuỗi bất khả quy và phi chu kỳ, mọi hàng của P^n hội tụ về cùng phân bố ổn định. Từ góc nhìn tuyến tính, P có giá trị riêng 1 và không gian riêng tương ứng của P^T cho phân bố ổn định sau chuẩn hóa.

Với chuỗi khả quy, tách các trạng thái thành các thành phần thường hồi dạng hồ và các trạng thái tạm thời. Phần tạm thời có ma trận con Q với mọi giá trị riêng có mô-đun nhỏ hơn 1, nên $I - Q$ khả nghịch. Dùng $(I - Q)^{-1}$ để tính xác suất hấp thụ vào từng hồ, rồi trong mỗi hồ tính phân bố ổn định. Với random walk trên đồ thị vô hướng có trọng số, điều kiện cân bằng chi tiết cho

$$\pi_i P_{i,j} = \pi_j P_{j,i}, \quad \pi_i = \frac{d_i}{\sum_j d_j},$$

trong đó d_i là tổng trọng số cạnh kề i .

13.6.2 Chỉ số hội tụ và chu kỳ của đồ thị có hướng

Với ma trận kề A của đồ thị có hướng, xét đại số Boolean: phép nhân là $\wedge$, phép cộng là $\vee$. Tồn tại $p, k_0 > 0$ sao cho với mọi $k \geq k_0$, $A^{k+p} = A^k$. Số p là chu kỳ của đồ thị, còn k_0 là chỉ số hội tụ.

Chỉ cần xét từng thành phần liên thông mạnh; chu kỳ toàn đồ thị là bội chung nhỏ nhất của chu kỳ các thành phần. Chu kỳ này đúng bằng gcd độ dài mọi chu trình trong thành phần. Để tính nhanh, dựng một cây DFS. Với mỗi cạnh không cây, lấy hiệu độ sâu hai đầu cộng/trừ 1 theo loại cạnh, rồi lấy gcd của các giá trị đó. Lập luận là mọi chu trình đơn có thể biến đổi dần để giảm số cạnh không cây, và mỗi bước chỉ thay đổi bởi một trong các hiệu nói trên.

Chỉ số hội tụ có thể chặn bằng cách chọn một tập con độ dài chu trình có gcd bằng 1 và dùng bài toán biểu diễn số nguyên không âm bằng tổ hợp tuyến tính của chúng. Cụ thể, theo Bezout, tồn tại các hệ số nguyên w_k sao cho $\sum w_k t_k = 1$. Tách các hệ số dương và âm, ta có thể rút ra ba độ dài liên tiếp cùng thuộc tập độ dài đi được; từ ba số liên tiếp đó suy ra mọi độ dài đủ lớn cũng biểu diễn được. Vì vậy với mỗi thành phần liên thông mạnh tồn tại một ngưỡng k_0 , sau ngưỡng đó quan hệ đạt được chỉ phụ thuộc vào phần dư theo chu kỳ của thành phần.

13.7 Đếm

13.7.1 Đường chéo hóa

Khi cần tính lũy thừa lớn của một biến đổi tuyến tính, nếu A tương tự với ma trận đường chéo

$$D = \text{diag}(\lambda_1, \dots, \lambda_n),$$

thì $A^k = Q^{-1} D^k Q$. Nhiều bài đếm có thể đưa về việc tính trace hoặc định thức của lũy thừa ma trận; khi ma trận là tuần hoàn hoặc có cấu trúc Vandermonde, các giá trị riêng có thể viết qua DFT.

13.7.2 Đếm bằng trace

Bài viết đưa ra ví dụ UOJ699: gán trọng số nguyên trong $[1, m]$ cho n vị trí sao cho một điều kiện dạng $w_i + K > w_{i+1} \pmod m$ được thỏa, hỏi số phương án modulo 998244353. DP trực tiếp tương đương với một ma trận chuyển M và đáp án là $\text{tr}(M^n)$. Nếu biết đa thức đặc trưng hoặc các giá trị riêng của M , trace của lũy thừa được tính bằng tổng lũy thừa các giá trị riêng.

Phần chứng minh chuyển bài toán sang tổng các định thức con chính của M , rồi lợi dụng cấu trúc dưới tam giác theo từng khối kích thước K . Trong mỗi khối chỉ chọn được một hàng/cột, và điều kiện giữa hai khối kề nhau chuyển thành một bài toán chèn khe; hệ số đa thức nhận được là các số tổ hợp. Sau đó phần còn lại là tuyến tính truy hồi. Với bài CF1580F2, chỉ cần đổi đoạn sau thành phân khối ma trận và quan sát tương tự.

13.8 Cơ sở tuyến tính

Trong đại số tuyến tính, để biểu diễn một vector x theo cơ sở $v_1, \dots, v_k$, ta giải hệ

$$c_1 v_1 + \dots + c_k v_k = x.$$

Trong cài đặt, không thể chỉ viết “giải hệ” cho mọi truy vấn, nên ta dùng cấu trúc dữ liệu cơ sở tuyến tính: khử các vector thành dạng tam giác trên. Bài viết gọi phần tử trong cơ sở tuyến tính là phần tử khử, còn phần tử trong cơ sở toán học ban đầu là phần tử cơ sở.

13.8.1 Giao của hai cơ sở tuyến tính

Giả sử biết cơ sở của hai không gian tuyến tính A, B và cần tìm cơ sở của $A \cap B$. Trước hết ghép các vector cơ sở của A và B lại rồi tìm một quan hệ phụ thuộc tuyến tính

$$c_1 a_1 + \dots + c_r a_r + d_1 b_1 + \dots + d_s b_s = 0.$$

Nếu chỉ có nghiệm tầm thường thì giao bằng 0. Ngược lại, phần thuộc B cho một vector trong giao. Sau đó dùng vector giao vừa tìm để thay một vector cơ sở của B có hệ số khác 0 và loại nó đi; lặp lại sẽ thu được cơ sở của giao.

Ví dụ UOJ698 cho nhiều không gian tuyến tính và nhiều truy vấn vector, hỏi lần đầu vector không xuất hiện trong giao tiền tố nào. Ta duy trì các giao tiền tố; do chỉ có tối đa w không gian khác nhau thật sự khi chiều tổng là w , có thể chen chúng theo thời gian vào một cơ sở mới và truy ngược thời điểm vector còn biểu diễn được.

13.8.2 Cơ sở tuyến tính có xóa

Với xóa offline, duy trì cơ sở tuyến tính theo thời điểm xóa lớn nhất: các vector cơ sở được sắp theo thời gian xóa giảm dần. Khi chen một vector mới, nếu cần thay một phần tử ở pivot nào đó, ta đưa phần tử bị thay quay lại quá trình chen với thời điểm xóa cũ. Cách nhìn này tương đương với việc mỗi vector đã bị khử ghi nhận rằng nó thuộc không gian sinh bởi các phần tử có thời điểm xóa không nhỏ hơn.

Với xóa online, cần biết biểu diễn tuyến tính của mọi phần tử theo các phần tử cơ sở. Nếu xóa một phần tử không thuộc cơ sở, chỉ việc xóa nó. Nếu xóa một phần tử cơ sở v , tìm một phần tử ngoài cơ sở có biểu diễn chứa v để pivot thay thế, giống phép xoay trong quy hoạch tuyến tính. Nếu không có phần tử thay thế, kích thước cơ sở giảm một; các biểu diễn còn lại được cập nhật bằng cách dùng một phần tử khử chứa v để triệt v trong các biểu diễn khác. Độ phức tạp mỗi lần xóa theo mô tả bài gốc là bậc đa thức theo số phần tử và kích thước cơ sở; khi hai đại lượng cùng cỡ thì phương pháp này khá thực dụng. Ví dụ kinh điển là UOJ453.

13.9 Bài toán parity của matroid tuyến tính

Bài toán parity của matroid tuyến tính là một lớp quan trọng trong tối ưu tổ hợp, bao hàm nhiều bài toán như luồng cực đại và matching cực đại, đổi lại độ phức tạp cao hơn.

Định nghĩa. Cho một matroid tuyến tính, các phần tử được chia thành nhiều cặp rời nhau. Cần chọn nhiều cặp nhất sao cho toàn bộ phần tử trong các cặp đã chọn độc lập. Ký hiệu số cặp tối đa là $\nu(A)$.

Bài viết nêu hai cách dựng đại số.

Dạng thưa. Giả sử matroid có biểu diễn tuyến tính bằng các cột của ma trận A . Gọi số cột là m ; dựng ma trận Tutte T của đồ thị có tập đỉnh là các cột, cạnh là các cặp phần tử trong bài toán parity. Khi đó

$$2\nu(A) = \text{rank} \begin{pmatrix} O & A \\ -A^T & T \end{pmatrix} - m,$$

trong đó O là khối không có kích thước phù hợp.

Dạng gọn. Với mỗi cặp vector cột (b_i, c_i) , gán một biến độc lập x_i và dùng tích ngoài

$$b_i \wedge c_i = b_i c_i^T - c_i b_i^T.$$

Khi đó

$$2\nu(A) = \text{rank} \sum_i x_i (b_i \wedge c_i).$$

Dạng này có lợi thế tính toán rõ rệt.

Để chứng minh dạng thưa, bài viết dùng bổ đề cho ma trận hỗn hợp đối xứng lách $K + T$, trong đó K là số trị còn T là ma trận biến hình thức: $K + T$ không suy biến khi và chỉ khi tồn tại tập chỉ số S sao cho $K[S]$ và $T[\bar{S}]$ đều không suy biến. Bổ đề suy từ khai triển Pfaffian của $K + T$. Sau đó tính chất đặc biệt của đồ thị cặp buộc các tập chỉ số được chọn theo từng cặp, đúng với điều kiện độc lập của matroid. Dạng gọn nhận được bằng phần bù Schur từ dạng thưa.

Khi cần dựng nghiệm, trước hết lấy một ma trận con chính không suy biến của ma trận gọn. Ta có thể cắt các vector về tập hàng tương ứng để giả sử ma trận đang xét là không suy biến. Sau đó lần lượt thử loại mỗi cặp (b_i, c_i) và kiểm tra hạng có giảm hay không; vì $b_i \wedge c_i$ có hạng không quá 2, dùng SMW để cập nhật nhanh. Tổng thời gian trong bài gốc có dạng $O(n^\omega + mr^2)$, với r là kích thước đáp án.

Ví dụ 4: RK 3-subcactus. Chọn nhiều tam giác rời cạnh nhất sao cho đồ thị cảm sinh bởi các đỉnh trong các tam giác đã chọn không có chu trình nào ngoài chính các tam giác đó. Đây là một bài toán parity của matroid tuyến tính: lấy matroid đồ thị, với mỗi tam giác chọn tùy ý hai cạnh làm một cặp. Chọn nhiều cặp độc lập nhất tương ứng với việc các cạnh được chọn tạo thành rừng sinh của cactus cạnh.

13.10 Một số ứng dụng khác

13.10.1 Phương pháp đồ thị nối tiếp-song song tổng quát

Nhiều định thức đặc biệt trong đồ thị thu được từ tổng các ma trận con chính liên tiếp. Nếu sau biến đổi ma trận trở thành ma trận Kirchhoff của một đồ thị nối tiếp-song song tổng quát, có thể dùng phương pháp trên đồ thị này để tính định thức. Ví dụ trong bài D1T1 của kỳ tập huấn, sau nhiều lần sai phân nhận được ma trận Kirchhoff của một đồ thị; định thức của nó là số cây khung có trọng số.

13.10.2 Phương pháp lũy thừa

Ví dụ BZOJ4181: cho ma trận thực đối xứng A và điều kiện $x^T A x = 1$, hỏi giá trị nhỏ nhất của $x^T x$. Ma trận thực đối xứng trực giao tương tự với $D = \text{diag}(\lambda_i)$. Sau đổi biến trực giao, bài toán trở thành tìm giá trị riêng có trị tuyệt đối lớn nhất, tức bán kính phổ của A .

Phương pháp lũy thừa chọn vector ban đầu x_0 rồi lặp

$$x_{t+1} = \text{normalize}(Ax_t).$$

Nếu thành phần của x_0 trên không gian riêng ứng với trị riêng lớn nhất khác 0, tỉ lệ các thành phần còn lại suy giảm theo cấp số nhân. Khi tồn tại cả $\lambda_{\max}$ và $-\lambda_{\max}$ gây dao động, xét $A^2 = A^T A$ là nửa xác định dương rồi lấy căn bán kính phổ.

13.10.3 Nguyên lý chuyển vị

Nguyên lý chuyển vị biểu diễn một thuật toán tuyến tính dưới dạng Ax , khảo sát tác dụng của $A^T x$, rồi chuyển vị từng bước phân rã của A để thu được thuật toán cho bài toán đối ngẫu. Cùng tinh thần này cho “nguyên lý lấy nghịch đảo” trong các phép biến đổi như DFT và IDFT. Vì đã có bài viết gần đây trình bày nguyên lý chuyển vị, bài này chỉ nhắc lại ứng dụng.

13.11 Tổng kết

Bài viết giới thiệu nhiều ứng dụng của đại số tuyến tính trong OI, nhưng vẫn chỉ là phần khái quát. Chứng minh định lý Tutte–Berge bằng ma trận Tutte thể hiện kỳ vọng của tác giả đối với các thuật toán đại số.

Tác giả hy vọng độc giả quan tâm đến đại số có thể tìm thêm những ứng dụng sâu hơn, không chỉ trong suy luận chuỗi hình thức mà cả trong việc thiết kế thuật toán tuyến tính và kết hợp hai hướng này.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Huang Xinjun của Trường Trung học Ba Thục Trùng Khánh đã quan tâm và hướng dẫn; cảm ơn gia đình đã quan tâm, ủng hộ, khích lệ và chăm sóc trong cuộc sống; cảm ơn bạn Tang Shaoxuan đã đọc kiểm tra bài viết.

Tài liệu tham khảo

1. Kazuo Murota. *Matrices and Matroids for Systems Analysis*.
2. Cheung, Lau, Leung. *Algebraic Algorithms for Linear Matroid Parity Problems*.
3. PhWEIE. *Một số vấn đề liên quan tới định thức*. Tập luận văn đội tuyển quốc gia Trung Quốc 2017.
4. Yang Jiaqi. *Matching trên đồ thị tổng quát dựa trên đại số tuyến tính*. Tập luận văn đội tuyển quốc gia Trung Quốc 2017.
5. Chen Yu. *Giới thiệu đơn giản về nguyên lý chuyển vị*. Tập luận văn đội tuyển quốc gia Trung Quốc 2020.
6. Wang Zhanpeng. *Các bài toán liên quan tới định thức trong thi tin học*. Tập luận văn đội tuyển quốc gia Trung Quốc 2020.
7. Wang Xiuhuan. *Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị*. Tập luận văn đội tuyển quốc gia Trung Quốc 2019.
8. Zhang Bo. *Ứng dụng quá trình ngẫu nhiên*.
9. James Geelen, Satoru Iwata. *Matroid Matching via Mixed Skew-Symmetric Matrices*.
10. Joseph P. S. Kung. *Bimatroids and Invariants*.
11. R. B. Bapat. *König's Theorem and Bimatroids*.

14 Bàn về các bài toán tổ hợp chuỗi liên quan tới lý thuyết Lyndon

Wan Chengzhang, Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông

Tóm tắt

Chuỗi Lyndon có những tính chất tốt trong các bài toán biểu diễn nhỏ nhất của chuỗi, đồng thời bộc lộ nhiều cấu trúc sâu trong tổ hợp chuỗi. Trên cơ sở nhắc lại các tính chất cơ bản của chuỗi Lyndon, bài viết giới thiệu một số cấu trúc và thuật toán có thể suy ra từ chúng, rồi dùng để giải các bài toán tổ hợp liên quan tới chuỗi.

14.1 Mở đầu

Trong thi tin học, lý thuyết Lyndon là một phần quan trọng của lý thuyết chuỗi. Mức độ phổ biến của nó trong OI chưa cao, nhưng cả phạm vi ứng dụng lẫn cấu trúc nội tại của chuỗi Lyndon đều rất phong phú. Bài viết trước hết ôn lại các kiến thức cơ bản, sau đó trình bày một vài ứng dụng trong các bài toán chuỗi.

Mục 2 và 3 nêu ký hiệu, quy ước và kiến thức nền về border, chu kỳ, mảng hậu tố. Mục 4 trình bày phân tích Lyndon và ứng dụng cơ bản của nó. Mục 5 giới thiệu mảng Lyndon, runs và các bài toán liên quan tới chuỗi bình phương. Mục 6 bàn về necklace, biểu diễn nhỏ nhất và đếm chuỗi Lyndon.

14.2 Định nghĩa và quy ước

Với chuỗi S (trong bài thường dùng S, T, w, u, v để ký hiệu chuỗi):

- $|S|$ là độ dài của chuỗi;
- S_i là ký tự thứ i ;
- $S[l, r]$ là chuỗi con từ vị trí l tới r ; nếu $l > r$ thì đó là chuỗi rỗng;
- $\text{Pre}(S, i) = S[1, i]$ là tiền tố độ dài i ;
- $\text{Suf}(S, i) = S[|S| - i + 1, |S|]$ là hậu tố độ dài i .

Khi $i < |S|$, tiền tố hoặc hậu tố tương ứng được gọi là tiền tố/hậu tố thực sự. Tập ký tự của S là miền giá trị của mọi ký tự trong S , trong bài mặc định ký hiệu là Σ .

Hai chuỗi được so sánh theo thứ tự từ điển: $S < T$ khi S là tiền tố thực sự của T , hoặc tại vị trí đầu tiên khác nhau, ký tự của S nhỏ hơn ký tự của T . Ký hiệu $S + T$ là phép nối chuỗi, và S^k là nối k bản sao của S .

14.3 Kiến thức nền

14.3.1 Border và chu kỳ

Với chuỗi S , số nguyên p là một chu kỳ nếu

$$S_i = S_{i+p} \quad (1 \leq i \leq |S| - p).$$

Chu kỳ nhỏ nhất được ký hiệu $\text{minper}(S)$. Số nguyên $p \in [0, |S|)$ là một border nếu

$$\text{Pre}(S, p) = \text{Suf}(S, p).$$

Chu kỳ và border là hai cách nhìn cùng một cấu trúc.

Bổ đề 3.1. p là chu kỳ của S khi và chỉ khi $|S| - p$ là một border của S .

Bổ đề 3.2: bổ đề chu kỳ yếu. Nếu p, q đều là chu kỳ của S và $p + q < |S|$, thì $\text{gcd}(p, q)$ cũng là chu kỳ của S . Ý chứng minh là trước hết chỉ ra $|p - q|$ là chu kỳ, rồi lặp phép chia Euclid.

Bổ đề 3.3. Nếu $p < q$ đều là border của S , thì p là border của $\text{Pre}(S, q)$.

14.3.2 Cấu trúc dữ liệu hậu tố

Các cấu trúc mô tả mọi chuỗi con hoặc hậu tố gồm mảng hậu tố, automaton hậu tố và cây hậu tố. Bài viết chỉ cần mảng hậu tố. Với chuỗi S , sắp mọi hậu tố không rỗng theo thứ tự từ điển; vị trí bắt đầu của hậu tố hạng i là $\text{SA}(i)$, còn hoán vị ngược là $\text{ISA}(i)$. Mảng LCP lưu độ dài tiền tố chung dài nhất của hai hậu tố kề nhau theo thứ tự hậu tố; sau khi tiền xử lý RMQ, có thể trả lời LCP của hai hậu tố bất kỳ trong $O(1)$.

14.4 Phân tích Lyndon

14.4.1 Giới thiệu cơ bản

Định nghĩa 4.1: chuỗi Lyndon. Chuỗi w là chuỗi Lyndon nếu w nhỏ hơn mọi hậu tố thực sự của nó.

Định nghĩa 4.2: chuỗi gần Lyndon. Nếu w là chuỗi Lyndon và w' là một tiền tố của w , thì chuỗi dạng $w^k w'$ được gọi là gần Lyndon.

Với chuỗi w , nếu ta phân tích được

$$w = w_1 + w_2 + \dots + w_k$$

thành các chuỗi Lyndon và $w_i \geq w_{i+1}$ với mọi i , thì đây là phân tích Lyndon của w ; mỗi w_i là một nhân tử Lyndon. Chẳng hạn

$$\text{abbabbaba} = \text{abb} + \text{abb} + \text{ab} + \text{a}.$$

Bổ đề 4.1. Nếu $w = w_1 + \dots + w_k$ là phân tích Lyndon của w , thì hậu tố thực sự nhỏ nhất của w là w_k .

Bổ đề 4.2. Nếu u, v đều là chuỗi Lyndon và $u < v$, thì uv cũng là chuỗi Lyndon.

Từ bổ đề 4.2, có thể khởi đầu bằng các ký tự đơn lẻ rồi liên tục gộp hai nhân tử kề nhau thỏa thứ tự để thu được phân tích Lyndon. Kết hợp bổ đề 4.1 suy ra:

Định lý 4.1. Mọi chuỗi đều có đúng một phân tích Lyndon.

Thuật toán Duval tìm phân tích Lyndon trong $O(|w|)$. Khi quét từ trái sang phải, thuật toán duy trì một tiền tố chưa phân tích có dạng gần Lyndon $w^k w'$. Nếu ký tự mới y bằng ký tự cần so sánh x , ta kéo dài chuỗi gần Lyndon; nếu $x < y$, toàn bộ phần đang xét trở thành một chuỗi Lyndon lớn hơn; nếu $x > y$, ta tách ra một số bản sao của w rồi tiếp tục từ đoạn còn lại. Quá trình này cũng cho thấy mọi tiền tố của một chuỗi gần Lyndon vẫn là gần Lyndon.

14.4.2 Họ hậu tố nhỏ nhất

Phân tích Lyndon cho biết hậu tố thực sự nhỏ nhất là nhân tử Lyndon cuối cùng. Một cách tổng quát hơn, bài viết xét các hậu tố có thể trở thành nhỏ nhất sau khi nối thêm một chuỗi bên phải.

Định nghĩa 4.3: họ hậu tố nhỏ nhất. Với chuỗi w , một hậu tố u của w được gọi là hậu tố có ý nghĩa (*significant suffix*) nếu tồn tại một chuỗi v sao cho uv là nhỏ nhất trong số các hậu tố của w sau khi đều nối thêm v . Tập mọi hậu tố như vậy ký hiệu $SS(w)$.

Nếu $w = w_1 + \dots + w_k$ là phân tích Lyndon và đặt $s_i = w_i + \dots + w_k$, thì các hậu tố có ý nghĩa nằm trong dãy s_i . Bài viết chứng minh:

- nếu u không có dạng đuôi của một đoạn Lyndon phù hợp, nó không thể là hậu tố có ý nghĩa;
- nếu $u, v \in SS(w)$ và $|u| < |v|$, thì u là border của v và $2|u| < |v|$;
- do đó số hậu tố có ý nghĩa chỉ là $O(\log |w|)$.

Khi chạy Duval trên w , gọi λ là vị trí từ đó thuật toán lần đầu so sánh tới cuối chuỗi. Khi đó có thể lấy các s_i với $i \geq \lambda$ để thu được $SS(w)$ trong thời gian tuyến tính. Với một chuỗi nối thêm v , hậu tố nhỏ nhất của w sau khi nối v , ký hiệu $\text{Minsuf}(w, v)$, có thể tìm bằng nhị phân trên dãy hữu hạn này vì các

chuỗi so sánh sinh ra có tính đơn điệu. Cực trị đầu/cuối của phép nhị phân tương ứng với trường hợp chọn toàn bộ w hoặc chọn hậu tố cuối cùng.

Ví dụ 4.1: hậu tố lớn nhất. Nếu đảo thứ tự bảng chữ cái thì bài toán hậu tố lớn nhất gần giống hậu tố nhỏ nhất. Cần xử lý riêng trường hợp một hậu tố là tiền tố của hậu tố khác; thêm một ký tự cực đại giả z rồi tính $\text{Minsuf}(S, z)$ là đủ. Vì z lớn hơn mọi ký tự gốc, chỉ cần một lần Duval để có đáp án trong $O(|S|)$.

Ví dụ 4.2: ISOL 2019, Festival. Với mỗi tiền tố $\text{Pre}(S, i)$, cần tìm biểu diễn vòng nhỏ nhất của nó. Dù bài toán không viết trực tiếp thành $\text{Minsuf}(w, v)$, cấu trúc tương tự vẫn đúng: điểm bắt đầu của biểu diễn nhỏ nhất thuộc một trong các hậu tố có ý nghĩa. Duy trì phân tích Lyndon của tiền tố và so sánh thô các ứng viên cho thuật toán $O(|S| \log |S|)$. Có thể làm tuyến tính bằng cách quan sát trong quá trình Duval, một số đoạn của chuỗi gần Lyndon không bao giờ có thể làm điểm bắt đầu của biểu diễn nhỏ nhất; chỉ còn so sánh biểu diễn kế thừa từ tiền tố trước với biểu diễn bắt đầu tại đoạn mới.

14.5 Mảng Lyndon và runs

14.5.1 Chuỗi bình phương và runs

Định nghĩa 5.1: chuỗi nguyên thủy. Chuỗi S là nguyên thủy nếu nó không có chu kỳ nguyên thật sự.

Định nghĩa 5.2: chuỗi lũy thừa. Chuỗi dạng w^k với $k \geq 2$ là chuỗi lũy thừa; khi $k = 2$ gọi là chuỗi bình phương. Nếu w nguyên thủy thì đó là lũy thừa nguyên thủy.

Nếu S có chu kỳ nhỏ nhất p , thì mọi chuỗi con độ dài $2p$ nằm trong đoạn còn giữ chu kỳ p đều là bình phương nguyên thủy. Vì vậy cấu trúc bình phương gắn chặt với các đoạn cực đại có chu kỳ nhỏ.

Định nghĩa 5.3: run. Một bộ ba $r = (i, j, p)$ là một run của S nếu $S[i, j]$ có chu kỳ nhỏ nhất p , độ dài ít nhất $2p$, và không thể mở rộng sang trái hoặc sang phải mà vẫn giữ chu kỳ p . Chỉ số của run là

$$e_r = \frac{j - i + 1}{p}.$$

Bổ đề 5.1. Hai run có cùng chu kỳ giao nhau trên đoạn ngắn hơn chu kỳ đó.

Bổ đề 5.2. Mỗi run $r = (i, j, p)$ sinh ra $j - i + 2 - 2p$ chuỗi bình phương nguyên thủy, tức mọi chuỗi con độ dài $2p$ trong đoạn run. Ngược lại, mỗi bình phương nguyên thủy được sinh bởi đúng một run.

14.5.2 Gốc Lyndon và định lý runs

Xét hai thứ tự ngược nhau trên bảng chữ cái, ký hiệu $<_0$ và $<_1$. Với một run $r = (i, j, p)$, một chuỗi con độ dài p nằm trong run và là chuỗi Lyndon theo một trong hai thứ tự được gọi là gốc Lyndon của run. Vì chu kỳ là nguyên thủy, các gốc Lyndon tồn tại và tương ứng với biểu diễn vòng nhỏ nhất của chu kỳ.

Định nghĩa 5.5: mảng Lyndon. Với mỗi vị trí i và mỗi thứ tự $<_t$ ($t = 0, 1$), đặt $l_t(i)$ là đầu phải của chuỗi con Lyndon dài nhất bắt đầu tại i . Dãy l_t là mảng Lyndon của S .

Thêm một ký tự canh gác nhỏ nhất vào đầu chuỗi để tránh biên. Bài viết chứng minh rằng với mọi run, một gốc Lyndon thực sự của nó có dạng $S[u, l_t(u)]$, và hai run khác nhau không thể dùng chung đầu trái của gốc Lyndon thực sự. Do các đầu trái khả dĩ nằm trong $\{2, \dots, |S|\}$, suy ra:

Định lý 5.1: định lý runs. Nếu $\rho(n)$ là số run lớn nhất của một chuỗi độ dài n , còn $\sigma(n)$ là tổng chỉ số lớn nhất của các run, thì

$$\rho(n) < n, \quad \sigma(n) < 3n.$$

Phần thứ hai dùng thêm việc một run có chỉ số e_r chứa ít nhất khoảng $e_r - 2$ gốc Lyndon thực sự, rồi cộng trên mọi run.

14.5.3 Tìm tất cả runs

Bài viết nêu hai phương pháp.

Thuật toán một. Với mỗi chu kỳ p , xét các điểm mốc là bội của p . Một run chu kỳ p phải đi qua ít nhất hai điểm mốc kề nhau. Với mỗi cặp mốc, dùng LCP mở rộng sang phải và LCS mở rộng sang trái để khôi phục đoạn cực đại có chu kỳ p . Tổng độ phức tạp là

$$O(n) + O\left(\sum_{p=1}^n \frac{n}{p}\right) = O(n \log n).$$

Thuật toán hai. Dùng lý thuyết ở trên: tính mảng Lyndon, rồi mở rộng từng ứng viên $S[i, l_t(i)]$ bằng LCP/LCS để thu được run. Mảng Lyndon có thể tính bằng cách duyệt từ phải sang trái, duy trì phân tích Lyndon của hậu tố trong một ngăn xếp: khi thêm ký tự mới vào đầu, liên tục gộp các nhân tử ở đỉnh ngăn xếp nếu phép gộp vẫn tạo chuỗi Lyndon. Nếu có mảng hậu tố tuyến tính, toàn bộ thuật toán cũng có thể đạt $O(n)$.

14.5.4 Bình phương nguyên thủy

Bổ đề 5.7: Three Square Lemma. Nếu u^2, v^2, w^2 là các bình phương nguyên thủy, u^2 là tiền tố của v^2 và v^2 là tiền tố của w^2 , thì

$$|w| > |u| + |v|.$$

Chứng minh dùng bổ đề chu kỳ yếu. Giả sử $|w|$ không đủ lớn, các phần chồng của ba bình phương tạo ra những chu kỳ nhỏ hơn cho một trong các chuỗi nguyên thủy, mâu thuẫn với tính nguyên thủy.

Hệ quả là số tiền tố bình phương nguyên thủy của một chuỗi w chỉ là $O(\log |w|)$, và tổng số bình phương nguyên thủy theo vị trí là $O(|w| \log |w|)$. Kết hợp với bổ đề 5.2, có thể liệt kê các bình phương nguyên thủy sinh bởi runs trong cùng cấp độ phức tạp.

Bổ đề 5.8. Số chuỗi con bình phương nguyên thủy khác nhau về nội dung trong một chuỗi w là $O(|w|)$. Xét lần xuất hiện cuối cùng của mỗi bình phương nguyên thủy; tại mỗi vị trí kết thúc, Three Square Lemma cho thấy không thể có quá nhiều độ dài khác nhau, cụ thể chỉ cần một hằng số.

Ví dụ 5.1: ZJOI 2020 String. Mỗi bình phương do một run sinh ra tương ứng với một điểm (l, r) trên mặt phẳng hai chiều. Với cùng một bội k của chu kỳ trong một run, các điểm tạo thành một đoạn chéo 45 độ. Định lý runs đảm bảo tổng số đoạn là $O(n)$. Sau đó dùng băm để loại các bình phương trùng nội dung và chuyển bài toán thành cộng đoạn chéo, cộng điểm, hỏi tổng trên hình chữ nhật.

Ví dụ 5.2: bài tập đội tuyển 2018 BBS. Cần đếm số cách chia chuỗi thành các đoạn nguyên thủy không giảm theo thứ tự từ điển. Gán trọng số có dấu cho mỗi đoạn theo chu kỳ nhỏ nhất, rồi dùng bao hàm loại trừ để tự động triệt các cách chia có đoạn không nguyên thủy. Với các đoạn không nằm trong cấu trúc run, quy hoạch động chỉ cần tổng tiền tố; các đoạn đặc biệt sinh từ run được xử lý thêm, tổng số là $O(|S| \log |S|)$.

14.5.5 Cây Lyndon

Một chuỗi Lyndon độ dài lớn hơn 1 có thể phân tích chuẩn

$$w = u + v,$$

trong đó v là hậu tố thực sự nhỏ nhất của w , còn u, v đều là chuỗi Lyndon. Dùng phân tích chuẩn làm hai con trái/phải tạo ra cây nhị phân gọi là cây Lyndon. Với một chuỗi không nhất thiết là Lyndon, dựng cây cho từng nhân tử Lyndon sẽ được rừng Lyndon; thêm một ký tự cực nhỏ ở đầu có thể biến nó thành một cây duy nhất.

Có thể xây cây bằng cùng quá trình duyệt từ phải sang trái như khi tính mảng Lyndon. Mỗi lần thêm ký tự mới, các nhân tử Lyndon bị gộp ở đỉnh ngăn xếp được nối bằng một chuỗi nhánh trái, từ đó cập nhật rừng.

Nếu đánh số lá theo vị trí chuỗi, thì chuỗi con Lyndon bắt đầu tại i tương ứng với một cây con trong cây Lyndon; mảng Lyndon cũng xác định được cấu trúc cây. Nhìn từ mảng ISA, thuật toán dựng cây chính là ngăn xếp đơn điệu của mảng thứ hạng hậu tố.

Truy vấn chu kỳ ngắn trên đoạn. Cho nhiều truy vấn $S[l, r]$, cần biết chu kỳ nhỏ nhất có không quá nửa độ dài đoạn hay không. Nếu có, đoạn nằm trong một run. Dựng hai cây Lyndon ứng với hai thứ tự chữ cái ngược nhau; lấy LCA của các lá trong đoạn theo từng cây và kiểm tra xem con phải của LCA có là gốc Lyndon của một run bao đoạn hay không. Nếu có thì chu kỳ của run là đáp án. Chứng minh dựa trên việc gốc Lyndon thật sự của run phải là con phải của một nút tổ tiên phù hợp trong cây Lyndon.

14.6 Đếm tổ hợp liên quan tới chuỗi Lyndon

14.6.1 Necklace và biểu diễn nhỏ nhất

Định nghĩa 6.1. Với chuỗi w , các chuỗi

$$\text{Suf}(w, i) + \text{Pre}(w, |w| - i) \quad (0 \leq i < |w|)$$

là các biểu diễn vòng của w . Biểu diễn nhỏ nhất theo thứ tự từ điển ký hiệu là (w) . Nếu $(w) = w$, ta gọi w là một necklace.

Bổ đề 6.1. Mỗi chuỗi Lyndon là một necklace. Mỗi necklace phân tích duy nhất thành u^k , trong đó u là chuỗi Lyndon. Vì vậy mỗi chuỗi nguyên thủy có đúng một biểu diễn vòng là chuỗi Lyndon.

Để tính (w) , có thể dùng phân tích Lyndon của ww : trong các nhân tử Lyndon có đầu trái nằm trong đoạn $[1, |w|]$, lấy nhân tử cuối cùng; đầu trái của nó cho điểm bắt đầu của biểu diễn nhỏ nhất. Do đó có thể tìm biểu diễn nhỏ nhất và kiểm tra một chuỗi có là necklace hay không trong $O(|w|)$.

14.6.2 Necklace tiền nhiệm

Ký hiệu $PL(w)$ là necklace lớn nhất có cùng độ dài, không vượt quá w theo thứ tự từ điển. Nếu w đã là necklace thì đáp án là w . Nếu không, bài viết chứng minh

$$PL(w) = \text{Pre}(w, i - 1) + P(w_i) + z^{|w|-i}$$

với một vị trí i , trong đó $P(w_i)$ là ký tự đứng ngay trước w_i trong bảng chữ cái, còn z là ký tự lớn nhất. Quét các ứng viên từ phải sang trái cho cách làm bình phương; tối ưu hóa bằng cách chạy Duval một lần, vì các tiền tố không gần Lyndon chắc chắn không thể dẫn tới necklace hợp lệ. Kết quả là:

Bổ đề 6.4. Có thể tính $PL(w)$ trong $O(|w|)$.

14.6.3 Đếm và xếp hạng chuỗi Lyndon

Không xét ràng buộc đặc biệt, nếu $L(n)$ là số chuỗi Lyndon độ dài n trên bảng chữ cái Σ , còn $P(n)$ là số chuỗi nguyên thủy độ dài n , thì

$$L(n) = \frac{P(n)}{n}.$$

Mọi chuỗi độ dài n có một chu kỳ nguyên thủy độ dài $d \mid n$, nên

$$|\Sigma|^n = \sum_{d \mid n} P(d).$$

Theo đảo Mobius,

$$L(n) = \frac{1}{n} \sum_{d \mid n} \mu(d) |\Sigma|^{n/d}.$$

Bài toán xếp hạng Lyndon hỏi: với chuỗi w độ dài n , có bao nhiêu chuỗi Lyndon độ dài n không lớn hơn w ; ký hiệu là $\text{Rank}(w)$. Vì chuỗi Lyndon là necklace, trước hết thay w bằng $PL(w)$ mà không đổi đáp án.

Với mỗi ước d của n , cần đếm các chuỗi nguyên thủy độ dài d có biểu diễn nhỏ nhất không vượt quá $\text{Pre}(w, d)$. Bổ đề quan trọng là nếu w là necklace độ dài n và u có độ dài $d \mid n$, thì

$$u^{n/d} < w \iff u < \text{Pre}(w, d).$$

Nhờ đó việc đếm ràng buộc theo necklace chuyển thành bài toán trên tiền tố độ dài d .

14.6.4 Automaton nhận diện

Với một necklace w độ dài m , xét tập

$$R(w) = \{\text{Pre}(w, i) + x : i < m, x < w_{i+1}\} \cup \{w\}.$$

Bài toán là nhận diện các chuỗi có chứa một phần tử của $R(w)$ làm chuỗi con. Bài viết đưa ra DFA có các trạng thái $0, 1, \dots, m$, bắt đầu tại 0:

$$g(i, x) = \begin{cases} i + 1, & i < m, x = w_{i+1}, \\ 0, & i < m, x > w_{i+1}, \\ m, & i = m \text{ hoặc } (i < m, x < w_{i+1}). \end{cases}$$

Trạng thái m là trạng thái chấp nhận. DFA này liên hệ trực tiếp với automaton KMP của w : trước khi chấp nhận, trạng thái chính là độ dài dài nhất của một hậu tố đã đọc khớp với tiền tố của w . Vì w là necklace,

các chuyển về 0 trong automaton nhận diện trùng với các chuyển thất bại của KMP trong trường hợp ký tự lớn hơn ký tự mong đợi. Từ đó chứng minh DFA nhận đúng các chuỗi chứa một phần tử của $R(w)$.

Để tính số chuỗi u sao cho u^k được nhận diện, lấy bổ sung: đếm các chuỗi mà mọi lần lặp vẫn không tới trạng thái m . Khi j đủ lớn, trạng thái sau $\text{Pre}(u^\infty, j)$ lặp theo chu kỳ $|u|$; gọi trạng thái ổn định đó là trạng thái đặc trưng của u . Mỗi chuỗi tương ứng với một cặp gồm trạng thái đặc trưng và một chu trình độ dài $|u|$ trong automaton tránh trạng thái m .

Các chu trình trong automaton có cấu trúc đơn giản: chúng là ghép của các vòng con dạng

$$0 \rightarrow 1 \rightarrow \dots \rightarrow i-1 \rightarrow 0.$$

Nếu b_i là số ký tự gây cạnh ngược từ $i-1$ về 0, và f_i là số chu trình độ dài i xuất phát từ 0, thì

$$f_0 = 1, \quad f_i = \sum_{j=1}^{\min(m,i)} b_j f_{i-j}.$$

Dùng hàm sinh hoặc kỹ thuật nửa trực tuyến với FFT để tính mọi f_i trong $O(m \log m)$. Với trạng thái đặc trưng không nhất thiết là 0, nhân thêm số cách chọn điểm đánh dấu trên vòng con, nên số chuỗi không được nhận diện có dạng

$$\sum_{i=1}^m i b_i f_{n-i}.$$

Từ đó suy ra thuật toán tính các giá trị cần cho $\text{Rank}(w)$ trong tổng thời gian

$$O\left(\sum_{d|n} d \log d\right) = O(n \log^2 n).$$

Nếu phần hữu hiệu của necklace ngắn hơn nhiều so với n , có thể dùng truy hồi tuyến tính và phép lấy dư đa thức để tính hạng ở độ dài rất lớn nhanh hơn.

Ví dụ 6.1: CTS 2019 Repeat. Cho chuỗi w độ dài n , cần đếm chuỗi u độ dài m sao cho trong u^2 tồn tại một chuỗi con độ dài n nhỏ hơn w . Khi $m = n$, chỉ cần lấy tiền nhiệm w^- của w , chuẩn hóa thành $\text{PL}(w^-)$ rồi tính số chuỗi có biểu diễn nhỏ nhất không vượt quá nó. Khi $m < n$, nối thêm các ký tự lớn nhất vào w^- để đưa về độ dài n . Khi $m > n$, tách trường hợp biểu diễn nhỏ nhất nhỏ hơn tiền tố độ dài m của w và trường hợp bằng đúng tiền tố đó; trường hợp sau đã biết biểu diễn nhỏ nhất nên chỉ cần kiểm tra hợp lệ và đếm số điểm bắt đầu. Tổng độ phức tạp là $O((m+n) \log(m+n))$.

14.7 Tổng kết

Bài viết nhìn lý thuyết Lyndon từ nhiều góc: phân tích Lyndon, định lý runs, cây Lyndon, đếm và xếp hạng chuỗi Lyndon. Một số ứng dụng chỉ được phác thảo nhưng rất đáng chú ý, như dùng cây Lyndon để trả lời truy vấn chu kỳ ngắn trên đoạn, phân tích Lyndon của dãy de Bruijn nhỏ nhất, hay liên hệ giữa chuỗi Lyndon và đa thức bất khả quy. Tác giả hy vọng có thêm nhiều nghiên cứu về Lyndon đi vào tầm nhìn của OI và sinh ra các bài toán hay.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn thầy Jin Jing của Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông đã quan tâm và hướng dẫn; cảm ơn các bạn học đã trao đổi nội dung liên quan; cảm ơn gia đình đã quan tâm và ủng hộ.

Tài liệu tham khảo

1. BBS, CHK. *Phân tích Lyndon*. Tập luận văn đội tuyển quốc gia Trung Quốc 2021.
2. Tomasz Kociumaka. *Minimal Suffix and Rotation of a Substring in Optimal Time*. arXiv:1601.08051, 2016.
3. Hideo Bannai, Tomohiro I, Shunsuke Inenaga, Yuto Nakashima, Masayuki Takeda, Kazuya Tsuruta. *The Runs Theorem*. arXiv:1406.0263, 2015.
4. Tomasz Kociumaka, Jakub Radoszewski, Wojciech Rytter. *Efficient Ranking of Lyndon Words and Decoding Lexicographically Minimal de Bruijn Sequence*. arXiv:1510.02637, 2015.
5. zghtyarecrenj. *Từ Lyndon Words đến Three Square Lemma*. [Luogu blog](#).
6. command_block. *Lyndon & Runs*. [Luogu blog](#).
7. rgy. *LOJ3123 [CTS2019] #42*. [rgy.moe](#).

15 Báo cáo ra đề *Chuỗi của Cutie Duo*

Feng Shiyuan, Trường Trung học Thạch Thất Thành Đô

Tóm tắt

Bài viết giới thiệu lời giải và bối cảnh ra đề của một bài toán chuỗi do tác giả đặt trong kỳ thi đối kháng đội tuyển trên UOJ. Ngoài ra, bài viết đưa ra thêm ba ví dụ để tổng kết cách kết hợp cây hậu tố với *dsu on tree* trong một số bài toán chuỗi.

15.1 Bài toán

15.1.1 Một số quy ước

Với chuỗi S độ dài n , bài viết dùng các ký hiệu:

- $S[i]$ là ký tự thứ i ;
- $S[l, r]$ là chuỗi con từ l đến r ;
- $\text{lcp}(i, j)$ là độ dài tiền tố chung dài nhất của hai hậu tố $S[i, n]$ và $S[j, n]$;
- $\text{lcs}(i, j)$ là độ dài hậu tố chung dài nhất của hai tiền tố $S[1, i]$ và $S[1, j]$;
- nếu $S[1, i] = S[n - i + 1, n]$ thì $S[1, i]$ là một border của S .

Với mệnh đề P , ngoặc Iverson $[P]$ bằng 1 khi P đúng và bằng 0 khi P sai. Nếu không nói thêm, chuỗi đang xét có độ dài n .

15.1.2 Mô tả bài toán

Với một chuỗi độ dài n , thuật toán KMP tính mảng next , trong đó

$$\text{next}_i = \max\{x : x < i, S[1, x] = S[i - x + 1, i]\}.$$

Nối cạnh từ i tới next_i tạo thành một cây gốc 0, gọi là cây next ; độ sâu của 0 bằng 0.

Cho chuỗi S độ dài n và một mảng trọng số w . Có m truy vấn, mỗi truy vấn cho đoạn $[l, r]$. Đặt $S' = S[l, r]$, $n' = |S'|$, và $w' = (w_l, w_{l+1}, \dots, w_r)$. Cần tính tổng có trọng số của độ sâu trong cây next của S' :

$$\sum_{i=1}^{n'} \text{dep}_i w'_i.$$

Ký tự của chuỗi thuộc bảng chữ cái $\{0, 1\}$.

15.1.3 Dữ liệu

- Subtask 1, 5 điểm: $n, m \leq 10^4$.
- Subtask 2, 10 điểm: $n, m \leq 10^5$, $r = n$, $w_i = 1$.
- Subtask 3, 20 điểm: $n, m \leq 10^5$, $r = n$.
- Subtask 4, 10 điểm: $n, m \leq 10^5$, mỗi ký tự được chọn ngẫu nhiên trong $\{0, 1\}$ và $w_i = 1$.
- Subtask 5, 10 điểm: $n, m \leq 10^5$, mỗi ký tự được chọn ngẫu nhiên trong $\{0, 1\}$.
- Subtask 6, 5 điểm: $n, m \leq 10^5$, $w_i = 1$.
- Subtask 7, 20 điểm: $n, m \leq 10^5$.
- Subtask 8, 20 điểm: $n, m \leq 2 \cdot 10^5$.

Giới hạn thời gian và bộ nhớ là khoảng 6 giây và 1 GB.

15.1.4 Phân bố điểm

Trong kỳ thi có 2 thí sinh đạt 80 điểm, 1 thí sinh đạt 55 điểm, 1 thí sinh đạt 40 điểm, 1 thí sinh đạt 35 điểm, 8 thí sinh đạt 25 điểm, 1 thí sinh đạt 15 điểm và 11 thí sinh đạt 5 điểm.

15.2 Giới thiệu thuật toán

15.2.1 Thuật toán 1

Cách làm trực tiếp là chạy KMP cho từng truy vấn rồi tính đáp án. Độ phức tạp $O(nq)$, kỳ vọng đạt 5 điểm.

15.2.2 Cây hậu tố

Bài viết gọi cấu trúc cây của automaton hậu tố xây trên chuỗi đảo là cây hậu tố. Ký hiệu:

- fa_x là cha của nút x trên cây;
- len_x là độ dài lớn nhất của các chuỗi do nút x đại diện;
- Right_x là tập đầu nút phải của các lần xuất hiện trong chuỗi đảo, tương đương tập đầu nút trái trong chuỗi gốc;
- pt_i là nút trên cây hậu tố tương ứng với hậu tố $S[i, n]$.

Ta có

$$\text{lcp}(i, j) = \text{len}_{\text{lca}(\text{pt}_i, \text{pt}_j)}.$$

Cách xây automaton hậu tố và quan hệ với cây hậu tố được nhắc tới trong các tài liệu tham khảo [1] và [2].

15.2.3 Thuật toán 2

Thuật toán này xử lý trường hợp $r = n$ và mọi $w_i = 1$. Thay vì tính tổng độ sâu của mọi điểm trong cây next, ta tính tổng kích thước các cây con; hai cách đếm là tương đương vì mỗi nút được tính đúng theo số tổ tiên của nó.

Xét tính chất của cây next. Nếu $S[1, x]$ là border của $S[1, y]$, thì x nằm trong cây con của y ; mặt khác, y ứng với một lần xuất hiện của $S[1, x]$ trong $S[1, y]$. Vì vậy đóng góp của x bằng số lần xuất hiện của tiền tố $S[1, x]$ trong đoạn cần xét.

Do đó, bài toán trở thành thống kê số lần xuất hiện của mọi tiền tố của $S[1, n]$ trong đoạn. Xây automaton hậu tố từ phải sang trái; với mỗi vị trí, cộng trên đường từ pt_i lên gốc các lượng dạng

$$(\text{len}_x - \text{len}_{\text{fa}_x}) \cdot |\text{Right}_x|.$$

Dùng LCT để duy trì, độ phức tạp $O(n \log n)$, kỳ vọng đạt 15 điểm.

15.2.4 Thuật toán 3

Thuật toán này xử lý dữ liệu ngẫu nhiên với $w_i = 1$. Tiếp tục ý tưởng của thuật toán 2, ta cần tính số lần xuất hiện của mỗi tiền tố của $S[l, r]$ trong chính $S[l, r]$. Gọi x là nút trên cây hậu tố ứng với $S[l, r]$. Với mỗi nút trên đường từ x lên gốc và mỗi độ dài

$$\text{len}_{\text{fa}_x} + 1 \leq \ell \leq \text{len}_x,$$

cần truy vấn số phần tử của Right_x trong một đoạn vị trí. Có thể dùng hợp nhất cây đoạn để làm truy vấn này.

Với dữ liệu ngẫu nhiên, chọn ngưỡng $B > 2 \log n + \log q + c$. Xác suất hai chuỗi con độ dài B bằng nhau rất nhỏ. Tiền xử lý hash cho các chuỗi độ dài dưới B rồi đếm trực tiếp các chuỗi cùng hash trong đoạn. Vì số độ dài cần xét kỳ vọng là $O(\log n)$, độ phức tạp là $O(n \log n + q \log^2 n)$, kết hợp thuật toán 2 đạt khoảng 25 điểm.

15.2.5 Thuật toán 4

Thuật toán này xử lý dữ liệu ngẫu nhiên không còn giả sử $w_i = 1$. Khi xét độ dài ℓ , ta cần cộng các trọng số w của những vị trí có chuỗi con cùng hash với $S[l, l + \ell - 1]$.

Nếu dùng cây hậu tố, tại một nút x và độ dài ℓ , ta cần chuyển lần xuất hiện trong cây hậu tố của chuỗi đảo sang lần xuất hiện tương ứng trong cây hậu tố của chuỗi xuôi. Chọn một phần tử của Right_x , xây automaton hậu tố xuôi, rồi dùng nhảy bậc hai để tìm nút y ứng với đoạn kết thúc tương ứng. Tập Right_x và Right_y tương ứng một-một, nên truy vấn trở thành truy vấn tổng trọng số trong cây con của y . Bài viết gọi cách chuyển này là *cây hậu tố đối xứng*. Độ phức tạp $O(n \log n + q \log^2 n)$, kết hợp thuật toán 2 đạt khoảng 35 điểm.

15.2.6 Thuật toán 5

Thuật toán này xử lý trường hợp $r = n$. Với một tiền tố của chuỗi con truy vấn, đóng góp có thể viết thành tổng theo các vị trí xuất hiện của chuỗi con. Tách đường từ pt_i lên gốc bằng phân rã nặng nhẹ. Với mỗi heavy path, đặt z_j là nút đầu tiên trên heavy path gặp được khi đi từ pt_j lên gốc, và đặt độ dài tương ứng là ℓ'_j .

Nếu một nút nằm trong cây con của con nặng, ta dùng cây hậu tố đối xứng để đổi sang truy vấn cây con ở phía chuỗi xuôi. Với phần còn lại, cần xử lý các cây con nhẹ trên tiền tố của một heavy path. Dùng

dsu on tree: quét heavy path từ trên xuống, khi tới một nút thì đưa toàn bộ thông tin của các cây con nhẹ vào cây Fenwick theo chỉ số vị trí, với trọng số phù hợp. Mỗi nút chỉ bị đưa vào $O(\log n)$ lần, đúng bằng số cạnh nhẹ trên đường tới gốc. Độ phức tạp $O((n + q) \log^2 n)$, kết hợp thuật toán 4 đạt khoảng 55 điểm.

15.2.7 Thuật toán 6

Thuật toán này xử lý $w_i = 1$. Tiếp tục từ thuật toán 5, tổng cần tính rút về các biểu thức dạng

$$\sum_j \min(\text{lcp}(\dots), r - j + 1).$$

Khi đó bài toán gần với Codeforces 1098F, nên bài viết chỉ phác thảo lời giải chính thức.

Với một heavy path, mỗi lần xuất hiện có thể biểu diễn thành các điểm trên mặt phẳng. Số lần đóng góp bằng số điểm thỏa đồng thời các bất đẳng thức theo r và theo độ dài đang xét. Các điểm trên cùng cấu trúc tạo thành những đoạn thẳng; lấy sai phân thành hai tia, rồi dùng quét đường thẳng cùng cây Fenwick để trả lời. Độ phức tạp $O((n + q) \log^2 n)$, kết hợp thuật toán 5 đạt khoảng 60 điểm.

15.2.8 Thuật toán 7

Thuật toán này xử lý $n, m \leq 10^5$. Đặt s_i là tổng tiền tố của mảng trọng số. Biểu thức cần tính được viết lại qua các tổng tiền tố này và các điều kiện dạng $\ell'_j + j > r + 1$, $\ell'_j < \ell$.

Vẫn dùng *dsu on tree*. Tại một nút x trên heavy path, truy vấn trở thành một bài toán đếm điểm trong hình chữ nhật với trọng số. Số lần chèn và số lần truy vấn lần lượt là $O(n \log n)$ và $O(q \log n)$, nên dùng cấu trúc cây lồng cây cho độ phức tạp $O((n + q) \log^3 n)$. Các truy vấn cây con phát sinh $O(\log n)$ lần vẫn xử lý bằng kỹ thuật của thuật toán 5. Kỳ vọng đạt 80 điểm.

15.2.9 Thuật toán 8

Thuật toán này tối ưu thuật toán 7. Trước hết bỏ yêu cầu phần giao với heavy path phải nằm phía trên x . Với các điểm nằm phía dưới, đóng góp có thể xử lý bằng truy vấn cây con; chính việc này lại loại bỏ một chiều ràng buộc. Cây hậu tố đối xứng được dùng để trừ đi cây con nhẹ chứa điểm truy vấn.

Biểu thức sau biến đổi chỉ còn các trường hợp:

- nếu biên phải của đoạn bị chặn trước khi đạt đủ độ dài, dùng quét đường thẳng và cây Fenwick;
- nếu độ dài trên một phía nhỏ hơn độ dài trên phía kia, dùng cây Fenwick sau khi sắp thứ tự;
- trường hợp còn lại dùng truy vấn cây hậu tố đối xứng.

Độ phức tạp giảm còn $O((n + q) \log^2 n)$ và đạt 100 điểm.

15.2.10 Thuật toán 9

Bài viết còn đưa ra một cách nhìn khác cho thuật toán 7. Xem cặp (j, ℓ') là một điểm trên mặt phẳng. Các điều kiện trong truy vấn đều có dạng

$$x_j < a, \quad y_j < b, \quad x_j + y_j < c.$$

Nếu $c > a + b$ thì truy vấn chỉ là truy vấn hình chữ nhật. Nếu $c < a + b$, tách miền cần đếm thành ba tập:

$$A = \{j : x_j < a, x_j + y_j < c\}, \quad B = \{j : y_j < b, x_j + y_j < c\}, \quad C = \{j : x_j + y_j < c\},$$

rồi lấy $A + B - C$. Mỗi tập chỉ có một hoặc hai ràng buộc, nên đều xử lý được bằng quét đường thẳng. Các trường hợp còn lại cũng tách tương tự, từ đó thu được $O((n + q) \log^2 n)$.

15.3 Bối cảnh ra đề và tổng kết bài toán

Nguồn cảm hứng của tác giả đến từ bài *Simple KMP*, yêu cầu tính tổng độ sâu cây next trên mọi chuỗi con của một chuỗi. Bài đó có thể giải bằng LCT và automaton hậu tố. Tác giả thử sửa thành nhiều truy vấn trên một chuỗi con, nhưng biểu thức thu được lại trùng tình thần với Codeforces 1098F, nên tiếp tục tăng cường thành truy vấn tổng độ sâu next có trọng số.

Trong quá trình thử lời giải, ngoài *dsu on tree*, tác giả dùng thêm tư tưởng cây hậu tố đối xứng để đạt $O((n+q)\log^3 n)$, tức thuật toán 7. Sau đó anh Ma Yaohua đưa ra cách tối ưu hơn $O((n+q)\log^2 n)$, tức thuật toán 8.

Bài toán có độ phân hóa tốt: với dữ liệu ngẫu nhiên, chỉ cần hash đơn giản đã có thể vượt qua một phần lớn; thí sinh phát hiện thêm tính chất ở các trường hợp $r = n$ hoặc $w_i = 1$ cũng nhận được số điểm tương ứng. Ngoài hướng dựa trên cây hậu tố và *dsu on tree*, anh Peng Sijin còn đưa ra lời giải chia khối $O((n+q)\sqrt{n})$ ngắn gọn, xem thêm tài liệu [5].

15.4 *dsu on tree* trên cây hậu tố

Ba ví dụ sau đều dùng được phương pháp cây hậu tố kết hợp *dsu on tree*.

15.4.1 Ví dụ 1

Mô tả. Cho chuỗi S và số nguyên dương K , hỏi S có bao nhiêu chuỗi con có thể chia thành K đoạn bằng nhau.

Phân tích. Cách làm trong bài tính được, với mỗi vị trí bắt đầu i , có bao nhiêu chuỗi con bắt đầu tại i thỏa điều kiện. Điều kiện có thể đổi thành một ràng buộc trên lcp, rồi chuyển sang quan hệ giữa vị trí xuất hiện và độ dài trên cây hậu tố.

Khi xử lý từng heavy path từ trên xuống, DFS thô qua các cây con nhẹ và thực hiện cộng đoạn; tại vị trí i chỉ cần hỏi điểm. Với các truy vấn tới con nặng, dùng truy vấn cây con. Độ phức tạp $O(n\log^2 n)$. Bài này còn có lời giải tốt hơn nhưng không liên quan trực tiếp tới chủ đề nên không bàn tiếp.

15.4.2 Ví dụ 2: bốn cách tìm border

Mô tả. Cho chuỗi S và m truy vấn $[l, r]$, cần tìm border dài nhất của $S[l, r]$ khác chính nó.

Phân tích. Theo tính chất border, đáp án có thể viết thành vị trí nhỏ nhất $x \in (l, r]$ sao cho phần chồng từ x tới r khớp với tiền tố của đoạn. Dùng cùng phương pháp với cây hậu tố: đưa các điểm x vào cây đoạn, khóa theo giá trị $\ell'_x + x - 1$, duy trì giá trị lớn nhất rồi nhị phân trên cây đoạn để tìm x nhỏ nhất hợp lệ.

Với $O(\log n)$ truy vấn con nặng, dùng hợp nhất cây đoạn để lấy tập Right và truy vấn. Cây con nhẹ chứa chính điểm đang hỏi không cần loại riêng, vì đáp án thu được vẫn hợp lệ theo giới hạn LCA. Độ phức tạp $O((n+m)\log^2 n)$.

Nếu dùng tư tưởng của thuật toán 8, điều kiện có thể đổi thành truy vấn vị trí hợp lệ nhỏ nhất trong một đoạn, kèm điều kiện $x + \ell_x > r + 1$. Sắp bằng radix sort theo ngưỡng, giữ các điểm còn sống và dùng DSU để tìm vị trí hợp lệ kế tiếp; độ phức tạp gần $O((n+q)\alpha(n)\log n)$. Bài toán này còn có lời giải tốt hơn nhờ tính chất riêng của border.

15.4.3 Ví dụ 3

Mô tả. Hai chuỗi độ dài m được gọi là tương tự nếu chúng bằng nhau hoặc chỉ khác đúng một vị trí. Với mỗi chuỗi con độ dài m , hãy tính có bao nhiêu chuỗi con độ dài m khác tương tự với nó.

Phân tích. Cần đếm số vị trí j thỏa

$$\text{lcp}(i, j) + \text{lcs}(i + m - 1, j + m - 1) \geq m - 1.$$

Xây automaton hậu tố xuôi để biểu diễn phần lcs. Dùng *dsu on tree* như trước, ràng buộc chuyển thành điều kiện về lcp và độ dài trên heavy path. Với mảng hậu tố, từ thứ hạng của hậu tố có thể nhị phân ra đoạn hạng $[L, R]$ thỏa điều kiện LCP; mỗi lần gặp một vị trí j thì cộng 1 trên đoạn hạng đó, cuối cùng hỏi tại hạng của vị trí cần tính. Với các truy vấn con nặng, cũng nhị phân trên mảng hậu tố rồi hỏi số phần tử trong cây con. Độ phức tạp $O(n \log^2 n)$.

15.5 Tổng kết và triển vọng

Từ bài toán chính và các ví dụ, có thể thấy *dsu on tree* rất hiệu quả khi xử lý các biểu thức liên quan tới lcp. Lý do là trên một heavy path, lcp thường có thể viết đơn giản thành một độ dài len, sau đó kết hợp với cấu trúc dữ liệu để chuyển thành các bài toán đếm điểm. Những cây con đặc biệt xuất hiện $O(\log n)$ lần thường có dạng đủ đơn giản để trả lời bằng truy vấn cây con.

Nhìn chung, công thức sinh ra từ *dsu on tree* trên cây hậu tố khá gọn, không cần biến đổi quá dài, và sau vài phân loại ngắn thường trở thành bài toán cấu trúc dữ liệu quen thuộc. Bài viết tổng kết phương pháp này và đưa ra ba ví dụ nhằm gợi mở thêm các ứng dụng mới trên cây hậu tố.

Lời cảm ơn

Tác giả cảm ơn cha mẹ đã nuôi dưỡng; cảm ơn China Computer Federation đã cung cấp cơ hội giao lưu; cảm ơn các thầy Zeng Guisheng và Liang Dewei của Trường Trung học Thạch Thất Thành Đô đã hướng dẫn và giúp đỡ nhiều năm; cảm ơn các bạn Tu Xiaodi, Zhang Jiarui và Cao Yue đã đọc kiểm bài viết; cảm ơn UOJ cung cấp nền tảng thi và các quản trị viên UOJ đã hỗ trợ; cảm ơn mọi người đã giúp đỡ tác giả trên con đường OI.

Tài liệu tham khảo

1. Chen Lijie. *Automaton hậu tố*. Trao đổi trại đông, 2012.
2. Zhang Tianyang. *Automaton hậu tố và ứng dụng*. Tập luận văn đội tuyển quốc gia Trung Quốc 2015.
3. Codeforces Blog. <http://codeforces.com/blog/entry/44351>.
4. Codeforces Blog. <https://codeforces.com/blog/entry/64331>.
5. Blog UOJ của Itst. itst.blog.uoj.ac.
6. Wikipedia. *Suffix Array*. https://en.wikipedia.org/wiki/Suffix_array.

16 Bàn về bài toán mô phỏng luồng chi phí trong OI

Chen Kuowen, Trường Trung học Dư Diêu, Chiết Giang

Tóm tắt

Luồng chi phí là một mô hình kinh điển trong lý thuyết đồ thị. Trong thi tin học, nhiều bài toán có thể quy về luồng chi phí, nhưng việc chạy trực tiếp thuật toán luồng thường không đạt độ phức tạp mong muốn. Bài viết này xuất phát từ mô hình luồng chi phí cực tiểu, nhắc lại mạng dư, đường tăng, chu trình âm và thuật toán SSP, rồi tổng kết các cách “mô phỏng” quá trình tăng luồng bằng cấu trúc dữ liệu, tính lồi, đối ngẫu tuyến tính và các dạng đồ thị đặc biệt như chuỗi một chiều, chuỗi hai chiều và cây.

16.1 Mở đầu

Mô hình luồng chi phí có năng lực biểu diễn rất mạnh: chọn một số đối tượng, ghép cặp, phân phối tài nguyên, lập lịch, hoặc cân bằng nhiều điều kiện đều có thể viết thành một bài toán tìm luồng cực tiểu chi phí. Vấn đề là mạng trong OI thường có quy mô lớn hoặc có cấu trúc động; nếu chỉ dựng đồ thị rồi chạy thuật toán tổng quát thì số cạnh, số lần tăng luồng hoặc chi phí tìm đường ngắn đều quá cao.

Vì vậy mục tiêu của bài viết không phải trình bày một thuật toán luồng chi phí tổng quát mới, mà là phân tích bản chất của quá trình tăng luồng. Khi đồ thị có cấu trúc đặc biệt, đường tăng thường chỉ đi qua một số điểm mấu chốt, chi phí theo lượng luồng có tính lồi rời rạc, hoặc bài toán đối ngẫu có dạng DP dễ hơn. Nếu nhận ra các tính chất đó, ta có thể mô phỏng chính xác kết quả của luồng chi phí mà không cần duy trì toàn bộ mạng dư.

16.2 Tóm lược về luồng chi phí cực tiểu

16.2.1 Định nghĩa và quy ước

Cho đồ thị có hướng $G = (V, E)$, mỗi cạnh (u, v) có dung lượng $c(u, v)$ và chi phí đơn vị $a(u, v)$. Một luồng x phải thỏa bảo toàn luồng tại mọi đỉnh khác nguồn s và đích t :

$$\sum_{(u,v) \in E} x_{uv} = \sum_{(v,w) \in E} x_{vw} \quad (v \in V \setminus \{s, t\}),$$

đồng thời $0 \leq x_{uv} \leq c(u, v)$. Chi phí của luồng là

$$\text{cost}_G(x) = \sum_{(u,v) \in E} x_{uv} a(u, v),$$

còn giá trị luồng là lượng chảy ròng từ s tới t . Bài toán nguồn–đích cần tìm luồng khả thi có chi phí nhỏ nhất, có thể với giá trị luồng cố định hoặc không cố định tùy mô hình.

16.2.2 Mạng dư

Với một luồng hiện tại x , mạng dư G_x gồm cạnh thuận còn dung lượng $c(u, v) - x_{uv}$ với chi phí $a(u, v)$, và cạnh ngược có dung lượng x_{uv} với chi phí $-a(u, v)$. Nếu x' là một luồng khả thi trên mạng dư, ký hiệu $x \oplus x'$ là kết quả cộng luồng theo cạnh thuận và trừ theo cạnh ngược. Khi đó chi phí biến đổi đúng bằng chi phí của phần luồng dư:

$$\text{cost}_G(x \oplus x') = \text{cost}_G(x) + \text{cost}_{G_x}(x').$$

Đẳng thức này là cơ sở để xem mỗi bước cải thiện nghiệm như một bài toán tìm đường hoặc chu trình có chi phí âm trong mạng dư.

16.2.3 Đường tăng và chu trình âm

Một đường từ s tới t trong mạng dư với mọi cạnh còn dung lượng dương gọi là đường tăng; dung lượng của đường là dung lượng nhỏ nhất trên các cạnh của nó. Một chu trình trong mạng dư có tổng chi phí âm gọi là chu trình âm. Nếu tồn tại chu trình âm, ta có thể đẩy luồng quanh chu trình đó để giảm chi phí mà không đổi giá trị luồng. Nếu tồn tại đường tăng âm từ s tới t , ta có thể tăng giá trị luồng đồng thời giảm chi phí biên.

Bổ đề. Các cải thiện trong mạng dư có thể phân rã thành đường tăng giữa s, t và các chu trình. Vì vậy, một luồng chỉ tối ưu khi mạng dư không chứa chu trình âm và không chứa đường tăng có chi phí âm phù hợp với ràng buộc giá trị luồng đang xét. Đây là dạng quen thuộc của điều kiện tối ưu nguyên thủy–đối ngẫu.

16.2.4 Thuật toán SSP

Thuật toán *successive shortest path* bắt đầu từ luồng không. Trong mỗi vòng, tìm đường ngắn nhất từ s tới t trên mạng dư, đẩy một đơn vị hoặc đẩy theo dung lượng nghẽn, rồi cập nhật mạng dư. Nếu dùng thể năng để bảo đảm trọng số không âm sau mỗi lần chạy đường ngắn, ta nhận được cách cài đặt kinh điển của min-cost max-flow.

Thuật toán 1: *successive shortest path*.

1. Khởi tạo luồng hiện tại $x^* \leftarrow 0$.
2. Trong khi nghiệm tối ưu của mạng dư $G(x^*)$ còn có chi phí âm:
 - (a) dùng Bellman–Ford trên $G(x^*)$ để tìm một đường ngắn nhất P từ s tới t ;
 - (b) dựng luồng khả thi x^0 tương ứng với đường P và có dung lượng 1;
 - (c) cập nhật $x^* \leftarrow x^* + x^0$.
3. Trả về x^* .

Trong quá trình chạy, thuật toán không tạo chu trình âm hoặc đường tăng âm từ t về s , và chi phí $\text{cost}_{G(x^*)}(x^0)$ của mỗi lần tăng không giảm.

Bài viết nhấn mạnh rằng trong các bài OI, ta thường không chạy SSP nguyên dạng. Thay vào đó, ta chứng minh các đường ngắn nhất có hình dạng đặc biệt rồi duy trì chúng bằng heap, cây đoạn, DSU hoặc các hàm lồi rời rạc. Sau k lần tăng, chi phí nhận được chính là hàm $f(k)$: chi phí nhỏ nhất của luồng có giá trị đúng k .

16.3 Kỹ thuật và hướng suy nghĩ

16.3.1 Mô phỏng đường tăng

Khi đồ thị có nhiều đỉnh phụ nhưng đường tăng luôn đi qua một số lớp hoặc điểm mấu chốt, ta không cần giữ toàn bộ đồ thị. Chỉ cần biết đường ngắn nhất giữa các điểm mấu chốt, rồi ghép các đoạn đó lại. Nếu mỗi đoạn có thể được lấy bằng heap hoặc cấu trúc dữ liệu, một vòng SSP có thể giảm từ tuyến tính theo số cạnh xuống còn logarithm hoặc gần tuyến tính tổng cộng.

Ví dụ 1: AGCO18C Coins. Bài toán yêu cầu chia n người vào ba nhóm để tối đa hóa tổng vàng, bạc và đồng thu được. Một cách dựng luồng là ban đầu giả sử mọi người ở nhóm thứ nhất, sau đó mỗi người có các cạnh biểu diễn lợi ích khi chuyển sang nhóm thứ hai hoặc thứ ba. Đường tăng chỉ cần xét qua các điểm mẫu chốt tương ứng với ba nhóm và các lựa chọn người. Ta duy trì các ứng viên chuyển nhóm bằng heap; mỗi lần chọn đường tăng tốt nhất thì cập nhật trạng thái của đúng những người liên quan. Như vậy quá trình giống SSP nhưng không hề dựng mạng dư đầy đủ.

Ví dụ 2: NOI2019 Sequence. Cho hai dãy a_i, b_i , cần chọn hai dãy con độ dài k sao cho phần giao có kích thước ít nhất L và tổng giá trị lớn nhất. Nếu cố định số phần tử giao, bài toán có thể dựng thành luồng qua các lớp “chỉ chọn a ”, “chỉ chọn b ” và “chọn cả hai”. Cách đầu tiên là liệt kê số phần tử giao rồi mô phỏng đường tăng giữa các lớp bằng heap. Khi chuyển từ tham số $d - 1$ sang d , dung lượng của một số cạnh thay đổi rất ít nên có thể tái sử dụng cấu trúc dữ liệu.

Cách thứ hai nhìn bài toán qua quy hoạch tuyến tính. Sau khi nói lỏng tính nguyên, ma trận ràng buộc vẫn cho nghiệm nguyên; bài toán đối ngẫu dẫn tới một dạng chọn các chênh lệch tốt nhất. Từ đó ta lại thu được cùng các điểm mẫu chốt nhưng chứng minh ngắn hơn và có thể đạt $O(n \log n)$.

16.3.2 Tính lỗi

Gọi $f(k)$ là chi phí nhỏ nhất để gửi đúng k đơn vị luồng. Với mạng có dung lượng nguyên, dãy $f(k)$ là lời rời rạc trên miền xác định:

$$f(k+1) - f(k) \geq f(k) - f(k-1).$$

Trực giác là đường tăng thứ $k+1$ không thể rẻ hơn đường tăng thứ k sau khi các cơ hội rẻ đã bị dùng. Tính lỗi này cho phép tìm lượng luồng tối ưu bằng tam phân rời rạc, nhị phân trên độ dốc hoặc WQS binary search.

Ví dụ 3: bài toán khăn ăn. Trong mô hình kế hoạch khăn ăn, mỗi ngày cần một số khăn sạch, có thể mua mới hoặc giặt bằng các dịch vụ khác nhau với thời gian và chi phí khác nhau. Dựng đồ thị theo thời gian, cạnh mua mới và cạnh giặt đều là các lựa chọn chuyển trạng thái. Hàm chi phí theo số khăn luân chuyển là lồi, nên có thể tìm số lượng tối ưu bằng tìm kiếm trên lượng luồng, hoặc dùng tham lam dựa trên chênh lệch chi phí biên. Đây là ví dụ điển hình cho việc dùng tính lồi để tránh chạy luồng trên mọi giá trị.

16.3.3 Duy trì hàm tuyến tính từng đoạn

Nhiều mô hình luồng sau khi rút gọn không cần lưu từng đường tăng, mà chỉ cần lưu hàm chi phí của một khối. Hàm này thường lồi và tuyến tính từng đoạn. Khi ghép hai khối, ta cộng chập các hàm; khi thêm một cạnh dung lượng hữu hạn, ta cắt miền; khi thêm chi phí đơn vị, ta tịnh tiến độ dốc. Heap trái/phải, multiset hoặc cây cân bằng có thể duy trì các điểm gãy của hàm.

Đây là cách nhìn quan trọng trong các bài trên chuỗi một chiều và hai chiều ở phần sau: thay vì mô phỏng từng lần tăng, ta duy trì tập độ dốc còn hữu hiệu.

16.3.4 Đối ngẫu quy hoạch tuyến tính

Một mạng luồng chi phí có thể viết thành quy hoạch tuyến tính. Lấy đối ngẫu đôi khi biến bài toán từ “chọn các cạnh luồng” thành “gán thế năng cho đỉnh”. Nếu đồ thị có cấu trúc đường thẳng, cây hoặc khoảng, các ràng buộc đối ngẫu có thể trở thành DP hoặc bài toán hồi quy đơn điệu.

Ví dụ 4: ZJOI2020 Sequence. Bài toán cho phép thực hiện các thao tác trên đoạn chắn, đoạn lẻ hoặc toàn bộ đoạn để giảm dãy về 0, cần số thao tác ít nhất. Sau khi biến đổi, mô hình có thể viết như một luồng chi phí với các cạnh biểu diễn việc giảm trên từng lớp chỉ số. Lấy đối ngẫu đưa bài toán về chọn các biến thỏa ràng buộc lân cận, từ đó có lời giải DP gọn hơn mô hình luồng ban đầu.

Mô hình hồi quy đẳng trường. Một dạng thường gặp là cần chọn dãy đơn điệu $x_1 \leq x_2 \leq \dots \leq x_n$ để tối ưu tổng các hàm lồi theo từng vị trí. Trong ngôn ngữ luồng, các cạnh giữa vị trí liên tiếp ép điều kiện đơn điệu; trong đối ngẫu, lời giải có thể mô tả bằng cách gộp các đoạn có trung bình không đúng thứ tự. Thuật toán PAV và các biến thể heap chính là cách mô phỏng luồng chi phí trên mô hình này.

16.4 Một số đồ thị đặc biệt

16.4.1 Mô hình chuột vào hang

Bài viết dùng mô hình “chuột vào hang” để thống nhất nhiều bài. Trên một đường thẳng, mỗi con chuột phải đi tới một hang còn chỗ; chi phí là khoảng cách hoặc trọng số đường đi. Nếu mọi cạnh chỉ cho đi theo một chiều, ta có chuỗi một chiều; nếu được đi hai chiều, ta có chuỗi hai chiều. Các biến thể với dung lượng cạnh, mở/đóng vị trí hoặc truy vấn khoảng đều có thể xem như luồng chi phí trên đường.

16.4.2 Chuỗi một chiều

Trong chuỗi một chiều, luồng chỉ đi từ trái sang phải. Điều kiện tối ưu trở nên đơn giản: đường tăng thường nối một con chuột chưa ghép với một hang chưa đầy, hoặc đổi ghép qua một đoạn liên tiếp. Vì không có chu trình âm phức tạp, ta có thể dùng cây đoạn để duy trì đường tăng tốt nhất của mỗi đoạn và ghép thông tin từ hai con.

Ví dụ 5: Skydiving Survival. Mỗi người nhảy dù cần tìm vị trí hạ cánh hợp lệ trên một đường thẳng. Khi thêm một người hoặc một vị trí, trạng thái ghép thay đổi như một đường tăng trong mô hình chuột–hang. Cây đoạn lưu ứng viên “chuột thừa”, “hang thừa” và chi phí tốt nhất khi ghép qua biên. Khi một bên đã đầy, nó có thể được xem ngược lại như nhu cầu của phía kia; cách chuyển vai này chính là cạnh ngược trong mạng dư.

Một lời giải khác duy trì hàm lồi tuyến tính từng đoạn của mỗi tiền tố. Mỗi con chuột thêm một điểm gãy, mỗi hang xóa hoặc dịch một điểm gãy. Hai lời giải tương ứng với hai cách nhìn của cùng quá trình SSP.

Ví dụ 6: duy trì chuỗi một chiều. Khi có cập nhật điểm và truy vấn đáp án toàn cục, ta dùng cây đoạn. Với mỗi nút, lưu bốn loại thông tin: nhu cầu còn lại ở đầu trái, khả năng cung cấp ở đầu phải, chi phí tốt nhất nội bộ và đường tăng tốt nhất đi qua biên. Phép hợp hai nút là chạy một bước “mô phỏng luồng” giữa phần dư của con trái và con phải.

Ví dụ 7: Trade. Bài toán có nhiều truy vấn khoảng trên một chuỗi. Nếu tách theo khối, các cạnh nằm hoàn toàn trong khối được tiền xử lý, còn phần đi qua biên giữa hai khối được biểu diễn bởi hàm chi phí theo lượng luồng qua cạnh giữa. Do hàm này lồi, có thể tìm lượng luồng tối ưu bằng tam phân hoặc nhị phân trên độ dốc. Kết quả là một thuật toán chia khối kết hợp cây đoạn, nhanh hơn nhiều so với chạy luồng cho từng truy vấn.

Ví dụ 8: lập lịch công việc. Mỗi công việc có thời điểm phát hành, hạn chót và giá trị. Chọn các công việc để tối đa hóa lợi ích có thể dựng thành chuỗi một chiều: thời gian là các đỉnh, cạnh dung lượng biểu diễn số máy hoặc số vị trí còn trống, còn công việc là cạnh nhảy qua một khoảng thời gian. WQS binary search biến ràng buộc số lượng công việc thành phạt tuyến tính; sau đó DSU có thể tìm vị trí trống gần nhất để mô phỏng đường tăng.

Ví dụ 9: NOI2017 Vegetables. Mỗi loại rau có số lượng, giá trị giảm dần theo ngày và giới hạn bán. Cách nhìn luồng là gửi đơn vị hàng hóa từ ngày sản xuất tới ngày bán với chi phí âm bằng lợi nhuận. Lời giải thứ nhất dùng cây đoạn để mô phỏng việc lấy đơn vị lợi nhuận biên tốt nhất. Lời giải thứ hai biến mỗi đơn vị lợi nhuận thành một công việc có hạn chót, rồi dùng DSU để đặt nó vào ngày muộn nhất còn chỗ. Hai cách làm cùng phản ánh một mạng dư nhưng cách thứ hai ngắn và thực dụng hơn.

Ví dụ 10: Interval. Trong mô hình đoạn có dung lượng cạnh bằng 1, các thao tác thêm/xóa chuột, hàng hoặc sửa dung lượng đều làm thay đổi cục bộ mạng dư. Cây đoạn lưu đường tăng tốt nhất và chu trình âm tốt nhất của từng đoạn. Khi ghép hai đoạn, chỉ có những đường đi qua biên mới cần xét thêm. Cách này cho phép xử lý cập nhật động trong $O(\log n)$ hoặc $O(\log^2 n)$ tùy phiên bản.

16.4.3 Chuỗi hai chiều

Trong chuỗi hai chiều, luồng có thể đi cả trái lẫn phải. Điều kiện tối ưu phức tạp hơn vì đường tăng có thể đổi hướng, nhưng cấu trúc lời vẫn còn. Một cách là giữ hai heap cho các độ dốc bên trái và bên phải; khi thêm một điểm mới, ta đưa nó vào phía thích hợp rồi cân bằng bằng cách chuyển điểm gãy giữa hai heap.

Ví dụ 11: chuỗi hai chiều cơ bản. Với các chuột và hàng trên đường thẳng, chi phí ghép là khoảng cách. Lời giải cây đoạn lưu thông tin tốt nhất ở hai đầu đoạn; lời giải hàm lồi lưu các điểm gãy. Khi nối thêm một vị trí, thủ tục EXTEND cập nhật cặp heap và giá trị hằng của hàm. Nếu một điểm gãy vượt qua phía còn lại, nó tương ứng với một đường tăng vừa được thực hiện trong mạng dư.

Thuật toán 2: EXTEND(Q_0, Q_1, f_0, x, y, c). Hai heap nhỏ Q_0, Q_1 lần lượt duy trì các điểm gãy còn có thể dùng ở hai phía, còn f_0 là giá trị hằng hiện tại của hàm lồi.

1. Nếu Q_1 rỗng hoặc $Q_1.\text{top}() \geq -c - x$, đưa $c - y$ vào Q_0 .
2. Ngược lại:
 - (a) đặt $\text{val} \leftarrow Q_1.\text{top}() + c + x$;
 - (b) cập nhật $f_0 \leftarrow f_0 + \text{val}$;
 - (c) xóa phần tử nhỏ nhất của Q_1 ;
 - (d) đưa $-c - x$ vào Q_1 ;
 - (e) đưa $c - y - \text{val}$ vào Q_0 .

Ví dụ 12: UER #8 BRS. Bài toán thêm dung lượng trên các đoạn của chuỗi hai chiều. Với một hàng có dung lượng lớn hơn 1, ta gom các điểm gãy cùng giá trị thành từng cặp (giá trị, số lượng). Thủ tục mở rộng trở thành:

Thuật toán 3: EXTEND($Q_0, Q_1, f_0, x, y, c, cnt$).

1. Ghi nhớ $cnt_s \leftarrow cnt$.
2. Trong khi $cnt > 0$:
 - (a) nếu Q_1 rỗng hoặc $Q_1.top().first \geq -c - x$, đưa $(c - y, cnt)$ vào Q_0 rồi dừng vòng lặp;
 - (b) ngược lại, lấy $s \leftarrow Q_1.top()$ và xóa nó khỏi Q_1 ;
 - (c) đặt $val \leftarrow s.first + c + x$ và $cnt_0 \leftarrow \min(s.second, cnt)$;
 - (d) cập nhật $f_0 \leftarrow f_0 + val \cdot cnt_0$;
 - (e) đưa $(c - y - val, cnt_0)$ vào Q_0 ;
 - (f) đặt $s.second \leftarrow s.second - cnt_0$ và $cnt \leftarrow cnt - cnt_0$;
 - (g) nếu $s.second > 0$, đưa s trở lại Q_1 .
3. Nếu $cnt_s - cnt > 0$, đưa $(-c - x, cnt_s - cnt)$ vào Q_1 .

Khi thêm chuột, vòng lặp chỉ chạy một lần. Khi thêm hàng, những vòng lặp trung gian đều làm giảm số phần tử trong Q_1 ; với thể năng $\Phi(D) = |Q_1|$, phần chi phí thừa được khấu hao. Do đó tổng số vòng lặp là $O(n + m)$, và tổng độ phức tạp là $\Theta((n + m) \log(n + m))$.

Ví dụ 13: IOI2017 Wiring. Bài *Wiring* là mô hình ghép hai màu điểm trên đường thẳng với chi phí khoảng cách. Lời giải DP chính thức có thể được hiểu như mô phỏng luồng trên chuỗi hai chiều sau khi gom các đoạn cùng màu. Các đoạn đơn sắc tạo ra hàm lồi; khi chuyển sang đoạn màu khác, ta thực hiện một loạt đường tăng tốt nhất giữa hai tập điểm.

Ví dụ 14: CTSC2010 Product Sales. Các truy vấn bán sản phẩm có thể xem như cố định một phần luồng trên chuỗi rồi hỏi chi phí tối ưu của phần còn lại. Nếu biết trước cạnh nào bị ép lưu lượng, ta chia chuỗi thành các đoạn độc lập; mỗi đoạn duy trì hàm lồi riêng và ghép qua các điều kiện biên. Cách này biến một bài luồng động thành bài toán ghép hàm tuyến tính từng đoạn.

Ví dụ 15: truy vấn cố định lưu lượng cạnh. Khi mỗi truy vấn yêu cầu cố định lưu lượng trên một cạnh, phần bên trái và bên phải của cạnh chỉ tương tác qua đúng một biến. Hàm chi phí theo biến đó là lồi, nên đáp án là tổng hai hàm lồi cộng thêm hằng số. Nếu tiền xử lý prefix/suffix của các hàm, mỗi truy vấn có thể trả lời bằng tìm điểm gãy thích hợp thay vì chạy lại luồng.

16.4.4 Cây

Trên cây, đường giữa hai điểm là duy nhất nên mô hình chuột–hàng mở rộng khá tự nhiên. Khó khăn nằm ở chỗ nhiều nhánh cùng tranh dung lượng qua một cạnh. Thông thường ta xử lý từ lá lên gốc; mỗi cây con trả về một hàm lồi hoặc một tập điểm gãy biểu diễn chi phí khi còn dư bao nhiêu đơn vị cần chuyển lên cha.

Ví dụ 16: LibreOJ NOI Round #2 Gold Miner. Bài toán khai thác vàng trên cây có thể dựng thành việc chọn một số luồng đi qua các cạnh với lợi ích ở đỉnh và chi phí di chuyển. Mỗi cây con trả về hàm chi phí theo số đơn vị gửi lên cha. Khi gộp các con, ta cộng chập các hàm lồi; heap hoặc hàng đợi ưu tiên dùng để lấy các chênh lệch tốt nhất. Đây là phiên bản cây của mô phỏng đường tăng.

Ví dụ 17: ICPC WF 2018 Conquer The World. Bài này có các thành phố cần phân phối quân hoặc tài nguyên trên cây với chi phí theo khoảng cách. Xem phần dư ở mỗi đỉnh là chuột hoặc hang, ta ghép từ dưới lên: phần cùng cây con được triệt tiêu trước, phần dư mới đi qua cạnh cha và chịu thêm chi phí. Dùng multiset hoặc heap có lazy tag để cộng chi phí cạnh cho toàn bộ phần dư, ta thu được lời giải gần tuyến tính.

16.5 Tổng kết

Các ví dụ trên cho thấy “mô phỏng luồng chi phí” không phải một kỹ thuật đơn lẻ, mà là một cách đọc mô hình. Trước hết dựng hoặc tưởng tượng mạng luồng để hiểu điều kiện tối ưu. Sau đó quan sát mạng dư: đường tăng đi qua những điểm nào, chi phí theo lượng luồng có lỗi không, đối ngẫu có dạng DP không, và đồ thị có phải đường thẳng hoặc cây không. Khi câu trả lời là có, ta thay thuật toán luồng tổng quát bằng cấu trúc dữ liệu đặc thù nhưng vẫn giữ được chứng minh tối ưu từ luồng chi phí.

Lời cảm ơn

Tác giả cảm ơn các thầy cô, bạn bè và những người đã trao đổi về các mô hình luồng chi phí, đặc biệt là các lời giải và ghi chép công khai về mô phỏng luồng chi phí trong OI. Các ví dụ trong bài lấy từ nhiều kỳ thi và tài liệu khác nhau; một số tên bài được giữ theo tên tiếng Anh hoặc tên quen dùng khi cần.

Tài liệu tham khảo

1. Bài giảng và ghi chép về thuật toán luồng chi phí cực tiểu trong OI.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest và Clifford Stein. *Introduction to Algorithms*.
3. Các lời giải công khai của NOI2017 *Vegetables*, IOI2017 *Wiring* và những bài liên quan tới mô hình chuột–hang.
4. Yang Yuchen. *Ghi chép về mô phỏng luồng chi phí*. Blog Luogu, <https://www.luogu.com.cn/>.